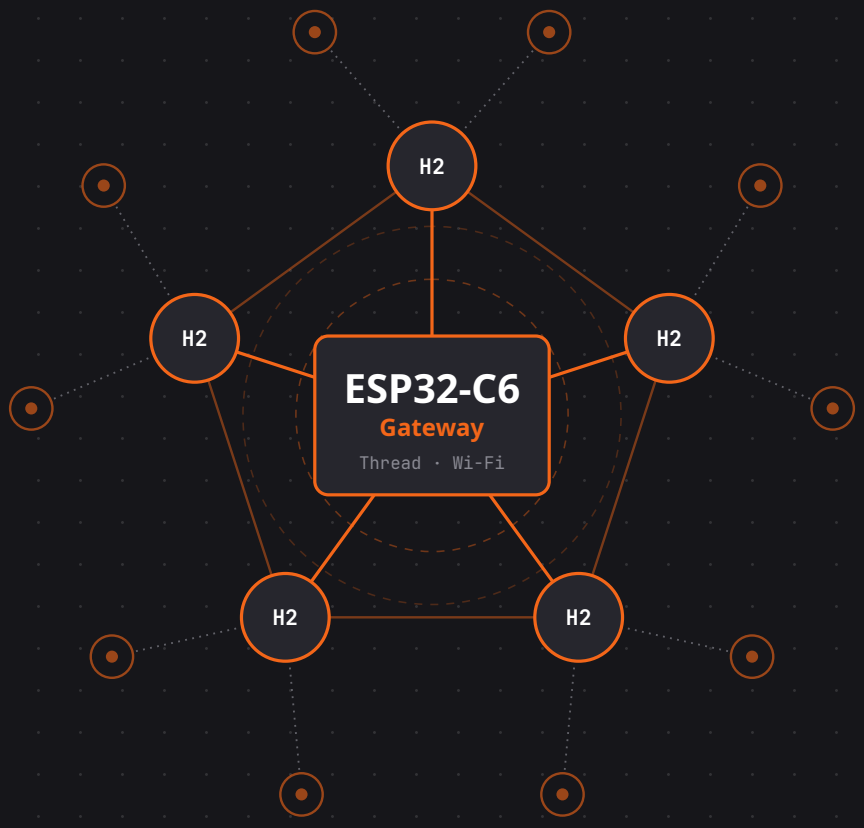
EDISI MAHASISWA

IOT COMMUNICATION LAB · BUKU KERJA LABORATORIUM

Petunjuk Praktikum Wireless Sensor Network & Internet of Things

ESP32-H2 & ESP32-C6 Version - Buku Pegangan untuk Mahasiswa



BLE · IEEE 802.15.4 · Zigbee · Thread · Wi-Fi/MQTT

PENULIS

Akhmad Hendriawan

TAHUN

2026

Lisensi Dokumen

Copyright © 2026 oleh Akhmad Hendriawan.

Dokumen ini disusun sebagai petunjuk praktikum mata kuliah Wireless Sensor Network dan Internet of Things. Seluruh isinya dapat digunakan dan disebarkan secara bebas untuk tujuan bukan komersial (nonprofit), dengan syarat tidak menghapus atau mengubah atribut penulis dan pernyataan copyright yang disertakan di dalamnya.

Seluruh angka hasil pengujian yang dicantumkan pada buku ini berasal dari pengukuran pada perangkat nyata, bukan nilai contoh. Kode sumber setiap modul berada pada repositori praktikum, di dalam folder `weekNN_nama` yang disebut pada awal tiap modul.

Departemen Teknik Elektro, Politeknik Elektronika Negeri Surabaya

Daftar Isi

Pendahuluan 19
Tentang Lab Ini 19
Struktur Setiap Modul 19
Keterkaitan Antar-Modul 20
Capaian Pembelajaran 20
Arsitektur Sistem Lab 21
Daftar Modul 21
Status Verifikasi Perangkat Keras 23
Koeksistensi Wi-Fi dan 802.15.4 pada ESP32-C6 24
Perangkat Keras 26
Perangkat Lunak 26
Kode Sumber dan Repositori 26
Aturan Laboratorium dan Keselamatan 26
Sistem Penilaian 27

1 Modul 01 — Komunikasi BLE Point-to-Point 28
1.1 Pendahuluan 28
1.2 Capaian Pembelajaran 28
1.3 Dasar Teori (Secukupnya) 29
1.4 Topologi 30
1.5 Alat yang Digunakan 30
1.6 Kode Program 30
1.7 Build dan Flash 33
1.8 Percobaan 33
1.8.1 EXP-01 — Advertising dan Scanning 33
1.8.2 EXP-02 — Pembentukan Tautan 34
1.8.3 EXP-03 — Ketahanan Tautan 35
1.9 Pengukuran 35
1.10 Analisis 36
1.11 Concept Check 36
1.12 Challenge (Tugas Modifikasi) 36
1.13 Laporan 37

2 Modul 02 — Pertukaran Data Dua Arah via BLE 38
2.1 Pendahuluan 38
2.2 Capaian Pembelajaran 38
2.3 Dasar Teori (Secukupnya) 39
2.4 Topologi 40

2.5 Alat yang Digunakan . . . 40
2.6 Kode Program . . . 40
2.7 Build dan Flash . . . 44
2.8 Percobaan . . . 44
2.8.1 EXP-01 — Connect dan Subscribe . . . 44
2.8.2 EXP-02 — Downlink: Server ke Client (notify) . . . 45
2.8.3 EXP-03 — Uplink dan Echo: Client ke Server (write) . . . 45
2.9 Verifikasi Hardware — Log Referensi . . . 46
2.10 Pengukuran . . . 46
2.11 Analisis . . . 47
2.12 Concept Check . . . 47
2.13 Challenge (Tugas Modifikasi) . . . 47
2.14 Laporan . . . 48

3 Modul 03 — GATT Server dan Client Terstruktur . . . 49
3.1 Pendahuluan . . . 49
3.2 Capaian Pembelajaran . . . 49
3.3 Dasar Teori (Secukupnya) . . . 50
3.4 Topologi . . . 51
3.5 Alat yang Digunakan . . . 51
3.6 Kode Program . . . 51
3.7 Build dan Flash . . . 55
3.8 Percobaan . . . 55
3.8.1 EXP-01 — Service Discovery . . . 55
3.8.2 EXP-02 — Read: Client Menarik Data . . . 55
3.8.3 EXP-03 — Write: Client Mengirim Perintah . . . 56
3.9 Verifikasi Hardware — Log Referensi . . . 56
3.10 Pengukuran . . . 57
3.11 Analisis . . . 57
3.12 Concept Check . . . 58
3.13 Challenge (Tugas Modifikasi) . . . 58
3.14 Laporan . . . 58

4 Modul 04 — Telemetry Berkala lewat Notify . . . 59
4.1 Pendahuluan . . . 59
4.2 Capaian Pembelajaran . . . 60
4.3 Dasar Teori (Secukupnya) . . . 60
4.4 Topologi . . . 61
4.5 Alat yang Digunakan . . . 61
4.6 Kode Program . . . 61
4.7 Build dan Flash . . . 64
4.8 Percobaan . . . 65
4.8.1 EXP-01 — Subscribe dan Stream Start . . . 65
4.8.2 EXP-02 — Aliran Telemetri . . . 65
4.8.3 EXP-03 — Kontinuitas dan Pemulihan . . . 66

4.9 Verifikasi Hardware — Log Referensi . . . 66
4.10 Pengukuran . . . 67
4.11 Analisis . . . 67
4.12 Concept Check . . . 68
4.13 Challenge (Tugas Modifikasi) . . . 68
4.14 Laporan . . . 68

5 Modul 05 — Multi-Node BLE (Topologi Bintang) . . . 69
5.1 Pendahuluan . . . 69
5.2 Capaian Pembelajaran . . . 70
5.3 Dasar Teori (Secukupnya) . . . 70
5.4 Topologi . . . 71
5.5 Alat yang Digunakan . . . 71
5.6 Kode Program . . . 71
5.7 Build dan Flash . . . 75
5.8 Percobaan . . . 76
5.8.1 EXP-01 — Menyalakan Dua Peripheral . . . 76
5.8.2 EXP-02 — Dua Koneksi dan Subscribe . . . 76
5.8.3 EXP-03 — Laju dan Ketahanan . . . 77
5.9 Verifikasi Hardware — Log Referensi . . . 77
5.10 Pengukuran . . . 78
5.11 Analisis . . . 78
5.12 Concept Check . . . 79
5.13 Challenge (Tugas Modifikasi) . . . 79
5.14 Laporan . . . 79

6 Modul 05B — Mini Project: Smart Sensor Jendela dan Pintu . . . 81
6.1 Pendahuluan . . . 81
6.2 Capaian Pembelajaran . . . 82
6.3 Dasar Teori (Secukupnya) . . . 82
6.4 Topologi . . . 83
6.5 Alat yang Digunakan . . . 83
6.6 Kode Program . . . 85
6.7 Build dan Flash . . . 94
6.7.1 Memantau Satu per Satu dengan picocom . . . 94
6.7.2 Memantau Ketiga Board Sekaligus . . . 95
6.8 Percobaan . . . 95
6.8.1 EXP-01 — Sensor Bekerja Sendiri (tanpa hub) . . . 95
6.8.2 EXP-02 — Hub Memantau Dua Sensor . . . 96
6.8.3 EXP-03 — Keandalan Sistem Alarm . . . 97
6.9 Verifikasi Build (Referensi) . . . 98
6.10 Pengukuran . . . 98
6.11 Analisis . . . 99
6.12 Concept Check . . . 99

6.13 Challenge (Tugas Modifikasi) . . . 100
6.14 Laporan . . . 100

7 Modul 05C — Pager Restoran: Notifikasi BLE One-to-Many . . . 101
7.1 Pendahuluan . . . 101
7.2 Capaian Pembelajaran . . . 102
7.3 Dasar Teori (Secukupnya) . . . 102
7.4 Topologi . . . 104
7.5 Alat yang Digunakan . . . 104
7.6 Kode Program . . . 106
7.7 Build dan Flash . . . 115
7.7.1 Membuka Terminal Pengujian . . . 115
7.8 Percobaan . . . 117
7.8.1 EXP-01 — Pager Menyala Sendiri-sendiri . . . 117
7.8.2 EXP-02 — Panggilan Terarah . . . 117
7.8.3 EXP-03 — ACK dan Waktu Tanggap . . . 118
7.8.4 EXP-04 — Perintah yang Harus Ditolak . . . 119
7.9 Verifikasi Hardware — Log Referensi . . . 119
7.10 Pengukuran . . . 120
7.11 Analisis . . . 121
7.12 Concept Check . . . 121
7.13 Challenge (Tugas Modifikasi) . . . 121
7.14 Laporan . . . 122

8 Modul 06 — Relay Multi-Hop di atas BLE . . . 123
8.1 Pendahuluan . . . 123
8.2 Capaian Pembelajaran . . . 123
8.3 Dasar Teori (Secukupnya) . . . 124
8.4 Topologi . . . 125
8.5 Alat yang Digunakan . . . 125
8.6 Kode Program . . . 125
8.7 Build dan Flash . . . 130
8.8 Percobaan . . . 130
8.8.1 EXP-01 — Menyalakan Relay (Node B) . . . 130
8.8.2 EXP-02 — Menutup Rantai (Node A dan Node C) . . . 131
8.8.3 EXP-03 — Mode Kegagalan Rantai . . . 131
8.9 Verifikasi Hardware — Log Referensi . . . 132
8.10 Pengukuran . . . 132
8.11 Analisis . . . 133
8.12 Concept Check . . . 133
8.13 Challenge (Tugas Modifikasi) . . . 133
8.14 Laporan . . . 134

9 Modul 07 — IEEE 802.15.4 Raw Frame (P2P) . . . 135
9.1 Pendahuluan . . . 135

9.2 Capaian Pembelajaran . . . 135
9.3 Dasar Teori (Secukupnya) . . . 136
9.4 Topologi . . . 137
9.5 Alat yang Digunakan . . . 138
9.6 Kode Program . . . 138
9.7 Build dan Flash . . . 143
9.8 Percobaan . . . 144
9.8.1 EXP-01 — Konfigurasi Radio dan Anatomi Frame . . . 144
9.8.2 EXP-02 — Pertukaran PING dan PONG . . . 144
9.8.3 EXP-03 — Isolasi Channel dan RTT . . . 145
9.8.4 EXP-04 — Efek Parameter: PAN ID, Channel, Broadcast . . . 146
9.8.5 EXP-05 — Promiscuous Mode: Ada dan Tidaknya Filter MAC . . . 147
9.9 Verifikasi Hardware — Log Referensi . . . 148
9.10 Pengukuran . . . 149
9.11 Analisis . . . 150
9.12 Concept Check . . . 150
9.13 Challenge (Tugas Modifikasi) . . . 150
9.14 Laporan . . . 151

10 Modul 08 — Zigbee P2P: Join dan Binding . . . 152
10.1 Pendahuluan . . . 152
10.2 Capaian Pembelajaran . . . 152
10.3 Dasar Teori (Secukupnya) . . . 153
10.4 Topologi . . . 156
10.5 Alat yang Digunakan . . . 156
10.6 Kode Program . . . 156
10.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 160
10.7 Build dan Flash . . . 161
10.8 Percobaan . . . 162
10.8.1 EXP-01 — Pembentukan Jaringan (Coordinator) . . . 162
10.8.2 EXP-02 — Join dan Binding (End Device) . . . 162
10.8.3 EXP-03 — Keandalan Kontrol . . . 163
10.9 Verifikasi Hardware — Log Referensi . . . 164
10.10 Pengukuran . . . 164
10.11 Analisis . . . 165
10.12 Concept Check . . . 165
10.13 Challenge (Tugas Modifikasi) . . . 166
10.14 Laporan . . . 166

11 Modul 09 — Zigbee Multi-Node dan Binding Table . . . 167
11.1 Pendahuluan . . . 167
11.2 Capaian Pembelajaran . . . 167
11.3 Dasar Teori (Secukupnya) . . . 168
11.4 Topologi . . . 169
11.5 Alat yang Digunakan . . . 169

11.6 Kode Program . . . 169
11.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 173
11.7 Build dan Flash . . . 174
11.8 Percobaan . . . 175
11.8.1 EXP-01 — Pembentukan Jaringan dan Join Ganda . . . 175
11.8.2 EXP-02 — Binding Table dan Kontrol Otomatis . . . 175
11.8.3 EXP-03 — Jarak dan Kehilangan Node . . . 177
11.9 Verifikasi Hardware — Log Referensi . . . 177
11.10 Pengukuran . . . 178
11.11 Analisis . . . 179
11.12 Concept Check . . . 179
11.13 Challenge (Tugas Modifikasi) . . . 179
11.14 Laporan . . . 179

12 Modul 10 — Zigbee Mesh: Routing Multi-Hop . . . 181
12.1 Pendahuluan . . . 181
12.2 Capaian Pembelajaran . . . 181
12.3 Dasar Teori (Secukupnya) . . . 182
12.4 Topologi . . . 183
12.5 Alat yang Digunakan . . . 183
12.6 Kode Program . . . 183
12.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 187
12.7 Build dan Flash . . . 188
12.8 Percobaan . . . 188
12.8.1 EXP-01 — Join Bertingkat . . . 188
12.8.2 EXP-02 — Kontrol Multi-Hop . . . 189
12.8.3 EXP-03 — Varian Topologi (inti modul) . . . 190
12.9 Verifikasi Hardware — Log Referensi . . . 191
12.10 Pengukuran . . . 191
12.11 Analisis . . . 192
12.12 Concept Check . . . 192
12.13 Challenge (Tugas Modifikasi) . . . 193
12.14 Laporan . . . 193

13 Modul 11 — Thread: Datagram UDP di atas IPv6 . . . 194
13.1 Pendahuluan . . . 194
13.2 Capaian Pembelajaran . . . 194
13.3 Dasar Teori (Secukupnya) . . . 195
13.4 Topologi . . . 196
13.5 Alat yang Digunakan . . . 197
13.6 Kode Program . . . 198
13.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 201
13.7 Build dan Flash . . . 202
13.8 Percobaan . . . 202
13.8.1 EXP-01 — Pembentukan Jaringan dan Attach . . . 202

13.8.2 EXP-02 — PING Multicast dan PONG Unicast . . . 203
13.8.3 EXP-03 — Jarak, Kehilangan Peer, dan Latency . . . 203
13.9 Verifikasi Hardware — Log Referensi . . . 204
13.10 Pengukuran . . . 204
13.11 Analisis . . . 205
13.12 Concept Check . . . 205
13.13 Challenge (Tugas Modifikasi) . . . 206
13.14 Laporan . . . 206

14 Modul 12 — Mesh Thread IPv6 (Many-to-Many) . . . 207
14.1 Pendahuluan . . . 207
14.2 Capaian Pembelajaran . . . 207
14.3 Dasar Teori (Secukupnya) . . . 208
14.4 Topologi . . . 209
14.5 Alat yang Digunakan . . . 209
14.6 Kode Program . . . 210
14.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 215
14.7 Build dan Flash . . . 215
14.8 Percobaan . . . 216
14.8.1 EXP-01 — Pembentukan Mesh dan Pemilihan Role . . . 216
14.8.2 EXP-02 — Multicast Many-to-Many . . . 216
14.8.3 EXP-03 — Perubahan Topologi dan Kegagalan Node . . . 217
14.9 Verifikasi Hardware — Log Referensi . . . 218
14.10 Pengukuran . . . 218
14.11 Analisis . . . 219
14.12 Concept Check . . . 219
14.13 Challenge (Tugas Modifikasi) . . . 219
14.14 Laporan . . . 220

15 Modul 13 — Gateway Thread ke Wi-Fi (H2 + C6) . . . 221
15.1 Pendahuluan . . . 221
15.2 Capaian Pembelajaran . . . 221
15.3 Dasar Teori (Secukupnya) . . . 222
15.4 Topologi . . . 223
15.5 Alat yang Digunakan . . . 223
15.6 Kode Program . . . 224
15.6.1 Channel antar Kelompok (wajib kelas paralel) . . . 229
15.6.2 Server HTTP Lokal — `http_sink.py` . . . 230
15.7 Build dan Flash . . . 231
15.8 Percobaan . . . 232
15.8.1 EXP-01 — Menyalakan Jaringan Thread . . . 232
15.8.2 EXP-02 — Telemetri lewat Thread . . . 232
15.8.3 EXP-03 — Penerusan ke Wi-Fi (HTTP POST) . . . 234
15.9 Verifikasi Hardware — Log Referensi . . . 235
15.9.1 Koeksistensi Wi-Fi dan 802.15.4 . . . 235
15.9.2 Delapan Perbaikan Kode yang Lahir dari Uji Ini . . . 236

15.10 Pengukuran . . . 237
15.11 Analisis . . . 237
15.12 Concept Check . . . 237
15.13 Challenge (Tugas Modifikasi) . . . 238
15.14 Laporan . . . 238

16 Modul 14 — MQTT: Publish dan Subscribe . . . 239
16.1 Pendahuluan . . . 239
16.2 Capaian Pembelajaran . . . 239
16.3 Dasar Teori (Secukupnya) . . . 240
16.4 Topologi . . . 241
16.5 Alat yang Digunakan . . . 241
16.6 Kode Program . . . 242
16.7 Build dan Flash . . . 244
16.8 Percobaan . . . 244
16.8.1 EXP-01 — Koneksi Wi-Fi dan Broker . . . 244
16.8.2 EXP-02 — Publish Berkala . . . 244
16.8.3 EXP-03 — Perintah Turun dan Pemulihan . . . 245
16.9 Verifikasi Hardware — Log Referensi . . . 246
16.10 Pengukuran . . . 247
16.11 Analisis . . . 247
16.12 Concept Check . . . 247
16.13 Challenge (Tugas Modifikasi) . . . 248
16.14 Laporan . . . 248

17 Modul 15 — Pipeline IoT End-to-End . . . 249
17.1 Pendahuluan . . . 249
17.2 Capaian Pembelajaran . . . 249
17.3 Dasar Teori (Secukupnya) . . . 250
17.4 Topologi . . . 251
17.5 Alat yang Digunakan . . . 251
17.6 Kode Program . . . 252
17.7 Build dan Flash . . . 256
17.8 Percobaan . . . 257
17.8.1 EXP-01 — Inisialisasi Gateway . . . 257
17.8.2 EXP-02 — Thread TX lalu Forward MQTT . . . 258
17.8.3 EXP-03 — Verifikasi End-to-End . . . 258
17.9 Verifikasi Hardware — Log Referensi . . . 259
17.9.1 Rantai Penuh Terbukti — dengan Satu Syarat Wajib . . . 259
17.9.2 Perbaikan Kode yang Lahir dari Uji Ini . . . 261
17.10 Pengukuran . . . 261
17.11 Analisis . . . 262
17.12 Concept Check . . . 262
17.13 Challenge (Tugas Modifikasi) . . . 262
17.14 Laporan . . . 263

18 Modul 16 — Proyek: Benchmark Protokol IoT 264
18.1 Pendahuluan 264
18.2 Capaian Pembelajaran 265
18.3 Dasar Teori (Secukupnya) 265
18.4 Topologi 266
18.5 Alat yang Digunakan 267
18.6 Kode Program 267
18.7 Build dan Flash 271
18.8 Percobaan 271
18.8.1 EXP-01 — Sensor BLE dan Advertising 271
18.8.2 EXP-02 — Gateway: Scan, Connect, Subscribe 272
18.8.3 EXP-03 — Verifikasi End-to-End dan Variasi 273
18.9 Verifikasi Hardware — Log Referensi 273
18.10 Pengukuran 274
18.11 Analisis 275
18.12 Concept Check 276
18.13 Challenge (Tugas Modifikasi) 276
18.14 Laporan 276

Bagian Pelengkap 278

A Skematik Board ESP32-H2 278
A.1 Pin yang Dipakai di Buku Ini 278

B Modul 00A — Blinky ESP32-H2 280
B.1 Pendahuluan 280
B.2 Capaian Pembelajaran 280
B.3 Dasar Teori (Secukupnya) 281
B.4 Alat yang Digunakan 281
B.5 Kode Program 282
B.6 Build dan Flash 282
B.7 Percobaan 283
B.7.1 EXP-01 — Build dan Flash Pertama 283
B.7.2 EXP-02 — Warna sebagai Data 283
B.8 Analisis 284
B.9 Concept Check 284
B.10 Challenge (Tugas Modifikasi) 284
B.11 Laporan 284

C Modul 00B — Tombol BOOT ke LED 285
C.1 Pendahuluan 285
C.2 Capaian Pembelajaran 285
C.3 Dasar Teori (Secukupnya) 286
C.4 Alat yang Digunakan 286
C.5 Kode Program 287

C.6 Build dan Flash **287**
C.7 Percobaan **288**
C.7.1 EXP-01 — Input Mengendalikan Output 288
C.7.2 EXP-02 — GPIO9 sebagai Strapping Pin 288
C.8 Analisis **289**
C.9 Concept Check **289**
C.10 Challenge (Tugas Modifikasi) **289**
C.11 Laporan **289**

Daftar Gambar

7.1 Sesi pengujian: monitor tiga pager di jendela kiri, terminal kasir cutecom di jendela kanan 117
A.1 Skematik Waveshare ESP32-H2-DEV-KIT-N4 279

Daftar Tabel

P.1 Sebelas bagian baku pada setiap modul praktikum . . . 19
P.2 Kontrak data yang dipertahankan lintas modul . . . 20
P.3 Pembagian blok protokol sepanjang satu semester . . . 21
P.4 Daftar modul praktikum beserta folder kode sumbernya . . . 21
P.5 Status verifikasi tiap modul pada perangkat nyata . . . 23
P.6 Perbaikan kode hasil pengujian pada ESP32-H2 . . . 24
P.7 Keberhasilan rantai end-to-end Modul 15 pada tiga access point . . . 24
P.8 Perbandingan dua pipeline end-to-end pada kondisi uji yang sama . . . 24
P.9 Perbaikan kode hasil pengujian pada ESP32-C6 . . . 25
P.10 Perkakas pendukung pada folder `tools/` . . . 25
P.11 Perangkat keras yang diperlukan sepanjang satu semester . . . 26
P.12 Bobot penilaian praktikum . . . 27
1.1 Peta modul blok BLE . . . 28
1.2 Istilah kerja Modul 01 . . . 29
1.3 Peran dan identitas radio tiap node Modul 01 . . . 30
1.4 Alat dan bahan Modul 01 . . . 30
1.5 Lembar pengamatan EXP-01 . . . 34
1.6 Lembar pengamatan EXP-03 . . . 35
1.7 Referensi interpretasi RSSI . . . 36
1.8 Lembar pengukuran jarak, RSSI, dan keberhasilan koneksi . . . 36
2.1 Peta modul blok BLE . . . 38
2.2 Istilah kerja Modul 02 . . . 39
2.3 Peran dan aksi periodik tiap node Modul 02 . . . 40
2.4 Peta UUID Modul 02 . . . 40
2.5 Alat dan bahan Modul 02 . . . 40
2.6 Lembar pengamatan EXP-01 . . . 44
2.7 Lembar pengamatan EXP-03 . . . 46
2.8 Hasil verifikasi hardware Modul 02 . . . 46
2.9 Referensi interpretasi RSSI . . . 47
2.10 Lembar pengukuran loss dua arah Modul 02 . . . 47
3.1 Peta modul blok BLE . . . 49
3.2 Istilah kerja Modul 03 . . . 50
3.3 Peran dan aksi periodik tiap node Modul 03 . . . 51
3.4 Peta UUID Modul 03 . . . 51
3.5 Alat dan bahan Modul 03 . . . 51
3.6 Lembar pengamatan EXP-01 . . . 55
3.7 Lembar pengamatan EXP-03 . . . 56
3.8 Hasil verifikasi hardware Modul 03 . . . 57
3.9 Referensi interpretasi RSSI . . . 57
3.10 Lembar pengukuran keberhasilan read dan write Modul 03 . . . 57
4.1 Peta modul blok BLE . . . 59
4.2 Istilah kerja Modul 04 . . . 60
4.3 Peran tiap node Modul 04 . . . 61
4.4 Peta UUID Modul 04 . . . 61
4.5 Alat dan bahan Modul 04 . . . 61
4.6 Lembar pengamatan EXP-01 . . . 65
4.7 Lembar pengamatan EXP-02 . . . 65
4.8 Lembar pengamatan EXP-03 . . . 66
4.9 Hasil verifikasi hardware Modul 04 . . . 67

4.10 Lembar pengukuran aliran telemetri Modul 04 . . . 67
4.11 Tabel pembanding polling dan notify . . . 67
5.1 Peta modul blok BLE . . . 69
5.2 Istilah kerja Modul 05 . . . 70
5.3 Peran tiap node Modul 05 . . . 71
5.4 Alat dan bahan Modul 05 . . . 71
5.5 Lembar pengamatan EXP-01 . . . 76
5.6 Lembar pengamatan EXP-03 . . . 77
5.7 Hasil verifikasi hardware Modul 05 . . . 78
5.8 Lembar pengukuran per node Modul 05 . . . 78
5.9 Lembar pengukuran skenario asimetris . . . 78
6.1 Perbedaan Modul 05 dan Modul 05B . . . 81
6.2 Istilah kerja Modul 05B . . . 82
6.3 Peran tiap node Modul 05B . . . 83
6.4 Alat dan bahan Modul 05B . . . 83
6.5 Pemetaan pin Modul 05B . . . 84
6.6 Pemetaan environment ke port Modul 05B . . . 94
6.7 Lembar pengamatan EXP-01 . . . 95
6.8 Lembar pengamatan EXP-02 . . . 96
6.9 Skenario kegagalan EXP-03 . . . 97
6.10 Lembar pengamatan EXP-03 . . . 97
6.11 Lembar pengukuran A — latency kejadian . . . 98
6.12 Lembar pengukuran B — uji beban event-driven . . . 98
6.13 Lembar pengukuran C — skenario asimetris . . . 99
6.14 Lembar pengukuran D — waktu pemulihan sambung ulang . . . 99
7.1 Perbedaan Modul 05, 05B, dan 05C . . . 101
7.2 Istilah kerja Modul 05C . . . 102
7.3 Peran tiap node Modul 05C . . . 104
7.4 Alat dan bahan Modul 05C . . . 104
7.5 Pemetaan pin Modul 05C . . . 104
7.6 Terminal pengujian tiap peran . . . 115
7.7 Perintah yang tersedia di terminal kasir . . . 116
7.8 Lembar pengamatan EXP-01 . . . 117
7.9 Lembar pengamatan EXP-02 . . . 118
7.10 Lembar pengamatan EXP-03 — waktu tanggap dari dua sisi . . . 118
7.11 Daftar uji penolakan perintah EXP-04 . . . 119
7.12 Hasil verifikasi hardware Modul 05C . . . 120
7.13 Lembar pengukuran A — latency perintah . . . 120
7.14 Lembar pengukuran B — pengaruh slot terpakai . . . 120
8.1 Peta modul blok BLE . . . 123
8.2 Istilah kerja Modul 06 . . . 124
8.3 Peran tiap node Modul 06 . . . 125
8.4 Alat dan bahan Modul 06 . . . 125
8.5 Lembar pengamatan EXP-01 . . . 130
8.6 Lembar pengamatan EXP-03 . . . 132
8.7 Hasil verifikasi hardware Modul 06 . . . 132
8.8 Lembar pengukuran jarak per hop Modul 06 . . . 132
8.9 Lembar pengukuran loss per hop . . . 133
9.1 Titik balik lapisan pada Modul 07 . . . 135
9.2 Istilah kerja Modul 07 . . . 136
9.3 Tiga aturan API `esp_ieee802154_*` dan gejala pelanggarannya . . . 137
9.4 Peran tiap node Modul 07 . . . 137
9.5 Alat dan bahan Modul 07 . . . 138
9.6 Konfigurasi radio Modul 07 . . . 138
9.7 Lembar pengamatan EXP-01 . . . 144
9.8 Lembar pengamatan EXP-03 . . . 145

9.9 Matriks uji efek parameter EXP-04 . . . 146
9.10 Lembar pengamatan EXP-04 . . . 146
9.11 Matriks uji promiscuous mode EXP-05 . . . 147
9.12 Lembar pengamatan EXP-05 . . . 148
9.13 Hasil verifikasi hardware Modul 07 . . . 149
9.14 Lembar pengukuran jarak Modul 07 . . . 149
9.15 Lembar pengukuran per-node Modul 07 . . . 149
10.1 Peta modul blok Zigbee . . . 152
10.2 Istilah kerja Modul 08 . . . 153
10.3 Padanan parameter XBee dan ESP32-H2 pada Modul 08 . . . 154
10.4 Class dasar yang dipakai pada Modul 08 . . . 155
10.5 Class endpoint pada library `Zigbee` . . . 155
10.6 Peran tiap node Modul 08 . . . 156
10.7 Alat dan bahan Modul 08 . . . 156
10.8 Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri . 160
10.9 Lembar pengamatan EXP-01 . . . 162
10.10 Lembar pengamatan EXP-03 . . . 164
10.11 Hasil verifikasi hardware Modul 08 . . . 164
10.12 Lembar pengukuran jarak Modul 08 . . . 164
10.13 Lembar pengukuran per-node Modul 08 . . . 165
10.14 Lembar pengukuran waktu join dan binding . . . 165
11.1 Peta modul blok Zigbee . . . 167
11.2 Istilah kerja Modul 09 . . . 168
11.3 Peran tiap node Modul 09 . . . 169
11.4 Alat dan bahan Modul 09 . . . 169
11.5 Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri . 173
11.6 Lembar pengamatan EXP-01 . . . 175
11.7 Lembar pengamatan EXP-03 . . . 177
11.8 Hasil verifikasi hardware Modul 09 . . . 177
11.9 Lembar pengukuran jarak Modul 09 . . . 178
11.10 Lembar pengukuran per-node Modul 09 . . . 178
11.11 Tabel pembanding BLE bintang dan Zigbee multi-node . . . 178
12.1 Peta modul blok Zigbee . . . 181
12.2 Istilah kerja Modul 10 . . . 182
12.3 Peran tiap node Modul 10 . . . 183
12.4 Alat dan bahan Modul 10 . . . 183
12.5 Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri . 187
12.6 Lembar pengamatan EXP-01 . . . 189
12.7 Lembar pengamatan EXP-03 . . . 190
12.8 Hasil verifikasi hardware Modul 10 . . . 191
12.9 Lembar pengukuran jarak Modul 10 . . . 191
12.10 Lembar pengukuran per-hop Modul 10 . . . 192
12.11 Tabel pembanding relay manual dan routing otomatis . . . 192
13.1 Peta modul blok Thread . . . 194
13.2 Istilah kerja Modul 11 . . . 195
13.3 Padanan alamat XBee, Zigbee, dan Thread . . . 196
13.4 Peran tiap node Modul 11 . . . 197
13.5 Alat dan bahan Modul 11 . . . 197
13.6 Parameter jaringan Thread Modul 11 . . . 197
13.7 Jatah channel Thread untuk enam kelompok — satu nilai untuk semua node kelompok. . 201
13.8 Lembar pengamatan EXP-01 . . . 202
13.9 Lembar pengamatan EXP-03 . . . 204
13.10 Hasil verifikasi hardware Modul 11 . . . 204
13.11 Lembar pengukuran jarak Modul 11 . . . 204
13.12 Tabel pembanding jangkauan dan overhead M07, M08, dan M11 . . . 205

14.1 Peta modul blok Thread . . . 207
14.2 Istilah kerja Modul 12 . . . 208
14.3 Peran tiap node Modul 12 . . . 209
14.4 Alat dan bahan Modul 12 . . . 209
14.5 Parameter jaringan Thread Modul 12 . . . 210
14.6 Jatah channel Thread untuk enam kelompok — satu nilai untuk semua node kelompok. . 215
14.7 Lembar pengamatan EXP-01 . . . 216
14.8 Lembar pengamatan EXP-03 . . . 217
14.9 Hasil verifikasi hardware Modul 12 . . . 218
14.10 Lembar pengukuran jarak Modul 12 . . . 218
14.11 Lembar pengukuran per-node Modul 12 . . . 218
14.12 Tabel pembanding self-healing M06, M10, dan M12 . . . 219
15.1 Peta modul blok integrasi . . . 221
15.2 Istilah kerja Modul 13 . . . 222
15.3 Peran tiap node Modul 13 . . . 223
15.4 Alat dan bahan Modul 13 . . . 223
15.5 Konfigurasi jaringan Modul 13 . . . 224
15.6 Jatah channel Thread untuk enam kelompok — satu nilai untuk kedua firmware . . . 229
15.7 Lembar pengamatan EXP-01 . . . 232
15.8 Lembar pengamatan EXP-03 . . . 234
15.9 Hasil verifikasi hardware Modul 13 . . . 235
15.10 Arti kode kegagalan HTTP pada gateway . . . 236
15.11 Delapan perbaikan kode hasil pengujian perangkat nyata . . . 236
15.12 Lembar pengukuran end-to-end Modul 13 . . . 237
15.13 Lembar pengukuran loss per hop Modul 13 . . . 237
16.1 Peta modul blok integrasi . . . 239
16.2 Istilah kerja Modul 14 . . . 240
16.3 Elemen sistem Modul 14 . . . 241
16.4 Alat dan bahan Modul 14 . . . 241
16.5 Konfigurasi MQTT Modul 14 . . . 242
16.6 Lembar pengamatan EXP-01 . . . 244
16.7 Lembar pengamatan EXP-03 . . . 245
16.8 Hasil verifikasi hardware Modul 14 . . . 246
16.9 Perbaikan kode hasil pengujian perangkat nyata . . . 246
16.10 Lembar pengukuran latency dan success rate Modul 14 . . . 247
16.11 Tabel pembanding HTTP POST dan MQTT publish . . . 247
17.1 Peta modul blok integrasi . . . 249
17.2 Istilah kerja Modul 15 . . . 250
17.3 Peran tiap elemen Modul 15 . . . 251
17.4 Alat dan bahan Modul 15 . . . 251
17.5 Konfigurasi jaringan Modul 15 . . . 252
17.6 Lembar pengamatan EXP-01 . . . 257
17.7 Lembar pengamatan EXP-03 . . . 259
17.8 Hasil verifikasi hardware Modul 15 . . . 259
17.9 Hasil uji pada tiga access point berbeda . . . 260
17.10 Pembanding pipeline BLE dan Thread pada kondisi jaringan identik . . . 260
17.11 Perbaikan kode hasil pengujian perangkat nyata Modul 15 . . . 261
17.12 Lembar pengukuran end-to-end Modul 15 . . . 261
17.13 Lembar counter per tahap Modul 15 . . . 261
17.14 Lembar dekomposisi latency per hop . . . 262
18.1 Peta seluruh seri dan asal angka pengukuran . . . 264
18.2 Variabel kontrol dan variabel bebas benchmark . . . 265
18.3 Istilah kerja Modul 16 . . . 265
18.4 Empat metrik dan pertanyaan yang dijawabnya . . . 266
18.5 Peran tiap elemen Modul 16 . . . 266

18.6 Alat dan bahan Modul 16 . . . 267
18.7 Lembar pengamatan EXP-01 . . . 271
18.8 Lembar pengamatan EXP-03 . . . 273
18.9 Hasil verifikasi hardware Modul 16 . . . 273
18.10 Pembanding langsung pipeline BLE dan Thread . . . 274
18.11 Perbaikan kode hasil pengujian perangkat nyata Modul 16 . . . 274
18.12 Lembar pengukuran pipeline BLE Modul 16 . . . 275
18.13 Tabel perbandingan protokol — deliverable utama Modul 16 . . . 275
18.14 Metrik pendukung pembanding BLE dan Thread . . . 275
A.1 Pin board yang dipakai pada Modul 00A dan 00B . . . 278
B.1 Istilah kerja Modul 00A . . . 281
B.2 Alat dan bahan Modul 00A . . . 281
B.3 Pemetaan pin Modul 00A . . . 281
B.4 Lembar pengamatan EXP-01 . . . 283
B.5 Lembar pengamatan EXP-02 — pemetaan nilai warna . . . 283
C.1 Istilah kerja Modul 00B . . . 286
C.2 Alat dan bahan Modul 00B . . . 286
C.3 Pemetaan pin Modul 00B . . . 286
C.4 Lembar pengamatan EXP-01 . . . 288
C.5 Lembar pengamatan EXP-02 . . . 288

Pendahuluan

Tentang Lab Ini

Buku ini bukan kumpulan tutorial Arduino, melainkan buku kerja laboratorium. Setiap minggu merupakan satu misi rekayasa dengan target sukses yang terukur, prosedur eksperimen yang tertulis, dan data yang harus dikumpulkan sendiri oleh praktikan.

Fokusnya adalah protokol komunikasi — mulai dari Bluetooth Low Energy, IEEE 802.15.4, Zigbee, Thread, hingga integrasi Wi-Fi/MQTT — di atas board ESP32-H2 dan ESP32-C6. Kompetensi dibangun bertahap: point-to-point, kemudian client-server, multi-node, dan mesh, pada tiga keluarga protokol, lalu ditutup dengan integrasi end-to-end ke Internet dan proyek perbandingan protokol berbasis data.

Struktur Setiap Modul

Seluruh modul memakai format yang sama, terdiri atas sebelas bagian seperti pada Tabel P.1.

No	Bagian	Isi
1	**Pendahuluan**	Identitas modul (misi, durasi, mode, level, instrumen), keterkaitan dengan modul lain, prasyarat, dan apa yang dipakai lagi sesudahnya — seluruhnya dalam bentuk kalimat, ditutup peta blok dan kontrak data
2	**Capaian Pembelajaran**	4–5 capaian **terukur** beserta kriteria keberhasilan
3	**Dasar Teori (secukupnya)**	Hanya istilah yang dipakai di percobaan, beserta sekuens protokol
4	**Topologi**	Diagram jaringan bernama board, peran tiap node, peta alamat
5	**Alat yang Digunakan**	Platform, alat dan bahan sampai versi/port, `platformio.ini`, pre-flight, perintah deploy
6	**Percobaan**	EXP-01 sampai EXP-03 dengan **CHECKPOINT** di tiap tahap, beserta log referensi hasil uji nyata
7	**Pengukuran**	Tabel yang diisi sendiri dan tabel pembanding lintas modul
8	**Analisis**	Pertanyaan yang hanya bisa dijawab dari tabel Pengukuran
9	**Concept Check**	Pertanyaan konseptual, bukan hafalan
10	**Challenge**	Tugas **modifikasi kode**, bukan “jelaskan hasilnya”
11	**Laporan**	Daftar deliverable

Tabel P.1: Sebelas bagian baku pada setiap modul praktikum

Tiga hal membedakan format ini dari panduan praktikum biasa.

CHECKPOINT di tengah percobaan Mahasiswa memverifikasi progres sebelum lanjut, bukan baru ketahuan salah di akhir sesi.

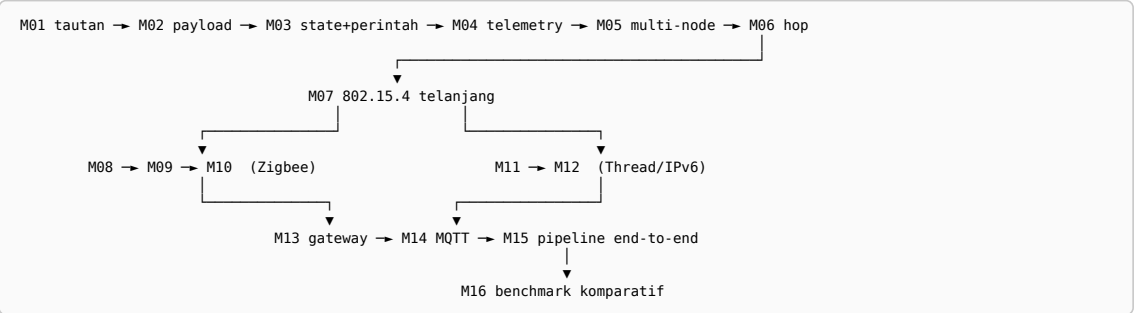
“Buka abstraksinya” Satu kotak per modul yang menyuruh mahasiswa membongkar satu baris kode yang tampak sepele (misalnya mengomentari `subscribe()` lalu melihat notify berhenti) — menghubungkan API dengan apa yang sebenarnya terjadi di udara.

Teori dibatasi Panduan ini menjawab **bagaimana**; pembahasan mendalam (**mengapa**) berada pada buku teori terpisah. Tiap bagian Dasar Teori menyebut batas itu secara eksplisit.

Keterkaitan Antar-Modul

IoT adalah stack berlapis, sehingga modulnya bukan pulau-pulau terpisah. Dua mekanisme dipakai untuk mengikatnya.

Pertama, rantai prasyarat. Tiap modul menyatakan apa yang harus sudah dikuasai, apa yang ditambahkannya, dan modul mana yang akan memakainya lagi.



Kedua, kontrak data yang konsisten. Beberapa keputusan sengaja dipertahankan lintas modul supaya datanya dapat dibandingkan pada M16, seperti tercantum pada Tabel P.2.

Kontrak	Diperkenalkan	Dipakai lagi di
Service UUID `4fafc201-…`	M01	M02–M06, M16
Payload bernomor urut (`SEQ=`, `#n`) untuk menghitung loss	M02 (CH-1)	M04, M07, M09, M12, M13–M16
Pemisahan kanal **data** dan kanal **perintah**	M03	M08 (cluster), M14 (topic)
Penanda identitas sumber pada payload/alamat	M05 (`A:`/`B:`)	M09 (short addr), M12 (`NODE_ID`), M15
Transparent forwarding — payload tidak diubah saat di-hop	M06	M13, M15, M16
Radio 802.15.4 channel 15 sebagai basis bersama	M07	M08–M13
Grup multicast `ff03::abcd` : 5050	M11	M12, M13, M15
Topic MQTT `praktikum/h2/telemetri`	M14	M15, M16
Variabel kontrol benchmark (payload, interval, gateway, broker)	M15	M16

Tabel P.2: Kontrak data yang dipertahankan lintas modul

Konsekuensinya, **angka pengukuran modul awal dipakai lagi di modul akhir.** Transaksi per menit M03 dibandingkan dengan M04; latency relay M06 dengan routing M10 dan mesh M12; latency per hop M11 dan M14 dijumlahkan lalu dicek terhadap latency end-to-end M15. Mahasiswa yang membuang data modul lama akan kesulitan pada M16 — dan hal itu memang disengaja.

Capaian Pembelajaran

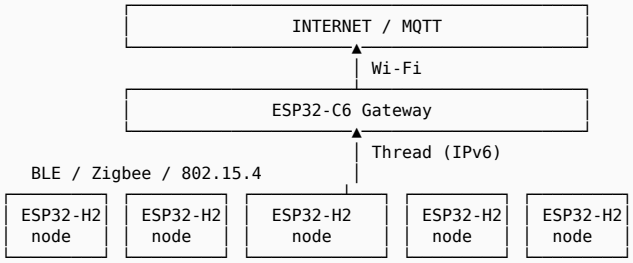
Setelah menyelesaikan seluruh modul, praktikan mampu:

1. menjelaskan karakteristik protokol BLE, 802.15.4, Zigbee, Thread, dan MQTT;
2. membangun komunikasi P2P, client-server, multi-node, dan mesh antar board;
3. melakukan pengukuran RSSI, latency, dan packet loss secara ilmiah;
4. menganalisis data pengujian untuk menilai kecocokan protokol terhadap kasus;
5. membangun sistem IoT end-to-end dari sensor node hingga broker MQTT.

Kompetensi pendukung

- Menggunakan PlatformIO/Arduino core 3.x untuk ESP32-H2/C6.
- Membaca Serial Monitor sebagai instrumen pengukuran, bukan sekadar log.
- Merancang eksperimen jarak–RSSI–packet loss dan mencatatnya secara sistematis.
- Bekerja berpasangan atau berkelompok dengan pembagian peran perangkat.

Arsitektur Sistem Lab



Blok	Modul	Cakupan
BLE	01–06	P2P, pertukaran data, GATT client-server, telemetry, multi-node, mesh/relay
802.15.4	07	Raw frame di atas radio telanjang
Zigbee	08–10	P2P, multi-node, mesh routing
Thread	11–12	P2P IPv6, mesh IPv6
Integrasi	13–15	Gateway Thread ke Wi-Fi, MQTT, pipeline end-to-end
Proyek	16	Benchmark komparatif BLE/Zigbee/Thread

Tabel P.3: Pembagian blok protokol sepanjang satu semester

Daftar Modul

Modul	Folder	Misi	Link komunikasi	Board	Level
00A	`week00_blinky`	Verify the Toolchain	— (single node)	ESP32-H2	Basic
00B	`week00_btn`	Read the First Input	— (single node)	ESP32-H2	Basic
01	`week01_ble_p2p`	Establish a BLE Link	koneksi H2 ke H2	ESP32-H2	Basic
02	`week02_ble_p2p_data`	Exchange Data	dua arah (notify + write)	ESP32-H2	Basic
03	`week03_ble_client_server`	Build a BLE Service	GATT read/write	ESP32-H2	Intermediate
04	`week04_ble_telemetry`	Stream Telemetry	server ke client (notify)	ESP32-H2	Intermediate
05	`week05_ble_multinode`	Connect Multiple Devices	1 pusat ke beberapa node	ESP32-H2	Intermediate
05B	`week05b_ble_multinode_project`	Build a Smart Sensor System	2 sensor bukaan ke 1 hub	ESP32-H2	Intermediate

Modul	Folder	Misi	Link komunikasi	Board	Level
05C	`week05c_ble_pager`	Notify One Device, Not All	1 controller ke N pager	ESP32-H2	Intermediate
06	`week06_ble_mesh`	Build a BLE Mesh	relay H2–H2–H2	ESP32-H2	Intermediate
07	`week07_802154_p2p`	Speak Raw 802.15.4	raw frame H2 ke H2	ESP32-H2	Intermediate
08	`week08_zigbee_p2p`	Join a Zigbee Network	Coordinator ke End Device	ESP32-H2	Advanced
09	`week09_zigbee_multinode`	Scale the Network	Coordinator ke beberapa ED	ESP32-H2	Intermediate
10	`week10_zigbee_mesh`	Route Through the Mesh	Coordinator ke Router ke ED	ESP32-H2	Advanced
11	`week11_thread_p2p`	Speak IPv6 over Thread	UDP IPv6 H2 ke H2	ESP32-H2	Intermediate
12	`week12_thread_mesh`	Mesh the Internet	IPv6 mesh multi-node	ESP32-H2	Advanced
13	`week13_thread_wifi_gateway`	Bridge Thread to Wi-Fi	H2 ke C6 ke Wi-Fi	H2 + C6	Advanced
14	`week14_mqtt`	Publish to the Cloud	C6 ke MQTT broker	ESP32-C6	Intermediate
15	`week15_e2e_iot`	Build an End-to-End IoT System	H2 → Thread → C6 → MQTT	H2 + C6	Advanced
16	`week16_comparative`	Prove Your Protocol	BLE/ Zigbee/ Thread → MQTT	H2 + C6	Project

Tabel P.4: Daftar modul praktikum beserta folder kode sumbernya

CATATAN

Modul 00A dan 00B adalah warm-up yang dikerjakan sebelum M01. Keduanya tidak memuat protokol komunikasi; fungsinya memastikan toolchain, board, dan rantai build–flash–monitor sudah terbukti bekerja, sehingga kegagalan pada modul komunikasi tidak lagi bercampur dengan masalah dasar. Karena bukan modul inti, keduanya dimuat di bagian lampiran (Lampiran B. untuk Modul 00A dan Lampiran C. untuk Modul 00B).

> **CATATAN**
> **Modul 05B adalah mini project**, bukan modul inti — 16 modul utama tetap 01–16. Isinya menerapkan topologi bintang M05 pada kasus nyata: dua smart sensor bukaan (jendela dan pintu, tombol BOOT sebagai proximity switch simulasi) melapor ke satu hub. Di sinilah trafik berubah dari periodik menjadi *event-driven*.

> **CATATAN**
> **Modul 05C adalah mini project** yang melanjutkan M05 dari arah sebaliknya: pusat mengirim perintah ke **satu** node terpilih (sistem pager restoran). Titik ajarnya adalah bahwa pengiriman terarah pada BLE bukan broadcast, melainkan penulisan ke satu objek koneksi tertentu.

Rantai sistem end-to-end pada Modul 15 adalah sebagai berikut.

```
H2 → Thread → C6 → Wi-Fi → MQTT → Dashboard
```

> **PENTING**
> **Modul 16 bukan sekadar demo.** Praktikan harus membuktikan dengan data, protokol mana yang paling sesuai untuk skenario IoT tertentu.

Status Verifikasi Perangkat Keras

Modul 02–16 sudah dikompilasi **dan** dijalankan pada perangkat nyata (2 buah ESP32-H2 DevKitM-1 dan 1 buah ESP32-C6 DevKitC-1, dengan capture Serial Monitor otomatis per modul). Log referensi hasil uji terdapat pada bagian “Verifikasi hardware” di masing-masing modul.

Modul	Build	Uji perangkat	Board dipakai	Catatan
02	lulus	lulus — 0 % loss dua arah	2 × ESP32-H2	—
03	lulus	lulus — read/write 100 %	2 × ESP32-H2	—
04	lulus	lulus — 24/24 notify	2 × ESP32-H2	—
05	lulus	lulus — 2 koneksi simultan	3 × ESP32-H2	laju A 30/mnt, B 20/mnt
05B	lulus	lulus — 8/8 kejadian + reconnect	3 × ESP32-H2	pulih 4,6 s (cepat) atau 28,3 s (via scan ulang); kedip LED belum diverifikasi
05C	lulus	lulus — panggilan terarah + ACK	4 × ESP32-H2	0 baris bocor ke pager lain; **batas 2 koneksi serentak** ditemukan di sini; buzzer belum diuji akustik
06	lulus	lulus — relay A ke B ke C, 9/9	3 × ESP32-H2	—
07	lulus	lulus — 12/12 PING–PONG	2 × ESP32-H2	perlu tiga perbaikan kode
08	lulus	lulus — 13/13 perintah	2 × ESP32-H2	erase NVS sebelum flash
09	lulus	lulus — 2 light, 14/14	3 × ESP32-H2	`short addr 0xFFFF` itu normal
10	lulus	lulus — 15/15 ke ZR dan ZED	3 × ESP32-H2	multi-hop perlu formasi garis
11	lulus	lulus — 12/12 PING–PONG	2 × ESP32-H2	perlu prefix mesh-local tetap
12	lulus	lulus — 3 node saling terima	3 × ESP32-H2	perlu prefix mesh-local tetap
13	lulus	lulus — rantai penuh	H2 + **C6**	POST sampai server; balasan HTTP kadang timeout (`-11`)
14	lulus	lulus — publish + perintah	**C6**	8/8 publish, 3/3 perintah; broker lokal
15	lulus	sebagian — rantai jalan, hop Wi-Fi rapuh	H2 + **C6**	end-to-end 0–36 % tergantung AP (koeksistensi)

Modul	Build	Uji perangkat	Board dipakai	Catatan
16	lulus	lulus — BLE ke MQTT	H2 + **C6**	47/48 notify, 47/47 publish tiba; end-to-end 98 %

Tabel P.5: Status verifikasi tiap modul pada perangkat nyata

> **CATATAN**
> Pada berkas sumber, kolom Build dan Uji perangkat ditandai dengan lambang centang dan lambang peringatan. Di buku ini lambang tersebut ditulis sebagai kata “lulus” dan “sebagian” agar tetap terbaca ketika dokumen dicetak hitam-putih. Isi penilaiannya tidak diubah.

Perbaikan kode yang lahir dari pengujian ini dirangkum pada Tabel P.6.

Modul	Masalah di perangkat nyata	Perbaikan
05C	Controller reboot saat koneksi BLE ketiga (`npl_freertos_callout_init`)	Controller ESP32-H2 hanya mencadangkan memori untuk dua koneksi dan tidak dapat dinaikkan lewat build flag; koneksi dibuka saat memanggil lalu dilepas setelah ACK
07	Node2 tidak pernah menerima frame	`esp_ieee802154_receive()` dipanggil di `setup()`
07	Payload sesekali berisi sampah RAM	buffer TX dijadikan `static` (transmit bersifat asinkron)
07	Panjang frame salah	byte `Len` kini menghitung 2 byte FCS
11, 12, 13, 15	Node attach tetapi tidak ada paket multicast yang tiba	prefix mesh-local dipaksa sama (`otThreadSetMeshLocalPrefix()`), dataset selalu di-commit ulang

Tabel P.6: Perbaikan kode hasil pengujian pada ESP32-H2

Koeksistensi Wi-Fi dan 802.15.4 pada ESP32-C6

Menjalankan Thread dan Wi-Fi bersamaan pada satu chip (Modul 13 dan 15) membutuhkan tiga hal yang tidak ada pada kode awal: **urutan inisialisasi** (Wi-Fi disambungkan di antara `OThread.begin()` dan `OThread.start()`), **prioritas radio** (`esp_coex_preference_set(ESP_COEX_PREFER_WIFI)`), dan **timer retry MQTT yang terpisah** dari timer retry Wi-Fi. Tanpa ketiganya, C6 mendapatkan IP yang benar tetapi seluruh TCP keluar gagal.

Setelah diperbaiki, rantai penuh Modul 13 dan 15 terbukti berjalan — tetapi hop Wi-Fi tetap tidak andal. Hasil pengujian pada tiga access point diberikan pada Tabel P.7.

AP uji	Kanal Wi-Fi	RSSI	M15 end-to-end
AP-1	1	−82 dBm	0 %
AP-2	12	−81 dBm	36 %
AP-3	9	−71 dBm	5 %

Tabel P.7: Keberhasilan rantai end-to-end Modul 15 pada tiga access point

Sinyal terkuat justru bukan yang terbaik, dan mengganti channel 802.15.4 (15 ke 25 ke 11) tidak menolong — jadi persoalannya bukan link budget melainkan pembagian airtime satu antena. Pembanding yang menentukan, pada access point dan jarak yang sama, diberikan pada Tabel P.8.

Pipeline	Modul	Hop sensor	Hop Wi-Fi/MQTT	End-to-end
BLE ⟶ C6 ⟶ MQTT	16	47/48 (98 %)	47/47 (100 %)	**98 %**

Pipeline	Modul	Hop sensor	Hop Wi-Fi/MQTT	End-to-end
Thread ⟶ C6 ⟶ MQTT	15	30/39 (77 %)	2/7 (29 %)	**5 %**

Tabel P.8: Perbandingan dua pipeline end-to-end pada kondisi uji yang sama

Kombinasi BLE dan Wi-Fi nyaris tanpa ongkos koeksistensi, sedangkan Thread dan Wi-Fi sangat mahal. Angka ini menjadi bahan utama analisis Modul 16.

> **PENTING**
> **Catatan metodologi yang lahir dari sini:** `mqtt.publish()` yang mengembalikan `true` bukan bukti pesan sampai, karena QoS 0 hanya menulis ke buffer socket. Semua tabel "broker menerima" pada lab ini wajib diisi dari log broker atau subscriber.

Perbaikan kode tambahan dari uji ESP32-C6 dirangkum pada Tabel P.9.

Modul	Masalah di perangkat nyata	Perbaikan
13, 15	C6 panic: `Failed to create OpentThread event loop`, lalu `assert failed: otTaskletsSignalPending`	`OThread.begin()` dipanggil sebelum Wi-Fi
13, 15	Wi-Fi tidak pernah asosiasi bila `OThread.start()` sudah berjalan	Wi-Fi disambungkan di antara `OThread.begin()` dan `OThread.start()`
13–16	`while (WiFi.status() != WL_CONNECTED)` menggantung selamanya	dibatasi `WIFI_TIMEOUT_MS`, lalu lanjut dan dicoba ulang berkala
14	`reconnectMQTT()` memblokir `loop()` tanpa batas	dibatasi `MQTT_TIMEOUT_MS`, keluar bila Wi-Fi belum siap
15, 16	`mqtt.connect()` dipanggil tiap iterasi `loop()` sehingga banjir `DNS Failed` menenggelamkan log	`maintainNetwork()` dengan jeda dan pemeriksaan Wi-Fi lebih dulu
13, 15, 16	`WiFi.disconnect()` tiap retry membatalkan asosiasi yang sedang berjalan	hanya `WiFi.begin()` ulang, dengan jeda lebih panjang
13, 15	Seluruh TCP keluar gagal saat stack Thread aktif	`esp_coex_preference_set(ESP_COEX_PREFER_WIFI)` sebelum `OThread.start()`
13	Balasan HTTP timeout padahal data sudah sampai server	`http.setConnectTimeout(8000)` dan `setTimeout(8000)`
15, 16	Retry MQTT ikut terkunci jeda retry Wi-Fi (20 s) padahal Wi-Fi sehat	timer `WIFI_RETRY_MS` dan `MQTT_RETRY_MS` dipisah
15	Baris `Publish MQTT [...]` dicetak walau broker tidak terhubung — laporan palsu	publish hanya bila `mqtt.connected()`, kegagalan dicetak apa adanya

Tabel P.9: Perbaikan kode hasil pengujian pada ESP32-C6

Perkakas pendukung pada folder `tools/` lahir karena jaringan uji memblokir port 1883 dan port 80 keluar.

Berkas	Guna
`tools/mqtt_broker.py`	Broker MQTT 3.1.1 lokal, Python murni tanpa install atau sudo; sekaligus pengganti `mosquitto_sub -v`
`tools/http_sink.py`	Server penerima POST lokal, pengganti `httpbin.org` untuk Modul 13

Tabel P.10: Perkakas pendukung pada folder `tools/`

Perangkat Keras

No	Peralatan	Jumlah	Keterangan
1	ESP32-H2 DevKitM-1	≥ 3	node sensor / BLE / Zigbee / Thread
2	ESP32-C6 DevKitC-1	≥ 1	gateway Wi-Fi / MQTT
3	Kabel USB data	sesuai board	flash dan serial monitor
4	PC/Laptop	1 per kelompok	terinstal PlatformIO
5	Akses Wi-Fi/hotspot	1	modul 13–16
6	Broker MQTT (tes/lokal)	1	modul 14–16

Tabel P.11: Perangkat keras yang diperlukan sepanjang satu semester

Perangkat Lunak

- PlatformIO dengan platform **pioarduino** `espressif32` 55.03.311 (Arduino core **3.3.11**, ESP-IDF 5.5.5). Platform resmi `platformio/espressif32` **tidak menyediakan board ESP32-H2**, sehingga tiap `platformio.ini` memakai URL rilis pioarduino secara eksplisit. Toolchain terunduh otomatis pada build pertama.
- Modul 13, 15, dan 16 memakai `board_build.partitions = huge_app.csv` pada gateway ESP32-C6 — firmware Thread/BLE + Wi-Fi + MQTT/HTTP melebihi partisi app default 1,25 MB.
- Zigbee dan OpenThread **bawaan** Arduino core 3.x; mode Zigbee diatur lewat `build_flags` (`-DZIGBEE_MODE_ED`, `-DZIGBEE_MODE_ZCZR`).
- Modul 07 dan 11–13 memakai API ESP-IDF (`esp_ieee802154.h`, `OThread`) yang dapat dipanggil langsung dari Arduino core 3.x.
- Serial Monitor 115200 baud.
- Library: NimBLE-Arduino (BLE) dan PubSubClient (MQTT).
- Perkakas lokal pada `tools/` bila jaringan memblokir broker atau HTTP publik — lihat `tools/README.md`.

Deploy per modul

```
# contoh: upload dua role pada Module 02
pio run -d week02_ble_p2p_data -e node1 -t upload
pio run -d week02_ble_p2p_data -e node2 -t upload
```

Kode Sumber dan Repositori

Buku ini dirancang agar dapat dipakai berdiri sendiri: **seluruh berkas sumber setiap modul dimuat lengkap di dalamnya**, bukan sekadar potongan. Setiap `platformio.ini`, tiap `main.cpp` per peran, tabel partisi Zigbee, maupun skrip bantu Python tercetak utuh pada bagian Kode Program bab yang bersangkutan. Listing-listing itu dibaca langsung dari salinan berkas aslinya saat buku dibangun, sehingga isinya dijamin identik dengan yang dijalankan di perangkat — tidak ada transkripsi manual yang bisa menyimpang.

Aturan Laboratorium dan Keselamatan

1. Kabel USB dicabut atau diamankan saat memindah posisi board (eksperimen jarak).
2. Antena board tidak ditempelkan ke permukaan logam saat pengukuran.
3. Gunakan daya dari PC atau laptop; hindari power bank bila tidak perlu.
4. Data pengukuran wajib hasil eksperimen sendiri, bukan salinan kelompok lain.
5. Laporan dan analisis ditulis dengan bahasa sendiri; menyalin teori atau kode tanpa pemahaman dinilai nol pada bagian analisis.

Sistem Penilaian

Komponen	Bobot	Sumber penilaian
Persiapan dan eksekusi percobaan (semua CHECKPOINT terlewati)	25 %	bagian Alat yang Digunakan dan Percobaan
Data pengukuran	20 %	bagian Pengukuran
Analisis berbasis data	20 %	bagian Analisis
Challenge (modifikasi kode)	15 %	bagian Challenge
Concept check	10 %	bagian Concept Check
Laporan akhir dan kesimpulan	10 %	bagian Laporan

Tabel P.12: Bobot penilaian praktikum

Catatan penilaian:

- **Capaian pembelajaran adalah rubriknya.** Tiap capaian ditulis agar dapat dinilai lulus atau tidak dari bukti yang dilampirkan, bukan dari kesan.
- **Angka tanpa asal-usul dianggap tidak ada.** Setiap sel tabel pengukuran harus dapat ditunjuk log atau kondisi ukurnya — hal inilah yang diuji habis-habisan pada Modul 16.
- **Challenge dinilai dari kode yang berubah**, bukan dari penjelasan tentang kode yang tidak diubah.

> **CATATAN**
> Setiap modul berada pada folder `weekNN_nama` berisi `platformio.ini`, kode sumber per peran (role), dan `README.md`. Buku ini adalah hasil konversi berkas `README.md` tersebut, dengan seluruh kode sumber tiap modul dimuat lengkap di bagian Kode Program masing-masing bab. Log serial mentah hasil uji perangkat tetap berada pada repositori praktikum.

September 2026

Akhmad Hendriawan

1. Modul 01 — Komunikasi BLE Point-to-Point

IoT COMMUNICATION LAB · MODUL 01

Establish a BLE Link — Komunikasi BLE Point-to-Point

ESP32-H2 · BLE · P2P · level Basic · 3 × 50 menit · folder kode `week01_ble_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 01 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat dasar. Misinya membangun tautan BLE point-to-point yang stabil dan terverifikasi lewat instrumen, bukan sekadar tersambung sekali lalu dianggap selesai. Percobaan berjalan dalam mode P2P antara satu peripheral dan satu central, dengan Serial Monitor 115200 baud sebagai instrumen utama dan aplikasi nRF Connect sebagai pembanding opsional.

1.1 Pendahuluan

BLE adalah stack berlapis. Modul ini adalah lapisan paling bawah: tautan radio itu sendiri — belum ada payload aplikasi. Modul-modul berikutnya menumpuk di atas tautan yang dibangun di sini, sehingga modul ini perlu dikerjakan sampai benar-benar solid sebelum melangkah lebih jauh.

Bekal yang diperlukan hanya dasar bahasa C dan pemahaman alur build PlatformIO; sebagai modul pembuka, tidak ada modul BLE yang mendahuluinya. Yang dibangun di sini adalah advertising, active scanning, pembentukan koneksi P2P, dan pengukuran RSSI terhadap jarak. Keempatnya dipakai lagi secara berurutan pada M02 (payload di atas tautan ini), M03 (GATT read/write), M04 (telemetry via notify), hingga M05 ketika jumlah node melampaui dua.

Peta modul blok BLE

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01 (ini)	**Tautan BLE P2P terbentuk dan stabil**
02	Pertukaran payload aplikasi lewat tautan M01
03	GATT characteristic — read/write data terstruktur
04	Telemetry via notify (push berkala tanpa polling)
05	Skala lebih dari dua node (menuju ranah WSN)
06	Relay A⟶B⟶C — jangkauan diperluas lewat hop

Tabel 1.1: Peta modul blok BLE

Kontrak data lab ini. Modul ini belum mengirim payload, tetapi sudah menetapkan dua hal yang dipakai seterusnya: **Service UUID** `4fafc201-1fb5-459e-8fcc-c5c9c331914b` (dipakai ulang M02–M06 dan M16) dan **Serial Monitor sebagai instrumen ukur**, bukan sekadar tempat log.

1.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membangun tautan BLE point-to-point yang terverifikasi

1. Menjelaskan perbedaan peran BLE Peripheral (advertiser/server) dan Central (scanner/client).
2. Menyusun proyek PlatformIO dua environment (`node1`, `node2`) dan memilih source per board lewat `build_src_filter`.
3. Menjalankan advertising dengan Device Name dan Service UUID tertentu, lalu memverifikasinya dari paket yang benar-benar mengudara.
4. Membangun koneksi BLE point-to-point melalui active scanning dari sisi Central.
5. Mengukur RSSI terhadap jarak dari log node sendiri dan menentukan ambang RSSI saat koneksi mulai gagal.

Kriteria keberhasilan

- ☐ Node2 menemukan `NODE1_H2` dan membentuk koneksi.
- ☐ Kedua node mencetak heartbeat status terhubung tiap 5 detik.
- ☐ Uji disconnect (EXP-03) membuktikan tautan dua arah — bukan sekadar heartbeat lokal.
- ☐ Tabel jarak–RSSI–keberhasilan terisi lengkap dari pengukuran sendiri.

1.3 Dasar Teori (Secukupnya)

Teori di sini dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (mengapa BLE hemat daya, detail lapisan protokol) berada di buku teori terpisah; panduan ini fokus pada "bagaimana". Istilah kerja yang diperlukan dirangkum pada Tabel 1.2.

Istilah	Definisi kerja di lab ini
BLE	Varian Bluetooth berdaya rendah untuk komunikasi data berkala.
Peripheral / Advertiser	Device yang menyiarkan paket advertising berisi nama device dan Service UUID.
Central / Scanner	Device yang memindai paket advertising dan memulai koneksi.
Advertising	Proses menyiarkan keberadaan device (nama `NODE1_H2` dan Service UUID).
Active scan	Scan dengan scan-request tambahan agar data advertising lengkap.
Connection	Tautan BLE setelah Central meminta koneksi ke Peripheral.
GATT / Service	Struktur data di atas koneksi; modul ini membuat service **tanpa** characteristic.
RSSI	Kuat sinyal terima (dBm); makin mendekati nol makin kuat — −50 lebih kuat dari −90.

Tabel 1.2: Istilah kerja Modul 01

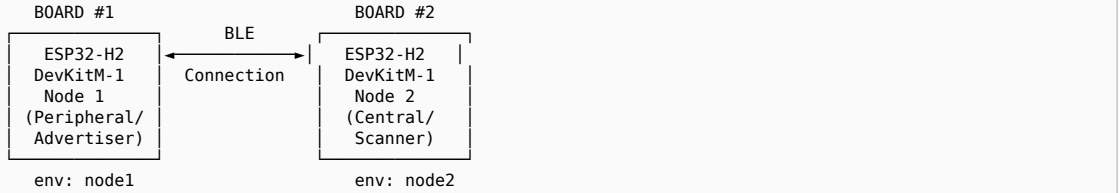
Mengapa ESP32-H2? H2 mendukung BLE 5 dan 802.15.4 (Thread/Zigbee), tetapi tidak punya Wi-Fi maupun Bluetooth Classic. Jadi seluruh komunikasi di seri lab ini murni BLE (dan 802.15.4 di modul lanjutan) — bukan kebetulan, melainkan pilihan board yang menegaskan fokus lab.

Mengapa keluaran modul ini sedikit? Service pada Node1 sengaja dibuat tanpa characteristic, sehingga tidak ada data aplikasi yang dikirim. Setelah banner boot dan pesan koneksi, kedua node hanya mencetak satu baris heartbeat tiap 5 detik. Itu bukan tanda program berhenti; itu memang bentuk keberhasilan Modul 01. Karena heartbeat dicetak dari status **lokal** masing-masing node, bukti tautan dua arah baru diperoleh di EXP-03.

Sekuens protokol yang diamati



1.4 Topologi



Node	Board	Environment	Peran	Identitas radio
Node 1	ESP32-H2 DevKitM-1	`node1`	BLE Peripheral (advertise)	`NODE1_H2`
Node 2	ESP32-H2 DevKitM-1	`node2`	BLE Central (scan dan connect)	`NODE2_H2`

Tabel 1.3: Peran dan identitas radio tiap node Modul 01

Service UUID kedua node: `4fafc201-1fb5-459e-8fcc-c5c9c331914b`

1.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` via `lib_deps`	—
5	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (Arduino core 3.3.11)	—
6	nRF Connect (opsional)	Android/iOS — *cross-check* paket advertising	1

Tabel 1.4: Alat dan bahan Modul 01

1.6 Kode Program

Kunci agar dua board tidak salah flash. Dengan dua ESP32-H2 terpasang bersamaan, auto-detect port bisa mengirim firmware `node1` ke board yang dimaksudkan sebagai `node2`. Pin port tiap environment, dan gunakan `build_src_filter` untuk memilih source per node seperti pada Kode 1.1.

```
[env]
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps = h2zero/NimBLE-Arduino@^2.2.3

[env:node1]
build_src_filter = +<node1/*.cpp>   ; memilih src/node1/main.cpp
upload_port  = /dev/ttyACM0         ; Windows: COM3 -- board Node1
monitor_port = /dev/ttyACM0

[env:node2]
build_src_filter = +<node2/*.cpp>   ; memilih src/node2/main.cpp
upload_port  = /dev/ttyACM2         ; Windows: COM4 -- board Node2
monitor_port = /dev/ttyACM2
```

Kode 1.1: `platformio.ini` dengan port yang dipin per environment

Berkas `platformio.ini` pada repositori praktikum (Kode 1.2) tidak memin `upload_port`, sehingga port dipilih lewat opsi `--upload-port` pada perintah `pio run`. Gunakan salah satu cara, jangan keduanya sekaligus.

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>
```

Kode 1.2: `platformio.ini` Modul 01 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"

static bool connected = false;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    connected = true;
    Serial.println("Node2 terhubung (link P2P aktif)");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    connected = false;
    Serial.println("Node2 terputus, mulai advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("Node1 (BLE Peripheral) starting...");

  NimBLEDevice::init("NODE1_H2");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());
```

```
  // Service tanpa characteristic: modul ini hanya menguji pembentukan tautan
  NimBLEService *pService = pServer->createService(SERVICE_UUID);

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("NODE1_H2");
  pAdvertising->start();

  Serial.println("Advertise sebagai NODE1_H2, menunggu Node2...");
}

void loop() {
  static unsigned long lastPrint = 0;
  if (connected && millis() - lastPrint > 5000) {
    lastPrint = millis();
    Serial.println("Status: H2 <-> H2 terhubung");
  }
}
```

Kode 1.3: `src/node1/main.cpp` — BLE Peripheral `NODE1_H2`

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress node1Addr;
static bool doConnect = false;
static bool connected = false;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override {
    connected = true;
    Serial.println("Terhubung ke Node1");
  }
  void onDisconnect(NimBLEClient *pClient, int reason) override {
    connected = false;
    Serial.println("Terputus dari Node1");
  }
};

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *advertisedDevice) override {
    if (advertisedDevice->getName() == "NODE1_H2") {
      Serial.println("Node1 ditemukan");
      node1Addr = advertisedDevice->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToNode1() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(node1Addr)) {
    Serial.println("Gagal terhubung");
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service tidak ditemukan");
    pClient->disconnect();
    return false;
  }
  return true;
}
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("Node2 (BLE Central) starting...");

  NimBLEDevice::init("NODE2_H2");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Scanning Node1...");
}

void loop() {
  if (doConnect) {
    if (connectToNode1()) {
      Serial.println("Koneksi berhasil");
    }
    doConnect = false;
  }

  static unsigned long lastPrint = 0;
  if (connected && millis() - lastPrint > 5000) {
    lastPrint = millis();
    Serial.println("Status: H2 <-> H2 terhubung");
  }
}
```

Kode 1.4: `src/node2/main.cpp` — BLE Central `NODE2_H2`

1.7 Build dan Flash

```
pio device list                                  # catat port dulu
pio run -d week01_ble_p2p -e node1 -t upload
pio run -d week01_ble_p2p -e node2 -t upload -t monitor
```

PENTING

Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Dengan demikian satu board memakai `/dev/ttyACM0`, dua board memakai `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board memakai `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ Jalankan `pio device list`, catat port tiap board, isi `upload_port`/`monitor_port` di atas.
- ☐ Board ESP32-H2 pertama terhubung (akan diflash `node1`), board kedua terhubung (`node2`).
- ☐ Toolchain PlatformIO siap; firmware `node1` dan `node2` berhasil build tanpa galat.
- ☐ Serial Monitor 115200 baud dibuka **lebih dulu**, lalu tekan RESET agar sekuens boot terekam.

1.8 Percobaan

1.8.1 EXP-01 — Advertising dan Scanning

Node1 advertise sebagai `NODE1_H2` membawa Service UUID. Node2 melakukan active scan 5 detik dan mencetak hasil temuan beserta RSSI.

```
Node1: [ADV] "NODE1_H2" + SERVICE_UUID
                  |
                  ▼
Node2: Scan 5 detik ──► onResult() ──► nama cocok? ──► stop scan
```

Data capture

Parameter	Hasil
Nama device yang ditemukan	
Service UUID pada paket advertising	
RSSI saat ditemukan (dBm)	
Waktu scanning (detik)	
MAC address Node1 (jika ada)	

Tabel 1.5: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Sebelum lanjut, pindai `NODE1_H2` menggunakan nRF Connect dan amati isi paket advertising: Device Name, Service UUID, dan flags. Cocokkan tiap field dengan baris di `src/node1/main.cpp` — baris kode mana yang menyetel masing-masing? Ini menghubungkan API NimBLE dengan byte yang benar-benar mengudara.

CHECKPOINT

Node2 mencetak `Node1 ditemukan` dan sebuah nilai RSSI. Jika tidak: pastikan board `node1` menyala dan filter nama di `ScanCallbacks` Node2 benar.

1.8.2 EXP-02 — Pembentukan Tautan

Setelah Node1 ditemukan, Node2 menjalankan alur koneksi. Verifikasi tiap tahap di Serial Monitor kedua node.

```
Scan ──► Node1 ditemukan ──► Connect ──► Service check ──► Connected
```

Expected output — Node1 (`NODE1_H2`)

```
Node1 (BLE Peripheral) starting...
Advertise sebagai NODE1_H2, menunggu Node2...
Node2 terhubung (link P2P aktif)
Status: H2 <-> H2 terhubung
Status: H2 <-> H2 terhubung
Status: H2 <-> H2 terhubung
```

Expected output — Node2 (`NODE2_H2`)

```
Node2 (BLE Central) starting...
Scanning Node1...
Node1 ditemukan
Terhubung ke Node1
Koneksi berhasil
Status: H2 <-> H2 terhubung
Status: H2 <-> H2 terhubung
Status: H2 <-> H2 terhubung
```

CHECKPOINT

Kedua node mencetak baris `Status: ... terhubung` berulang tiap 5 detik. Nilai RSSI baru ikut tercetak setelah modifikasi pada bagian Pengukuran diterapkan.

1.8.3 EXP-03 — Ketahanan Tautan

Modul ini belum mempertukarkan payload aplikasi. Uji ketahanan tautan dan amati perilaku dua arahnya.

1. **Reset Node1** — Node2 mencetak `Terputus dari Node1` setelah supervision timeout (± 2–3 detik), lalu berhenti mencetak status. Node1 hanya mencetak banner boot-nya, bukan `Node2 terputus`, karena baru restart.
2. **Reset Node2** — Node1 mencetak `Node2 terputus, mulai advertise ulang`, lalu `Node2 terhubung` saat Node2 selesai boot dan scan lagi. Ini satu-satunya skenario yang memicu `onDisconnect` di Node1.
3. **Reconnect** — biarkan Node2 jalan, restart Node1: koneksi tidak terbentuk sendiri karena Node2 hanya scan sekali (5 detik) di `setup()`. Node2 perlu di-reset. Hal ini akan diperbaiki di CH-4.
4. **Filter test** — ubah filter nama di `ScanCallbacks` Node2, amati Node2 tak lagi menemukan Node1, lalu kembalikan kode.

Data capture

Parameter	Hasil
Waktu rata-rata scan sampai ditemukan (ms)	
Pesan saat disconnect pada Node1	
Pesan saat disconnect pada Node2	
Interval cetak status (ms)	

Tabel 1.6: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa reset Node1 dan reset Node2 memicu pesan yang berbeda — itu bukti tautan diamati dari dua sisi.

1.9 Pengukuran

RSSI dibaca dari log Node2 sendiri, bukan aplikasi luar. Tambahkan cetak RSSI koneksi pada heartbeat Node2 seperti Kode 1.5.

```
// Node2 - loop(), cetak tiap 5000 ms saat terhubung
if (pClient && pClient->isConnected()) {
  Serial.printf("Status: H2 <-> H2 terhubung | RSSI: %d dBm\n",
                pClient->getRssi());
}
```

Kode 1.5: Cetak RSSI koneksi pada heartbeat Node2

nRF Connect boleh dipakai sebagai *cross-check* opsional. Gunakan Tabel 1.7 untuk menilai kualitas tautan.

RSSI (dBm)	Interpretasi
> −70	Kuat — koneksi andal
−70 s.d. −85	Baik — masih stabil
−85 s.d. −95	Marginal — mulai rawan putus
< −95	Tidak reliable — koneksi sering gagal

Tabel 1.7: Referensi interpretasi RSSI

Ukur pada beberapa jarak, diisi dari Serial Monitor Node2 (Tabel 1.8).

Jarak	RSSI (dBm)	Latency koneksi (ms)	Success (ya/tidak)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 1.8: Lembar pengukuran jarak, RSSI, dan keberhasilan koneksi

Metrik turunan: latency rata-rata scan sampai connected, dan ambang RSSI saat koneksi mulai gagal.

1.10 Analisis

Jawab berdasarkan tabel bagian Pengukuran, bukan berdasarkan teori saja.

1. Bagaimana pengaruh jarak terhadap nilai RSSI?
2. Pada nilai RSSI berapa koneksi mulai gagal? Bandingkan dengan tabel referensi.
3. Berapa latency rata-rata dari scan sampai connected (ms)?
4. Apakah halangan (tembok atau tubuh) memengaruhi keberhasilan koneksi?
5. Apakah BLE P2P cocok untuk aplikasi yang butuh koneksi cepat dan hemat daya? Jelaskan dari data hasil pengukuran.

1.11 Concept Check

1. Apa perbedaan Peripheral dan Central pada BLE?
2. Apa saja yang dimuat dalam paket advertising?
3. Apa beda active scan dan passive scan?
4. Mengapa Node1 harus advertise ulang setelah Node2 disconnect?
5. Apa fungsi Service UUID dalam koneksi BLE?

1.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Memperkuat dan mengukur tautan

CH-1 — Interval heartbeat Ubah interval heartbeat dari 5000 ms ke 1000 ms; amati dampaknya pada keterbacaan log dan beban.

CH-2 — Indikator koneksi Tambahkan LED pada Node1 yang menyala saat ada Central terhubung.

CH-3 — Latency koneksi Ukur waktu dari Node2 start sampai `Koneksi berhasil` menggunakan `millis()`, 5 percobaan, lalu hitung rata-ratanya.

Level 3 — Tantangan · Tautan yang memulihkan diri

CH-4 — Self-healing Buat Node2 melakukan scan ulang otomatis saat `onDisconnect` agar tautan *self-healing* (fondasi robustness untuk M04 dan M05), seperti pada Kode 1.6.

```
// CH-4 - di ClientCallbacks::onDisconnect(), picu scan ulang
void onDisconnect(NimBLEClient* c, int reason) override {
  Serial.println("Terputus - scan ulang...");
  NimBLEDevice::getScan()->start(5000, false);
}
```

Kode 1.6: Kerangka CH-4 — scan ulang otomatis saat terputus

1.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori secara ringkas.
3. Konfigurasi — `platformio.ini`, environment, dan UUID.
4. Hasil eksperimen — log Serial Monitor kedua node (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel bagian Pengukuran.
6. Analisis dan concept check.
7. Challenge — termasuk CH-4.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

2. Modul 02 — Pertukaran Data Dua Arah via BLE

IoT COMMUNICATION LAB · MODUL 02

Exchange Data — Pertukaran Data Dua Arah via BLE

ESP32-H2 · BLE GATT · NOTIFY/WRITE · level Basic · 3 × 50 menit · folder kode `week02_ble_p2p_data`

PENGANTAR MODUL · Gambaran Umum

Modul 02 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat dasar. Misinya memindahkan payload aplikasi ke dua arah di atas tautan BLE, lalu mengukur apa yang benar-benar selamat sampai tujuan. Percobaan berjalan dalam mode P2P antara server yang menyediakan notify dan write dengan sebuah client, diamati melalui dua terminal Serial Monitor pada 115200 baud.

2.1 Pendahuluan

Modul 01 hanya membuktikan **tautan** terbentuk — belum ada satu byte aplikasi pun yang lewat. Modul ini memakai tautan itu untuk membawa data, dan memperkenalkan dua mekanisme yang akan dipakai terus sampai Modul 16: **notify** (server mendorong) dan **write** (client mengirim).

Prasyaratnya adalah M01: tautan BLE P2P sudah terbentuk dan Serial Monitor sudah terbaca sebagai instrumen. Yang dibangun di sini adalah dua characteristic dengan peran berbeda (TX/NOTIFY dan RX/WRITE), mekanisme subscribe, callback `onWrite` dan `onNotify`, pola echo, serta pengukuran loss dua arah. Semuanya dipakai lagi pada M03 ketika characteristic menjadi data terstruktur, M04 ketika notify berkembang menjadi telemetry berkala, M05 saat jumlah node bertambah, dan M16 ketika loss serta latency menjadi metrik pembanding antarprotokol.

Peta modul blok BLE

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan BLE P2P terbentuk dan stabil
02 (ini)	**Payload aplikasi mengalir dua arah lewat tautan M01**
03	GATT characteristic — read/write data terstruktur
04	Telemetry via notify (push berkala tanpa polling)
05	Lebih dari dua node — satu central, banyak peripheral
06	Relay A⟶B⟶C, jangkauan diperluas lewat hop

Tabel 2.1: Peta modul blok BLE

Kontrak data lab ini. Mulai modul ini setiap payload membawa penanda yang bisa dihitung — di sini `millis()`, mulai CH-1 berupa nomor urut `SEQ=<n>`. Nomor urut itulah yang membuat *packet loss* bisa dihitung, dan format yang sama akan muncul lagi di Zigbee (M09), Thread (M12), dan MQTT (M14).

2.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mengalirkan payload aplikasi dua arah dan mengukurnya

1. Membuat dua characteristic pada satu service BLE dengan property berbeda (`NOTIFY` untuk TX, `WRITE` untuk RX) dan menunjukkan baris kode yang menetapkannya.
2. Menjalankan subscribe dari sisi client sehingga notify dari server benar-benar diterima, dibuktikan dari log kedua node.
3. Mengirim data dari client ke server memakai write dan menjelaskan jalur echo-nya (`onWrite` → `notify` → `onNotify`) dari urutan timestamp di dua Serial Monitor.
4. Menghitung packet loss tiap arah secara terpisah (notify hilang dibandingkan write hilang) pada minimal 4 jarak berbeda.

Kriteria keberhasilan

- ☐ Client subscribe ke characteristic TX dan menerima notify tiap 2 detik.
- ☐ Write dari client tiba di server dan di-echo balik utuh (isi sama persis).
- ☐ Tabel jarak–RSSI–loss terisi dari pengukuran sendiri, minimal 4 jarak.
- ☐ CH-1 selesai: payload memakai `SEQ=<n>` sehingga loss terhitung dari lompatan nomor.

2.3 Dasar Teori (Secukupnya)

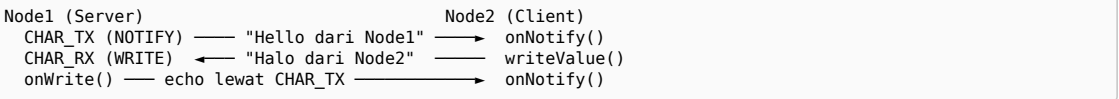
Teori di sini dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (mengapa GATT dirancang berlapis, detail ATT/L2CAP) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 2.2.

Istilah	Definisi kerja di lab ini
GATT	Skema data hierarkis Server → Service → Characteristic.
Service	Wadah characteristic; UUID `4fafc201-1fb5-459e-8fcc-c5c9c331914b`.
Characteristic	Unit data; punya UUID sendiri dan satu atau lebih property.
Property WRITE	Client boleh menulis nilai ke characteristic milik server.
Property NOTIFY	Server mendorong nilai ke client tanpa diminta — tetapi hanya setelah client subscribe.
Subscribe	Client mengaktifkan notify pada characteristic TX (menulis ke CCCD).
`onWrite` / `onNotify`	Callback yang dipicu saat data ditulis ke server atau notifikasi tiba di client.
Echo	Server mengirim balik isi write lewat notify — dipakai untuk membuktikan jalur pulang.

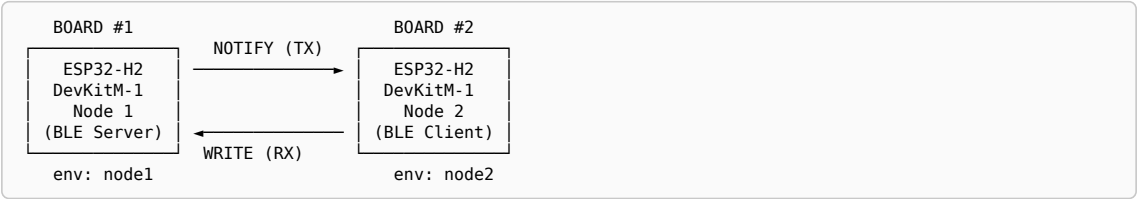
Tabel 2.2: Istilah kerja Modul 02

Mengapa dua characteristic, bukan satu? Satu characteristic hanya bisa mengalir efisien ke satu arah: `NOTIFY` adalah dorongan server ke client, `WRITE` adalah kiriman client ke server. Memisahkan TX dan RX membuat arah data terbaca langsung dari UUID-nya — pola yang sama dipakai profil serial BLE (Nordic UART Service) di dunia nyata.

Sekuens protokol yang diamati



2.4 Topologi



Node	Board	Environment	Peran	Aksi periodik
Node 1	ESP32-H2 DevKitM-1	`node1`	BLE Server (`NODE1_H2`)	notify `Hello dari Node1 (<millis>)` tiap 2 s
Node 2	ESP32-H2 DevKitM-1	`node2`	BLE Client	write `Halo dari Node2 (<millis>)` tiap 3 s

Tabel 2.3: Peran dan aksi periodik tiap node Modul 02

Kedua peran berjalan di **ESP32-H2** dengan radio Bluetooth LE — modul ini tidak membutuhkan ESP32-C6. Pesan yang diterima server di-echo kembali ke client lewat notify.

Address map (identik di kedua node)

Objek	UUID
Service	`4fafc201-1fb5-459e-8fcc-c5c9c331914b`
CHAR_TX (NOTIFY)	`beb5483e-36e1-4688-b7f5-ea07361b26a8`
CHAR_RX (WRITE)	`beb5483e-36e1-4688-b7f5-ea07361b26a9`

Tabel 2.4: Peta UUID Modul 02

2.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	USB-A/C ke micro-USB, **kabel data** (bukan kabel *charge-only*)	2
3	PC/Laptop	PlatformIO Core/IDE terpasang, 2 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` — otomatis via `lib_deps`	—
5	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (Arduino core 3.3.11)	—

Tabel 2.5: Alat dan bahan Modul 02

2.6 Kode Program

Kunci agar dua board tidak salah flash. Dengan dua ESP32-H2 terpasang bersamaan, auto-detect port bisa mengirim firmware `node1` ke board yang dimaksudkan sebagai `node2`. Pin port tiap environment seperti Kode 2.1.

```
[env:node1]
build_src_filter = +<node1/*.cpp>
upload_port  = /dev/ttyACM0     ; Windows: COM3  -- board Node1
monitor_port = /dev/ttyACM0

[env:node2]
build_src_filter = +<node2/*.cpp>
upload_port  = /dev/ttyACM2     ; Windows: COM4  -- board Node2
monitor_port = /dev/ttyACM2
```

Kode 2.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>
```

Kode 2.2: `platformio.ini` Modul 02 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TX_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define CHAR_RX_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a9"

static bool deviceConnected = false;
static NimBLECharacteristic *pTxCharacteristic = nullptr;
static unsigned long lastSend = 0;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Client terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Client terputus, mulai advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// Dipanggil saat Node2 menulis ke CHAR_RX; isinya digemakan balik lewat notify
class RxCallbacks : public NimBLECharacteristicCallbacks {
public:
  void onWrite(NimBLECharacteristic *pCharacteristic, NimBLEConnInfo &connInfo) override {
    NimBLEAttValue value = pCharacteristic->getValue();
    if (value.length() > 0) {
      Serial.print("RX dari Node2: ");
      Serial.println(value.c_str());

      pTxCharacteristic->setValue(value.data(), value.length());
      pTxCharacteristic->notify();
    }
  }
};

void setup() {
  Serial.begin(115200);
```

```
  Serial.println("Node1 (BLE Server) starting...");

  NimBLEDevice::init("NODE1_H2");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);

  pTxCharacteristic = pService->createCharacteristic(CHAR_TX_UUID, NIMBLE_PROPERTY::NOTIFY);
  NimBLECharacteristic *pRxCharacteristic = pService->createCharacteristic(CHAR_RX_UUID,
NIMBLE_PROPERTY::WRITE);
  pRxCharacteristic->setCallbacks(new RxCallbacks());
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("NODE1_H2");
  pAdvertising->start();

  Serial.println("Menunggu koneksi dari Node2...");
}

void loop() {
  if (deviceConnected && millis() - lastSend > 2000) {
    lastSend = millis();
    String msg = "Hello dari Node1 (" + String(millis()) + ")";
    pTxCharacteristic->setValue((const uint8_t *)msg.c_str(), msg.length());
    pTxCharacteristic->notify();
    Serial.println("TX ke Node2: " + msg);
  }
}
```

Kode 2.3: `src/node1/main.cpp` — BLE Server dengan CHAR_TX dan CHAR_RX

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TX_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define CHAR_RX_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a9"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress node1Addr;
static bool doConnect = false;
static bool connected = false;
static NimBLEClient *pClient = nullptr;
static NimBLERemoteCharacteristic *pRxCharacteristic = nullptr;
static unsigned long lastSend = 0;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override {
    connected = true;
    Serial.println("Terhubung ke Node1");
  }
  void onDisconnect(NimBLEClient *pClient, int reason) override {
    connected = false;
    Serial.println("Terputus dari Node1");
  }
};

// Callback notify NimBLE 2.x: sebuah fungsi, bukan objek callback
static void onNotify(NimBLERemoteCharacteristic *pChar, uint8_t *pData,
                     size_t length, bool isNotify) {
  Serial.print("RX dari Node1: ");
  Serial.write(pData, length);
  Serial.println();
}

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *advertisedDevice) override {
    if (advertisedDevice->getName() == "NODE1_H2") {
```

```
      Serial.println("Node1 ditemukan");
      node1Addr = advertisedDevice->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToNode1() {
  pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(node1Addr)) {
    Serial.println("Gagal terhubung");
    NimBLEDevice::deleteClient(pClient);
    pClient = nullptr;
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pTx = pService->getCharacteristic(CHAR_TX_UUID);
  pRxCharacteristic = pService->getCharacteristic(CHAR_RX_UUID);
  if (pTx == nullptr || pRxCharacteristic == nullptr) {
    Serial.println("Characteristic tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  if (pTx->canNotify()) {
    pTx->subscribe(true, onNotify);
  }

  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node2 (BLE Client) starting...");

  NimBLEDevice::init("NODE2_H2");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Scanning Node1...");
}

void loop() {
  if (doConnect) {
    if (connectToNode1()) {
      Serial.println("Koneksi berhasil");
    }
    doConnect = false;
  }

  if (connected && pRxCharacteristic && millis() - lastSend > 3000) {
    lastSend = millis();
    String msg = "Halo dari Node2 (" + String(millis()) + ")";
    // response = true: CHAR_RX hanya punya property WRITE (bukan WRITE_NR)
    pRxCharacteristic->writeValue((const uint8_t *)msg.c_str(), msg.length(), true);
    Serial.println("TX ke Node1: " + msg);
  }
}
```

Kode 2.4: `src/node2/main.cpp` — BLE Client yang subscribe dan write

2.7 Build dan Flash

```
pio device list                                    # catat port dulu
pio run -d week02_ble_p2p_data -e node1 -t upload
pio run -d week02_ble_p2p_data -e node2 -t upload -t monitor
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ Jalankan `pio device list`, catat port tiap board, isi `upload_port`/`monitor_port` di atas.
- ☐ Board pertama terhubung (akan diflash `node1`, BLE Server), board kedua terhubung (`node2`, BLE Client).
- ☐ Firmware `node1` dan `node2` berhasil build tanpa galat.
- ☐ Dua Serial Monitor 115200 baud dibuka lebih dulu, lalu tekan RESET agar sekuens boot terekam.

2.8 Percobaan

2.8.1 EXP-01 — Connect dan Subscribe

Node1 advertise `NODE1_H2`; Node2 scan, konek, mencari service dan kedua characteristic, lalu subscribe ke TX.

```
Node2: Scan → NODE1_H2 ditemukan → Connect → getService()
      → getCharacteristic(TX/RX) → subscribe(TX, notify)
```

Data capture

Parameter	Hasil
Nama device server yang ditemukan	
Service UUID ditemukan	
Jumlah characteristic ditemukan	
Waktu dari scan sampai subscribe selesai (s)	

Tabel 2.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Di `src/node2/main.cpp`, `pTx->subscribe(true, onNotify)` tampak seperti satu baris biasa. Sebenarnya baris itu **menulis ke CCCD** (Client Characteristic Configuration Descriptor) milik server. Cari di `src/node1/main.cpp` di mana descriptor itu dibuat — jawabannya: tidak ada, NimBLE menambahkannya otomatis begitu property `NOTIFY` dipasang. Buktikan dengan nRF Connect: buka characteristic TX dan lihat descriptor `0x2902`.

CHECKPOINT

Node2 mencetak `Koneksi berhasil`. Jika berhenti di `Scanning Node1...`: pastikan board `node1` menyala dan nama pada `advertisedDevice->getName()` di Node2 sama persis dengan `pAdvertising->setName()` di Node1.

2.8.2 EXP-02 — Downlink: Server ke Client (notify)

Setelah terhubung, Node1 mengirim notify setiap 2000 ms.

Expected output — Node1 (Server)

```
Node1 (BLE Server) starting...
Menunggu koneksi dari Node2...
Client terhubung
TX ke Node2: Hello dari Node1 (2005)
RX dari Node2: Halo dari Node2 (3001)
TX ke Node2: Hello dari Node1 (4006)
TX ke Node2: Hello dari Node1 (6007)
```

Expected output — Node2 (Client)

```
Node2 (BLE Client) starting...
Scanning Node1...
Node1 ditemukan
Terhubung ke Node1
Koneksi berhasil
RX dari Node1: Hello dari Node1 (2005)
RX dari Node1: Halo dari Node2 (3001)
TX ke Node1: Halo dari Node2 (3001)
RX dari Node1: Hello dari Node1 (4006)
RX dari Node1: Hello dari Node1 (6007)
```

CHECKPOINT

Baris `RX dari Node1` muncul di Node2 dengan jarak ±2 detik dan nilai `millis()` yang sama persis dengan `TX ke Node2` di Node1. Jika nilainya berbeda, log yang sedang dibaca tidak sinkron — hentikan dan periksa dulu, jangan lanjut ke EXP-03.

2.8.3 EXP-03 — Uplink dan Echo: Client ke Server (write)

Node2 menulis ke characteristic RX tiap 3000 ms. Server menerima (`onWrite`), mencetak, lalu meng-echo balik lewat notify pada characteristic TX.

```
Node2 ── WRITE "Halo dari Node2 (6002)" ──► Node1 CHAR_RX
Node1 ── print "RX dari Node2: ..."
Node1 ── NOTIFY (echo) ──► Node2 print "RX dari Node1: Halo dari Node2 (6002)"
```

Expected output — Node1

```
TX ke Node2: Hello dari Node1 (6007)
RX dari Node2: Halo dari Node2 (6002)
TX ke Node2: Hello dari Node1 (8008)
```

Echo tidak dicetak sebagai baris `TX` tersendiri di Node1: `onWrite()` hanya mencetak `RX dari Node2: ...`, lalu langsung meneruskannya lewat notify.

Expected output — Node2

```
RX dari Node1: Halo dari Node2 (6002)
TX ke Node1: Halo dari Node2 (6002)
```

Baris echo `RX dari Node1` tercetak **sebelum** `TX ke Node1` karena Node2 baru mencetak `TX` setelah `writeValue()` selesai, sedangkan notify echo dari Node1 sudah tiba lebih dulu.

Data capture

Parameter	Hasil
Interval notify Node1 (ms)	2000
Interval write Node2 (ms)	3000
Contoh payload notify Node1	
Contoh payload write Node2	
Apakah echo diterima Node2? (ya/tidak)	

Tabel 2.7: Lembar pengamatan EXP-03

CHECKPOINT

Pada Node2 muncul **dua jenis** baris `RX dari Node1`: yang berisi `Hello ...` (notify periodik) dan yang berisi `Halo ...` (echo dari write yang dikirim sendiri). Jika hanya satu jenis yang muncul, satu arah belum jalan — perbaiki sebelum masuk ke pengukuran.

2.9 Verifikasi Hardware — Log Referensi

Modul ini sudah dijalankan pada 2 × **ESP32-H2 DevKitM-1** (jarak ±20 cm, capture 25 detik). Log di bawah adalah hasil sebenarnya, bukan ilustrasi.

```
# Node 1 (ESP32-H2, env node1)          # Node 2 (ESP32-H2, env node2)
[0.200] Node1 (BLE Server) starting...  [0.401] Node2 (BLE Client) starting...
[0.401] Menunggu koneksi dari Node2...  [0.401] Scanning Node1...
[0.601] Client terhubung                [0.401] Node1 ditemukan
[2.205] TX ke Node2: Hello ... (2005)   [0.601] Terhubung ke Node1
[3.407] RX dari Node2: Halo ... (3001)  [2.404] RX dari Node1: Hello ... (2005)
                                        [3.406] TX ke Node1: Halo ... (3001)
                                        [3.406] RX dari Node1: Halo ... (3001)  <- echo
```

Parameter	Hasil terukur
Waktu scan sampai subscribe	± 1,4 s
Notify Node1 diterima Node2	12 / 12 (0 % loss)
Write Node2 diterima Node1	8 / 8 (0 % loss)
Echo kembali ke Node2	ya, payload utuh

Tabel 2.8: Hasil verifikasi hardware Modul 02

2.10 Pengukuran

Pindahkan Node1 menjauh; hitung jumlah pesan yang diterima Node2 per interval pengamatan. **Hitung tiap arah terpisah** — inti modul ini adalah dua arah tidak selalu gagal bersamaan.

RSSI dibaca dari log Node2 sendiri. Tambahkan pada heartbeat Node2 seperti Kode 2.5.

```
// Node2 - di loop(), saat terhubung
Serial.printf("RSSI: %d dBm\n", pClient->getRssi());
```

Kode 2.5: Cetak RSSI pada heartbeat Node2

RSSI (dBm)	Interpretasi
> −70	Kuat — koneksi andal
−70 s.d. −85	Baik — masih stabil
−85 s.d. −95	Marginal — mulai rawan putus
< −95	Tidak reliable — sering gagal

Tabel 2.9: Referensi interpretasi RSSI

Jarak	RSSI (dBm)	Notify /30 s (harapan 15)	Write ter-echo /30 s (harapan 10)	Loss down (%)	Loss up (%)
1 m					
3 m					
5 m					
10 m					
15 m					

Tabel 2.10: Lembar pengukuran loss dua arah Modul 02

Latency diestimasi dari selisih nilai `millis()` pada payload terhadap waktu kedatangan di Node2.

2.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran, bukan berdasarkan teori saja.

1. Bagaimana pengaruh jarak terhadap nilai RSSI pada data pengukuran yang diperoleh?
2. Pada RSSI berapa pesan mulai hilang? Bandingkan dengan tabel referensi.
3. Berapa latency rata-rata satu arah (Node1 ke Node2)?
4. Apakah loss downlink dan uplink mulai naik pada jarak yang sama? Jika berbeda, apa dugaan penyebabnya (daya pancar, `writeValue(..., true)` yang menunggu response, atau posisi antena)?
5. Untuk data periodik, mana yang lebih hemat: notify atau client yang mem-*polling* dengan read berulang? Dukung dengan jumlah transaksi per menit dari data hasil pengukuran.

2.12 Concept Check

1. Apa perbedaan property READ, WRITE, dan NOTIFY pada characteristic?
2. Mengapa client harus subscribe sebelum bisa menerima notify?
3. Apa yang terjadi jika server memanggil `notify()` saat tidak ada client terhubung?
4. Bagaimana arsitektur GATT menentukan arah aliran data?
5. `writeValue(..., true)` menunggu response dari server, `false` tidak. Kapan masing-masing lebih tepat dipakai?

2.13 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Payload bernomor urut dan penghitungan loss

CH-1 — Nomor urut (wajib, dipakai modul berikutnya) Ganti payload notify Node1 menjadi `SEQ=<n>` dengan nomor urut bertambah tiap kirim, interval 500 ms, seperti Kode 2.6.

CH-2 — Deteksi loss otomatis Di Node2, simpan nomor urut terakhir dan cetak peringatan saat ada lompatan, lalu hitung loss-nya seperti Kode 2.7. Contoh pelaporan: 30 dikirim, 27 diterima, sehingga loss = (30 − 27)/30 × 100 % = 10 %.

```
// Node1 - loop()
static uint32_t seq = 0;
String msg = "SEQ=" + String(++seq);
```

Kode 2.6: Kerangka CH-1 — payload bernomor urut

```
// Node2 - di onNotify()
static uint32_t last = 0;
uint32_t n = atoi(strchr((char*)pData, '=') + 1);
if (last && n != last + 1) Serial.printf("LOSS: %lu paket\n", n - last - 1);
last = n;
```

Kode 2.7: Kerangka CH-2 — deteksi lompatan nomor urut

Level 3 — Tantangan · Uji arah balik dan batas laju

CH-3 — Uplink berpenanda Terapkan hal yang sama pada arah write (Node2 ke Node1) sehingga loss uplink terhitung otomatis di Node1. Bandingkan angkanya dengan loss downlink pada jarak yang sama.

CH-4 — Backpressure Turunkan interval notify Node1 ke 50 ms. Amati apakah Node2 masih menerima semua nomor urut. Pada interval berapa mulai ada yang hilang, dan mengapa? Petunjuk: connection interval BLE.

2.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (GATT, property, subscribe, echo).
3. Konfigurasi — `platformio.ini`, environment, UUID TX/RX.
4. Hasil eksperimen — log Serial Monitor kedua arah (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel bagian Pengukuran, loss dua arah terpisah.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2, sertakan potongan kode yang diubah.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

3. Modul 03 — GATT Server dan Client Terstruktur

IoT COMMUNICATION LAB · MODUL 03

Build a BLE Service — GATT Server dan Client Terstruktur

ESP32-H2 · BLE GATT · READ/WRITE · level Intermediate · 3 × 50 menit · folder kode `week03_ble_client_server`

PENGANTAR MODUL · Gambaran Umum

Modul 03 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya merancang service GATT sendiri, dengan satu characteristic berperan sebagai data dan satu lagi sebagai kanal perintah. Percobaan berjalan dalam mode client-server memakai operasi GATT read dan write, diamati melalui dua terminal Serial Monitor pada 115200 baud, dengan nRF Connect sebagai pembanding opsional.

3.1 Pendahuluan

Modul 02 memindahkan *string* bebas lewat dua characteristic. Modul ini menaikkan satu tingkat abstraksi: characteristic tidak lagi sekadar pipa, tetapi **mewakili keadaan perangkat** — `CHAR_COUNTER` adalah nilai yang bisa dibaca kapan saja, `CHAR_CMD` adalah kanal perintah. Inilah pola *sensor dan aktuator* yang nanti muncul lagi sebagai atribut Zigbee (M08) dan topic MQTT (M14).

Prasyaratnya adalah M02: service, characteristic, property, dan pola callback sudah dikenali. Yang dibangun di sini adalah rancangan service dengan dua characteristic berbeda peran, akses `READ` on-demand oleh client, kanal perintah `WRITE`, serta perbandingan biaya antara polling dan push. Rancangan itu dipakai lagi pada M04 ketika characteristic yang sama diubah menjadi push/notify, M08 pada perintah ON/OFF versi Zigbee, dan M14 ketika topic data dipisahkan dari topic perintah pada MQTT.

Peta modul blok BLE

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan BLE P2P terbentuk dan stabil
02	Payload aplikasi mengalir dua arah
03 (ini)	**Characteristic mewakili state dan perintah — bukan sekadar pipa**
04	State yang sama didorong berkala (notify), polling dihapus
05	Lebih dari dua node
06	Relay A→B→C

Tabel 3.1: Peta modul blok BLE

Kontrak data lab ini. Pemisahan **kanal data** (`CHAR_COUNTER`) dan **kanal perintah** (`CHAR_CMD`) adalah pola yang dipertahankan sampai modul terakhir: Zigbee memisahkannya menjadi cluster/endpoint, MQTT memisahkannya menjadi `praktikum/h2/telemetri` dan `praktikum/h2/perintah`.

3.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Merancang service GATT berisi data dan perintah

1. Merancang satu service GATT berisi dua characteristic dengan property berbeda (`READ` untuk data, `WRITE` untuk perintah) dan menjelaskan alasan pemisahannya.
2. Membaca nilai characteristic dari sisi client secara berkala (`readValue()`) dan membuktikan nilainya berubah mengikuti counter di server.
3. Mengirim perintah dari client ke server dan menunjukkan jejaknya di log server (`onWrite`).
4. Menghitung jumlah transaksi radio per menit pada skema polling dan membandingkannya dengan skema notify Modul 04 memakai angka sendiri.

Kriteria keberhasilan

- ☐ Client mencetak `READ counter = <n>` tiap 2 detik dengan nilai yang naik.
- ☐ Selisih counter antar-read konsisten dengan rasio interval read banding interval counter.
- ☐ Server mencetak `Perintah dari client: ON` tiap 5 detik.
- ☐ Tabel jarak–RSSI–keberhasilan read terisi dari pengukuran sendiri.

3.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (model atribut GATT, ATT MTU, hubungan dengan Bluetooth SIG profile) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 3.2.

Istilah	Definisi kerja di lab ini
GATT Server	Pihak yang **memiliki** data; di sini ESP32-H2 dengan counter.
GATT Client	Pihak yang **meminta** data; melakukan read dan write.
Property READ	Client boleh membaca nilai kapan saja atas inisiatifnya sendiri.
Property WRITE	Client boleh mengubah nilai; server bereaksi lewat `onWrite`.
Polling	Client membaca berulang pada interval tetap — tiap read adalah satu transaksi radio.
`readValue()`	Permintaan baca dari client; server menjawab dengan nilai terkini.
Handle / UUID	Alamat characteristic di dalam service; UUID dipakai untuk mencarinya.

Tabel 3.2: Istilah kerja Modul 03

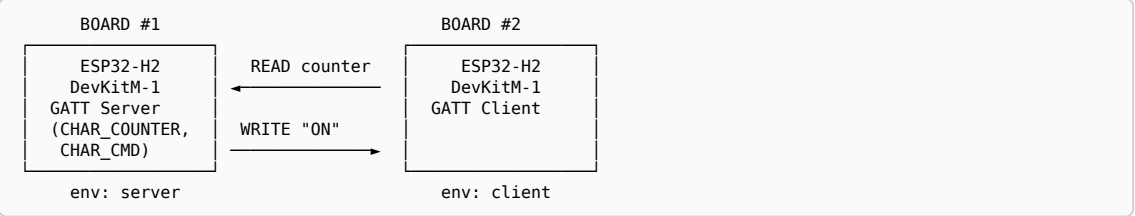
Mengapa counter, bukan sensor? Counter naik 1 tiap 1000 ms secara pasti, jadi nilai yang hilang atau read yang gagal langsung terlihat sebagai lompatan angka. Sensor asli akan menyembunyikan kesalahan di balik fluktuasi nilai.

Mengapa server hanya menaikkan counter saat ada client? Agar nilai counter merepresentasikan **lama koneksi**, bukan lama board menyala. Periksa `src/server/main.cpp` dan tunjukkan baris yang menegakkan aturan ini.

Sekuens protokol yang diamati

```
Server                                      Client
 counter++ tiap 1000 ms
        ◂──── READ CHAR_COUNTER (tiap 2000 ms) ────
 kirim nilai ──────────────────────────────▸  "READ counter = 12"
        ◂──── WRITE "ON" ke CHAR_CMD (tiap 5000 ms) ─
 onWrite() → "Perintah dari client: ON"
```

3.4 Topologi



Node	Board	Environment	Peran	Aksi periodik
Server	ESP32-H2 DevKitM-1	`server`	GATT Server (`GATT_SERVER`)	counter++ tiap 1000 ms
Client	ESP32-H2 DevKitM-1	`client`	GATT Client (`GATT_CLIENT`)	read tiap 2000 ms, write `ON` tiap 5000 ms

Tabel 3.3: Peran dan aksi periodik tiap node Modul 03

Kedua peran berjalan di **ESP32-H2** memakai radio Bluetooth LE; ESP32-C6 baru dipakai mulai Modul 13 (gateway Wi-Fi).

Address map

Objek	UUID
Service	`4fafc201-1fb5-459e-8fcc-c5c9c331914b`
CHAR_COUNTER (READ)	`beb5483e-36e1-4688-b7f5-ea07361b26b0`
CHAR_CMD (WRITE)	`beb5483e-36e1-4688-b7f5-ea07361b26b1`

Tabel 3.4: Peta UUID Modul 03

3.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` via `lib_deps`	—
5	nRF Connect (opsional)	Android/iOS — untuk melihat struktur GATT dari luar	1

Tabel 3.5: Alat dan bahan Modul 03

3.6 Kode Program

Pin port agar tidak salah flash (Kode 3.1).

```
[env:server]
build_src_filter = +<server/*.cpp>
upload_port  = /dev/ttyACM0     ; Windows: COM3  -- board Server
monitor_port = /dev/ttyACM0

[env:client]
build_src_filter = +<client/*.cpp>
upload_port  = /dev/ttyACM2     ; Windows: COM4  -- board Client
monitor_port = /dev/ttyACM2
```

Kode 3.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = server

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:server]
build_src_filter = +<server/*.cpp>

[env:client]
build_src_filter = +<client/*.cpp>
```

Kode 3.2: `platformio.ini` Modul 03 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_COUNTER_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b0"
#define CHAR_CMD_UUID     "beb5483e-36e1-4688-b7f5-ea07361b26b1"

static uint32_t counter = 0;
static bool deviceConnected = false;
static NimBLECharacteristic *pCounter = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Client terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Client terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// Tulis nilai counter terkini ke characteristic (sebagai teks)
static void publishCounter() {
  char buf[12];
  snprintf(buf, sizeof(buf), "%lu", (unsigned long)counter);
  pCounter->setValue((const uint8_t *)buf, strlen(buf));
}

// Dipanggil saat client menulis ke CHAR_CMD (perintah ON/OFF)
class CmdCallbacks : public NimBLECharacteristicCallbacks {
public:
  void onWrite(NimBLECharacteristic *pCharacteristic, NimBLEConnInfo &connInfo) override {
    NimBLEAttValue value = pCharacteristic->getValue();
    Serial.print("Perintah dari client: ");
    Serial.println(value.c_str());
  }
};
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("GATT Server starting...");

  NimBLEDevice::init("GATT_SERVER");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);

  // Characteristic READ: client membaca nilai ini (counter)
  // Nilai disimpan sebagai teks agar client dapat mencetaknya langsung.
  pCounter = pService->createCharacteristic(
    CHAR_COUNTER_UUID, NIMBLE_PROPERTY::READ);
  publishCounter();

  // Characteristic WRITE: client menulis perintah ke sini
  NimBLECharacteristic *pCmd = pService->createCharacteristic(
    CHAR_CMD_UUID, NIMBLE_PROPERTY::WRITE);
  pCmd->setCallbacks(new CmdCallbacks());
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("GATT_SERVER");
  pAdvertising->start();

  Serial.println("Menunggu client...");
}

void loop() {
  // Update counter tiap 1 detik agar terbaca berubah oleh client
  static unsigned long last = 0;
  if (deviceConnected && millis() - last > 1000) {
    last = millis();
    counter++;
    publishCounter();  // tanpa ini, client selalu membaca nilai lama
  }
}
```

Kode 3.3: `src/server/main.cpp` — GATT Server dengan counter dan kanal perintah

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_COUNTER_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b0"
#define CHAR_CMD_UUID     "beb5483e-36e1-4688-b7f5-ea07361b26b1"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress serverAddr;
static bool doConnect = false;
static bool connected = false;
static NimBLERemoteCharacteristic *pCounter = nullptr;
static NimBLERemoteCharacteristic *pCmd = nullptr;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override {
    connected = true;
    Serial.println("Terhubung ke server");
  }
  void onDisconnect(NimBLEClient *pClient, int reason) override {
    connected = false;
    Serial.println("Terputus dari server");
  }
};

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *advertisedDevice) override {
    if (advertisedDevice->getName() == "GATT_SERVER") {
      Serial.println("Server ditemukan");
```

```
      serverAddr = advertisedDevice->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToServer() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(serverAddr)) {
    Serial.println("Gagal terhubung");
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  pCounter = pService->getCharacteristic(CHAR_COUNTER_UUID);
  pCmd = pService->getCharacteristic(CHAR_CMD_UUID);
  if (pCounter == nullptr || pCmd == nullptr) {
    Serial.println("Characteristic tidak ditemukan");
    pClient->disconnect();
    return false;
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("GATT Client starting...");

  NimBLEDevice::init("GATT_CLIENT");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Scanning server...");
}

void loop() {
  if (doConnect) {
    if (connectToServer()) {
      Serial.println("Koneksi berhasil");
    }
    doConnect = false;
  }

  if (!connected) return;

  static unsigned long lastRead = 0;
  if (millis() - lastRead > 2000) {
    lastRead = millis();
    NimBLEAttValue value = pCounter->readValue();
    Serial.print("READ counter = ");
    Serial.println(value.c_str());
  }

  static unsigned long lastWrite = 0;
  if (millis() - lastWrite > 5000) {
    lastWrite = millis();
    const char *cmd = "ON";
    // response = true: CHAR_CMD hanya punya property WRITE (bukan WRITE_NR)
    pCmd->writeValue((const uint8_t *)cmd, strlen(cmd), true);
    Serial.println("WRITE perintah: ON");
  }
}
```

Kode 3.4: `src/client/main.cpp` — GATT Client yang melakukan read dan write

3.7 Build dan Flash

```
pio device list
pio run -d week03_ble_client_server -e server -t upload
pio run -d week03_ble_client_server -e client -t upload -t monitor
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, port tiap board dicatat dan diisikan di atas.
- ☐ Board pertama terhubung (env `server`), board kedua terhubung (env `client`).
- ☐ Firmware `server` dan `client` berhasil build tanpa galat.
- ☐ Dua Serial Monitor 115200 baud dibuka lebih dulu, lalu RESET kedua board.

3.8 Percobaan

3.8.1 EXP-01 — Service Discovery

Client scan, connect, lalu mencari service dan kedua characteristic berdasarkan UUID. Catat apa yang terjadi jika UUID tidak ditemukan (lihat penanganan `nullptr` di `src/client/main.cpp`).

Data capture

Parameter	Hasil
Nama device server	
Service UUID ditemukan	
Characteristic ditemukan (jumlah dan UUID)	
Waktu scan sampai `Koneksi berhasil` (s)	

Tabel 3.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Buka `GATT_SERVER` dengan nRF Connect dan lihat daftar characteristic beserta property-nya (`READ`, `WRITE`). Cocokkan tiap baris di aplikasi dengan baris `createCharacteristic(...)` di `src/server/main.cpp`. Property yang muncul di aplikasi **berasal dari argumen kedua** fungsi itu — ubah salah satunya menjadi `NIMBLE_PROPERTY::READ` saja lalu amati apa yang hilang di aplikasi.

CHECKPOINT

Client mencetak `Koneksi berhasil`. Jika muncul `Characteristic tidak ditemukan`, berarti UUID di client dan server berbeda — samakan dulu sebelum lanjut.

3.8.2 EXP-02 — Read: Client Menarik Data

Server menaikkan counter tiap 1000 ms **hanya saat ada client terhubung**. Client membaca tiap 2000 ms.

Expected output — Server

```
GATT Server starting...
Menunggu client...
Client terhubung
Perintah dari client: ON
Perintah dari client: ON
```

Expected output — Client

```
GATT Client starting...
Scanning server...
Server ditemukan
Terhubung ke server
Koneksi berhasil
READ counter = 2
READ counter = 4
WRITE perintah: ON
READ counter = 6
READ counter = 8
WRITE perintah: ON
```

CHECKPOINT

Selisih dua `READ counter` berturut-turut adalah **2** (interval read 2000 ms dibagi interval counter 1000 ms). Jika selisihnya tidak konsisten, ada read yang gagal — catat kejadiannya sebagai data pengukuran.

3.8.3 EXP-03 — Write: Client Mengirim Perintah

Client menulis `"ON"` ke `CHAR_CMD` tiap 5000 ms; server mencetak penerimaannya.

```
Server: counter++ tiap 1000 ms
Client: READ  ──────────────► "READ counter = 12"
Client: WRITE "ON" ─────────► Server: "Perintah dari client: ON"
```

Data capture

Parameter	Hasil
Interval read client (ms)	2000
Interval write client (ms)	5000
Interval update counter server (ms)	1000
Selisih nilai counter antar-read (rata-rata)	
Perintah yang dikirim client	ON
Jumlah transaksi radio per menit (read dan write)	

Tabel 3.7: Lembar pengamatan EXP-03

CHECKPOINT

Baris `Perintah dari client: ON` muncul di server kira-kira tiap 5 detik, dan **di antara** dua baris itu ada 2–3 baris `READ counter` di client. Urutan inilah bukti dua kanal berjalan independen.

3.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 2 × **ESP32-H2 DevKitM-1**, capture 25 detik.

```
# Server (ESP32-H2, env server)        # Client (ESP32-H2, env client)
[0.200] GATT Server starting...        [0.401] GATT Client starting...
[0.401] Menunggu client...             [0.401] Server ditemukan
[0.601] Client terhubung               [0.601] Terhubung ke server
[5.409] Perintah dari client: ON       [2.405] READ counter = 2
[10.418] Perintah dari client: ON      [4.408] READ counter = 4
                                       [5.409] WRITE perintah: ON
                                       [6.410] READ counter = 6
```

Parameter	Hasil terukur
Selisih counter antar-read	2 (read 2000 ms dibagi counter 1000 ms)
READ berhasil	12 / 12
WRITE `ON` sampai ke server	4 / 4
Transaksi radio per menit	30 read + 12 write = 42

Tabel 3.8: Hasil verifikasi hardware Modul 03

3.10 Pengukuran

RSSI dibaca dari log client sendiri (`pClient->getRssi()`).

RSSI (dBm)	Interpretasi
> −70	Kuat — koneksi andal
−70 s.d. −85	Baik — masih stabil
−85 s.d. −95	Marginal — mulai rawan putus
< −95	Tidak reliable — sering gagal

Tabel 3.9: Referensi interpretasi RSSI

Jarak	RSSI (dBm)	READ /30 s (harapan 15)	WRITE /30 s (harapan 6)	Loss read (%)
1 m				
3 m				
5 m				
10 m				
15 m				

Tabel 3.10: Lembar pengukuran keberhasilan read dan write Modul 03

Metrik turunan yang wajib dihitung: jumlah transaksi radio per menit pada skema polling ini. Angka ini akan dibandingkan langsung dengan skema notify Modul 04.

3.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana pengaruh jarak terhadap RSSI dan terhadap keberhasilan read?
2. Apakah read dan write mulai gagal pada jarak yang sama? Jika tidak, apa dugaan penyebabnya?

3. Berapa transaksi radio per menit yang dihasilkan skema polling ini? Berapa persen di antaranya membawa nilai counter yang **sama** dengan read sebelumnya (data mubazir)?
4. Jika interval read dipercepat menjadi 200 ms, berapa transaksi per menit dan apa dampaknya pada konsumsi daya client?
5. Untuk data yang jarang berubah, mana yang lebih efisien: polling atau notify? Dukung dengan angka dari poin sebelumnya.

3.12 Concept Check

1. Apa perbedaan peran GATT Server dan GATT Client? Apakah server selalu yang mengirim data?
2. Mengapa satu service bisa memuat banyak characteristic dengan property berbeda?
3. Apa yang terjadi bila client melakukan read pada characteristic yang tidak punya property `READ`?
4. Mengapa server hanya menaikkan counter saat ada client terhubung, dan apa akibatnya jika aturan itu dihapus?
5. Dalam sistem nyata, mana yang lebih tepat memegang "state" perangkat: server atau client? Mengapa?

3.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Perintah nyata dan status yang terbaca

CH-1 — Perintah nyata Ubah `CHAR_CMD` agar menerima `"ON"` dan `"OFF"` serta menyalakan atau mematikan LED bawaan server. Client mengirim keduanya bergantian.

CH-2 — Characteristic status Tambahkan characteristic ketiga (`READ`) berisi status LED, lalu buktikan dari client bahwa nilainya mengikuti perintah yang barusan dikirim.

Level 3 — Tantangan · Mengukur biaya polling dan memvalidasi masukan

CH-3 — Ukur biaya polling Hitung jumlah read per menit, lalu naikkan interval read ke 200 ms dan turunkan ke 10 detik. Catat untuk tiap kasus: transaksi per menit, jumlah nilai counter yang terlewat, dan jumlah read yang mubazir. Sajikan sebagai tabel.

CH-4 — Validasi perintah Tolak perintah selain `ON` dan `OFF` di `onWrite`, lalu cetak `Perintah tidak dikenal: <isi>`. Uji dengan mengirim string acak dari client.

3.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (GATT server/client, property READ/WRITE, polling).
3. Konfigurasi — `platformio.ini`, environment, UUID service dan characteristic.
4. Hasil eksperimen — log kedua node (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel bagian Pengukuran beserta hitungan transaksi per menit.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-3, sertakan tabel biaya polling.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

4. Modul 04 — Telemetry Berkala lewat Notify

IoT COMMUNICATION LAB · MODUL 04

Stream Telemetry — Telemetry Berkala lewat Notify

ESP32-H2 · BLE NOTIFY · TELEMETRY · level Intermediate · 3 × 50 menit · folder kode `week04_ble_telemetry`

PENGANTAR MODUL · Gambaran Umum

Modul 04 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya mengubah pola tarik (polling) menjadi pola dorong (notify), lalu membuktikan penghematannya dengan angka. Percobaan berjalan sebagai telemetry satu arah dari sensor yang berperan sebagai server menuju monitor yang berperan sebagai client, diamati melalui dua terminal Serial Monitor pada 115200 baud.

4.1 Pendahuluan

Modul 03 membuat client **menarik** data berulang-ulang, sebagian besar mubazir. Modul ini membalik arah inisiatif: sensor mendorong nilai baru begitu tersedia, monitor hanya menunggu. Pola inilah yang dipakai semua telemetri di sisa lab — Zigbee attribute report (M09), Thread UDP periodik (M11–M13), dan MQTT publish (M14). Angka transaksi per menit yang dihitung pada M03 dipakai lagi di sini sebagai pembanding.

Prasyaratnya adalah M03: service GATT dan hitungan transaksi radio pada skema polling. Yang dibangun di sini adalah notify sebagai telemetry berkala, mekanisme subscribe dari sisi client, pengamatan kontinuitas aliran data, dan perilaku sistem saat pengirim mati lalu hidup kembali. Keempatnya dipakai lagi pada M05 ketika satu central menerima notify dari banyak sensor, M06 ketika notify diteruskan hop demi hop, serta M14 dan M15 ketika telemetri yang sama berakhir di broker MQTT.

Peta modul blok BLE

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan BLE P2P terbentuk dan stabil
02	Payload aplikasi mengalir dua arah
03	Characteristic mewakili state dan perintah (polling)
04 (ini)	**Polling dihapus — sensor mendorong data sendiri (notify)**
05	Lebih dari dua node
06	Relay A→B→C

Tabel 4.1: Peta modul blok BLE

Kontrak data lab ini. Payload telemetri di lab ini selalu berupa **nilai terukur dalam bentuk string ringkas** (`"26.3"`, nanti `"suhu:26.3"`). Format itu dipertahankan agar hop terakhir — publish MQTT di M14/M15 — tidak perlu mengubah isinya sama sekali (*transparent forwarding*).

4.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mengganti polling dengan telemetry berbasis notify

1. Mengimplementasikan characteristic `NOTIFY` yang mengirim nilai sensor tiap 1000 ms hanya saat ada client yang subscribe.
2. Membuktikan dari log bahwa monitor tidak melakukan satu pun permintaan baca, namun tetap menerima seluruh nilai.
3. Menghitung jumlah transaksi radio per menit skema notify dan membandingkannya dengan skema polling Modul 03 dalam satu tabel.
4. Menjelaskan perilaku sistem saat sensor mati mendadak, berdasarkan log kedua node — bukan berdasarkan dugaan.

Kriteria keberhasilan

- ☐ Monitor menerima ± 60 nilai per menit tanpa pernah memanggil `readValue()`.
- ☐ Interval kedatangan terukur ± 1000 ms dan tercatat sebarannya.
- ☐ Uji gangguan (sensor di-reset) dilakukan dan perilaku kedua node tercatat.
- ☐ Tabel perbandingan polling (M03) dan notify (M04) terisi dari angka sendiri.

4.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (siklus connection event BLE, hubungan interval notify dengan konsumsi arus) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 4.2.

Istilah	Definisi kerja di lab ini
Telemetry	Pengiriman nilai terukur dari perangkat ke pemantau secara berkala.
Notify	Server mendorong nilai ke client tanpa diminta, tanpa acknowledgement.
Indication	Seperti notify tetapi menunggu balasan client — lebih andal, lebih mahal. Tidak dipakai di sini.
Subscribe / CCCD	Client mengaktifkan notify; tanpa langkah ini `notify()` di server tidak berefek apa pun.
Push dan pull	Push berarti pengirim menentukan kapan data dikirim; pull berarti penerima yang meminta (M03).
Sensor simulasi	Nilai suhu dibangkitkan `random()`, mulai 25,0 °C, fluktuasi ±1,0 °C, reset ke 25,0 bila keluar rentang 20–40 °C.

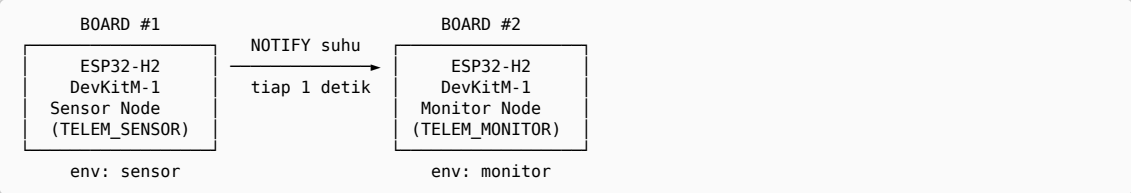
Tabel 4.2: Istilah kerja Modul 04

Mengapa sensornya disimulasi? Karena yang diuji modul ini adalah **kanal telemetri**, bukan akurasi sensor. Nilai simulasi menghilangkan variabel kalibrasi sehingga setiap kelainan pada log pasti berasal dari kanal radio. Mengganti `readSensor()` dengan sensor asli adalah CH-3.

Sekuens protokol yang diamati

```
Sensor (Server)                          Monitor (Client)
  advertise TELEM_SENSOR
        ◄──── connect + subscribe(CCCD) ────
  tiap 1000 ms:
    readSensor() → "26.3"
    notify() ──────────────────────────►  onNotify() → cetak
  (tidak ada permintaan baca sama sekali dari monitor)
```

4.4 Topologi



Node	Board	Environment	Peran	Aksi
Sensor	ESP32-H2 DevKitM-1	`sensor`	GATT Server (`TELEM_SENSOR`)	notify suhu tiap 1000 ms
Monitor	ESP32-H2 DevKitM-1	`monitor`	GATT Client (`TELEM_MONITOR`)	subscribe dan cetak tiap notifikasi

Tabel 4.3: Peran tiap node Modul 04

Keduanya **ESP32-H2** (radio Bluetooth LE). Tidak ada ESP32-C6 pada modul ini.

Address map

Objek	UUID
Service	`4fafc201-1fb5-459e-8fcc-c5c9c331914b`
CHAR telemetry (NOTIFY)	`beb5483e-36e1-4688-b7f5-ea07361b26b2`

Tabel 4.4: Peta UUID Modul 04

4.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` via `lib_deps`	—
5	Data Modul 03	tabel transaksi per menit skema polling — dipakai sebagai pembanding	1

Tabel 4.5: Alat dan bahan Modul 04

> **CATATAN**
> Suhu pada kode adalah **simulasi** (`random`), tidak memakai sensor fisik.

4.6 Kode Program

Pin port agar tidak salah flash (Kode 4.1).

```
[env:sensor]
build_src_filter = +<sensor/*.cpp>
upload_port  = /dev/ttyACM0     ; Windows: COM3  -- board Sensor
monitor_port = /dev/ttyACM0

[env:monitor]
build_src_filter = +<monitor/*.cpp>
upload_port  = /dev/ttyACM2     ; Windows: COM4  -- board Monitor
monitor_port = /dev/ttyACM2
```

Kode 4.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = sensor

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:sensor]
build_src_filter = +<sensor/*.cpp>

[env:monitor]
build_src_filter = +<monitor/*.cpp>
```

Kode 4.2: `platformio.ini` Modul 04 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID     "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TELEM_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26b2"

static bool deviceConnected = false;
static NimBLECharacteristic *pTelemetry = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Monitor terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Monitor terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// Simulasi pembacaan sensor (mis. suhu)
static float readSensor() {
  static float temp = 25.0;
  temp += (random(0, 20) - 10) / 10.0;  // fluktuasi +-1.0 C
  if (temp > 40.0) temp = 25.0;
  if (temp < 20.0) temp = 25.0;
  return temp;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Sensor Node (BLE Telemetry) starting...");

  NimBLEDevice::init("TELEM_SENSOR");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
```

```
  pTelemetry = pService->createCharacteristic(CHAR_TELEM_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("TELEM_SENSOR");
  pAdvertising->start();

  Serial.println("Menunggu monitor...");
}

void loop() {
  static unsigned long lastSend = 0;
  if (deviceConnected && millis() - lastSend > 1000) {
    lastSend = millis();
    float suhu = readSensor();
    String payload = String(suhu, 1);  // contoh: "26.3"
    pTelemetry->setValue((const uint8_t *)payload.c_str(), payload.length());
    pTelemetry->notify();
    Serial.println("Notify: suhu = " + payload + " C");
  }
}
```

Kode 4.3: `src/sensor/main.cpp` — sensor node yang mendorong telemetri

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID     "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TELEM_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26b2"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress sensorAddr;
static bool doConnect = false;
static bool connected = false;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override {
    connected = true;
    Serial.println("Terhubung ke sensor");
  }
  void onDisconnect(NimBLEClient *pClient, int reason) override {
    connected = false;
    Serial.println("Terputus dari sensor");
  }
};

// Callback notify NimBLE 2.x: sebuah fungsi, bukan objek callback
static void onTelemetry(NimBLERemoteCharacteristic *pChar, uint8_t *pData,
                        size_t length, bool isNotify) {
  Serial.print("Telemetry diterima: suhu = ");
  Serial.write(pData, length);
  Serial.println(" C");
}

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *advertisedDevice) override {
    if (advertisedDevice->getName() == "TELEM_SENSOR") {
      Serial.println("Sensor ditemukan");
      sensorAddr = advertisedDevice->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToSensor() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(sensorAddr)) {
    Serial.println("Gagal terhubung");
```

```
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pTelemetry = pService->getCharacteristic(CHAR_TELEM_UUID);
  if (pTelemetry == nullptr) {
    Serial.println("Characteristic tidak ditemukan");
    pClient->disconnect();
    return false;
  }
  if (pTelemetry->canNotify()) {
    pTelemetry->subscribe(true, onTelemetry);
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Monitor Node starting...");

  NimBLEDevice::init("TELEM_MONITOR");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Scanning sensor...");
}

void loop() {
  if (doConnect) {
    if (connectToSensor()) {
      Serial.println("Koneksi berhasil, menunggu telemetry...");
    }
    doConnect = false;
  }
}
```

Kode 4.4: `src/monitor/main.cpp` — monitor node yang hanya subscribe

4.7 Build dan Flash

```
pio device list
pio run -d week04_ble_telemetry -e sensor  -t upload
pio run -d week04_ble_telemetry -e monitor -t upload -t monitor
```

PENTING

Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, port tiap board dicatat dan diisikan di atas.
- ☐ Board pertama terhubung (env `sensor`), board kedua terhubung (env `monitor`).
- ☐ Firmware `sensor` dan `monitor` berhasil build tanpa galat.
- ☐ Dua Serial Monitor 115200 baud dibuka lebih dulu, lalu RESET kedua board.
- ☐ Angka transaksi per menit dari Modul 03 sudah di tangan.

4.8 Percobaan

4.8.1 EXP-01 — Subscribe dan Stream Start

Monitor scan, connect, lalu subscribe. Aliran data baru dimulai **setelah** subscribe berhasil — bukan setelah connect.

Data capture

Parameter	Hasil
Nama device sensor	
Waktu scan sampai `Koneksi berhasil` (s)	
Waktu connect sampai nilai pertama tiba (s)	
Apakah ada `readValue()` di kode monitor? (ya/tidak)	

Tabel 4.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Sebelum lanjut, komentari baris `pTx->subscribe(...)` di `src/monitor/main.cpp`, flash ulang, dan amati: koneksi tetap terbentuk, sensor tetap memanggil `notify()`, tetapi monitor **tidak menerima apa pun**. Kembalikan kodenya. Ini membuktikan notify bukan sekadar "server mengirim", melainkan kontrak dua pihak yang dicatat di CCCD.

CHECKPOINT

Monitor mencetak `Koneksi berhasil, menunggu telemetry...` lalu baris `Telemetry diterima` pertama muncul kurang dari 1,5 detik kemudian. Jika baris kedua tidak pernah muncul, subscribe gagal — periksa dulu.

4.8.2 EXP-02 — Aliran Telemetri

Sensor mengirim nilai tiap 1000 ms selama monitor terhubung.

Expected output — Sensor

```
Sensor Node (BLE Telemetry) starting...
Menunggu monitor...
Monitor terhubung
Notify: suhu = 24.7 C
Notify: suhu = 25.1 C
Notify: suhu = 24.6 C
```

Expected output — Monitor

```
Monitor Node starting...
Scanning sensor...
Sensor ditemukan
Terhubung ke sensor
Koneksi berhasil, menunggu telemetry...
Telemetry diterima: suhu = 24.7 C
Telemetry diterima: suhu = 25.1 C
Telemetry diterima: suhu = 24.6 C
```

Data capture

Parameter	Hasil
Interval notify (ms)	1000

Parameter	Hasil
Format payload	`s.d` (contoh: 26.3)
Rentang nilai suhu teramati (°C)	
Jumlah telemetry diterima per 60 detik	
Interval kedatangan terkecil dan terbesar (ms)	

Tabel 4.7: Lembar pengamatan EXP-02

CHECKPOINT

Tiap nilai `Notify:` di sensor punya pasangan `Telemetry diterima:` di monitor dengan angka yang sama persis. Cocokkan minimal 10 baris berturut-turut sebelum masuk EXP-03.

4.8.3 EXP-03 — Kontinuitas dan Pemulihan

1. Reset sensor selama ± 5 detik lalu nyalakan lagi. Amati `Monitor terputus, advertise ulang` di sensor dan apa yang dilakukan monitor.
2. Catat apakah aliran pulih sendiri atau monitor perlu di-reset, dan **jelaskan dari kode** mengapa demikian.
3. Jauhkan sensor perlahan sampai ada nilai yang hilang; catat interval kedatangan saat mulai tidak teratur.

Data capture

Parameter	Hasil
Pesan di sensor saat monitor hilang	
Pesan di monitor saat sensor hilang	
Aliran pulih otomatis? (ya/tidak)	
Alasan berdasarkan kode	

Tabel 4.8: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menunjukkan baris kode yang menentukan apakah monitor melakukan scan ulang atau tidak. Jika belum bisa, jangan lanjut ke analisis — jawabannya ada di `setup()` monitor.

4.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 2 × **ESP32-H2 DevKitM-1**, capture 25 detik.

```
# Sensor (ESP32-H2, env sensor)        # Monitor (ESP32-H2, env monitor)
[1.404] Notify: suhu = 24.3 C          [1.405] Telemetry diterima: suhu = 24.3 C
[2.406] Notify: suhu = 23.4 C          [2.407] Telemetry diterima: suhu = 23.4 C
[3.407] Notify: suhu = 23.8 C          [3.409] Telemetry diterima: suhu = 23.8 C
```

Parameter	Hasil terukur
Notify dikirim sensor	24
Telemetry diterima monitor	24 (0 % loss)
Interval kedatangan	1001 ± 2 ms
Rentang suhu teramati	21,3 – 24,3 °C
Selisih waktu sensor ke monitor	1–2 ms

Tabel 4.9: Hasil verifikasi hardware Modul 04

4.10 Pengukuran

Jarak	RSSI (dBm)	Telemetry /60 s (harapan 60)	Loss (%)	Interval terbesar (ms)
1 m				
3 m				
5 m				
10 m				
15 m				

Tabel 4.10: Lembar pengukuran aliran telemetri Modul 04

Tabel pembanding wajib — polling (M03) dan notify (M04). Isi kolom kiri dari data Modul 03 dan kolom kanan dari modul ini.

Aspek	Polling (M03)	Notify (M04)
Transaksi radio per menit		
Nilai baru yang benar-benar terbawa per menit		
Transaksi mubazir (nilai tak berubah) per menit		
Siapa yang menentukan waktu kirim	client	sensor
Konsekuensi bila data jarang berubah		

Tabel 4.11: Tabel pembanding polling dan notify

4.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa jumlah telemetry yang diterima per menit, dan berapa persen dari nilai yang dikirim sensor?
2. Bandingkan transaksi radio per menit skema polling dan notify dari tabel pembanding. Berapa persen penghematannya?
3. Pada jarak berapa interval kedatangan mulai tidak teratur, dan bagaimana bentuk ketidakteraturannya (melar atau melompat)?
4. Jika data hanya berubah tiap 10 detik, skema mana yang lebih boros? Tunjukkan dengan angka.

5. Apa risiko notify dibanding indication untuk data yang tidak boleh hilang, misalnya alarm?

4.12 Concept Check

1. Apa beda notify dan indication, dan kapan masing-masing dipakai?
2. Mengapa `notify()` tidak berefek apa pun sebelum client subscribe?
3. Apa yang disimpan di CCCD dan siapa yang menulisnya?
4. Mengapa sensor sebaiknya berhenti mengirim saat tidak ada client terhubung?
5. Bagaimana pola push ini nanti diterjemahkan ke MQTT pada Modul 14?

4.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Telemetri berpenanda dan hemat transaksi

CH-1 — Telemetri berpenanda (wajib) Ubah payload menjadi `suhu:26.3,#<seq>` dengan nomor urut naik. Monitor menghitung loss dari lompatan nomor. Format ini dipakai lagi di M13–M16, lihat Kode 4.5.

CH-2 — Interval adaptif Kirim notify hanya bila nilai berubah lebih dari 0,5 °C dari nilai terakhir yang dikirim, dengan batas maksimal 10 detik tanpa kirim (*heartbeat*). Ukur berapa banyak transaksi yang dihemat dalam 2 menit.

```
// Sensor - loop()
static uint32_t seq = 0;
char msg[32];
snprintf(msg, sizeof(msg), "suhu:%.1f,#%lu", readSensor(), (unsigned long)++seq);
```

Kode 4.5: Kerangka CH-1 — telemetri dengan nomor urut

Level 3 — Tantangan · Sensor nyata dan pemulihan otomatis

CH-3 — Sensor asli Ganti `readSensor()` dengan pembacaan sensor nyata (DHT22 atau DS18B20 bila tersedia). Bandingkan sebaran nilainya dengan versi simulasi dan catat berapa lama satu pembacaan memblokir `loop()`.

CH-4 — Self-healing Buat monitor melakukan scan ulang otomatis pada `onDisconnect` sehingga aliran pulih tanpa reset manual. Ukur waktu pemulihannya, 5 percobaan, lalu hitung rata-ratanya.

4.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (telemetry, notify dibanding indication, push dibanding pull).
3. Konfigurasi — `platformio.ini`, environment, UUID characteristic telemetri.
4. Hasil eksperimen — log kedua node (EXP-01 sampai EXP-03 beserta checkpoint), termasuk hasil uji ketika subscribe dikomentari.
5. Data pengukuran — tabel bagian Pengukuran **dan** tabel pembanding polling dengan notify.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2, sertakan angka penghematan transaksi.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

5. Modul 05 — Multi-Node BLE (Topologi Bintang)

IoT COMMUNICATION LAB · MODUL 05

Connect Multiple Devices — Multi-Node BLE (Topologi Bintang)

ESP32-H2 · BLE · STAR / 2 koneksi · level Intermediate · 3 × 50 menit · folder kode `week05_ble_multinode`

PENGANTAR MODUL · Gambaran Umum

Modul 05 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menahan dua koneksi sekaligus dari satu central dan membuktikan kedua aliran data tetap utuh. Percobaan berjalan dalam topologi bintang dengan satu central dan dua peripheral, diamati melalui tiga terminal Serial Monitor pada 115200 baud.

5.1 Pendahuluan

Empat modul pertama hanya pernah menangani **satu** lawan bicara. Di sini jumlah node menjadi variabel — dan bersamanya muncul pertanyaan khas WSN: apakah pusat jaringan sanggup melayani semua node, dan bagaimana cara membedakan sumber tiap pesan. Jawaban modul ini (satu objek client per node, laju per node dihitung terpisah) adalah versi sederhana dari binding table Zigbee (M09) dan mesh Thread (M12).

Prasyaratnya adalah M04: notify sebagai telemetry, mekanisme subscribe, dan pengukuran loss. Yang dibangun di sini adalah dua objek `NimBLEClient` yang hidup bersamaan, pemisahan aliran data per sumber, perhitungan laju tiap node secara terpisah, serta pengamatan ketahanan sistem saat salah satu node hilang. Semuanya dipakai lagi pada M06 ketika node ketiga berperan sebagai hop alih-alih cabang, M09 pada satu coordinator dengan banyak end device, M12 pada komunikasi many-to-many, dan M16 ketika batas skala menjadi bahan pembanding antarprotokol.

Peta modul blok BLE

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan BLE P2P terbentuk dan stabil
02	Payload aplikasi mengalir dua arah
03	Characteristic mewakili state dan perintah
04	Telemetry via notify
05 (ini)	**Jumlah node jadi variabel — satu pusat, banyak sumber**
06	Node ketiga dipakai sebagai relay, bukan cabang

Tabel 5.1: Peta modul blok BLE

Kontrak data lab ini. Setiap peripheral memberi **prefiks identitas** pada payloadnya (`A:n`, `B:n`). Tanpa penanda sumber, pesan dari banyak node tidak bisa dipisahkan di pusat — masalah yang persis sama muncul lagi di M09 (`short addr`), M12 (`NODE_ID`), dan M15 (`node2` pada payload MQTT).

5.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Melayani dua koneksi sekaligus dari satu central

1. Membangun satu central BLE yang memelihara dua koneksi aktif bersamaan dan menunjukkan objek atau state yang memisahkan keduanya di dalam kode.
2. Menghitung laju pesan tiap node secara terpisah (pesan per menit) dan mencocokkannya dengan interval yang diprogram (A: 30 per menit, B: 20 per menit).
3. Menghitung packet loss **per node**, bukan gabungan, pada minimal 4 jarak.
4. Menjelaskan perilaku sistem saat salah satu peripheral hilang dan kembali, berdasarkan log ketiga board.

Kriteria keberhasilan

- ☐ Central memegang dua koneksi aktif bersamaan (`Koneksi ke kedua node selesai`).
- ☐ Laju tiap node sesuai perhitungan interval (A: 30 per menit, B: 20 per menit).
- ☐ Loss per node terukur terpisah dan tercatat di tabel.
- ☐ Uji ketahanan (Node B dimatikan lalu dinyalakan) dilakukan dan hasilnya dicatat.

5.3 Dasar Teori (Secukupnya)

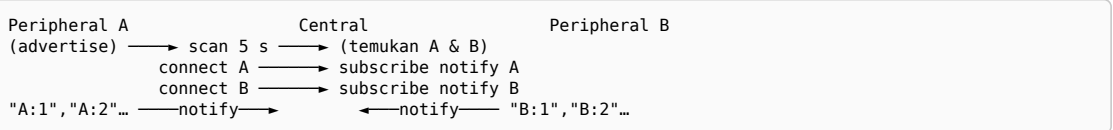
Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (penjadwalan connection event multi-link, batas jumlah koneksi NimBLE) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 5.2.

Istilah	Definisi kerja di lab ini
Topologi bintang	Satu pusat berkomunikasi dengan banyak node; semua lalu lintas lewat pusat.
Advertising	Peripheral menyiarkan nama dan Service UUID agar ditemukan central.
Active scan	Central mengirim scan-request agar nama device ikut terbaca.
Multi-connection	Satu central memelihara beberapa objek `NimBLEClient` sekaligus (di sini 2).
Notify	Peripheral mendorong data ke central setelah di-subscribe.
Time-sharing radio	Satu radio melayani dua koneksi bergantian — sumber utama penurunan laju bila node bertambah banyak.

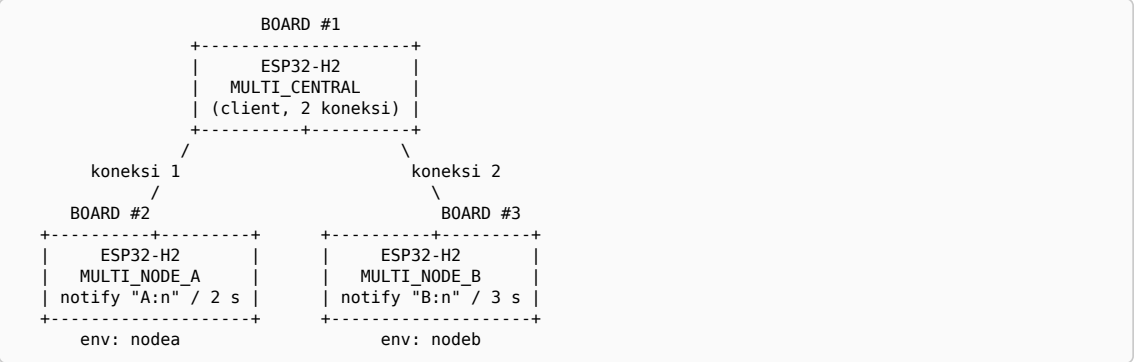
Tabel 5.2: Istilah kerja Modul 05

Mengapa interval A dan B sengaja dibedakan? Jika keduanya sama, log central akan berselang-seling rapi sehingga "central melayani dua node dengan benar" tidak dapat dibedakan dari "central hanya mencatat bergantian". Interval 2 s dan 3 s menghasilkan pola tak beraturan yang hanya cocok bila kedua aliran memang independen.

Sekuens protokol yang diamati



5.4 Topologi



Node	Board	Environment	Peran	Payload / interval
Central	ESP32-H2 DevKitM-1	`central`	BLE Central, 2 koneksi	—
Node A	ESP32-H2 DevKitM-1	`nodea`	Peripheral `MULTI_NODE_A`	`A:n` tiap 2000 ms
Node B	ESP32-H2 DevKitM-1	`nodeb`	Peripheral `MULTI_NODE_B`	`B:n` tiap 3000 ms

Tabel 5.3: Peran tiap node Modul 05

Ketiga peran memakai board yang sama, **ESP32-H2 DevKitM-1**, dengan radio Bluetooth LE. ESP32-C6 tidak dipakai pada modul ini.

5.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` via `lib_deps`	—
5	Ruang uji	jarak awal ±1 m antar board, bebas logam di dekat antena	—

Tabel 5.4: Alat dan bahan Modul 05

Bila port USB kurang dari tiga, monitor boleh dibuka bergantian — tetapi minimal central harus terus termonitor selama pengukuran.

5.6 Kode Program

Pin port agar tidak salah flash (Kode 5.1).

```
[env:central]
build_src_filter = +<central/*.cpp>
upload_port  = /dev/ttyACM0     ; Windows: COM3
monitor_port = /dev/ttyACM0

[env:nodea]
build_src_filter = +<nodea/*.cpp>
upload_port  = /dev/ttyACM2     ; Windows: COM4
monitor_port = /dev/ttyACM2

[env:nodeb]
build_src_filter = +<nodeb/*.cpp>
upload_port  = /dev/ttyACM4     ; Windows: COM5
monitor_port = /dev/ttyACM4
```

Kode 5.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = central

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:central]
build_src_filter = +<central/*.cpp>

[env:nodea]
build_src_filter = +<nodea/*.cpp>

[env:nodeb]
build_src_filter = +<nodeb/*.cpp>
```

Kode 5.2: `platformio.ini` Modul 05 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress addrA;
static NimBLEAddress addrB;
static bool foundA = false;
static bool foundB = false;
static NimBLEClient *clientA = nullptr;
static NimBLEClient *clientB = nullptr;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  ClientCallbacks(const char *name) : _name(name) {}
  void onConnect(NimBLEClient *pClient) override { Serial.printf("%s terhubung\n", _name); }
  void onDisconnect(NimBLEClient *pClient, int reason) override { Serial.printf("%s terputus\n", _name); }
private:
  const char *_name;
};

// Kumpulkan hasil scan untuk A dan B (tanpa menghentikan scan)
class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    if (d->getName() == "MULTI_NODE_A" && !foundA) {
      addrA = d->getAddress();
      foundA = true;
      Serial.println("Node A ditemukan");
    }
    if (d->getName() == "MULTI_NODE_B" && !foundB) {
```

```
      addrB = d->getAddress();
      foundB = true;
      Serial.println("Node B ditemukan");
    }
  }
};

static bool connectAndSubscribe(const NimBLEAddress &addr, const char *name,
                                NimBLEClient *&client) {
  client = NimBLEDevice::createClient();
  client->setClientCallbacks(new ClientCallbacks(name));

  if (!client->connect(addr)) {
    Serial.printf("Gagal terhubung ke %s\n", name);
    NimBLEDevice::deleteClient(client);
    client = nullptr;
    return false;
  }

  NimBLERemoteService *pService = client->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.printf("Service %s tidak ditemukan\n", name);
    client->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pChar = pService->getCharacteristic(CHAR_NOTIFY_UUID);
  if (pChar == nullptr) {
    Serial.printf("Characteristic %s tidak ditemukan\n", name);
    client->disconnect();
    return false;
  }

  if (pChar->canNotify()) {
    // notify_callback NimBLE 2.x adalah std::function -> lambda boleh menangkap nama node
    pChar->subscribe(true, [name](NimBLERemoteCharacteristic *c, uint8_t *data,
                                  size_t length, bool isNotify) {
      Serial.printf("[%s] RX: %.*s\n", name, (int)length, (const char *)data);
    });
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Central (multi-node) starting...");

  NimBLEDevice::init("MULTI_CENTRAL");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // scan 5 detik (ms di NimBLE 2.x), kumpulkan A dan B

  Serial.println("Scanning node A dan B...");
}

void loop() {
  // Setelah scan selesai, konek ke node yang ditemukan
  static bool connecting = true;
  if (connecting) {
    if (foundA && foundB) {
      NimBLEDevice::getScan()->stop();
      connectAndSubscribe(addrA, "NodeA", clientA);
      connectAndSubscribe(addrB, "NodeB", clientB);
      connecting = false;
      Serial.println("Koneksi ke kedua node selesai");
    }
  }
  delay(100);
}
```

Kode 5.3: `src/central/main.cpp` — central dengan dua objek client

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
```

```cpp
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

static bool deviceConnected = false;
static NimBLECharacteristic *pNotify = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Central terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Central terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("Node A (peripheral) starting...");

  NimBLEDevice::init("MULTI_NODE_A");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pNotify = pService->createCharacteristic(CHAR_NOTIFY_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("MULTI_NODE_A");
  pAdvertising->start();

  Serial.println("Menunggu central...");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (deviceConnected && millis() - last > 2000) {
    last = millis();
    String msg = "A:" + String(++count);
    pNotify->setValue((const uint8_t *)msg.c_str(), msg.length());
    pNotify->notify();
    Serial.println("Notify: " + msg);
  }
}
```

Kode 5.4: `src/nodea/main.cpp` — peripheral `MULTI_NODE_A`

```cpp
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

static bool deviceConnected = false;
static NimBLECharacteristic *pNotify = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Central terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Central terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("Node B (peripheral) starting...");

  NimBLEDevice::init("MULTI_NODE_B");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pNotify = pService->createCharacteristic(CHAR_NOTIFY_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("MULTI_NODE_B");
  pAdvertising->start();

  Serial.println("Menunggu central...");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (deviceConnected && millis() - last > 3000) {
    last = millis();
    String msg = "B:" + String(++count);
    pNotify->setValue((const uint8_t *)msg.c_str(), msg.length());
    pNotify->notify();
    Serial.println("Notify: " + msg);
  }
}
```

Kode 5.5: `src/nodeb/main.cpp` — peripheral `MULTI_NODE_B`

5.7 Build dan Flash

Flash **peripheral dulu**, central belakangan, agar saat central melakukan scan 5 detik keduanya sudah mengudara.

```
pio run -d week05_ble_multinode -e nodea   -t upload
pio run -d week05_ble_multinode -e nodeb   -t upload
pio run -d week05_ble_multinode -e central -t upload -t monitor
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat dan diisikan di atas.
- ☐ Ketiga ESP32-H2 terpasang dan terdeteksi.
- ☐ Environment `central`, `nodea`, dan `nodeb` dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud siap (3 terminal).
- ☐ Penempatan tiga perangkat direncanakan (jarak awal ±1 m).

5.8 Percobaan

5.8.1 EXP-01 — Menyalakan Dua Peripheral

Unggah firmware peripheral ke dua board, verifikasi keduanya advertise dan menunggu central.



Data capture

Parameter	Hasil
Nama BLE Node A	
Nama BLE Node B	
Pesan `Menunggu central...` muncul di keduanya?	
Interval kirim A (ms)	
Interval kirim B (ms)	

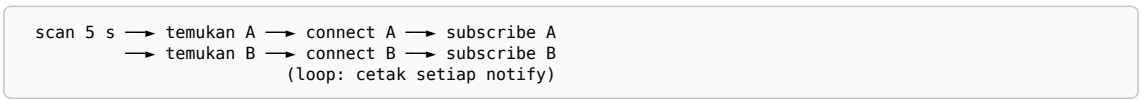
Tabel 5.5: Lembar pengamatan EXP-01

CHECKPOINT

Kedua peripheral mencetak `Menunggu central...` dan belum mencetak `Notify:` sama sekali. Jika sudah ada `Notify:` padahal central belum menyala, berarti node mengirim tanpa penerima — periksa syarat `deviceConnected` di kodenya.

5.8.2 EXP-02 — Dua Koneksi dan Subscribe

Unggah environment `central` ke board ketiga. Central melakukan active scan 5 detik, menemukan `MULTI_NODE_A` dan `MULTI_NODE_B`, lalu membuka dua koneksi dan subscribe notify pada masing-masing.



Expected output — Central

```
Central (multi-node) starting...
Scanning node A dan B...
Node A ditemukan
Node B ditemukan
NodeA terhubung
NodeB terhubung
Koneksi ke kedua node selesai
[NodeA] RX: A:1
[NodeB] RX: B:1
[NodeA] RX: A:2
[NodeB] RX: B:2
[NodeA] RX: A:3
[NodeA] RX: A:4
[NodeB] RX: B:3
```

Expected output — Peripheral (contoh Node A)

```
Node A (peripheral) starting...
Menunggu central...
Central terhubung
Notify: A:1
Notify: A:2
```

BUKA ABSTRAKSINYA
Di `src/central/main.cpp`, cari **berapa objek `NimBLEClient` yang dibuat** dan bagaimana callback notify tahu pesan ini datang dari A atau dari B. Jawabannya bukan dari isi payload — payload hanya kebetulan diberi prefiks. Telusuri hingga menemukan mekanisme sebenarnya, lalu jawab: jika kedua node mengirim payload identik `"data"`, apakah central masih bisa membedakannya?

CHECKPOINT

Central mencetak `Koneksi ke kedua node selesai` dan setelah itu muncul baris `[NodeA]` **dan** `[NodeB]` bergantian tak beraturan. Jika hanya satu label yang pernah muncul, koneksi kedua gagal — ulangi scan dengan mereset central (peripheral tetap menyala).

5.8.3 EXP-03 — Laju dan Ketahanan

Biarkan sistem berjalan 2–3 menit, ukur laju pesan tiap node, lalu uji ketahanan: reset (atau cabut USB) Node B, amati `NodeB terputus` di central dan `Central terputus, advertise ulang` di Node B, lalu nyalakan kembali dan verifikasi pemulihan.

Data capture

Parameter	Hasil
Jumlah pesan A per menit (harapan 30)	
Jumlah pesan B per menit (harapan 20)	
Apakah laju A berubah saat B mati?	
Perilaku central saat Node B reset	
Apakah koneksi B pulih otomatis?	

Tabel 5.6: Lembar pengamatan EXP-03

CHECKPOINT

Saat Node B dimatikan, aliran `[NodeA]` **tidak boleh** ikut berhenti. Jika ikut berhenti, central kehilangan kedua koneksi — itu temuan penting, catat dan jelaskan di analisis.

5.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (jarak ±20 cm), capture 30 detik.

```
# Central (ESP32-H2, env central)
[0.401] Node B ditemukan
[0.401] Node A ditemukan
[0.802] NodeA terhubung
[1.604] NodeB terhubung
[2.205] Koneksi ke kedua node selesai
[2.406] [NodeA] RX: A:1
[3.408] [NodeB] RX: B:1
[4.409] [NodeA] RX: A:2
[6.413] [NodeA] RX: A:3
[6.413] [NodeB] RX: B:2
```

Parameter	Hasil terukur
Waktu scan sampai dua koneksi aktif	2,2 s
Node A: notify dikirim / diterima central	14 / 14 (0 % loss)
Node B: notify dikirim / diterima central	9 / 9 (0 % loss)
Laju Node A	2,00 s per pesan, yaitu 30 pesan per menit
Laju Node B	3,00 s per pesan, yaitu 20 pesan per menit

Tabel 5.7: Hasil verifikasi hardware Modul 05

Dua koneksi simultan tidak menggeser interval kedua node — pada beban ringan ini central masih mampu melayani keduanya tanpa kehilangan pesan.

5.10 Pengukuran

Ukur **per node**, jangan digabung. Ini inti modul: dua node pada satu central tidak selalu terdegradasi bersamaan.

Jarak	RSSI A	RSSI B	Pesan A /60 s (30)	Pesan B /60 s (20)	Loss A (%)	Loss B (%)
1 m						
3 m						
5 m						
10 m						

Tabel 5.8: Lembar pengukuran per node Modul 05

Skenario asimetris (wajib, minimal satu baris) — dekatkan A, jauhkan B.

Posisi A	Posisi B	Loss A (%)	Loss B (%)	Kesimpulan
1 m	10 m			

Tabel 5.9: Lembar pengukuran skenario asimetris

5.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana hubungan jarak terhadap RSSI dan keberhasilan penerimaan dari kedua node?
2. Apakah jumlah pesan per menit sesuai perhitungan dari interval tiap node? Jelaskan bila ada selisih.

3. Berapa latency dari `Notify` di peripheral hingga `RX` di central? Apakah bertambah pada jarak jauh?
4. Pada skenario asimetris, apakah node yang jauh menurunkan kualitas node yang dekat? Apa artinya bagi desain WSN?
5. Apakah topologi bintang BLE cocok untuk WSN banyak node? Bandingkan dengan koneksi P2P modul sebelumnya, dan perkirakan apa yang terjadi pada 10 node.

5.12 Concept Check

1. Apa perbedaan peran central dan peripheral dalam BLE?
2. Mengapa central perlu subscribe pada tiap characteristic notify secara terpisah?
3. Bagaimana central membedakan pesan dari Node A dan Node B — dari payload atau dari sesuatu yang lain?
4. Apa yang terjadi pada peripheral saat koneksi terputus, dan bagaimana pemulihannya?
5. Apa yang membatasi jumlah koneksi pada topologi bintang BLE, dari sisi stack dan dari sisi radio?

5.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah node dan menghitung loss per sumber

CH-1 — Node ketiga Tambahkan Node C (peripheral ketiga, interval 5 s, pesan `C:n`). Hitung persentase pesan tiap node terhadap total di central dan periksa apakah laju A dan B berubah setelah C bergabung.

CH-2 — Loss otomatis per node Hitung packet loss dari nomor urut, terpisah untuk tiap node, seperti Kode 5.6. Contoh: counter Node A mencapai 120 dalam 4 menit tetapi central menerima 116, sehingga loss = (120 − 116)/120 × 100 % = 3,33 %.

```
// Central - di callback notify, simpan last[] per node
static uint32_t last[2] = {0, 0};
uint32_t n = atoi(strchr(buf, ':') + 1);
if (last[idx] && n != last[idx] + 1)
    Serial.printf("LOSS %c: %lu paket\n", 'A' + idx, n - last[idx] - 1);
last[idx] = n;
```

Kode 5.6: Kerangka CH-2 — penghitung loss per node

Level 3 — Tantangan · Mencari batas skala dan memulihkan koneksi

CH-3 — Cari batas skala Turunkan interval kedua node ke 100 ms. Catat pada interval berapa central mulai kehilangan pesan, dan node mana yang lebih dulu terdampak. Kaitkan dengan konsep *time-sharing radio*.

CH-4 — Reconnect otomatis Buat central melakukan scan ulang dan menyambung kembali node yang hilang tanpa perlu reset. Ukur waktu pemulihannya, 5 percobaan.

5.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (central/peripheral, multi-connection, topologi bintang).
3. Konfigurasi — environment `central`, `nodea`, `nodeb`, UUID, dan interval.
4. Hasil eksperimen — log serial tiga perangkat (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel per node **dan** tabel skenario asimetris.

6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

6. Modul 05B — Mini Project: Smart Sensor Jendela dan Pintu

IoT COMMUNICATION LAB · MODUL 05B

Mini Project — Smart Sensor Jendela dan Pintu

ESP32-H2 · BLE · STAR / 2 sensor · level Intermediate · 3 × 50 menit · folder kode `week05b_ble_multinode_project`

PENGANTAR MODUL · Gambaran Umum

Modul 05B adalah mini project lanjutan dari `week05_ble_multinode`, dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya mengubah topologi bintang M05 dari sekadar pengirim counter menjadi **sistem keamanan sederhana**: dua smart sensor bukaan yang melapor ke satu hub. Percobaan berjalan dalam topologi bintang dengan satu hub dan dua sensor, diamati melalui tiga terminal Serial Monitor pada 115200 baud dengan LED RGB onboard sebagai penanda visual.

6.1 Pendahuluan

M05 membuktikan satu central sanggup memegang dua koneksi, tetapi datanya masih counter (`A:1`, `B:1`) yang tidak berarti apa-apa. Modul ini mengganti counter itu dengan **kejadian nyata dari sensor**, dan di situlah karakter lalu lintas berubah total: dari periodik menjadi *event-driven*.

Prasyaratnya adalah M05: dua koneksi simultan dan pemisahan aliran per sumber. Yang dibangun di sini adalah pemakaian input digital sebagai sumber data, transmisi berbasis kejadian, payload berstruktur `ID:STATE:seq`, penyimpanan state ruangan di hub, serta indikator lokal dan terpusat. Semuanya dipakai lagi pada M09 ketika end device melaporkan state alih-alih counter, M14 dan M15 ketika payload sensor naik ke MQTT, dan M16 ketika perbandingan protokol memakai beban event-driven.

Yang berubah dari M05

Aspek	M05	M05B (ini)
Sumber data	`millis()` — timer	Tombol BOOT (GPIO9) — proximity switch simulasi
Pola trafik	Periodik, 2 s dan 3 s	Event-driven, hanya saat status berubah
Payload	`A:n` / `B:n`	`JENDELA1:OPEN:3` / `PINTU1:CLOSED:4`
Output di node	Serial saja	Serial dan kedip LED RGB (GPIO8)
Peran central	Mencetak apa yang diterima	Menyimpan state ruangan dan lampu status
Sensor hilang	Koneksi mati sampai central di-reset	Disambungkan ulang otomatis, waktu pemulihan dicetak

Tabel 6.1: Perbedaan Modul 05 dan Modul 05B

Kontrak data lab ini. Payload berbentuk `<ID>:<STATE>:<nomor_event>`: identitas sensor, statusnya, dan nomor urut kejadian. Nomor urut inilah yang membuat *kejadian yang hilang* bisa dideteksi — pada trafik event-driven, pesan yang hilang tidak terlihat sebagai jeda seperti pada trafik periodik.

6.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membangun sistem alarm event-driven dua sensor

1. Menggabungkan input digital (proximity switch simulasi) dan output visual (LED WS2812) dengan komunikasi BLE dalam satu firmware node.
2. Menjelaskan perbedaan trafik periodik dan event-driven, serta konsekuensinya pada deteksi packet loss.
3. Merancang payload berstruktur yang membawa identitas, status, dan nomor urut, lalu memparsing-nya di sisi hub.
4. Menjelaskan bahwa hub mengetahui asal pesan dari **objek koneksi**, bukan dari isi payload — dan mengapa prefiks pada payload tetap diperlukan.
5. Menganalisis keandalan sistem alarm sederhana: kejadian hilang, sensor terputus, dan state hub yang tidak lagi mencerminkan kondisi nyata.
6. Menjelaskan cara kerja sambung ulang otomatis (retry ke alamat lama lalu scan ulang) dan membedakan apa yang dipulihkannya (tautan) dari apa yang tidak (data yang telanjur hilang).

Kriteria keberhasilan

☐ Hub memegang dua koneksi aktif (`Semua sensor terpantau — sistem siaga`).
☐ Tombol BOOT di node jendela membuat LED node berkedip **dan** hub mencetak `[ALARM] Jendela 1 TERBUKA`.
☐ Tombol BOOT di node pintu membuat hub mencetak `[ALARM] Pintu 1 TERBUKA`, tanpa tertukar label.
☐ LED hub kuning saat ada sensor hilang, merah saat ada bukaan, hijau saat semua aman.
☐ Sensor yang dimatikan lalu dihidupkan kembali tersambung **otomatis** (`[PULIH]`), tanpa reset hub.
☐ Waktu pemulihan tercatat, minimal 3 percobaan.

6.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Proximity / reed switch	Sakelar yang menutup dan membuka mengikuti posisi daun jendela atau pintu. Di lab disimulasikan tombol BOOT.
Active low	Tombol BOOT tersambung ke GND lewat `Key2` dengan pull-up 10K di board: tidak ditekan berarti `HIGH`, ditekan berarti `LOW`.
Debounce	Kontak mekanis memantul beberapa milidetik saat ditekan; tanpa penyaringan, satu tekanan terbaca sebagai banyak kejadian.
Event-driven	Node hanya mengirim saat **status berubah**, bukan tiap interval tetap. Hemat energi, tetapi tiap paket jadi berharga.
Notify	Peripheral mendorong data ke hub setelah di-subscribe — cocok untuk alarm karena hub tidak perlu polling.
State di hub	Salinan kondisi lapangan yang disimpan hub. Bisa **basi** bila sebuah kejadian hilang.
Sambung ulang otomatis	Hub mencoba menyambung lagi tiap 3 detik ke sensor yang hilang; setelah 3 kali gagal, alamatnya dicari ulang lewat scan.
CCCD	Penanda "aku mau notify" di sisi sensor. Hilang saat koneksi putus (tidak ada bonding), jadi **subscribe wajib diulang** tiap kali menyambung.

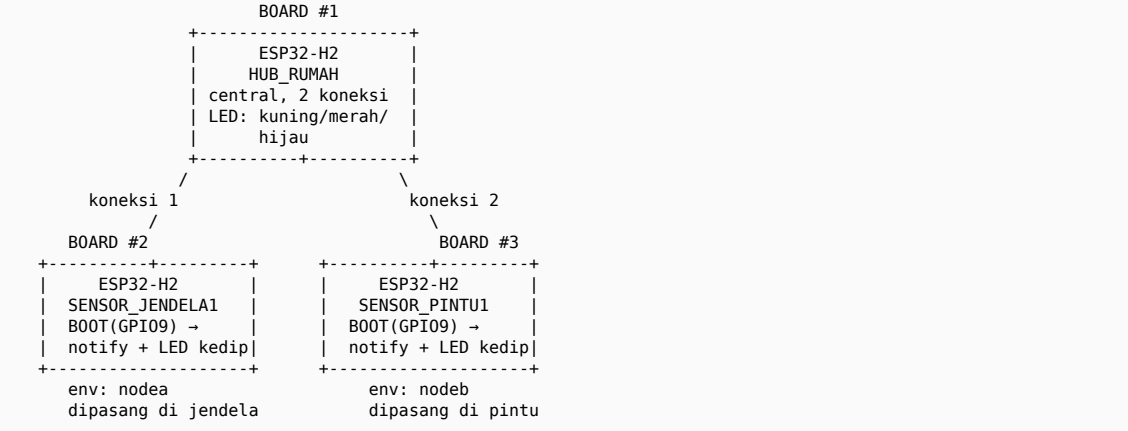
Tabel 6.2: Istilah kerja Modul 05B

Mengapa event-driven lebih rawan daripada periodik? Pada M05, kehilangan satu `A:7` langsung terlihat — nomor urut melompat dan pengiriman berikutnya datang 2 detik lagi. Di sini, jika kejadian `OPEN` hilang, tidak ada pengiriman berikutnya sampai seseorang menyentuh jendela lagi. Hub akan terus menampilkan "tertutup" padahal jendela terbuka: **kegagalan yang senyap**. Itulah alasan nomor urut disertakan di payload.

Sekuens yang diamati

```
Sensor Jendela            Hub (central)             Sensor Pintu
(advertise) ──────► scan 5 s ──────► (temukan keduanya)
             connect ──────────────► subscribe notify
[tombol ditekan]
LED kedip merah
"JENDELA1:OPEN:1" ──notify──►  [ALARM] Jendela 1 TERBUKA
                               LED hub → MERAH
                                         ◄──notify── "PINTU1:OPEN:1"
[tombol dilepas]
LED kedip hijau
"JENDELA1:CLOSED:2" ─notify─►  [INFO ] Jendela 1 tertutup kembali
                               LED hub tetap MERAH (pintu masih terbuka)
```

6.4 Topologi



Node	Environment	Nama BLE	Peran	Payload
Hub	`central`	`HUB_RUMAH`	BLE Central, 2 koneksi, lampu status	—
Sensor jendela	`nodea`	`SENSOR_JENDELA1`	Peripheral	`JENDELA1:OPEN:n` / `JENDELA1:CLOSED:n`
Sensor pintu	`nodeb`	`SENSOR_PINTU1`	Peripheral	`PINTU1:OPEN:n` / `PINTU1:CLOSED:n`

Tabel 6.3: Peran tiap node Modul 05B

6.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO, NimBLE-Arduino, dan Adafruit NeoPixel.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	Waveshare ESP32-H2-DEV-KIT-N4 atau DevKitM-1	3
2	Kabel USB data	kabel data, bukan *charge-only*	3

No	Peralatan	Spesifikasi	Jumlah
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Library	`h2zero/NimBLE-Arduino@^2.2.3`, `adafruit/Adafruit NeoPixel@^1.15.5`	—

Tabel 6.4: Alat dan bahan Modul 05B

Pin yang dipakai (dari skematik board, sama di ketiga node).

Fungsi	GPIO	Keterangan
Proximity switch simulasi	**GPIO9**	Tombol BOOT (`Key2`), ke GND saat ditekan, pull-up 10K di board sehingga *active low*
Indikator sensor / status	**GPIO8**	LED RGB WS2812 (`RGB_CTRL`), 1 pixel, urutan byte RGB

Tabel 6.5: Pemetaan pin Modul 05B

Tidak ada wiring tambahan — tombol dan LED sudah ada di board. Bila ingin sensor sungguhan, reed switch cukup dipasang antara GPIO9 dan GND (lihat CH-4). Skematik lengkap board tersedia pada Bab A.

PERINGATAN
GPIO9 juga *strapping pin* mode download. Menahan tombol **saat board reset** membuat board masuk mode flash, bukan menjalankan program. Tekan tombol hanya setelah firmware berjalan.

Struktur proyek

```
week05b_ble_multinode_project/
├── platformio.ini          ← port tiap board sudah di-pin di sini
├── monitor_serial.py       ← pemantau 3 port serial sekaligus
├── logserial.md            ← log referensi hasil uji perangkat
└── src/
    ├── central/main.cpp    ← hub: 2 koneksi, parsing event, lampu status
    ├── nodea/main.cpp      ← sensor jendela
    └── nodeb/main.cpp      ← sensor pintu
```

6.6 Kode Program

```
[platformio]
default_envs = central

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3
    adafruit/Adafruit NeoPixel@^1.15.5

; Port di-pin agar tidak salah flash saat tiga board terpasang bersamaan.
; Sesuaikan dengan hasil `pio device list` di komputer masing-masing —
; Windows memakai COMx, Linux /dev/ttyACMx.
[env:nodea]
build_src_filter = +<nodea/*.cpp>
upload_port  = /dev/ttyACM0     ; sensor jendela  · Windows: COM3
monitor_port = /dev/ttyACM0

[env:nodeb]
build_src_filter = +<nodeb/*.cpp>
upload_port  = /dev/ttyACM2     ; sensor pintu    · Windows: COM4
monitor_port = /dev/ttyACM2

[env:central]
build_src_filter = +<central/*.cpp>
upload_port  = /dev/ttyACM4     ; hub             · Windows: COM5
monitor_port = /dev/ttyACM4
```

Kode 6.1: `platformio.ini` Modul 05B — port tiap board sudah dipin

```
// ============================================================================
// MODUL 05B — Hub Smart Home (Central)
//
// Peran : BLE central, memelihara dua koneksi sekaligus ke smart sensor
//         jendela (Node A) dan pintu (Node B), lalu menerjemahkan payload
//         mentah menjadi status ruangan yang bisa dibaca manusia.
//         Sensor yang hilang disambungkan kembali otomatis, tanpa reset hub.
// Aktuator: LED RGB WS2812 onboard (GPIO8) sebagai lampu status ruangan —
//         kuning bila ada sensor hilang, merah bila ada bukaan, hijau bila
//         semua sensor terpantau dan tertutup.
// ============================================================================
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <Adafruit_NeoPixel.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

#define PIN_LED 8

#define SCAN_MS 5000            // lama satu putaran scan
#define RETRY_MS 3000           // jeda antar-percobaan sambung ulang
#define CONNECT_TIMEOUT_MS 4000 // batas satu percobaan connect (default lib 30 s)
#define RETRIES_BEFORE_RESCAN 3 // gagal sekian kali -> alamat dicari ulang

// Daftar sensor yang dilayani hub. Menambah sensor ketiga cukup menambah satu
// baris di sini — sisa kode sudah berbasis indeks.
struct SensorInfo {
  const char *bleName;  // nama advertising yang dicari saat scan
  const char *label;    // nama yang ditampilkan di log
};

static const SensorInfo SENSORS[] = {
    {"SENSOR_JENDELA1", "Jendela 1"},
    {"SENSOR_PINTU1", "Pintu 1"},
};
static const int SENSOR_COUNT = sizeof(SENSORS) / sizeof(SENSORS[0]);

// State per sensor. Alamat disalin (bukan pointer hasil scan) agar tetap valid
// setelah scan berhenti.
static NimBLEAddress addr[SENSOR_COUNT];
```

```
static bool found[SENSOR_COUNT] = {false};       // alamatnya sudah diketahui
static bool connected[SENSOR_COUNT] = {false};   // tautan sedang hidup
static bool isOpen[SENSOR_COUNT] = {false};      // jendela/pintu terbuka?
static NimBLEClient *client[SENSOR_COUNT] = {nullptr};

// Bahan sambung ulang otomatis
static uint8_t retries[SENSOR_COUNT] = {0};
static unsigned long nextTry[SENSOR_COUNT] = {0};
static unsigned long lostAt[SENSOR_COUNT] = {0};

Adafruit_NeoPixel strip(1, PIN_LED, NEO_RGB + NEO_KHZ800);

// Lampu status ruangan. Urutan prioritas dipilih supaya kondisi paling
// meragukan yang menang: sensor hilang lebih penting daripada "semua aman",
// karena hub tidak tahu apa yang terjadi di sisi sensor itu.
static void updateStatusLed() {
  bool anyLost = false, anyOpen = false;
  for (int i = 0; i < SENSOR_COUNT; i++) {
    anyLost |= !connected[i];
    anyOpen |= isOpen[i];
  }
  if (anyLost)      strip.setPixelColor(0, strip.Color(255, 150, 0));  // kuning
  else if (anyOpen) strip.setPixelColor(0, strip.Color(255, 0, 0));    // merah
  else              strip.setPixelColor(0, strip.Color(0, 255, 0));    // hijau
  strip.show();
}

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  ClientCallbacks(int idx) : _idx(idx) {}
  void onConnect(NimBLEClient *pClient) override {
    Serial.printf("%s terhubung\n", SENSORS[_idx].label);
  }
  // Callback ini berjalan di task NimBLE. Di sini hanya ditandai; percobaan
  // sambung ulang dikerjakan loop() — memanggil connect() dari dalam callback
  // stack BLE adalah cara cepat membuat hub hang.
  void onDisconnect(NimBLEClient *pClient, int reason) override {
    connected[_idx] = false;
    lostAt[_idx] = millis();
    nextTry[_idx] = millis() + RETRY_MS;
    retries[_idx] = 0;
    updateStatusLed();
    Serial.printf("[WARN ] %s terputus (reason %d) - mencoba sambung ulang\n",
                  SENSORS[_idx].label, reason);
  }

private:
  int _idx;
};

// Kumpulkan hasil scan untuk semua sensor (tanpa menghentikan scan)
class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    for (int i = 0; i < SENSOR_COUNT; i++) {
      if (!found[i] && d->getName() == SENSORS[i].bleName) {
        addr[i] = d->getAddress();
        found[i] = true;
        nextTry[i] = 0;  // alamat baru: coba sambung pada iterasi loop berikutnya
        Serial.printf("%s ditemukan (RSSI %d dBm)\n", SENSORS[i].label, d->getRSSI());
      }
    }
  }
};

// Payload node berbentuk "<ID>:<STATE>:<nomor event>", mis. "PINTU1:OPEN:3".
// Nomor event dibandingkan dengan yang terakhir diterima untuk menemukan
// kejadian yang hilang di udara.
static void handleEvent(int idx, const char *payload, size_t len) {
  static uint32_t lastSeq[SENSOR_COUNT] = {0};

  char buf[48];
  size_t n = len < sizeof(buf) - 1 ? len : sizeof(buf) - 1;
  memcpy(buf, payload, n);
  buf[n] = '\0';

  const char *state = strchr(buf, ':');
  const char *seqStr = state ? strchr(state + 1, ':') : nullptr;
  if (!state || !seqStr) {
    Serial.printf("[WARN ] payload tidak dikenal dari %s: %s\n", SENSORS[idx].label, buf);
    return;
  }
```

```
  bool opened = (strncmp(state + 1, "OPEN", 4) == 0);
  uint32_t seq = strtoul(seqStr + 1, nullptr, 10);

  if (lastSeq[idx] && seq > lastSeq[idx] + 1)
    Serial.printf("[LOSS ] %s: %lu kejadian tidak sampai ke hub\n", SENSORS[idx].label,
                  (unsigned long)(seq - lastSeq[idx] - 1));
  lastSeq[idx] = seq;

  isOpen[idx] = opened;
  updateStatusLed();

  Serial.printf("%s [%7.3f s] %s %s (event #%lu)\n", opened ? "[ALARM]" : "[INFO ]",
                millis() / 1000.0, SENSORS[idx].label,
                opened ? "TERBUKA" : "tertutup kembali", (unsigned long)seq);
}

static void ensureScanning() {
  NimBLEScan *pScan = NimBLEDevice::getScan();
  if (!pScan->isScanning()) pScan->start(SCAN_MS, false);
}

// Sambung + subscribe. Dipakai untuk koneksi pertama maupun sambung ulang;
// objek client dipakai ulang karena jumlah client NimBLE terbatas.
static bool connectAndSubscribe(int idx) {
  const char *label = SENSORS[idx].label;

  if (client[idx] == nullptr) {
    client[idx] = NimBLEDevice::createClient();
    client[idx]->setClientCallbacks(new ClientCallbacks(idx));
    // Default library 30 detik: satu percobaan ke sensor yang mati akan
    // menahan loop() selama itu, sehingga logika retry/scan-ulang di bawah
    // praktis tidak pernah jalan. 4 detik membuatnya benar-benar berputar.
    client[idx]->setConnectTimeout(CONNECT_TIMEOUT_MS);
  }

  // Scan dan connect berebut radio yang sama — hentikan dulu.
  NimBLEScan *pScan = NimBLEDevice::getScan();
  if (pScan->isScanning()) pScan->stop();

  if (!client[idx]->connect(addr[idx])) {
    Serial.printf("Gagal terhubung ke %s\n", label);
    return false;
  }

  NimBLERemoteService *pService = client[idx]->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.printf("Service %s tidak ditemukan\n", label);
    client[idx]->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pChar = pService->getCharacteristic(CHAR_NOTIFY_UUID);
  if (pChar == nullptr) {
    Serial.printf("Characteristic %s tidak ditemukan\n", label);
    client[idx]->disconnect();
    return false;
  }

  if (pChar->canNotify()) {
    // Subscribe WAJIB diulang tiap kali menyambung: nilai CCCD hilang saat
    // koneksi putus karena kedua sisi tidak melakukan bonding.
    // notify_callback NimBLE 2.x adalah std::function -> lambda boleh menangkap
    // indeks sensor. Inilah yang membuat hub tahu asal pesan — bukan prefiks
    // pada payload.
    if (!pChar->subscribe(true, [idx](NimBLERemoteCharacteristic *c, uint8_t *data, size_t length,
                                      bool isNotify) { handleEvent(idx, (const char *)data, length); })) {
      Serial.printf("Gagal subscribe ke %s\n", label);
      client[idx]->disconnect();
      return false;
    }
  }

  connected[idx] = true;
  retries[idx] = 0;
  updateStatusLed();
  return true;
}

// Dipanggil tiap iterasi loop: jaga agar semua sensor tetap tersambung.
static void maintainLinks() {
  unsigned long now = millis();
```

```
  for (int i = 0; i < SENSOR_COUNT; i++) {
    if (connected[i]) continue;

    if (!found[i]) {  // alamat belum/tidak lagi diketahui -> cari lewat scan
      ensureScanning();
      continue;
    }
    if ((long)(now - nextTry[i]) < 0) continue;
    nextTry[i] = now + RETRY_MS;

    if (connectAndSubscribe(i)) {
      if (lostAt[i])
        Serial.printf("[PULIH] %s tersambung lagi setelah %.1f s\n", SENSORS[i].label,
                      (millis() - lostAt[i]) / 1000.0);
      lostAt[i] = 0;
    } else if (++retries[i] >= RETRIES_BEFORE_RESCAN) {
      // Board sensor mungkin sudah restart dengan alamat lain — buang alamat
      // lama dan cari ulang lewat scan.
      Serial.printf("%s tidak menjawab %d kali - scan ulang\n", SENSORS[i].label, retries[i]);
      found[i] = false;
      retries[i] = 0;
      ensureScanning();
    }
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("Hub Smart Home (BLE central) starting...");

  strip.begin();
  strip.setBrightness(50);
  updateStatusLed();  // kuning: belum ada sensor yang terpantau

  NimBLEDevice::init("HUB_RUMAH");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(SCAN_MS, false);  // scan 5 detik (ms di NimBLE 2.x)

  Serial.println("Mencari smart sensor jendela dan pintu...");
}

void loop() {
  static bool announced = false;

  maintainLinks();

  bool allUp = true;
  for (int i = 0; i < SENSOR_COUNT; i++) allUp &= connected[i];
  if (allUp && !announced) {
    Serial.println("Semua sensor terpantau - sistem siaga");
    announced = true;
  } else if (!allUp) {
    announced = false;  // supaya pesan siaga muncul lagi setelah pulih
  }

  delay(100);
}
```

Kode 6.2: `src/central/main.cpp` — hub dengan state ruangan dan sambung ulang

```
// ============================================================================
// MODUL 05B — Smart Sensor Jendela (Node A)
//
// Peran : BLE peripheral, dipasang di JENDELA 1.
// Sensor: tombol BOOT (GPIO9) mensimulasikan proximity/reed switch jendela.
//         Ditekan  -> jendela dianggap TERBUKA
//         Dilepas  -> jendela dianggap TERTUTUP
// Aktuator: LED RGB WS2812 onboard (GPIO8) berkedip sebentar tiap kali sensor
//         berubah, sebagai bukti visual bahwa sensor benar-benar terbaca.
// ============================================================================
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <Adafruit_NeoPixel.h>
```

```
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

// Identitas sensor — dipakai sebagai prefiks payload agar hub bisa memisahkan
// sumber pesan tanpa bergantung pada urutan koneksi.
#define SENSOR_ID "JENDELA1"
#define BLE_NAME "SENSOR_JENDELA1"

// Waveshare ESP32-H2-DEV-KIT-N4: tombol BOOT (Key2) di GPIO9, aktif LOW
// dengan pull-up 10K di board; LED RGB WS2812 di GPIO8, urutan byte RGB.
#define PIN_SWITCH 9
#define PIN_LED 8
#define DEBOUNCE_MS 50
#define LED_ON_MS 300

Adafruit_NeoPixel strip(1, PIN_LED, NEO_RGB + NEO_KHZ800);

static bool deviceConnected = false;
static NimBLECharacteristic *pNotify = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Hub terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Hub terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// Kedip non-blocking: warna diset sekarang, dimatikan oleh ledTick() nanti.
// delay() tidak dipakai agar loop tetap bisa membaca sensor selama LED menyala.
static unsigned long ledOffAt = 0;

static void ledFlash(uint8_t r, uint8_t g, uint8_t b) {
  strip.setPixelColor(0, strip.Color(r, g, b));
  strip.show();
  ledOffAt = millis() + LED_ON_MS;
}

static void ledTick() {
  if (ledOffAt && millis() >= ledOffAt) {
    strip.setPixelColor(0, 0);
    strip.show();
    ledOffAt = 0;
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("Smart sensor JENDELA 1 (peripheral) starting...");

  pinMode(PIN_SWITCH, INPUT_PULLUP);
  strip.begin();
  strip.setBrightness(50);
  strip.clear();
  strip.show();

  NimBLEDevice::init(BLE_NAME);

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pNotify = pService->createCharacteristic(CHAR_NOTIFY_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName(BLE_NAME);
  pAdvertising->start();

  Serial.println("Menunggu hub... (tekan tombol BOOT = jendela dibuka)");
}
```

```
void loop() {
  ledTick();

  // --- Pembacaan proximity switch dengan debounce ------------------------
  static bool lastStable = HIGH;      // HIGH = tidak ditekan = jendela tertutup
  static bool lastRaw = HIGH;
  static unsigned long lastChange = 0;
  static uint32_t eventCount = 0;

  bool raw = digitalRead(PIN_SWITCH);
  if (raw != lastRaw) {
    lastRaw = raw;
    lastChange = millis();
  }
  if (millis() - lastChange < DEBOUNCE_MS || raw == lastStable) return;

  lastStable = raw;
  bool isOpen = (raw == LOW);  // tombol ditekan -> LOW -> jendela terbuka
  ++eventCount;

  // Indikator lokal: merah saat terbuka, hijau saat tertutup kembali.
  if (isOpen) ledFlash(255, 0, 0);
  else        ledFlash(0, 255, 0);

  // Payload: "<ID>:<STATE>:<nomor event>" — nomor event dipakai hub untuk
  // mendeteksi kejadian yang hilang saat pengukuran jarak.
  String msg = String(SENSOR_ID) + (isOpen ? ":OPEN:" : ":CLOSED:") + String(eventCount);

  if (deviceConnected) {
    pNotify->setValue((const uint8_t *)msg.c_str(), msg.length());
    pNotify->notify();
    Serial.println("Notify: " + msg);
  } else {
    // Sensor tetap bekerja (LED berkedip) walau hub belum tersambung —
    // pesannya yang hilang, bukan pembacaannya.
    Serial.println("Sensor aktif tapi hub belum tersambung: " + msg);
  }
}
```

Kode 6.3: `src/nodea/main.cpp` — sensor jendela

```
// ============================================================================
// MODUL 05B — Smart Sensor Pintu (Node B)
//
// Peran : BLE peripheral, dipasang di PINTU 1.
// Sensor: tombol BOOT (GPIO9) mensimulasikan proximity/reed switch pintu.
//         Ditekan  -> pintu dianggap TERBUKA
//         Dilepas  -> pintu dianggap TERTUTUP
// Aktuator: LED RGB WS2812 onboard (GPIO8) berkedip sebentar tiap kali sensor
//         berubah, sebagai bukti visual bahwa sensor benar-benar terbaca.
// ============================================================================
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <Adafruit_NeoPixel.h>

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_NOTIFY_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"

// Identitas sensor — dipakai sebagai prefiks payload agar hub bisa memisahkan
// sumber pesan tanpa bergantung pada urutan koneksi.
#define SENSOR_ID "PINTU1"
#define BLE_NAME "SENSOR_PINTU1"

// Waveshare ESP32-H2-DEV-KIT-N4: tombol BOOT (Key2) di GPIO9, aktif LOW
// dengan pull-up 10K di board; LED RGB WS2812 di GPIO8, urutan byte RGB.
#define PIN_SWITCH 9
#define PIN_LED 8
#define DEBOUNCE_MS 50
#define LED_ON_MS 300

Adafruit_NeoPixel strip(1, PIN_LED, NEO_RGB + NEO_KHZ800);

static bool deviceConnected = false;
static NimBLECharacteristic *pNotify = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
```

```
    deviceConnected = true;
    Serial.println("Hub terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    deviceConnected = false;
    Serial.println("Hub terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// Kedip non-blocking: warna diset sekarang, dimatikan oleh ledTick() nanti.
// delay() tidak dipakai agar loop tetap bisa membaca sensor selama LED menyala.
static unsigned long ledOffAt = 0;

static void ledFlash(uint8_t r, uint8_t g, uint8_t b) {
  strip.setPixelColor(0, strip.Color(r, g, b));
  strip.show();
  ledOffAt = millis() + LED_ON_MS;
}

static void ledTick() {
  if (ledOffAt && millis() >= ledOffAt) {
    strip.setPixelColor(0, 0);
    strip.show();
    ledOffAt = 0;
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("Smart sensor PINTU 1 (peripheral) starting...");

  pinMode(PIN_SWITCH, INPUT_PULLUP);
  strip.begin();
  strip.setBrightness(50);
  strip.clear();
  strip.show();

  NimBLEDevice::init(BLE_NAME);

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pNotify = pService->createCharacteristic(CHAR_NOTIFY_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName(BLE_NAME);
  pAdvertising->start();

  Serial.println("Menunggu hub... (tekan tombol BOOT = pintu dibuka)");
}

void loop() {
  ledTick();

  // --- Pembacaan proximity switch dengan debounce ------------------------
  static bool lastStable = HIGH;      // HIGH = tidak ditekan = pintu tertutup
  static bool lastRaw = HIGH;
  static unsigned long lastChange = 0;
  static uint32_t eventCount = 0;

  bool raw = digitalRead(PIN_SWITCH);
  if (raw != lastRaw) {
    lastRaw = raw;
    lastChange = millis();
  }
  if (millis() - lastChange < DEBOUNCE_MS || raw == lastStable) return;

  lastStable = raw;
  bool isOpen = (raw == LOW);  // tombol ditekan -> LOW -> pintu terbuka
  ++eventCount;

  // Indikator lokal: merah saat terbuka, hijau saat tertutup kembali.
  if (isOpen) ledFlash(255, 0, 0);
  else        ledFlash(0, 255, 0);
```

```
  // Payload: "<ID>:<STATE>:<nomor event>" — nomor event dipakai hub untuk
  // mendeteksi kejadian yang hilang saat pengukuran jarak.
  String msg = String(SENSOR_ID) + (isOpen ? ":OPEN:" : ":CLOSED:") + String(eventCount);

  if (deviceConnected) {
    pNotify->setValue((const uint8_t *)msg.c_str(), msg.length());
    pNotify->notify();
    Serial.println("Notify: " + msg);
  } else {
    // Sensor tetap bekerja (LED berkedip) walau hub belum tersambung —
    // pesannya yang hilang, bukan pembacaannya.
    Serial.println("Sensor aktif tapi hub belum tersambung: " + msg);
  }
}
```

Kode 6.4: `src/nodeb/main.cpp` — sensor pintu

```
#!/usr/bin/env python3
"""Monitor tiga port serial sekaligus untuk Modul 05B.

Menampilkan aktivitas hub dan kedua sensor dalam SATU jendela terminal, dengan
timestamp bersama, sehingga urutan kejadian antar-board langsung terbaca —
sesuatu yang tidak terlihat kalau tiap board dibuka di terminal terpisah.

    python3 week05b_ble_multinode_project/monitor_serial.py
    python3 week05b_ble_multinode_project/monitor_serial.py --log sesi1.txt
    python3 week05b_ble_multinode_project/monitor_serial.py --port JENDELA=/dev/ttyACM1

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C — ringkasan jumlah baris per board dicetak saat keluar.

DUA HAL SOAL DTR/RTS pada board Waveshare ESP32-H2 (rangkaian auto program:
RTS → EN, DTR → IO9, pin yang sama dengan tombol BOOT):

1. Membuka port **me-reset board**, sama seperti `pio device monitor`. Kernel
   mengaktifkan DTR/RTS saat `open()`, sebelum program sempat mencegahnya, dan
   itu tidak bisa dihindari dari sisi Python. Karena itu jalankan monitor ini
   **lebih dulu**, baru amati — banner startup ketiga board justru ikut
   terekam. Jangan membukanya di tengah percobaan panjang.
2. Setelah terbuka, kedua jalur ditahan **tidak aktif** (`dtr=False`,
   `rts=False`). Ini wajib: DTR yang dibiarkan aktif menahan IO9 tetap LOW,
   dan sensor akan membaca tombol seolah-olah ditekan terus.
"""
import argparse
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

DEFAULT_PORTS = [
    ("JENDELA", "/dev/ttyACM0", "\033[36m"),  # cyan
    ("PINTU", "/dev/ttyACM2", "\033[33m"),    # kuning
    ("HUB", "/dev/ttyACM4", "\033[32m"),      # hijau
]
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
stop = threading.Event()
t0 = time.time()


def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"{DIM}[{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
```

```
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()


def open_port(port, baud):
    """Buka port dengan DTR/RTS ditahan tidak aktif.

    Board tetap ter-reset saat open() (lihat catatan di docstring modul), tetapi
    setelah itu IO9 dibiarkan tinggi sehingga tombol tidak terbaca tertekan.
    """
    s = serial.Serial()
    s.port = port
    s.baudrate = baud
    s.timeout = 0.2
    s.dtr = False
    s.rts = False
    s.open()
    return s


def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip
        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:
            text = line.decode("utf-8", "replace").rstrip("\r")
            if text.strip():
                counts[name] += 1
                show(name, color, text, use_color, logfile)
    ser.close()


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyACMx",
                    help="ganti/tambah port; boleh diulang")
    ap.add_argument("--baud", type=int, default=115200)
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    ports = {name: (port, color) for name, port, color in DEFAULT_PORTS}
    for item in args.port or []:
        if "=" not in item:
            sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyACMx)")
        name, port = item.split("=", 1)
        _, color = ports.get(name.upper(), (None, "\033[35m"))
        ports[name.upper()] = (port, color)

    use_color = not args.no_color and sys.stdout.isatty()
    logfile = open(args.log, "w") if args.log else None

    # SIGTERM (mis. dijalankan lewat `timeout 30 ...`) diperlakukan sama dengan
    # Ctrl-C supaya ringkasan tetap tercetak.
    signal.signal(signal.SIGTERM, lambda *_: (_ for _ in ()).throw(KeyboardInterrupt))

    print(f"Monitor 3 board — Ctrl-C untuk berhenti"
          f"{' · log: ' + args.log if args.log else ''}")
    print("-" * 60)

    threads = [threading.Thread(target=reader,
                                args=(name, port, color, args.baud, use_color, logfile),
```

```
                                daemon=True)
                for name, (port, color) in ports.items()]
    for t in threads:
        t.start()

    try:
        while any(t.is_alive() for t in threads):
            time.sleep(0.2)
    except KeyboardInterrupt:
        pass
    finally:
        stop.set()
        for t in threads:
            t.join(timeout=1.0)
        print("\n" + "-" * 60)
        print(f"Durasi: {time.time() - t0:.1f} s")
        for name in ports:
            print(f"  {name:<7} : {counts.get(name, 0)} baris")
        if logfile:
            logfile.close()
            print(f"Log tersimpan di {args.log}")


if __name__ == "__main__":
    main()
```

Kode 6.5: `monitor_serial.py` — pemantau tiga port serial dalam satu sumbu waktu

6.7 Build dan Flash

Flash **sensor dulu**, hub belakangan, agar saat hub melakukan scan 5 detik kedua sensor sudah mengudara.

Port tiap environment sudah dipin di `platformio.ini`, jadi tidak perlu `--upload-port` selama urutan port pada komputer yang dipakai sama (Tabel 6.6).

Environment	Port	Board
`nodea`	`/dev/ttyACM0`	sensor jendela
`nodeb`	`/dev/ttyACM2`	sensor pintu
`central`	`/dev/ttyACM4`	hub

Tabel 6.6: Pemetaan environment ke port Modul 05B

```
pio run -d week05b_ble_multinode_project -e nodea   -t upload
pio run -d week05b_ble_multinode_project -e nodeb   -t upload
pio run -d week05b_ble_multinode_project -e central -t upload
```

PENTING
Pilih port USB-to-UART, bukan USB native. Board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Flash dilakukan lewat **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Jika port pada komputer yang dipakai berbeda, jalankan `pio device list` lalu sesuaikan `upload_port` dan `monitor_port` pada `platformio.ini` (Windows memakai `COMx`).

6.7.1 Memantau Satu per Satu dengan picocom

Apabila pemantauan lebih nyaman dilakukan satu terminal per board, gunakan picocom (keluar: `Ctrl-A` lalu `Ctrl-X`).

```
picocom /dev/ttyACM4 -b115200 --imap crcrlf,lfcrlf   # central / hub
picocom /dev/ttyACM0 -b115200                        # nodea - sensor jendela
picocom /dev/ttyACM2 -b115200                        # nodeb - sensor pintu
```

`--imap crcrlf,lfcrlf` hanya perlu di central, dan sebabnya ada di kode, bukan di kabelnya: hub mencetak lewat `Serial.printf("...\n")` yang mengirim **LF saja**, sehingga kursor turun tanpa kembali ke kolom pertama — barisnya bertingkat seperti tangga. Kedua node memakai `Serial.println()` yang mengirim CR+LF, jadi tampil rapi tanpa opsi tambahan. Menambahkan `--imap` pada node juga tidak merusak apa pun, jadi aman dipakai seragam jika lebih mudah diingat.

6.7.2 Memantau Ketiga Board Sekaligus

Membuka tiga Serial Monitor terpisah membuat urutan kejadian antar-board sulit dibaca — tiap terminal punya timestamp sendiri. Skrip `monitor_serial.py` (Kode 6.5) menggabungkan ketiganya dalam satu jendela dengan satu sumbu waktu.

```
python3 week05b_ble_multinode_project/monitor_serial.py
python3 week05b_ble_multinode_project/monitor_serial.py --log sesi1.txt
python3 week05b_ble_multinode_project/monitor_serial.py --port JENDELA=/dev/ttyACM6
```

Contoh tampilan — sensor dan hub berdampingan, jadi latency kejadian bisa dibaca langsung dari selisih timestamp (log hasil uji nyata ada di `logserial.md`).

```
[   0.408] HUB     | Mencari smart sensor jendela dan pintu...
[   0.809] HUB     | Jendela 1 terhubung
[   1.610] PINTU   | Hub terhubung
[   2.211] HUB     | Semua sensor terpantau - sistem siaga
[  12.480] JENDELA | Notify: JENDELA1:OPEN:1
[  12.495] HUB     | [ALARM] [ 12.480 s] Jendela 1 TERBUKA (event #1)
```

Ctrl-C menghentikannya dan mencetak ringkasan jumlah baris per board.

> **PERINGATAN**
> **Membuka monitor me-reset ketiga board** — persis seperti `pio device monitor`. Kernel mengaktifkan DTR/RTS saat port dibuka, dan pada board ini RTS terhubung ke EN. Karena itu **jalankan monitor lebih dulu**, baru mulai percobaan; banner startup ketiga board justru ikut terekam. Jangan membukanya di tengah percobaan yang sedang berjalan.

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat.
- ☐ Ketiga ESP32-H2 terpasang dan terdeteksi.
- ☐ Serial Monitor 115200 baud siap (3 terminal).
- ☐ Label fisik ditempel di board: **JENDELA** dan **PINTU**, supaya tidak tertukar saat pengujian.

6.8 Percobaan

6.8.1 EXP-01 — Sensor Bekerja Sendiri (tanpa hub)

Nyalakan **hanya** kedua node sensor. Tekan tombol BOOT pada masing-masing.

Expected output — sensor jendela

```
Smart sensor JENDELA 1 (peripheral) starting...
Menunggu hub... (tekan tombol BOOT = jendela dibuka)
Sensor aktif tapi hub belum tersambung: JENDELA1:OPEN:1
Sensor aktif tapi hub belum tersambung: JENDELA1:CLOSED:2
```

Data capture

Parameter	Hasil
LED berkedip saat tombol ditekan? (warna)	

Parameter	Hasil
LED berkedip saat tombol dilepas? (warna)	
Satu tekanan menghasilkan berapa baris di Serial?	
Apakah nomor event naik satu per perubahan?	

Tabel 6.7: Lembar pengamatan EXP-01

CHECKPOINT

Satu tekanan harus menghasilkan **tepat satu** baris `OPEN` dan satu pelepasan menghasilkan **tepat satu** `CLOSED`. Jika muncul berkali-kali, debounce gagal — periksa `DEBOUNCE_MS` di `src/nodea/main.cpp`.

6.8.2 EXP-02 — Hub Memantau Dua Sensor

Nyalakan hub. Hub melakukan active scan 5 detik, menemukan kedua sensor, membuka dua koneksi, dan subscribe notify.

Expected output — hub

```
Hub Smart Home (BLE central) starting...
Mencari smart sensor jendela dan pintu...
Jendela 1 ditemukan (RSSI -30 dBm)
Pintu 1 ditemukan (RSSI -29 dBm)
Jendela 1 terhubung
Pintu 1 terhubung
Semua sensor terpantau - sistem siaga
[ALARM] [ 43.851 s] Jendela 1 TERBUKA (event #1)
[INFO ] [ 45.051 s] Jendela 1 tertutup kembali (event #2)
[ALARM] [ 59.062 s] Pintu 1 TERBUKA (event #3)
[INFO ] [ 61.112 s] Pintu 1 tertutup kembali (event #4)
```

Nomor event dihitung terpisah per sensor. Pada rekaman ini event pintu mulai dari #3 karena dua kejadian pertamanya terpicu sebelum perekaman dimulai.

Data capture

Parameter	Hasil
Waktu scan sampai dua koneksi aktif (s)	
RSSI jendela dan pintu (dBm)	
Label hub cocok dengan board yang ditekan?	
Warna LED hub saat jendela terbuka	
Warna LED hub saat keduanya tertutup	
Warna LED hub saat jendela ditutup **tetapi** pintu masih terbuka	
Warna LED hub saat satu sensor dicabut	

Tabel 6.8: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA
Di `src/central/main.cpp`, `handleEvent()` menerima `idx` sebagai argumen pertama. Cari dari mana `idx` itu berasal (petunjuk: lihat lambda pada `pChar->subscribe(...)`). Lalu jawab: jika kedua sensor mengirim payload identik `"OPEN"` tanpa prefiks ID, apakah hub masih bisa membedakannya? Jika ya, mengapa prefiks `JENDELA1:` tetap dipakai?

CHECKPOINT

Tekan tombol di board jendela, pastikan yang muncul `Jendela 1`, bukan `Pintu 1`. Jika tertukar, kedua node kemungkinan di-flash dengan environment yang sama — cek ulang `-e nodea` dan `-e nodeb`.

6.8.3 EXP-03 — Keandalan Sistem Alarm

Uji tiga skenario kegagalan dan catat apa yang dilihat "pemilik rumah", yaitu apa yang ditampilkan hub.

No	Skenario	Langkah	Hasil di hub
1	Sensor mati saat tertutup	Cabut USB node pintu	
2	Jendela dibuka saat sensornya mati	Cabut USB node jendela, tekan tombolnya, pasang lagi	
3	Kejadian di luar jangkauan	Bawa node jendela ±15 m atau balik dinding, tekan tombol, kembali	

Tabel 6.9: Skenario kegagalan EXP-03

Data capture

Parameter	Hasil
Pesan hub saat sensor dicabut	
Apakah sensor lain tetap terpantau?	
Waktu pemulihan (`[PULIH] ... setelah X s`)	
Berapa kali `Gagal terhubung` sebelum pulih?	
Apakah `scan ulang` sempat terpicu?	
Setelah skenario 2, apakah state di hub sesuai kondisi nyata?	
Apakah baris `[LOSS ]` pernah muncul?	

Tabel 6.10: Lembar pengamatan EXP-03

Expected output — hub saat sensor mati lalu hidup lagi

```
[WARN ] Jendela 1 terputus (reason 520) - mencoba sambung ulang
Gagal terhubung ke Jendela 1
Gagal terhubung ke Jendela 1
Gagal terhubung ke Jendela 1
Jendela 1 tidak menjawab 3 kali - scan ulang
Jendela 1 ditemukan (RSSI -27 dBm)
Jendela 1 terhubung
[PULIH] Jendela 1 tersambung lagi setelah 28.3 s
Semua sensor terpantau - sistem siaga
```

BUKA ABSTRAKSINYA
Hub menyambung ulang lewat dua jalur berbeda: mencoba **alamat lama** (cepat), dan bila gagal 3 kali, **scan ulang** untuk mencari alamat baru. Cari keduanya di `maintainLinks()`. Lalu jawab: mengapa jalur kedua diperlukan padahal alamat BLE board biasanya tidak berubah? Dan mengapa `connect()` dibatasi 4 detik, bukan 30 detik bawaan library?

CHECKPOINT

Skenario 2 adalah inti modul ini. Koneksinya memang pulih sendiri (`[PULIH]`), tetapi **kejadian yang terjadi selama sensor mati tetap hilang selamanya**: hub menampilkan "aman" padahal jendela terbuka, dan tidak ada baris `[LOSS ]` karena lompatan nomor event baru ketahuan pada kejadian berikutnya. Catat berapa lama kondisi salah ini bertahan, lalu simpulkan: sambung ulang otomatis memperbaiki **tautan**, bukan **data**.

6.9 Verifikasi Build (Referensi)

```
Environment    Status    Duration
central        SUCCESS   00:00:03.756     Flash 54.8%  RAM 6.8%
nodea          SUCCESS   00:00:02.581     Flash 54.1%  RAM 6.8%
nodeb          SUCCESS   00:00:02.651     Flash 54.1%  RAM 6.8%
```

6.10 Pengukuran

A. Latency kejadian — dari LED node berkedip sampai baris muncul di hub. Ukur dengan merekam video kedua layar, atau bandingkan timestamp Serial node dan hub. Perhatikan bahwa tiap board punya `millis()` sendiri, jadi yang dibandingkan adalah **selisih antar-kejadian**, bukan nilai absolutnya.

Jarak	RSSI jendela	RSSI pintu	Latency rata-rata (ms, 10 tekanan)	Kejadian hilang / 10
1 m				
5 m				
10 m				
Balik dinding				

Tabel 6.11: Lembar pengukuran A — latency kejadian

B. Uji beban event-driven — tekan tombol secepat mungkin selama 30 detik pada satu node, lalu bandingkan nomor event terakhir di node dan di hub.

Node	Event terakhir di node	Event terakhir di hub	Kejadian hilang	Loss (%)
Jendela				
Pintu				

Tabel 6.12: Lembar pengukuran B — uji beban event-driven

C. Skenario asimetris (wajib) — jendela didekatkan, pintu dijauhkan.

Posisi jendela	Posisi pintu	Loss jendela (%)	Loss pintu (%)	Kesimpulan
1 m	10 m			

Tabel 6.13: Lembar pengukuran C — skenario asimetris

D. Waktu pemulihan sambung ulang — matikan satu sensor, hidupkan lagi, catat angka pada baris `[PULIH]`. Ulangi 5 kali; dua di antaranya dengan sensor dimatikan lebih dari 15 detik, supaya jalur *scan ulang* ikut teruji.

No	Sensor	Lama dimatikan (s)	`Gagal terhubung` (kali)	Scan ulang?	Waktu pemulihan (s)
1					
2					
3					
4					
5					

Tabel 6.14: Lembar pengukuran D — waktu pemulihan sambung ulang

Selama sensor itu mati, tekan tombol pada sensor **yang lain** dan pastikan alarmnya tetap tampil di hub — pemulihan satu tautan tidak boleh membekukan tautan lainnya.

6.11 Analisis

1. Berapa latency dari tombol ditekan hingga alarm tampil di hub? Apakah cukup cepat untuk sistem keamanan? Bandingkan dengan waktu respons manusia.
2. Pada uji beban (B), pada laju tekanan berapa kejadian mulai hilang? Apa penyebabnya — radio, debounce, atau kecepatan `loop()`?
3. Skenario EXP-03 nomor 2 menghasilkan state hub yang salah tanpa satu pun pesan error. Rancang mekanisme yang membuat kegagalan ini **terdeteksi** (petunjuk: apa yang dilakukan sistem alarm komersial saat sensor diam terlalu lama?).
4. Sensor ini hanya mengirim saat ada perubahan. Bandingkan konsumsi energi dan keandalannya dengan pendekatan M05 yang mengirim tiap 2 detik. Kapan masing-masing lebih tepat?
5. Dari tabel D, apa yang paling menentukan lama pemulihan — `RETRY_MS`, `CONNECT_TIMEOUT_MS`, atau lama sensor mati? Apabila pemulihan dituntut selalu di bawah 5 detik, nilai mana yang diubah dan apa konsekuensinya?
6. Untuk rumah dengan 12 jendela dan 5 pintu, apakah topologi bintang BLE ini masih layak? Apa batasannya, dan protokol mana (Zigbee atau Thread, M08–M12) yang lebih cocok? Berikan alasan teknis.

6.12 Concept Check

1. Mengapa tombol BOOT dibaca dengan `INPUT_PULLUP` dan dianggap aktif saat `LOW`?
2. Apa yang terjadi bila `DEBOUNCE_MS` diset 0? Dan bila diset 2000?
3. Mengapa LED node dikedipkan dengan `millis()` (non-blocking) dan bukan `delay(300)`? Apa yang hilang jika memakai `delay()`?
4. Dari mana hub tahu sebuah notify berasal dari sensor jendela — dari isi payload, atau dari sesuatu yang lain?

5. Nomor event (`seq`) tidak dipakai oleh logika alarm sama sekali. Mengapa tetap dikirim?
6. Mengapa `subscribe()` harus dipanggil lagi setiap kali menyambung ulang, padahal objek `NimBLEClient`-nya sama?
7. Mengapa percobaan `connect()` dijalankan dari `loop()` dan bukan langsung di dalam callback `onDisconnect()`?
8. Sensor tetap mengedipkan LED walau hub belum tersambung. Apakah itu keputusan desain yang tepat untuk sebuah sistem keamanan? Jelaskan dengan alasan.

6.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memperluas sistem dan menutup kegagalan senyap

CH-1 — Sensor ketiga Tambahkan `nodec` sebagai `SENSOR_PINTU2` (pintu belakang). Di hub, cukup tambahkan satu baris pada array `SENSORS[]` — buktikan tidak ada perubahan lain yang diperlukan, lalu jelaskan mengapa desain berbasis array itu penting.

CH-2 — Heartbeat anti-kegagalan senyap Sambung ulang otomatis menyelamatkan tautan, tetapi tidak menyelamatkan kejadian yang terjadi saat sensor mati. Buat tiap sensor mengirim status terkininya (`JENDELA1:ALIVE:OPEN`) setiap 30 detik, dan buat hub **menimpa** state-nya dengan isi heartbeat itu. Uji dengan skenario EXP-03 nomor 2 dan tunjukkan bahwa state hub kini kembali benar setelah pemulihan.

Level 3 — Tantangan · Alarm terpusat, sensor nyata, dan kanal balik

CH-3 — Alarm terpusat Buat LED hub **berkedip** merah (bukan menyala tetap) selama masih ada bukaan, dan tambahkan hitungan berapa lama tiap bukaan terbuka (`Jendela 1 sudah terbuka 45 detik`).

CH-4 — Sensor sungguhan Ganti tombol BOOT dengan reed switch magnetik (atau sensor IR proximity) antara GPIO9 dan GND. Bandingkan perilaku bounce-nya dengan tombol dan sesuaikan `DEBOUNCE_MS`. Catat hasil pengukurannya.

CH-5 — Perintah balik Tambahkan characteristic `WRITE` pada node agar hub bisa mengirim perintah `MUTE` yang mematikan kedip LED sensor. Ini mengubah sistem dari satu arah menjadi dua arah — kaitkan dengan M03.

6.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (event-driven dibanding periodik, active low, debounce, state di hub).
3. Konfigurasi — environment `central`, `nodea`, `nodeb`, UUID, pin GPIO8 dan GPIO9, serta format payload.
4. Hasil eksperimen — log serial tiga perangkat (EXP-01 sampai EXP-03 beserta checkpoint), foto atau video LED.
5. Data pengukuran — tabel A, B, dan C pada bagian Pengukuran.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri, khususnya soal keandalan sistem alarm event-driven.

7. Modul 05C — Pager Restoran: Notifikasi BLE One-to-Many

IoT COMMUNICATION LAB · MODUL 05C

Mini Project — Pager Restoran, Notifikasi BLE One-to-Many

ESP32-H2 · BLE · 1 controller / N pager · level Intermediate · 3 × 50 menit · folder kode `week05c_ble_pager`

PENGANTAR MODUL · Gambaran Umum

Modul 05C adalah mini project lanjutan dari `week05_ble_multinode`, dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya membangun sistem pager restoran: kasir memanggil pemilik pesanan tertentu, pager milik pelanggan itu berbunyi dan menyala, dan panggilan berhenti ketika pelanggan menekan tombol ACK. Percobaan berjalan dengan satu controller dan tiga pager, dikendalikan lewat baris perintah di Serial Monitor kasir, dengan LED RGB onboard dan buzzer sebagai penanda di sisi pelanggan.

7.1 Pendahuluan

Koneksi tidak dipelihara terus-menerus ke seluruh pager, melainkan dibuka saat sebuah pager dipanggil dan ditutup setelah pelanggan menekan ACK — pilihan rancangan yang lahir dari batas perangkat keras, dan dibahas tersendiri di bagian Dasar Teori.

Modul 05 membuktikan satu central sanggup memegang beberapa koneksi sekaligus, tetapi arah datanya hanya satu: node mengirim, pusat menerima. Modul ini membalik arahnya dan menambahkan satu syarat yang jauh lebih ketat — **pesan harus sampai ke satu node tertentu saja**. Dari sinilah muncul pertanyaan yang menjadi inti modul: bagaimana memilih satu penerima di antara banyak node, dan apa bedanya dengan menyiarkan pesan ke semua orang lalu membiarkan mereka menyaring sendiri.

Prasyaratnya adalah M05 untuk koneksi ganda dan pemisahan aliran per sumber, M03 untuk characteristic sebagai kanal perintah, serta M00B untuk pembacaan tombol. Yang dibangun di sini adalah pengiriman perintah terarah lewat objek koneksi tertentu, karakteristik `WRITE` sebagai kanal perintah turun berdampingan dengan `NOTIFY` sebagai kanal status naik, pengelolaan slot koneksi yang jumlahnya terbatas, antarmuka baris perintah di sisi kasir, serta pengukuran waktu tanggap pelanggan. Semuanya dipakai lagi pada M08 ketika perintah ON/OFF Zigbee dialamatkan lewat binding, M12 ketika unicast IPv6 dibandingkan dengan multicast, dan M14 ketika topic MQTT memisahkan perintah turun dari telemetri naik.

Yang membedakan modul ini dari M05 dan M05B

Aspek	M05	M05B	M05C (ini)
Arah data utama	node ke pusat	node ke pusat	**pusat ke node**
Pemicu	timer	tombol di node	**perintah kasir**
Tujuan pesan	satu-satunya pusat	satu-satunya hub	**satu node terpilih dari banyak**
Peran `WRITE`	tidak dipakai	tidak dipakai	**kanal perintah utama**
Umpan balik	—	kejadian sensor	**ACK dari pelanggan dan waktu tanggap**

Tabel 7.1: Perbedaan Modul 05, 05B, dan 05C

Kontrak data lab ini. Perintah turun berupa kata kunci pendek (`READY`, `CANCEL`) yang ditulis ke characteristic perintah, dan status naik berbentuk `<id>:ACK:<milidetik>`. Perhatikan bahwa perintah **tidak memuat nomor pager sama sekali** — nomor tujuan sudah terkandung dalam pilihan koneksi. Ini kebalikan dari pola M05 yang menaruh identitas di dalam payload, dan perbedaan itulah yang dibahas di bagian Analisis.

7.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mengirim perintah terarah ke satu node di antara banyak

1. Menjelaskan mengapa pengiriman ke satu node pada BLE **bukan** broadcast, dan menunjukkan di kode letak pemilihan tujuannya.
2. Membangun central yang memegang beberapa koneksi sekaligus dan mengirim perintah hanya melalui koneksi yang dipilih.
3. Merancang service dengan dua characteristic berlawanan arah: `WRITE` untuk perintah turun dan `NOTIFY` untuk status naik.
4. Membuat antarmuka operator berbasis baris perintah pada Serial Monitor, lengkap dengan penanganan perintah yang tidak sah.
5. Mengukur waktu tanggap pelanggan dari dua sisi (pager dan kasir) dan menjelaskan sumber selisihnya.

Kriteria keberhasilan

- ☐ Seluruh pager terdaftar di controller, dan `READY` membuka koneksi ke pager tujuan saat itu juga.
- ☐ Perintah `READY <id>` membuat **hanya** pager bernomor itu berbunyi; pager lain tidak mencetak apa pun.
- ☐ Tombol ACK menghentikan LED dan buzzer, lalu memunculkan baris `[ACK ]` di sisi kasir.
- ☐ Waktu tanggap tercatat di kedua sisi dan selisihnya dijelaskan.
- ☐ Slot koneksi dilepas otomatis setelah ACK, dan panggilan melebihi kapasitas ditolak dengan pesan yang jelas — bukan membuat controller reboot.

7.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Unicast	Pesan dikirim melalui satu koneksi menuju satu perangkat. Inilah yang dipakai modul ini.
Broadcast	Satu paket tersiar ke udara dan diterima siapa pun yang mendengarkan. Pada BLE hal ini hanya terjadi lewat advertising, bukan lewat koneksi.
Penyaringan di penerima	Pola alternatif: semua node menerima pesan yang sama, lalu membuang yang bukan miliknya. **Tidak** dipakai di sini — bagian Analisis membahas mengapa.
Characteristic `WRITE`	Kanal perintah turun. Central menulis nilai, peripheral menerimanya lewat callback `onWrite`.
Characteristic `NOTIFY`	Kanal status naik. Peripheral mendorong nilai ke central yang sudah subscribe.
Write with response	Penulisan yang ditunggu balasannya oleh central, sehingga kegagalan kirim langsung ketahuan. Dipakai agar kasir tahu perintahnya benar-benar diterima.
Slot koneksi	Jumlah koneksi BLE yang sanggup dipegang controller serentak. Pada ESP32-H2 di lab ini jumlahnya **dua**.

Istilah	Definisi kerja di lab ini
Sambung saat dibutuhkan	Pola pemakaian slot: koneksi dibuka ketika pager dipanggil dan ditutup setelah ACK, sehingga jumlah pager tidak dibatasi jumlah slot.
Waktu tanggap	Selang antara perintah `READY` dan penekanan tombol ACK — besaran yang diukur pada modul ini.

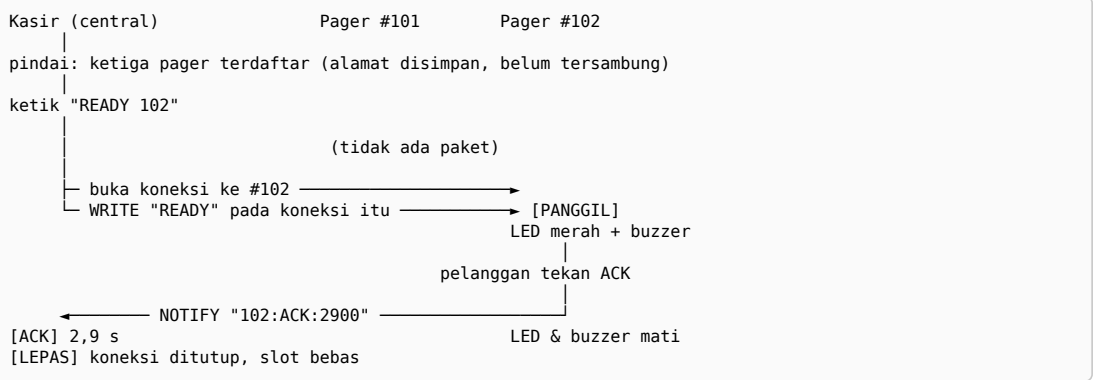
Tabel 7.2: Istilah kerja Modul 05C

Mengapa ini bukan broadcast. Secara naluri, memanggil satu pelanggan di antara banyak terasa seperti menyiarkan nomor lewat pengeras suara: satu suara, semua mendengar, hanya yang merasa dipanggil yang bereaksi. BLE berbasis koneksi bekerja sebaliknya. Controller memegang **satu tautan terpisah untuk tiap pager**, dan menulis perintah ke tautan yang dipilih saja. Pager lain tidak menyaring apa pun — paketnya memang tidak pernah dikirim kepada mereka. Konsekuensinya nyata dan bisa diamati: menambah pager berarti menambah koneksi, bukan menambah pendengar, sehingga batas jumlah pager ditentukan batas koneksi radio.

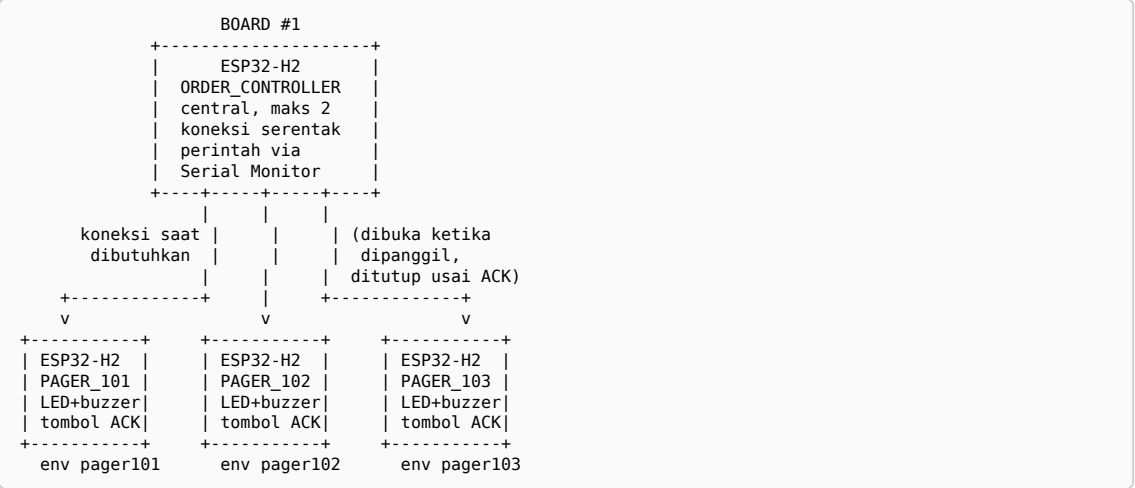
Batas dua koneksi, dan mengapa rancangannya menjadi seperti ini. Percobaan pertama modul ini memelihara koneksi ke ketiga pager sekaligus. Hasilnya: controller **reboot** tepat saat koneksi ketiga terbentuk, berulang kali, dengan pesan `assert failed: npl_freertos_callout_init`. Penyebabnya bukan kesalahan program, melainkan pustaka controller BLE ESP32-H2 pada Arduino core ini yang hanya menyediakan memori untuk dua koneksi serentak (`CONFIG_BT_LE_CONN_RESERVED_MEMORY_COUNT=2`). Nilai itu berada di pustaka yang sudah terkompilasi, sehingga menambahkan `-DCONFIG_BT_LE_MAX_CONNECTIONS=3` pada `build_flags` **tidak** menolong — sudah dicoba dan tetap reboot.

Jawabannya adalah mengubah cara slot dipakai, bukan memaksa menambah slot: controller menyimpan alamat seluruh pager hasil pemindaian, tetapi baru membuka koneksi ketika sebuah pager dipanggil, lalu menutupnya setelah ACK. Dengan begitu jumlah pager tidak lagi dibatasi jumlah slot — yang dibatasi hanya jumlah **panggilan yang berjalan bersamaan**. Inilah pelajaran terpentingnya: batas sebuah sistem WSN sering datang dari radio dan pustaka di bawahnya, bukan dari logika aplikasi di atasnya.

Sekuens yang diamati



7.4 Topologi



Node	Environment	Nama BLE	Peran
Kasir	`controller`	`ORDER_CONTROLLER`	Central, maksimum 2 koneksi serentak, antarmuka perintah
Pager 101	`pager101`	`PAGER_101`	Peripheral, LED dan buzzer serta tombol ACK
Pager 102	`pager102`	`PAGER_102`	Peripheral, sama
Pager 103	`pager103`	`PAGER_103`	Peripheral, sama

Tabel 7.3: Peran tiap node Modul 05C

Ketiga pager memakai **file source yang sama**; nomornya diberikan lewat build flag `-DPAGER_ID=<n>` di `platformio.ini`. Menambah pager keempat berarti menambah satu environment dan satu baris pada array `PAGER_ID[]` di controller — dan tidak menambah beban koneksi, karena slot hanya terpakai selama panggilan berlangsung.

7.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO, NimBLE-Arduino, dan Adafruit NeoPixel.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	Waveshare ESP32-H2-DEV-KIT-N4 atau DevKitM-1	4 (1 kasir + 3 pager)
2	Buzzer pasif	3,3 V, disambung ke GPIO10 dan GND	3
3	Kabel USB data	kabel data, bukan *charge-only*	4
4	PC/Laptop	PlatformIO Core/IDE	1

Tabel 7.4: Alat dan bahan Modul 05C

Percobaan tetap bisa dijalankan dengan dua pager saja; pager yang belum pernah terlihat muncul sebagai `BELUM TERLIHAT` pada perintah `LIST`, dan justru berguna untuk menguji penolakan perintah.

Pin yang dipakai (sama di ketiga pager).

Fungsi	GPIO	Keterangan
LED penanda	GPIO8	WS2812 onboard (`RGB_CTRL`), satu pixel, urutan byte RGB

Fungsi	GPIO	Keterangan
Buzzer	GPIO10	Pin header bebas, bukan *strapping pin*; digerakkan `tone()`
Tombol ACK	GPIO9	Tombol BOOT (`Key2`), aktif LOW, pull-up 10K di board

Tabel 7.5: Pemetaan pin Modul 05C

PERINGATAN
GPIO9 juga *strapping pin* mode download. Menahan tombol ACK **saat board direset** membuat board masuk mode flash, bukan menjalankan program.

Struktur proyek

```
week05c_ble_pager/
├── platformio.ini          ← 4 environment; ID pager lewat build flag
├── monitor_serial.py       ← pantau ketiga pager dalam satu sumbu waktu
├── logserial.md            ← log referensi hasil uji perangkat
└── src/
    ├── controller/main.cpp  ← central, N koneksi, antarmuka perintah kasir
    └── pager/main.cpp       ← satu source untuk seluruh pager
```

7.6 Kode Program

```
[platformio]
default_envs = controller

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3
    adafruit/Adafruit NeoPixel@^1.15.5

; Catatan batas perangkat keras: pada ESP32-H2 dengan Arduino core ini,
; controller BLE hanya menyediakan memori untuk DUA koneksi serentak.
; Koneksi ketiga membuat npl_freertos_callout_init() gagal dan board reboot.
; Batas itu berada di pustaka controller yang sudah terkompilasi, sehingga
; TIDAK dapat dinaikkan lewat build flag (-DCONFIG_BT_LE_MAX_CONNECTIONS
; sudah dicoba dan tidak berpengaruh). Karena itu controller memakai pola
; sambung-saat-dibutuhkan; lihat MAX_KONEKSI di src/controller/main.cpp.

; Port di-pin agar tidak salah flash saat beberapa board terpasang bersamaan.
; Pakai port jembatan USB-to-UART (Hardware ID 1A86:55D3) — pada Linux
; nomornya genap. Sesuaikan dengan hasil `pio device list`.
[env:controller]
build_src_filter = +<controller/*.cpp>
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Ketiga pager memakai source yang sama; hanya nomor pagernya yang berbeda.
; Inilah alasan ID diberikan lewat build flag, bukan lewat tiga salinan file.
[env:pager101]
build_src_filter = +<pager/*.cpp>
build_flags = -DPAGER_ID=101
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2

[env:pager102]
build_src_filter = +<pager/*.cpp>
build_flags = -DPAGER_ID=102
upload_port  = /dev/ttyACM4
monitor_port = /dev/ttyACM4

[env:pager103]
build_src_filter = +<pager/*.cpp>
build_flags = -DPAGER_ID=103
upload_port  = /dev/ttyACM6
monitor_port = /dev/ttyACM6
```

Kode 7.1: `platformio.ini` Modul 05C — empat environment, ID lewat build flag

```
// ============================================================================
// MODUL 05C — Order Controller / Kasir (central)
//
// Memegang satu koneksi BLE untuk tiap pager, lalu mengirim perintah hanya ke
// pager yang dituju. Perintah diketikkan kasir di Serial Monitor:
//
//     READY 102     panggil pemilik pesanan pager 102
//     CANCEL 102    batalkan panggilan
//     LIST          tampilkan status semua pager
//
// Catatan penting: ini BUKAN broadcast. Tidak ada satu paket pun yang tersiar
// ke semua pager. Controller memilih objek koneksi milik pager tujuan, lalu
// menulis ke characteristic perintah pada koneksi itu saja.
//
// Batas perangkat keras: ESP32-H2 pada Arduino core ini hanya sanggup memegang
// DUA koneksi BLE serentak (koneksi ketiga membuat controller reboot). Karena
// itu koneksi tidak dipelihara terus-menerus untuk semua pager, melainkan
// dibuka saat sebuah pager dipanggil dan ditutup setelah pelanggan menekan
// ACK. Alamat hasil pemindaian tetap disimpan agar penyambungan berikutnya
// tidak perlu memindai ulang.
// ============================================================================
#include <Arduino.h>
#include <NimBLEDevice.h>
```

```
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_CMD_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b1"     // controller -> pager
#define CHAR_STATUS_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"  // pager -> controller

#define SCAN_MS 5000
#define CONNECT_TIMEOUT_MS 4000

// Sengaja 2, bukan angka yang dikira-kira: koneksi ketiga membuat controller
// BLE ESP32-H2 gagal mengalokasikan callout dan board reboot.
#define MAX_KONEKSI 2

// Daftar pager yang dilayani. Menambah pager keempat cukup satu baris di sini
// plus satu environment baru di platformio.ini.
static const int PAGER_ID[] = {101, 102, 103};
static const int JUMLAH_PAGER = sizeof(PAGER_ID) / sizeof(PAGER_ID[0]);

static NimBLEAddress alamat[JUMLAH_PAGER];
static bool ditemukan[JUMLAH_PAGER] = {false};
static bool tersambung[JUMLAH_PAGER] = {false};
static bool memanggil[JUMLAH_PAGER] = {false};
static unsigned long mulaiPanggil[JUMLAH_PAGER] = {0};
static NimBLEClient *klien[JUMLAH_PAGER] = {nullptr};
static NimBLERemoteCharacteristic *charCmd[JUMLAH_PAGER] = {nullptr};

static bool putuskanSetelahAck[JUMLAH_PAGER] = {false};

static int jumlahTersambung() {
  int n = 0;
  for (int i = 0; i < JUMLAH_PAGER; i++) n += tersambung[i] ? 1 : 0;
  return n;
}

static int indeksDariId(int id) {
  for (int i = 0; i < JUMLAH_PAGER; i++)
    if (PAGER_ID[i] == id) return i;
  return -1;
}

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  ClientCallbacks(int idx) : _idx(idx) {}
  void onConnect(NimBLEClient *c) override {
    Serial.printf("Pager #%d terhubung\n", PAGER_ID[_idx]);
  }
  // Percobaan sambung ulang dikerjakan loop(), bukan di dalam callback ini:
  // memanggil connect() dari stack NimBLE membuat controller hang.
  void onDisconnect(NimBLEClient *c, int reason) override {
    bool sedangMemanggil = memanggil[_idx];
    tersambung[_idx] = false;
    charCmd[_idx] = nullptr;
    memanggil[_idx] = false;
    // Putus setelah ACK adalah hal biasa — slot koneksi memang sengaja
    // dilepas. Yang perlu diberitahukan hanya putus di tengah panggilan.
    if (sedangMemanggil)
      Serial.printf("[WARN ] Pager #%d terputus di tengah panggilan (reason %d)\n",
                    PAGER_ID[_idx], reason);
  }

private:
  int _idx;
};

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    for (int i = 0; i < JUMLAH_PAGER; i++) {
      char nama[16];
      snprintf(nama, sizeof(nama), "PAGER_%d", PAGER_ID[i]);
      if (!ditemukan[i] && d->getName() == nama) {
        alamat[i] = d->getAddress();
        ditemukan[i] = true;
        Serial.printf("Pager #%d ditemukan (RSSI %d dBm)\n", PAGER_ID[i], d->getRSSI());
      }
    }
  }
};

// Payload status dari pager: "<id>:ACK:<milidetik tanggap>"
static void tanganiStatus(int idx, const char *data, size_t len) {
  char buf[40];
  size_t n = len < sizeof(buf) - 1 ? len : sizeof(buf) - 1;
```

```
  memcpy(buf, data, n);
  buf[n] = '\0';

  if (strstr(buf, ":ACK") == nullptr) {
    Serial.printf("[WARN ] Status tidak dikenal dari pager #%d: %s\n", PAGER_ID[idx], buf);
    return;
  }

  double tanggapPager = 0;
  const char *p = strrchr(buf, ':');
  if (p) tanggapPager = atol(p + 1) / 1000.0;

  double tanggapKasir = memanggil[idx] ? (millis() - mulaiPanggil[idx]) / 1000.0 : 0;
  memanggil[idx] = false;
  // Pemutusan dikerjakan loop(), bukan di sini: memanggil disconnect() dari
  // dalam callback notify berarti membongkar koneksi dari stack NimBLE sendiri.
  putuskanSetelahAck[idx] = true;

  Serial.printf("[ACK  ] Pager #%d diambil pelanggan - %.1f s menurut pager, "
                "%.1f s menurut kasir\n",
                PAGER_ID[idx], tanggapPager, tanggapKasir);
}

static void pastikanScan() {
  NimBLEScan *pScan = NimBLEDevice::getScan();
  if (!pScan->isScanning()) pScan->start(SCAN_MS, false);
}

static bool sambungkan(int idx) {
  int id = PAGER_ID[idx];
  if (klien[idx] == nullptr) {
    klien[idx] = NimBLEDevice::createClient();
    klien[idx]->setClientCallbacks(new ClientCallbacks(idx));
    // Default library 30 detik: satu percobaan ke pager mati akan menahan
    // loop() selama itu, sehingga kasir tidak bisa mengetik perintah.
    klien[idx]->setConnectTimeout(CONNECT_TIMEOUT_MS);
  }

  NimBLEScan *pScan = NimBLEDevice::getScan();
  if (pScan->isScanning()) pScan->stop();

  if (!klien[idx]->connect(alamat[idx])) {
    Serial.printf("Gagal terhubung ke pager #%d\n", id);
    return false;
  }

  NimBLERemoteService *pService = klien[idx]->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.printf("Service pager #%d tidak ditemukan\n", id);
    klien[idx]->disconnect();
    return false;
  }

  charCmd[idx] = pService->getCharacteristic(CHAR_CMD_UUID);
  NimBLERemoteCharacteristic *pStatus = pService->getCharacteristic(CHAR_STATUS_UUID);
  if (charCmd[idx] == nullptr || pStatus == nullptr) {
    Serial.printf("Characteristic pager #%d tidak lengkap\n", id);
    klien[idx]->disconnect();
    return false;
  }

  // Subscribe wajib diulang tiap kali menyambung: nilai CCCD hilang saat
  // koneksi putus karena kedua sisi tidak melakukan bonding.
  pStatus->subscribe(true, [idx](NimBLERemoteCharacteristic *c, uint8_t *data, size_t len,
                                 bool isNotify) { tanganiStatus(idx, (const char *)data, len); });

  tersambung[idx] = true;
  return true;
}

// Lepaskan koneksi pager yang panggilannya sudah selesai, supaya slot koneksi
// kembali tersedia untuk pager berikutnya.
static void lepaskanSlot() {
  for (int i = 0; i < JUMLAH_PAGER; i++) {
    if (!putuskanSetelahAck[i]) continue;
    putuskanSetelahAck[i] = false;
    if (klien[i] && klien[i]->isConnected()) {
      klien[i]->disconnect();
      Serial.printf("[LEPAS] Koneksi pager #%d ditutup - slot koneksi kembali bebas\n",
                    PAGER_ID[i]);
    }
  }
```

```
}

static void kirimPerintah(int idx, const char *cmd) {
  // Inilah inti modul ini: perintah ditulis ke SATU objek koneksi. Pager lain
  // tidak menerima apa pun — bukan karena menyaring, tetapi karena paketnya
  // memang tidak pernah dikirim ke mereka.
  if (!charCmd[idx]->writeValue((const uint8_t *)cmd, strlen(cmd), true)) {
    Serial.printf("[GAGAL] Perintah %s ke pager #%d tidak terkirim\n", cmd, PAGER_ID[idx]);
    return;
  }
  Serial.printf("[KIRIM] %s -> pager #%d saja (pager lain tidak menerima apa pun)\n", cmd,
                PAGER_ID[idx]);
}

static void tampilkanDaftar() {
  Serial.printf("--- Status pager (%d/%d slot koneksi terpakai) ---\n", jumlahTersambung(),
                MAX_KONEKSI);
  for (int i = 0; i < JUMLAH_PAGER; i++) {
    const char *st = memanggil[i]    ? "MEMANGGIL"
                   : tersambung[i] ? "tersambung"
                   : ditemukan[i]  ? "terdaftar"
                                   : "BELUM TERLIHAT";
    Serial.printf("  #%d : %-14s", PAGER_ID[i], st);
    if (tersambung[i] && memanggil[i])
      Serial.printf(" (%.0f s berjalan)", (millis() - mulaiPanggil[i]) / 1000.0);
    Serial.println();
  }
}

static void bantuan() {
  Serial.printf("Perintah: READY <id> | CANCEL <id> | LIST | HELP  "
                "(maksimum %d panggilan berjalan bersamaan)\n", MAX_KONEKSI);
}

static void olahPerintah(String baris) {
  baris.trim();
  if (baris.length() == 0) return;
  baris.toUpperCase();

  if (baris == "LIST") {
    tampilkanDaftar();
    return;
  }
  if (baris == "HELP") {
    bantuan();
    return;
  }

  int spasi = baris.indexOf(' ');
  if (spasi < 0) {
    Serial.printf("[GAGAL] Perintah tidak dikenal: %s\n", baris.c_str());
    bantuan();
    return;
  }

  String kata = baris.substring(0, spasi);
  int id = baris.substring(spasi + 1).toInt();
  int idx = indeksDariId(id);

  if (idx < 0) {
    Serial.printf("[GAGAL] Pager #%d tidak terdaftar\n", id);
    return;
  }
  if (kata == "READY") {
    if (!ditemukan[idx]) {
      Serial.printf("[GAGAL] Pager #%d belum pernah terlihat - pastikan pager menyala\n", id);
      return;
    }
    if (!tersambung[idx] && jumlahTersambung() >= MAX_KONEKSI) {
      Serial.printf("[GAGAL] Slot koneksi penuh (%d/%d). Tunggu ACK panggilan berjalan "
                    "atau CANCEL salah satunya.\n",
                    jumlahTersambung(), MAX_KONEKSI);
      return;
    }
    if (!tersambung[idx] && !sambungkan(idx)) {
      Serial.printf("[GAGAL] Tidak bisa menyambung ke pager #%d\n", id);
      return;
    }
    kirimPerintah(idx, "READY");
    memanggil[idx] = true;
    mulaiPanggil[idx] = millis();
  } else if (kata == "CANCEL") {
```

```
    if (!tersambung[idx]) {
      Serial.printf("[GAGAL] Pager #%d tidak sedang dipanggil\n", id);
      return;
    }
    kirimPerintah(idx, "CANCEL");
    memanggil[idx] = false;
    putuskanSetelahAck[idx] = true;
  } else {
    Serial.printf("[GAGAL] Perintah tidak dikenal: %s\n", kata.c_str());
    bantuan();
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("Order Controller (kasir) starting...");

  NimBLEDevice::init("ORDER_CONTROLLER");

  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(SCAN_MS, false);

  Serial.println("Mencari pager...");
}

void loop() {
  static bool siapDiumumkan = false;
  static String baris = "";

  lepaskanSlot();

  // Pemindaian dibiarkan berjalan sampai seluruh pager pernah terlihat, supaya
  // alamatnya siap dipakai begitu kasir memanggil.
  bool semuaTerlihat = true;
  for (int i = 0; i < JUMLAH_PAGER; i++) semuaTerlihat &= ditemukan[i];
  if (!semuaTerlihat) pastikanScan();

  if (semuaTerlihat && !siapDiumumkan) {
    Serial.printf("Seluruh %d pager terdaftar - kasir siap menerima perintah\n", JUMLAH_PAGER);
    bantuan();
    siapDiumumkan = true;
  }

  // Baris perintah dari Serial Monitor kasir
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (baris.length()) {
        olahPerintah(baris);
        baris = "";
      }
    } else if (baris.length() < 32) {
      baris += c;
    }
  }
}
```

Kode 7.2: `src/controller/main.cpp` — controller kasir dengan antarmuka perintah

```
// ============================================================================
// MODUL 05C — Pager Pelanggan (peripheral)
//
// Satu unit pager restoran. Menunggu perintah dari controller kasir, lalu
// memanggil pelanggan dengan LED dan buzzer sampai tombol ACK ditekan.
//
// Nomor pager datang dari build flag (-DPAGER_ID=101) sehingga ketiga unit
// memakai file source yang sama persis.
// ============================================================================
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <Adafruit_NeoPixel.h>

#ifndef PAGER_ID
#define PAGER_ID 101
#endif
```

```
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_CMD_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b1"     // controller -> pager
#define CHAR_STATUS_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b3"  // pager -> controller

// Waveshare ESP32-H2-DEV-KIT-N4
#define PIN_LED 8      // WS2812 onboard, urutan byte RGB
#define PIN_BUZZER 10  // buzzer pasif di header, bukan strapping pin
#define PIN_ACK 9      // tombol BOOT (Key2), aktif LOW, pull-up 10K di board

#define DEBOUNCE_MS 50
#define BEEP_MS 250       // panjang satu nada dan satu jeda
#define BEEP_HZ 2400      // nada buzzer saat memanggil
#define BLINK_MS 250      // kedip LED saat memanggil

Adafruit_NeoPixel strip(1, PIN_LED, NEO_RGB + NEO_KHZ800);

static bool memanggil = false;         // sedang memanggil pelanggan?
static bool buzzerAktif = false;       // tone() sedang berjalan?
static unsigned long mulaiPanggil = 0; // untuk mengukur waktu tanggap
static bool terhubung = false;
static NimBLECharacteristic *pStatus = nullptr;

static void berhentiMemanggil() {
  memanggil = false;
  // noTone() pada pin yang tidak sedang berbunyi memunculkan galat di log,
  // dan panggilan bisa berhenti saat buzzer sedang di fase diam.
  if (buzzerAktif) {
    noTone(PIN_BUZZER);
    buzzerAktif = false;
  }
  strip.setPixelColor(0, 0);
  strip.show();
}

class CmdCallbacks : public NimBLECharacteristicCallbacks {
public:
  void onWrite(NimBLECharacteristic *pChar, NimBLEConnInfo &connInfo) override {
    String cmd = String(pChar->getValue().c_str());
    cmd.trim();

    if (cmd == "READY") {
      // Perintah ini hanya tiba di pager yang dituju controller. Tidak ada
      // penyaringan nomor di sini: pemilihan tujuan terjadi di sisi controller,
      // lewat koneksi mana perintah dikirim.
      memanggil = true;
      mulaiPanggil = millis();
      Serial.printf("[PANGGIL] Pesanan siap - menunggu tombol ACK\n");
    } else if (cmd == "CANCEL") {
      berhentiMemanggil();
      Serial.println("[BATAL  ] Panggilan dibatalkan kasir");
    } else {
      Serial.printf("[WARN   ] Perintah tidak dikenal: %s\n", cmd.c_str());
    }
  }
};

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    terhubung = true;
    Serial.println("Controller terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    terhubung = false;
    Serial.println("Controller terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

void setup() {
  Serial.begin(115200);
  Serial.printf("Pager #%d starting...\n", PAGER_ID);

  pinMode(PIN_ACK, INPUT_PULLUP);
  pinMode(PIN_BUZZER, OUTPUT);
  strip.begin();
  strip.setBrightness(60);
  strip.clear();
  strip.show();

  char nama[16];
  snprintf(nama, sizeof(nama), "PAGER_%d", PAGER_ID);
```

```
  NimBLEDevice::init(nama);

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);

  NimBLECharacteristic *pCmd =
      pService->createCharacteristic(CHAR_CMD_UUID, NIMBLE_PROPERTY::WRITE);
  pCmd->setCallbacks(new CmdCallbacks());

  pStatus = pService->createCharacteristic(CHAR_STATUS_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName(nama);
  pAdvertising->start();

  Serial.printf("Menunggu controller... (nama BLE: %s)\n", nama);
}

// Kedip LED dan bunyi buzzer dijalankan tanpa delay() supaya tombol ACK tetap
// terbaca selama pager sedang berbunyi.
static void tandaPanggilan() {
  static unsigned long tik = 0;
  static bool nyala = false;
  if (millis() - tik < BEEP_MS) return;
  tik = millis();
  nyala = !nyala;

  if (nyala) {
    strip.setPixelColor(0, strip.Color(255, 0, 0));
    tone(PIN_BUZZER, BEEP_HZ);
    buzzerAktif = true;
  } else {
    strip.setPixelColor(0, 0);
    if (buzzerAktif) {
      noTone(PIN_BUZZER);
      buzzerAktif = false;
    }
  }
  strip.show();
}

void loop() {
  if (memanggil) tandaPanggilan();

  // --- tombol ACK dengan debounce ---------------------------------------
  static bool stabil = HIGH, mentah = HIGH;
  static unsigned long berubah = 0;

  bool baca = digitalRead(PIN_ACK);
  if (baca != mentah) {
    mentah = baca;
    berubah = millis();
  }
  if (millis() - berubah < DEBOUNCE_MS || baca == stabil) return;
  stabil = baca;

  if (baca != LOW) return;  // hanya tepi turun (tombol mulai ditekan)

  if (!memanggil) {
    Serial.println("[INFO   ] Tombol ditekan, tetapi tidak ada panggilan aktif");
    return;
  }

  unsigned long tanggap = millis() - mulaiPanggil;
  berhentiMemanggil();

  char msg[32];
  snprintf(msg, sizeof(msg), "%d:ACK:%lu", PAGER_ID, tanggap);
  if (terhubung) {
    pStatus->setValue((const uint8_t *)msg, strlen(msg));
    pStatus->notify();
  }
  Serial.printf("[ACK    ] Pelanggan menekan tombol setelah %.1f s\n", tanggap / 1000.0);
}
```

Kode 7.3: `src/pager/main.cpp` — satu source untuk seluruh pager

```
#!/usr/bin/env python3
"""Monitor ketiga pager sekaligus untuk Modul 05C.

Menampilkan aktivitas pager 101, 102, dan 103 dalam SATU jendela terminal
dengan timestamp bersama, sehingga terlihat langsung bahwa hanya pager yang
dipanggil yang bereaksi — bukti bahwa perintah kasir bersifat unicast, bukan
broadcast. Hal itu sulit dinilai bila tiap pager dibuka di terminal terpisah.

    python3 week05c_ble_pager/monitor_serial.py
    python3 week05c_ble_pager/monitor_serial.py --log sesi1.txt
    python3 week05c_ble_pager/monitor_serial.py --port P103=/dev/ttyACM8

TERMINAL KASIR TIDAK DIBUKA DI SINI. Controller ada di /dev/ttyACM0 dan
dipegang cutecom (baud 115200, akhir baris LF), karena kasir perlu mengetik
perintah sedangkan skrip ini hanya membaca. Membuka port yang sama dari dua
program sekaligus membuat keduanya berebut data.

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C — ringkasan jumlah baris per board dicetak saat keluar.

DUA HAL SOAL DTR/RTS pada board Waveshare ESP32-H2 (rangkaian auto program:
RTS → EN, DTR → IO9, pin yang sama dengan tombol ACK pager):

1. Membuka port **me-reset board**, sama seperti `pio device monitor`. Kernel
   mengaktifkan DTR/RTS saat `open()`, sebelum program sempat mencegahnya, dan
   itu tidak bisa dihindari dari sisi Python. Karena itu jalankan monitor ini
   **lebih dulu**, baru amati — banner startup ketiga board justru ikut
   terekam. Jangan membukanya di tengah panggilan yang sedang berjalan —
   pager yang ter-reset memutus koneksinya ke kasir.
2. Setelah terbuka, kedua jalur ditahan **tidak aktif** (`dtr=False`,
   `rts=False`). Ini wajib: DTR yang dibiarkan aktif menahan IO9 tetap LOW,
   dan pager akan membaca tombol ACK seolah-olah ditekan terus.
"""
import argparse
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

DEFAULT_PORTS = [
    ("P101", "/dev/ttyACM2", "\033[36m"),  # cyan
    ("P102", "/dev/ttyACM4", "\033[33m"),  # kuning
    ("P103", "/dev/ttyACM6", "\033[35m"),  # magenta
]
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
stop = threading.Event()
t0 = time.time()


def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"{DIM}[{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()


def open_port(port, baud):
    """Buka port dengan DTR/RTS ditahan tidak aktif.

    Board tetap ter-reset saat open() (lihat catatan di docstring modul), tetapi
    setelah itu IO9 dibiarkan tinggi sehingga tombol tidak terbaca tertekan.
    """
```

```
    s = serial.Serial()
    s.port = port
    s.baudrate = baud
    s.timeout = 0.2
    s.dtr = False
    s.rts = False
    s.open()
    return s


def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip
        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:
            text = line.decode("utf-8", "replace").rstrip("\r")
            if text.strip():
                counts[name] += 1
                show(name, color, text, use_color, logfile)
    ser.close()


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyACMx",
                    help="ganti/tambah pager; boleh diulang (mis. P104=/dev/ttyACM8)")
    ap.add_argument("--baud", type=int, default=115200)
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    ports = {name: (port, color) for name, port, color in DEFAULT_PORTS}
    for item in args.port or []:
        if "=" not in item:
            sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyACMx)")
        name, port = item.split("=", 1)
        _, color = ports.get(name.upper(), (None, "\033[35m"))
        ports[name.upper()] = (port, color)

    use_color = not args.no_color and sys.stdout.isatty()
    logfile = open(args.log, "w") if args.log else None

    # SIGTERM (mis. dijalankan lewat `timeout 30 ...`) diperlakukan sama dengan
    # Ctrl-C supaya ringkasan tetap tercetak.
    signal.signal(signal.SIGTERM, lambda *_: (_ for _ in ()).throw(KeyboardInterrupt))

    print(f"Monitor 3 pager — kasir dipantau terpisah lewat cutecom · Ctrl-C untuk berhenti"
          f"{' · log: ' + args.log if args.log else ''}")
    print("-" * 60)

    threads = [threading.Thread(target=reader,
                                args=(name, port, color, args.baud, use_color, logfile),
                                daemon=True)
               for name, (port, color) in ports.items()]
    for t in threads:
        t.start()

    try:
        while any(t.is_alive() for t in threads):
            time.sleep(0.2)
    except KeyboardInterrupt:
        pass
    finally:
```

```
        stop.set()
        for t in threads:
            t.join(timeout=1.0)
        print("\n" + "-" * 60)
        print(f"Durasi: {time.time() - t0:.1f} s")
        for name in ports:
            print(f"  {name:<7} : {counts.get(name, 0)} baris")
        if logfile:
            logfile.close()
            print(f"Log tersimpan di {args.log}")


if __name__ == "__main__":
    main()
```

Kode 7.4: `monitor_serial.py` — pemantau ketiga pager dalam satu sumbu waktu

7.7 Build dan Flash

Flash pager lebih dahulu, controller belakangan, agar saat controller memindai kedua pager sudah mengudara.

```
pio run -d week05c_ble_pager -e pager101 -t upload
pio run -d week05c_ble_pager -e pager102 -t upload
pio run -d week05c_ble_pager -e pager103 -t upload
pio run -d week05c_ble_pager -e controller -t upload -t monitor
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board muncul sebagai dua port serial: jembatan CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Flash memakai jembatan UART karena jalur itulah yang tersambung ke rangkaian *auto program*. Pada Linux keduanya berselang-seling — port genap adalah UART — sehingga empat board memakai `/dev/ttyACM0`, `/dev/ttyACM2`, `/dev/ttyACM4`, dan `/dev/ttyACM6`. Verifikasi dengan `pio device list`.

7.7.1 Membuka Terminal Pengujian

Kasir dan pager memerlukan terminal yang berbeda sifatnya. Kasir harus **mengetik** perintah, sedangkan pager hanya diamati. Karena itu controller dibuka dengan cutecom yang menyediakan kotak kirim beserta pilihan akhir baris, dan pager dibuka dengan picocom.

Peran	Port	Terminal
Kasir	`/dev/ttyACM0`	cutecom — baud `115200`, opsi akhir baris **LF**
Pager 101	`/dev/ttyACM2`	`picocom /dev/ttyACM2 -b115200 --imap crcrlf,lfcrlf`
Pager 102	`/dev/ttyACM4`	`picocom /dev/ttyACM4 -b115200 --imap crcrlf,lfcrlf`
Pager 103	`/dev/ttyACM6`	`picocom /dev/ttyACM6 -b115200 --imap crcrlf,lfcrlf`

Tabel 7.6: Terminal pengujian tiap peran

Keluar dari picocom: `Ctrl-A` lalu `Ctrl-X`.

Alternatif: ketiga pager dalam satu jendela. Tiga jendela picocom menyulitkan penilaian "pager lain benar-benar diam", karena tiap jendela punya sumbu waktunya sendiri. Skrip `monitor_serial.py` menggabungkan ketiganya pada satu sumbu waktu, sementara kasir tetap dipegang cutecom.

```
python3 week05c_ble_pager/monitor_serial.py
python3 week05c_ble_pager/monitor_serial.py --log sesi1.txt
```

```
[  10.232] P102    | Controller terhubung
[  11.235] P102    | [PANGGIL] Pesanan siap - menunggu tombol ACK
[  14.242] P103    | Controller terhubung
[  15.245] P103    | [PANGGIL] Pesanan siap - menunggu tombol ACK
```

Pada cuplikan nyata di atas, `READY 102` lalu `READY 103` dikirim dari kasir, dan P101 tidak mencetak satu baris pun sepanjang sesi — persis bukti yang diminta CHECKPOINT EXP-02.

> **PERINGATAN**
> **Membuka monitor me-reset ketiga pager**, sama seperti `pio device monitor`: kernel mengaktifkan DTR/RTS saat port dibuka, dan RTS terhubung ke EN. Jalankan monitor **lebih dahulu**, baru mulai memanggil. Membukanya di tengah panggilan akan memutus koneksi pager ke kasir.

Dua opsi di atas bukan hiasan, keduanya memperbaiki gejala yang nyata.

`--imap crcrlf,lfcrlf` pada picocom Firmware pager mencetak lewat `Serial.printf("...\n")` yang mengirim **LF saja**, sehingga kursor turun tanpa kembali ke kolom pertama dan baris tampil bertingkat seperti tangga. Pemetaan ini memperbaikinya di sisi terminal, tanpa mengubah firmware.

Keluaran serial dijaga ASCII Cutecom tidak mendekode UTF-8, sehingga satu karakter tanda pisah panjang akan tampil sebagai tiga byte mentah dan mengotori hampir setiap baris penting. Karena itu seluruh pesan firmware pada modul ini memakai tanda pisah biasa. Aturan yang sama berlaku bila menambahkan pesan sendiri.

Opsi akhir baris LF pada cutecom Perintah kasir baru diproses setelah controller menerima penanda akhir baris. Controller menerima CR maupun LF, sehingga opsi **LF** aman dipakai; yang harus dihindari adalah pengaturan tanpa akhir baris sama sekali — perintah akan terkirim tetapi tidak pernah dieksekusi.

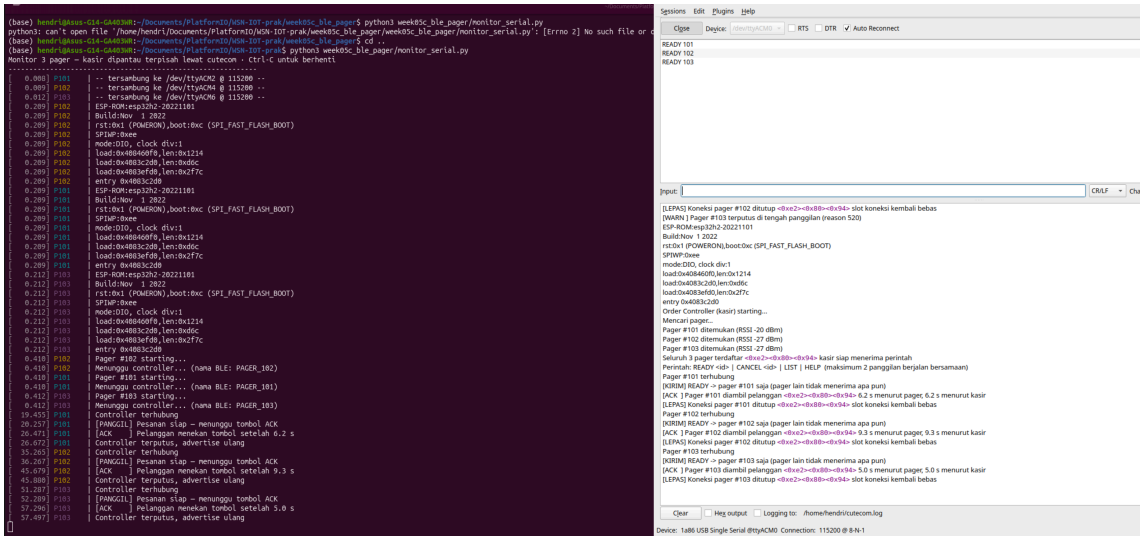
Bila cutecom tidak tersedia, picocom juga bisa dipakai untuk kasir: `picocom /dev/ttyACM0 -b115200 -c --imap crcrlf,lfcrlf`. Perintah diketik langsung lalu diakhiri Enter. Opsi `-c` menyalakan gema lokal — tanpa itu ketikan tidak tampak di layar, karena controller memang tidak menggemakan kembali karakter yang diterimanya.

Perintah kasir (diketik pada terminal controller, diakhiri Enter).

Perintah	Arti
`READY <id>`	Panggil pemilik pesanan pager `<id>`
`CANCEL <id>`	Hentikan panggilan tanpa menunggu ACK
`LIST`	Tampilkan status seluruh pager
`HELP`	Tampilkan daftar perintah

Tabel 7.7: Perintah yang tersedia di terminal kasir

Contoh sesi pengujian. Jendela kiri menjalankan `monitor_serial.py` untuk ketiga pager, jendela kanan adalah cutecom pada `/dev/ttyACM0` tempat kasir mengetik `READY 101`, `READY 102`, lalu `READY 103` secara berurutan. Terlihat tiap perintah hanya membangunkan satu pager — baris `[PANGGIL]` muncul bergantian di P101, P102, dan P103, tidak pernah serentak — sedangkan waktu tanggap ketiga pelanggan (6,2 s · 9,3 s · 5,0 s) tercatat sama persis di kedua sisi. Setiap `[LEPAS]` menandai slot koneksi yang kembali bebas, sehingga panggilan berikutnya dapat dilayani. Rangkaian byte mentah yang tampak pada tangkapan layar berasal dari versi firmware terdahulu yang memakai tanda pisah panjang; firmware sekarang hanya mencetak ASCII, sehingga baris yang sama tampil rapi.



Gambar 7.1: Sesi pengujian: monitor tiga pager di jendela kiri, terminal kasir cutecom di jendela kanan

7.8 Percobaan

7.8.1 EXP-01 — Pager Menyala Sendiri-sendiri

Nyalakan seluruh pager tanpa controller, lalu tekan tombol ACK pada salah satunya.

Expected output — pager

```
Pager #101 starting...
Menunggu controller... (nama BLE: PAGER_101)
[INFO   ] Tombol ditekan, tetapi tidak ada panggilan aktif
```

Data capture

Parameter	Hasil
Nama BLE tiap pager	
LED menyala tanpa perintah?	
Buzzer berbunyi tanpa perintah?	
Reaksi tombol saat tidak ada panggilan	

Tabel 7.8: Lembar pengamatan EXP-01

CHECKPOINT

Pager diam sepenuhnya sebelum ada perintah. LED atau buzzer yang menyala sendiri berarti kondisi awal `memanggil` tidak `false`; periksa `src/pager/main.cpp` sebelum melanjutkan.

7.8.2 EXP-02 — Panggilan Terarah

Nyalakan controller, tunggu seluruh pager tersambung, lalu ketik `READY 102`. Amati **ketiga** Serial Monitor sekaligus.

Expected output — kasir

```
Order Controller (kasir) starting...
Mencari pager...
Pager #101 ditemukan (RSSI -16 dBm)
Pager #102 ditemukan (RSSI -26 dBm)
Pager #103 ditemukan (RSSI -32 dBm)
Seluruh 3 pager terdaftar - kasir siap menerima perintah
Perintah: READY <id> | CANCEL <id> | LIST | HELP  (maksimum 2 panggilan berjalan bersamaan)
Pager #102 terhubung
[KIRIM] READY -> pager #102 saja (pager lain tidak menerima apa pun)
```

Data capture

Parameter	Hasil
Waktu pindai sampai seluruh pager terdaftar (s)	
Waktu `READY` sampai koneksi ke pager tujuan terbentuk (s)	
Pager yang berbunyi setelah `READY 102`	
Jumlah baris baru pada Serial pager 101	
Selang perintah diketik sampai LED pager menyala (s)	

Tabel 7.9: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA
Di `src/controller/main.cpp`, fungsi `kirimPerintah()` hanya menerima indeks dan kata perintah; nomor pager tidak pernah ikut terkirim ke udara. Telusuri dari mana indeks itu berasal, lalu jawab: seandainya ketiga pager memakai firmware yang sama persis **tanpa** `PAGER_ID`, apakah sistem ini masih bisa memanggil satu pelanggan tertentu? Jika ya, untuk apa nomor pager tetap disimpan di firmware pager?

CHECKPOINT

Serial pager yang tidak dipanggil harus **tidak bertambah satu baris pun**. Bertambahnya baris — walau isinya penolakan — berarti pesan tetap sampai ke sana, dan sistemnya bukan unicast seperti yang dirancang.

7.8.3 EXP-03 — ACK dan Waktu Tanggap

Panggil sebuah pager, biarkan berbunyi beberapa detik, lalu tekan tombol ACK. Ulangi lima kali dengan lama tunggu berbeda-beda.

Expected output

```
# pager
[PANGGIL] Pesanan siap - menunggu tombol ACK
[ACK    ] Pelanggan menekan tombol setelah 2.9 s

# kasir
[ACK  ] Pager #102 diambil pelanggan - 2.9 s menurut pager, 3.0 s menurut kasir
[LEPAS] Koneksi pager #102 ditutup - slot koneksi kembali bebas
```

Data capture

No	Lama menunggu (s)	Tanggap menurut pager	Tanggap menurut kasir	Selisih
1				
2				

No	Lama menunggu (s)	Tanggap menurut pager	Tanggap menurut kasir	Selisih
3				
4				
5				

Tabel 7.10: Lembar pengamatan EXP-03 — waktu tanggap dari dua sisi

CHECKPOINT

Kedua angka waktu tanggap harus berdekatan. Selisih yang membesar seiring lamanya menunggu menandakan salah satu papan menghitung waktu dari titik yang keliru — telusuri `mulaiPanggil` di kedua sisi.

7.8.4 EXP-04 — Perintah yang Harus Ditolak

Uji jalur kesalahan yang pasti terjadi di pemakaian nyata.

Uji	Perintah	Hasil yang diharapkan
Slot penuh	`READY 101`, `READY 103`, lalu `READY 102`	dua panggilan berjalan, panggilan ketiga **ditolak**, controller tetap hidup
Pager mati	matikan pager 103, reset controller, lalu `READY 103`	`[GAGAL] Pager #103 belum pernah terlihat`
Nomor asing	`READY 999`	`[GAGAL] Pager #999 tidak terdaftar`
Perintah ngawur	`MAKAN 101`	`[GAGAL] Perintah tidak dikenal`
Batal panggilan	`READY 102` lalu `CANCEL 102`	pager berhenti tanpa ACK, slot dilepas
Slot dilepas	`LIST` sesudah semua ACK	`0/2 slot koneksi terpakai`

Tabel 7.11: Daftar uji penolakan perintah EXP-04

CHECKPOINT

Uji slot penuh adalah yang terpenting. Panggilan ketiga harus **ditolak dengan pesan**, dan controller tetap menerima perintah berikutnya. Board yang justru reboot berarti pembatasan slot dilucuti dari kode — dan itu persis kegagalan yang ditemukan saat modul ini disusun (lihat `logserial.md`).

7.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 4 × ESP32-H2 (1 controller ditambah pager 101, 102, dan 103). Log lengkap ada di `logserial.md`.

```
[  0.40] KASIR| Seluruh 3 pager terdaftar - kasir siap menerima perintah
[ 10.30] KASIR| Pager #102 terhubung
[ 11.18] P102 | [PANGGIL] Pesanan siap - menunggu tombol ACK
[ 11.22] KASIR| [KIRIM] READY -> pager #102 saja (pager lain tidak menerima apa pun)
[ 14.12] P102 | [ACK    ] Pelanggan menekan tombol setelah 2.9 s
[ 14.16] KASIR| [ACK  ] Pager #102 diambil pelanggan - 2.9 s menurut pager, 3.0 s menurut kasir
[ 14.16] KASIR| [LEPAS] Koneksi pager #102 ditutup - slot koneksi kembali bebas
--- dua panggilan berjalan bersamaan, panggilan ketiga ditolak ---
[ 24.33] KASIR| --- Status pager (2/2 slot koneksi terpakai) ---
[ 24.33] KASIR|   #101 : MEMANGGIL      (6 s berjalan)
[ 24.33] KASIR|   #102 : terdaftar
[ 24.33] KASIR|   #103 : MEMANGGIL      (0 s berjalan)
[ 25.53] KASIR| [GAGAL] Slot koneksi penuh (2/2). Tunggu ACK panggilan berjalan atau CANCEL salah satunya.
```

Parameter	Hasil terukur
Pindai sampai tiga pager terdaftar	0,4 s
`READY` sampai koneksi terbentuk dan pager berbunyi	± 1,0 s
Baris baru di pager yang tidak dipanggil	**0**
Selisih waktu tanggap pager dan kasir	≤ 0,1 s
Dua panggilan bersamaan	berhasil
Panggilan ketiga	ditolak, controller tetap responsif
Build	controller Flash 53,0 % RAM 6,6 %; pager Flash 55,3 % RAM 6,8 %

Tabel 7.12: Hasil verifikasi hardware Modul 05C

7.10 Pengukuran

A. Latency perintah — dari Enter ditekan sampai LED pager menyala. Ukur dengan membandingkan stempel waktu pada log gabungan ketiga papan.

Jarak kasir ke pager	RSSI (dBm)	Latency perintah (ms, 10 percobaan)	Perintah gagal / 10
1 m			
5 m			
10 m			
Balik dinding			

Tabel 7.13: Lembar pengukuran A — latency perintah

B. Pengaruh panggilan yang sedang berjalan — ukur latency `READY` saat belum ada panggilan lain, lalu saat satu panggilan sedang berjalan (satu slot sudah terpakai).

Panggilan berjalan	Latency `READY` sampai pager berbunyi (ms)	Catatan
0		
1		
2	—	perintah ditolak, catat pesannya

Tabel 7.14: Lembar pengukuran B — pengaruh slot terpakai

C. Waktu tanggap pelanggan — gunakan tabel EXP-03, lalu hitung rata-rata dan sebarannya.

7.11 Analisis

1. Tunjukkan baris kode yang menentukan pager mana yang menerima perintah. Mengapa nomor pager tidak perlu ikut dikirim di dalam payload?
2. Rancangan alternatif: controller menyiarkan `READY:102` ke **semua** pager, dan tiap pager membuang pesan yang bukan nomornya. Bandingkan keduanya dari sisi lalu lintas radio, keamanan, konsumsi daya pager, dan kemudahan menambah pager.
3. Dari tabel B, apakah latency perintah bertambah ketika satu slot sudah terpakai? Kaitkan dengan konsep *time-sharing radio* pada M05.
4. Rancangan ini membuka koneksi hanya saat memanggil. Sebutkan dua kerugiannya dibanding koneksi yang dipelihara terus-menerus, lalu jelaskan mengapa kerugian itu tetap diterima di sini.
5. Waktu tanggap versi pager dan versi kasir dihitung dari dua jam yang berbeda. Mengapa keduanya tetap bisa dibandingkan, dan kapan cara ini akan menyesatkan?
6. Restoran sesungguhnya memakai 30 pager, dengan kemungkinan lima pesanan siap bersamaan. Bagian mana dari rancangan ini yang lebih dahulu menyerah — jumlah pager atau jumlah panggilan serentak? Usulkan protokol dari modul lain yang lebih sesuai beserta alasannya.

7.12 Concept Check

1. Apa perbedaan broadcast dan unicast, dan yang mana yang dipakai modul ini?
2. Mengapa `WRITE` dipilih untuk perintah turun, sedangkan `NOTIFY` untuk status naik? Bisakah keduanya ditukar?
3. Apa arti *write with response*, dan informasi apa yang hilang seandainya dipakai *write without response*?
4. Mengapa ketiga pager dapat memakai satu file source yang sama?
5. Pager tetap berbunyi ketika koneksi ke controller terputus di tengah panggilan. Apakah itu perilaku yang benar untuk sebuah pager restoran? Jelaskan dengan alasan.
6. Mengapa batas dua koneksi tidak dapat diatasi dengan menambah `-DCONFIG_BT_LE_MAX_CONNECTIONS=3` pada `build_flags`?
7. Mengapa kedip LED dan bunyi buzzer dijadwalkan dengan `millis()` alih-alih `delay()`?

7.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah pager dan memanggil berurutan

CH-1 — Pager keempat Tambahkan `PAGER_104`: satu environment baru pada `platformio.ini` dan satu angka pada array `PAGER_ID[]`. Buktikan tidak ada perubahan lain yang diperlukan, lalu jelaskan mengapa rancangan berbasis array dan build flag membuat hal itu mungkin.

CH-2 — Panggil semua Tambahkan perintah `READY ALL` yang memanggil seluruh pager secara bergiliran — ingat hanya dua slot koneksi yang tersedia, sehingga pager ketiga baru dapat dipanggil setelah satu slot dilepas. Perhatikan bahwa ini tetap **bukan** broadcast, dan ukur selisih waktu antara pager pertama dan terakhir yang berbunyi.

Level 3 — Tantangan · Pengingat, antrian, dan RSSI sebagai data

CH-3 — Pengingat otomatis Bila ACK tidak ditekan dalam 60 detik, buat pager mengubah pola bunyi dan controller mencetak peringatan agar pelayan mengantar pesanan ke meja.

CH-4 — Antrian pesanan Simpan daftar pesanan di controller (`ORDER <id>` untuk memasukkan antrean, `NEXT` untuk memanggil yang terdepan) sehingga kasir tidak perlu mengingat nomor.

CH-5 — Jarak sebagai data Catat RSSI tiap pager secara berkala dan tampilkan pada `LIST`, lalu selidiki apakah RSSI dapat menjadi petunjuk kasar apakah pelanggan masih berada di dalam ruangan.

7.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (unicast dibanding broadcast, `WRITE` dibanding `NOTIFY`, write with response).
3. Konfigurasi — environment, UUID, pin GPIO8, GPIO9, GPIO10, serta format perintah dan status.
4. Hasil eksperimen — log serial seluruh papan (EXP-01 sampai EXP-04 beserta checkpoint), foto atau video pager berbunyi.
5. Data pengukuran — tabel A, B, dan C pada bagian Pengukuran.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang disusun sendiri, khususnya mengenai batas skala rancangan berbasis koneksi.

8. Modul 06 — Relay Multi-Hop di atas BLE

IoT COMMUNICATION LAB · MODUL 06

Build a BLE Mesh — Relay Multi-Hop di atas BLE

ESP32-H2 · BLE · relay A⟶B⟶C · level Intermediate · 3 × 50 menit · folder kode `week06_ble_mesh`

PENGANTAR MODUL · Gambaran Umum

Modul 06 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya memperluas jangkauan lewat hop, sehingga pesan dari A sampai ke C yang tidak terjangkau langsung. Percobaan berjalan sebagai relay dua hop dengan A sebagai sumber, B sebagai relay, dan C sebagai penerima, diamati melalui tiga terminal Serial Monitor pada 115200 baud.

8.1 Pendahuluan

Modul 05 menambah node sebagai **cabang** — semua tetap bicara langsung ke pusat. Modul ini memakai node ketiga sebagai **jembatan**, sehingga jangkauan jaringan melampaui jangkauan satu radio. Konsep hop inilah yang membuat Zigbee (M10) dan Thread (M12) disebut mesh; bedanya, di sana routing dikerjakan stack, sedangkan di sini routing dituliskan sendiri di lapisan aplikasi — supaya mekanismenya terlihat telanjang sebelum disembunyikan protokol.

Prasyaratnya adalah M05: banyak koneksi, penanda sumber, dan pengukuran loss per node. Yang dibangun di sini adalah node dual-role yang menjadi server dan client sekaligus, penerusan pesan antar-hop, pengukuran loss per hop, serta pemetaan mode kegagalan rantai. Keempatnya dipakai lagi pada M10 ketika router Zigbee melakukan hal serupa secara otomatis, M12 pada mesh Thread yang self-healing, M13 ketika gateway berperan sebagai hop antar-protokol, dan M16 ketika jangkauan multi-hop menjadi kriteria pemilihan protokol.

Peta modul blok BLE (penutup blok)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan BLE P2P terbentuk dan stabil
02	Payload aplikasi mengalir dua arah
03	Characteristic mewakili state dan perintah
04	Telemetry via notify
05	Satu pusat, banyak sumber (bintang)
06 (ini)	**Jangkauan diperluas lewat hop — penutup blok BLE**
07	Turun ke lapisan MAC 802.15.4 telanjang — fondasi Zigbee dan Thread

Tabel 8.1: Peta modul blok BLE

Kontrak data lab ini. Payload `A:n` **tidak diubah** saat melewati relay — inilah *transparent forwarding*. Prinsip yang sama dipakai gateway di M13 dan M15: gateway meneruskan isi apa adanya, tidak memparsing ulang, sehingga hop tambahan tidak merusak data.

8.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Memperluas jangkauan lewat relay dua hop

1. Mengimplementasikan node dual-role yang menjalankan GATT server dan GATT client bersamaan, dan menunjukkan bagian kode yang menangani masing-masing peran.
2. Membuktikan pesan dari A tiba utuh di C dengan mencocokkan nomor pesan pada log ketiga node.
3. Menghitung packet loss **per hop** (A ke B dan B ke C) dan menentukan hop mana yang menjadi penyebab kehilangan.
4. Menjelaskan mode kegagalan rantai (relay mati, penerima akhir mati) berdasarkan log, dan menyebutkan konsekuensinya bagi desain jaringan.

Kriteria keberhasilan

- ☐ Rantai A ke B ke C terbentuk; C mencetak `Pesan tiba (via A -> B -> C)`.
- ☐ Nomor pesan di A, B, dan C dapat dicocokkan satu per satu.
- ☐ Tabel loss per hop terisi dari pengukuran sendiri.
- ☐ Kedua mode kegagalan (C mati, B mati) diuji dan hasilnya dicatat.

8.3 Dasar Teori (Secukupnya)

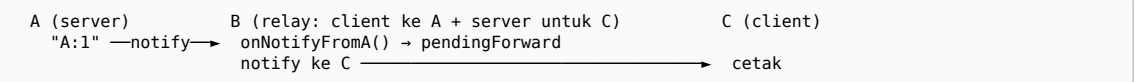
Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (BLE Mesh standar atau ESP-BLE-MESH, flooding dibanding routing, model publish-subscribe mesh) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 8.2.

Istilah	Definisi kerja di lab ini
Hop	Satu lompatan radio dari node ke node tetangga.
Relay	Node yang menerima pesan lalu meneruskannya ke node berikutnya.
Dual role	Satu board menjalankan GATT server (untuk hop berikutnya) dan GATT client (ke hop sebelumnya) bersamaan.
Transparent forwarding	Isi pesan diteruskan apa adanya, tanpa diubah relay.
Loss per hop	Kehilangan dihitung terpisah tiap lompatan, bukan hanya ujung-ke-ujung.
Latency kumulatif	Waktu total A ke C sama dengan latency hop 1 ditambah waktu proses relay ditambah latency hop 2.

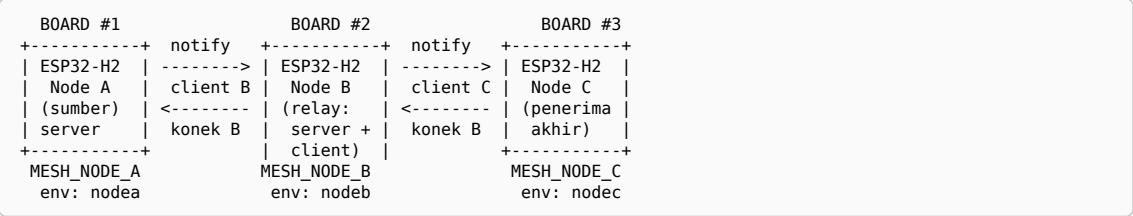
Tabel 8.2: Istilah kerja Modul 06

Ini bukan BLE Mesh standar. ESP-BLE-MESH (spesifikasi Bluetooth SIG) memakai flooding, provisioning, dan model publish-subscribe — jalur pesan ditentukan protokol. Di sini rantai A ke B ke C ditentukan **oleh kode aplikasi**. Keterbatasannya (tidak ada penemuan rute, tidak ada self-healing, arah tunggal) justru yang perlu dicatat, karena itulah yang dibereskan Thread di M12.

Sekuens protokol yang diamati



8.4 Topologi



Node	Board	Environment	Peran	Aksi
Node A	ESP32-H2 DevKitM-1	`nodea`	Sumber (server)	kirim `A:n` tiap 4 s saat relay terhubung
Node B	ESP32-H2 DevKitM-1	`nodeb`	Relay (server dan client)	terima dari A, teruskan ke C
Node C	ESP32-H2 DevKitM-1	`nodec`	Penerima akhir (client)	cetak pesan yang tiba

Tabel 8.3: Peran tiap node Modul 06

Ketiganya **ESP32-H2 DevKitM-1**. Relay di sini dibuat di lapisan aplikasi di atas BLE GATT — bukan BLE Mesh standar — sehingga tidak butuh board lain.

Susun posisi **A, B, dan C dalam satu garis**; A dan C tidak perlu (dan sebaiknya tidak) saling terjangkau.

8.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan PlatformIO dan pustaka NimBLE-Arduino.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Library NimBLE-Arduino	`h2zero/NimBLE-Arduino@^2.2.3` via `lib_deps`	—
5	Power bank atau catu daya USB	agar A dan C bisa dijauhkan dari meja	2 (opsional)
6	Ruang uji	lorong atau ruang panjang untuk formasi garis A–B–C	—

Tabel 8.4: Alat dan bahan Modul 06

8.6 Kode Program

Pin port agar tidak salah flash (Kode 8.1).

```
[env:nodea]
build_src_filter = +<nodea/*.cpp>
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:nodeb]
build_src_filter = +<nodeb/*.cpp>
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2

[env:nodec]
build_src_filter = +<nodec/*.cpp>
upload_port  = /dev/ttyACM4
monitor_port = /dev/ttyACM4
```

Kode 8.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = nodea

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:nodea]
build_src_filter = +<nodea/*.cpp>

[env:nodeb]
build_src_filter = +<nodeb/*.cpp>

[env:nodec]
build_src_filter = +<nodec/*.cpp>
```

Kode 8.2: `platformio.ini` Modul 06 pada repositori

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID     "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_RELAY_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26b4"

static bool relayConnected = false;
static NimBLECharacteristic *pRelay = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    relayConnected = true;
    Serial.println("Node B terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    relayConnected = false;
    Serial.println("Node B terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("Node A (sumber pesan) starting...");

  NimBLEDevice::init("MESH_NODE_A");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pRelay = pService->createCharacteristic(CHAR_RELAY_UUID, NIMBLE_PROPERTY::NOTIFY);
```

```
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("MESH_NODE_A");
  pAdvertising->start();

  Serial.println("Menunggu relay (Node B)...");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (relayConnected && millis() - last > 4000) {
    last = millis();
    String msg = "A:" + String(++count);
    pRelay->setValue((const uint8_t *)msg.c_str(), msg.length());
    pRelay->notify();
    Serial.println("Kirim ke B: " + msg);
  }
}
```

Kode 8.3: `src/nodea/main.cpp` — sumber pesan

```
#include <Arduino.h>
#include <NimBLEDevice.h>

// Node B = relay: client ke A (menerima) sekaligus server ke C (meneruskan)

#define SERVICE_UUID     "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_RELAY_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26b4"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress nodeAAddr;
static bool doConnect = false;
static bool connected = false;
static NimBLECharacteristic *pRelayOut = nullptr;  // notify ke C

static String pendingForward = "";
static volatile bool hasPending = false;

// --- Bagian server (untuk Node C) ---
class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    Serial.println("Node C terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
    Serial.println("Node C terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

// --- Bagian client (ke Node A) ---
class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override { connected = true; Serial.println("Terhubung ke Node
A"); }
  void onDisconnect(NimBLEClient *pClient, int reason) override { connected = false;
Serial.println("Terputus dari Node A"); }
};

// Callback notify NimBLE 2.x: sebuah fungsi, bukan objek callback
static void onNotifyFromA(NimBLERemoteCharacteristic *pChar, uint8_t *pData,
                          size_t length, bool isNotify) {
  char buf[64];
  size_t n = length < sizeof(buf) - 1 ? length : sizeof(buf) - 1;
  memcpy(buf, pData, n);
  buf[n] = '\0';
  Serial.printf("Terima dari A: %s (diteruskan)\n", buf);
  pendingForward = buf;
  hasPending = true;
}
```

```
class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    if (d->getName() == "MESH_NODE_A") {
      Serial.println("Node A ditemukan");
      nodeAAddr = d->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToA() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(nodeAAddr)) {
    Serial.println("Gagal terhubung ke A");
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service A tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pChar = pService->getCharacteristic(CHAR_RELAY_UUID);
  if (pChar == nullptr) {
    Serial.println("Characteristic A tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  if (pChar->canNotify()) {
    pChar->subscribe(true, onNotifyFromA);
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node B (relay) starting...");

  NimBLEDevice::init("MESH_NODE_B");

  // Server untuk C
  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());
  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pRelayOut = pService->createCharacteristic(CHAR_RELAY_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();
  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("MESH_NODE_B");
  pAdvertising->start();

  // Client ke A
  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Menunggu A dan C...");
}

void loop() {
  if (doConnect) {
    if (connectToA()) Serial.println("Koneksi ke A berhasil");
    doConnect = false;
  }

  if (hasPending) {
    hasPending = false;
    pRelayOut->setValue((const uint8_t *)pendingForward.c_str(), pendingForward.length());
    pRelayOut->notify();
```

```
    Serial.println("Teruskan ke C: " + pendingForward);
  }
}
```

Kode 8.4: `src/nodeb/main.cpp` — relay dual-role (server dan client)

```
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID     "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_RELAY_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26b4"

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress nodeBAddr;
static bool doConnect = false;
static bool connected = false;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override { connected = true; Serial.println("Terhubung ke Node
B"); }
  void onDisconnect(NimBLEClient *pClient, int reason) override { connected = false;
Serial.println("Terputus dari Node B"); }
};

// Callback notify NimBLE 2.x: sebuah fungsi, bukan objek callback
static void onNotifyFromB(NimBLERemoteCharacteristic *pChar, uint8_t *pData,
                          size_t length, bool isNotify) {
  Serial.printf("Pesan tiba (via A -> B -> C): %.*s\n", (int)length, (const char *)pData);
}

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    if (d->getName() == "MESH_NODE_B") {
      Serial.println("Node B ditemukan");
      nodeBAddr = d->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToB() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(nodeBAddr)) {
    Serial.println("Gagal terhubung ke B");
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("Service B tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pChar = pService->getCharacteristic(CHAR_RELAY_UUID);
  if (pChar == nullptr) {
    Serial.println("Characteristic B tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  if (pChar->canNotify()) {
    pChar->subscribe(true, onNotifyFromB);
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node C (penerima akhir) starting...");

  NimBLEDevice::init("MESH_NODE_C");
```

```
  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik

  Serial.println("Scanning Node B...");
}

void loop() {
  if (doConnect) {
    if (connectToB()) Serial.println("Koneksi ke B berhasil");
    doConnect = false;
  }
}
```

Kode 8.5: `src/nodec/main.cpp` — penerima akhir

8.7 Build dan Flash

Urutan penting: relay dulu, baru ujung-ujungnya.

```
pio run -d week06_ble_mesh -e nodeb -t upload
pio run -d week06_ble_mesh -e nodea -t upload
pio run -d week06_ble_mesh -e nodec -t upload -t monitor
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat dan diisikan di atas.
- ☐ Ketiga ESP32-H2 terpasang dan terdeteksi.
- ☐ Environment `nodea`, `nodeb`, dan `nodec` dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud siap (3 terminal).
- ☐ Formasi garis A, B, C sudah disiapkan.

8.8 Percobaan

8.8.1 EXP-01 — Menyalakan Relay (Node B)

Unggah environment `nodeb` lebih dulu, lalu verifikasi dual-role: server (advertise `MESH_NODE_B` untuk C) sekaligus client (scan `MESH_NODE_A`).

```
      server (untuk C)          client (ke A)
  advertise MESH_NODE_B       scan MESH_NODE_A
            \                       /
             +---- Node B -----+
```

Data capture

Parameter	Hasil
Pesan awal relay (`Node B (relay) starting...`)	

Parameter	Hasil
Nama BLE relay	
Characteristic yang dipakai (UUID akhir)	
Berapa peran radio yang dijalankan Node B?	

Tabel 8.5: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Di `src/nodeb/main.cpp`, tunjukkan **dua blok kode terpisah**: satu yang membuat `NimBLEServer` (untuk C) dan satu yang membuat `NimBLEClient` (ke A). Keduanya hidup di board yang sama dengan satu radio. Pertanyaan yang harus dijawab sebelum lanjut: saat B sedang mengirim notify ke C, apakah B masih bisa menerima notify dari A pada saat yang sama?

CHECKPOINT

Node B mencetak baris start-nya dan tidak crash. Jika board reboot berulang (boot loop), kemungkinan besar radio kehabisan resource — periksa jumlah koneksi maksimum di konfigurasi NimBLE.

8.8.2 EXP-02 — Menutup Rantai (Node A dan Node C)

Unggah `nodea` (server, kirim `A:n` tiap 4 s saat relay terhubung) dan `nodec` (client yang scan `MESH_NODE_B` lalu subscribe). Amati Serial Monitor ketiga node dan verifikasi jejak pesan per hop.

```
A: setup ⟶ advertise ⟶ B konek ke A ⟶ A kirim "A:n" / 4 s
B: onNotifyFromA() ⟶ pendingForward ⟶ notify ke C
C: onNotifyFromB() ⟶ cetak "Pesan tiba (via A -> B -> C)"
```

Expected output

Node A `Node A (sumber pesan) starting...`, `Menunggu relay (Node B)...`, `Node B terhubung`, `Kirim ke B: A:1`, dan seterusnya.

Node B `Node B (relay) starting...`, `Menunggu A dan C...`, `Node A ditemukan`, `Node C terhubung`, `Terhubung ke Node A`, `Koneksi ke A berhasil`, `Terima dari A: A:1 (diteruskan)`, `Teruskan ke C: A:1`, dan seterusnya. Node C dapat tersambung ke B sebelum B selesai tersambung ke A, sehingga `Node C terhubung` bisa muncul lebih dulu.

Node C `Node C (penerima akhir) starting...`, `Scanning Node B...`, `Node B ditemukan`, `Terhubung ke Node B`, `Koneksi ke B berhasil`, `Pesan tiba (via A -> B -> C): A:1`, dan seterusnya.

CHECKPOINT

Cocokkan **nomor pesan yang sama** di tiga Serial Monitor: `Kirim ke B: A:5` di A, `Teruskan ke C: A:5` di B, dan `Pesan tiba ...: A:5` di C. Jika nomor di C tertinggal jauh atau melompat, hentikan dan catat — itu loss per hop yang akan diukur pada bagian Pengukuran.

8.8.3 EXP-03 — Mode Kegagalan Rantai

1. **Matikan Node C** — verifikasi B tetap menerima dari A dan tetap memanggil notify, tetapi tidak ada penerima akhir.
2. **Matikan Node B** — amati Node A berhenti mengirim (syarat `relayConnected` tidak terpenuhi lagi).
3. Ukur dan bandingkan jumlah pesan di A dan di C selama 2 menit.

Data capture

Parameter	Hasil
Pesan terkirim A per 2 menit (harapan 30)	
Pesan tiba di C per 2 menit	
Perilaku saat Node C dimatikan	
Perilaku saat Node B dimatikan	
Apakah rantai pulih sendiri setelah B dinyalakan lagi?	

Tabel 8.6: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa matinya B menghentikan A (bukan sekadar "pesannya tidak sampai"), dengan menunjuk baris kode di `src/nodea/main.cpp`.

8.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** dalam satu garis, capture 40 detik.

```
# Node A (ESP32-H2)          # Node B (ESP32-H2, relay)        # Node C (ESP32-H2)
[0.601] Node B terhubung     [0.602] Node C terhubung          [0.602] Terhubung ke Node B
[4.210] Kirim ke B: A:1      [4.207] Terima dari A: A:1        [4.410] Pesan tiba
[8.217] Kirim ke B: A:2      [4.207] Teruskan ke C: A:1                (via A -> B -> C): A:1
```

Parameter	Hasil terukur
Rantai A ke B ke C terbentuk	1,4 s setelah boot
Pesan dikirim A	9
Pesan diteruskan B	9
Pesan tiba di C	9 (0 % loss ujung-ke-ujung)
Latency relay (B terima sampai C cetak)	< 200 ms

Tabel 8.7: Hasil verifikasi hardware Modul 06

8.10 Pengukuran

Geser jarak hop A–B dan B–C (ukur dari posisi Node B); catat RSSI dan success rate.

Jarak (per hop)	RSSI (dBm)	Latency A ke C (kasar)	Success (%)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 8.8: Lembar pengukuran jarak per hop Modul 06

Pengukuran per-hop (pengamatan 2 menit) — inti modul ini.

Hop	Pesan dikirim	Pesan diterima	Loss (%)
A ke B			
B ke C			
A ke C (ujung-ke-ujung)			

Tabel 8.9: Lembar pengukuran loss per hop

Periksa: apakah loss A ke C kira-kira sama dengan loss A ke B ditambah loss B ke C? Jelaskan jika tidak.

Uji jangkauan (wajib) — jauhkan A dan C sampai **tidak saling terjangkau langsung**, buktikan dengan mematikan B (pesan berhenti), lalu nyalakan B lagi (pesan kembali). Inilah bukti hop benar-benar menambah jangkauan.

8.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Apakah setiap pesan `A:n` yang dikirim Node A juga tiba di Node C? Hitung persentase kedatangan.
2. Bagaimana pengaruh jarak tiap hop terhadap RSSI dan keberhasilan relay?
3. Berapa tambahan latency karena melewati dua hop dibanding koneksi langsung satu hop? Bandingkan dengan data M04.
4. Pada hop mana loss lebih besar, dan apa penyebab yang mungkin?
5. Bandingkan relay multi-hop dengan topologi bintang M05: kapan relay lebih menguntungkan, dan apa harganya?

8.12 Concept Check

1. Apa yang dimaksud relay pada jaringan mesh?
2. Mengapa Node B harus menjalankan peran server dan client sekaligus?
3. Apa keuntungan multi-hop dibanding komunikasi langsung dari sisi jangkauan dan daya pancar?
4. Apa keterbatasan simulasi mesh ini dibanding BLE Mesh (ESP-BLE-MESH) sebenarnya? Sebut minimal tiga.
5. Apa yang terjadi pada aliran pesan bila relay mati — dan apa yang **seharusnya** terjadi pada mesh sungguhan?

8.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah hop dan mengukur loss tiap lompatan

CH-1 — Tiga hop Tambahkan Node D setelah C (A ke B ke C ke D): ubah C menjadi dual-role seperti B. Amati apakah pesan tetap utuh sampai D dan berapa tambahan latency per hop.

CH-2 — Loss per hop otomatis Beri nomor urut pada payload dan hitung loss di B dan di C secara otomatis. Contoh: B menerima 30 pesan, C hanya 27, sehingga loss hop 2 = (30 − 27)/30 × 100 % = 10 %. Bandingkan dengan loss kumulatif A ke D pada CH-1.

Level 3 — Tantangan · Jejak hop dan pemulihan rantai

CH-3 — Jejak hop Ubah relay agar menambahkan penanda pada payload (`A:5|B`) sehingga jalur yang dilalui terbaca di penerima akhir. Diskusikan: apa yang hilang dari sifat *transparent forwarding* akibat perubahan ini?

CH-4 — Self-healing sederhana Buat Node C melakukan scan ulang otomatis saat relay hilang, dan Node A menunggu relay kembali tanpa perlu reset. Ukur waktu pemulihan rantai, 5 percobaan.

8.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (hop, relay, dual role, transparent forwarding).
3. Konfigurasi — environment `nodea`, `nodeb`, `nodec`, UUID, dan interval 4 s.
4. Hasil eksperimen — log serial tiga node (EXP-01 sampai EXP-03 beserta checkpoint), termasuk pencocokan nomor pesan.
5. Data pengukuran — tabel jarak **dan** tabel per-hop, beserta hasil uji jangkauan.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

9. Modul 07 — IEEE 802.15.4 Raw Frame (P2P)

IoT COMMUNICATION LAB · MODUL 07

Speak Raw 802.15.4 — IEEE 802.15.4 Raw Frame (P2P)

ESP32-H2 · 802.15.4 · raw MAC frame · level Intermediate · 3 × 50 menit · folder kode `week07_802154_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 07 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menyusun sendiri frame MAC 802.15.4 byte demi byte dan membuatnya terbang tanpa bantuan stack apa pun. Percobaan berjalan sebagai pertukaran raw frame PING/PONG antara dua node, diamati melalui dua terminal Serial Monitor pada 115200 baud.

9.1 Pendahuluan

Enam modul pertama berjalan di atas BLE, tempat stack menyembunyikan semua detail radio. Modul ini **membuka lantai dasar**: tidak ada Zigbee, tidak ada Thread, tidak ada GATT — hanya PHY/MAC 802.15.4 dan array byte yang disusun sendiri. Setelah modul ini, setiap kali Zigbee (M08–M10) atau Thread (M11–M13) tampak "langsung bekerja", praktikan mengetahui persis apa yang sebenarnya dikerjakan protokol tersebut.

Prasyaratnya adalah M01–M06: alur build, pembacaan Serial Monitor sebagai instrumen, dan pengukuran loss. Yang dibangun di sini adalah pemahaman struktur MHR 802.15.4, pengaturan channel dan PAN ID, short address, penyusunan frame secara manual, serta penanganan callback penerimaan di konteks ISR. Semuanya dipakai lagi pada M08–M10 karena Zigbee memakai PHY/MAC yang sama, M11–M13 karena Thread pun demikian, dan M16 saat overhead antarprotokol dibandingkan di atas radio yang identik.

Peta modul — titik balik seri ini

Modul	Lapisan yang dikerjakan
01–06	BLE — stack menyembunyikan radio
07 (ini)	**802.15.4 telanjang — frame disusun sendiri**
08–10	Zigbee di atas 802.15.4 — stack mengurus join, binding, routing
11–13	Thread di atas 802.15.4 — stack mengurus IPv6, mesh, dataset

Tabel 9.1: Titik balik lapisan pada Modul 07

Kontrak data lab ini. Radio yang dipakai M07–M13 **sama persis** (IEEE 802.15.4, channel 15). Yang berbeda hanya lapisan di atasnya. Karena itu angka RSSI dan jangkauan yang diukur pada modul ini dapat dipakai sebagai garis dasar (*baseline*) saat membandingkan Zigbee dan Thread di M16.

9.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menyusun dan menerbangkan frame MAC 802.15.4 sendiri

1. Menyusun frame MAC 802.15.4 lengkap (Len ditambah MHR 11 byte ditambah payload) dan menunjukkan letak tiap field pada dump heksadesimal frame yang benar-benar tertangkap.
2. Menjelaskan tiga aturan API `esp_ieee802154_*` yang wajib dipatuhi (panjang `Len` termasuk FCS, `esp_ieee802154_receive()` di `setup()`, buffer TX harus `static`) beserta gejala kegagalannya.
3. Menjalankan pertukaran PING/PONG dua arah dan menghitung packet loss tiap arah secara terpisah.
4. Membuktikan pengaruh channel dan PAN ID terhadap keterhubungan dengan mengubahnya dan mengamati akibatnya.
5. Membuktikan bahwa penyaringan PAN ID dan alamat tujuan hanya aktif bila promiscuous mode dimatikan dengan `esp_ieee802154_set_promiscuous(false)`.

Kriteria keberhasilan

- ☐ Setiap `PING n` dibalas `PONG n` pada jarak dekat, isi payload utuh (bukan karakter acak).
- ☐ Komunikasi terbukti berhenti saat channel salah satu node dibedakan.
- ☐ PAN ID berbeda terbukti tersaring hanya bila promiscuous mode dimatikan (EXP-05).
- ☐ Loss tiap arah terukur terpisah (PING hilang dibanding PONG hilang).
- ☐ Dump frame heksadesimal dianalisis dan tiap field ditunjuk.

9.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (modulasi O-QPSK, CSMA-CA, superframe, beacon) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 9.2.

Istilah	Definisi kerja di lab ini
IEEE 802.15.4	Standar PHY/MAC nirkabel low-rate low-power; basis Zigbee dan Thread.
Channel	Frekuensi kerja radio (di sini 15); kedua node harus sama.
PAN ID	Identitas jaringan (`0xCAFE`); frame di luar PAN disaring hardware — **hanya bila promiscuous mode dimatikan**.
Promiscuous mode	Mode "dengar semua": filter PAN ID dan alamat tujuan di MAC dimatikan, setiap frame dengan FCS valid diteruskan ke callback. Driver ESP-IDF menyalakannya **secara default**, sehingga kode memanggil `esp_ieee802154_set_promiscuous(false)` di `setup()`.
Short address	Alamat 16-bit node (`0x0001` atau `0x0002`).
Frame / MHR	Header MAC `[Len][FC(2)][Seq(1)][DestPAN(2)][DestAddr(2)][SrcPAN(2)][SrcAddr(2)][payload]`.
Alamat pengirim	Dibaca callback dari `SrcAddr` (`frame[10..11]`, little-endian) — setara info alamat asal pada frame RX mode API XBee. Log `RX dari 0x....` mencetak alamat ini; bila frame bukan untuk node tersebut (broadcast atau promiscuous), tujuan dan PAN-nya ikut dicetak: `RX dari 0x0001 (ke 0x0002, PAN 0xCAFE)`. Node2 dan Node3 membalas ke alamat ini dan **hanya membalas PING**.
RSSI / LQI	Disalin callback dari `frame_info->rssi` dan `frame_info->lqi`, dicetak di akhir baris RX: `RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]`. RSSI (dBm) sudah dikoreksi driver ESP-IDF. LQI adalah nilai mentah hardware ESP32-H2 dengan skala khusus chip (teramati 6–11), **tidak** sebanding dengan LQI 0–255 radio lain — untuk pengukuran jarak pakai RSSI.
FCS	Checksum 2 byte; **isinya** dihitung hardware, tetapi **panjangnya tetap ikut** pada byte `Len`.

Istilah	Definisi kerja di lab ini
RX when idle	Radio kembali ke RX setiap selesai TX atau RX — bukan pengganti `esp_ieee802154_receive()`.
ISR	`esp_ieee802154_receive_done()` berjalan di konteks interupsi: salin data, set flag, jangan mencetak.

Tabel 9.2: Istilah kerja Modul 07

Tiga jebakan API `esp_ieee802154_*`. Ketiganya nyata dan ditemukan saat modul ini diuji di perangkat. Kode yang disediakan sudah memperhitungkannya — yang dituntut di sini adalah kemampuan menjelaskan gejalanya (Tabel 9.3).

Aturan	Gejala bila dilanggar
`Len` = MHR + payload + 2 (FCS)	Frame ditolak atau payload terpotong di penerima
`esp_ieee802154_receive()` wajib dipanggil di `setup()`	TX jalan, tetapi tidak ada satu pun frame diterima
Buffer TX harus tetap hidup sampai transmisi selesai (`static`, bukan variabel lokal)	Payload sesekali berubah menjadi sampah atau isi RAM lama

Tabel 9.3: Tiga aturan API `esp_ieee802154_*` dan gejala pelanggarannya

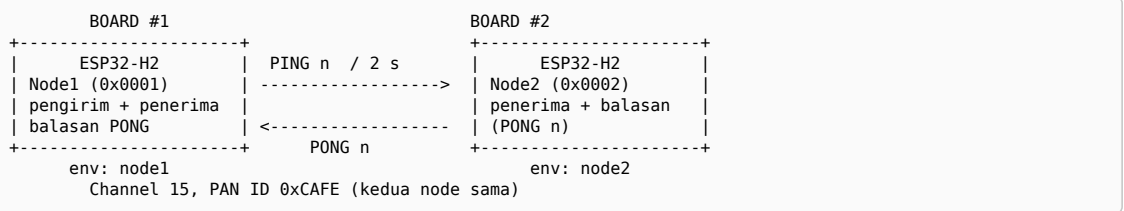
Aturan ketiga paling menipu: `esp_ieee802154_transmit()` bersifat **asinkron** dan hanya menyimpan *pointer* ke buffer. Bila buffer dideklarasikan sebagai variabel lokal di `loop()`, isinya sudah tertimpa stack frame lain saat radio benar-benar mengirim — sebagian frame berangkat berisi sampah, sebagian lain kebetulan masih utuh. Itulah mengapa gejalanya *intermiten*, bukan gagal total.

Catatan — filter PAN tidak aktif dengan sendirinya. `esp_ieee802154_set_panid()` dan `esp_ieee802154_set_short_address()` hanya mengisi register; nilai itu baru dipakai untuk menyaring bila promiscuous mode mati. Karena driver ESP-IDF menyalakan promiscuous secara default, tanpa `esp_ieee802154_set_promiscuous(false)` dua node dengan PAN ID berbeda tetap bisa saling bertukar pesan, dan node lain ikut menerima unicast yang bukan untuknya. Perilaku ini berbeda dengan XBee, yang firmware-nya selalu menyaring PAN ID dan alamat tujuan. Pengaruh baris ini dibuktikan pada EXP-05.

Sekuens protokol yang diamati

```
buildFrame()                        radio 802.15.4
[Len][FC][Seq][DPAN][DA][SPAN][SA][payload]
       |                                  |
       ▼                                  ▼
 esp_ieee802154_transmit() ─► udara ─► esp_ieee802154_receive_done() (ISR)
                                                  |
                                 flag hasRx ──► loop(): cetak "RX dari ..."
```

9.4 Topologi



Node	Board	Environment	Short addr	Aksi
Node1	ESP32-H2 DevKitM-1	`node1`	`0x0001`	TX `PING n` tiap 2 s, RX balasan
Node2	ESP32-H2 DevKitM-1	`node2`	`0x0002`	RX `PING n`, TX `PONG n`

Tabel 9.4: Peran tiap node Modul 07

Radio 802.15.4 dipakai **telanjang** (tanpa Zigbee atau Thread) di dua **ESP32-H2 DevKitM-1**. ESP32-C6 juga punya radio 802.15.4 dan bisa dipakai, tetapi lab ini menyimpannya untuk peran gateway Wi-Fi di Modul 13.

9.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan API ESP-IDF `esp_ieee802154.h`.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (API `esp_ieee802154.h` bawaan)	—

Tabel 9.5: Alat dan bahan Modul 07

Tidak ada library eksternal — modul ini memanggil API ESP-IDF langsung.

Radio config

Parameter	Nilai
Channel	15
PAN ID	`0xCAFE`
Short address	Node1 `0x0001`, Node2 `0x0002`
Interval PING	2000 ms

Tabel 9.6: Konfigurasi radio Modul 07

9.6 Kode Program

Pin port agar tidak salah flash (Kode 9.1).

```
[env:node1]
build_src_filter = +<node1/*.cpp>
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:node2]
build_src_filter = +<node2/*.cpp>
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2

[env:node3]
build_src_filter = +<node3/*.cpp>
; port diisi sesuai board ketiga (opsional, untuk EXP-04-e broadcast)
```

Kode 9.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>

[env:node3]
build_src_filter = +<node3/*.cpp>
```

Kode 9.2: `platformio.ini` Modul 07 pada repositori

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node1 = pengirim (short addr 0x0001)
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0001
#define PEER_ADDR 0x0002
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

// Buffer hasil terima (diisi ISR, dibaca loop)
static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

// Susun frame: [Len][FC(2)][Seq(1)][DestPAN(2)][DestAddr(2)][SrcPAN(2)][SrcAddr(2)][payload]
// FCS dihitung hardware otomatis, tetapi 2 byte-nya tetap ikut dihitung pada "Len".
static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = 11;                       // 2+1+2+2+2+2
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;                         // byte "Len" (PHR), termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;       // Frame Control (data, addr 16-bit)
  frame[3] = 0x00;                        // sequence number
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;       // dest PAN
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;          // dest addr
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;       // src PAN
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;   // src addr
  memcpy(&frame[12], payload, plen);
  return 1 + len;  // total byte dalam buffer
}

// ISR: frame diterima. [Len]...[MHR]...[payload][RSSI][LQI]
void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
```

```
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node1 (802.15.4 sender) starting...");

  uint8_t ext[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    printRx(rxSrc, rxDst, rxDstPan);
  }

  static unsigned long last = 0;
  static uint32_t count = 0;
  if (millis() - last > 2000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "PING %lu", (unsigned long)++count);
    // static: driver 802.15.4 mengirim secara asinkron, buffer harus tetap
    // hidup sampai transmisi selesai (buffer di stack akan tertimpa)
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, PEER_ADDR, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX ke 0x%04X: %s\n", PEER_ADDR, msg);
  }
}
```

Kode 9.3: `src/node1/main.cpp` — pengirim PING `0x0001`

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node2 = penerima (short addr 0x0002)
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0002
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = MHR_LEN;
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;   // byte Len termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;
  frame[3] = 0x00;
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;
  memcpy(&frame[12], payload, plen);
  return 1 + len;
```

```
}

void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node2 (802.15.4 receiver) starting...");

  uint8_t ext[] = {0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    uint16_t src = rxSrc;
    printRx(src, rxDst, rxDstPan);

    // Hanya PING yang dibalas; PONG milik node lain (tertangkap saat
    // promiscuous) diabaikan agar tidak terjadi balas-membalas tanpa henti
    if (strncmp(rxPayload, "PING ", 5) != 0) return;

    // Balas PONG ke alamat pengirim sebenarnya
    char msg[32];
    snprintf(msg, sizeof(msg), "PONG %s", rxPayload + 5);  // hilangkan "PING "
    // static: driver 802.15.4 mengirim secara asinkron, buffer harus tetap
    // hidup sampai transmisi selesai (buffer di stack akan tertimpa)
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, src, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX balasan ke 0x%04X: %s\n", src, msg);
  }
}
```

Kode 9.4: `src/node2/main.cpp` — penerima dan pembalas PONG `0x0002`

Berkas `src/node3/main.cpp` (Kode 9.5) adalah node ketiga bershort address `0x0003` yang dipakai pada uji broadcast EXP-04-e. Node2 dan Node3 tidak memakai `PEER_ADDR`: keduanya membalas ke alamat pengirim yang dibaca dari `SrcAddr` frame, dan hanya membalas frame berisi PING.

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node3 = penerima tambahan (short addr 0x0003)
// Dipakai untuk uji broadcast (EXP-04-e): menerima frame yang dikirim ke 0xFFFF
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0003
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = MHR_LEN;
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;   // byte Len termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;
  frame[3] = 0x00;
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;
  memcpy(&frame[12], payload, plen);
  return 1 + len;
}

void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node3 (802.15.4 receiver) starting...");

  uint8_t ext[] = {0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    uint16_t src = rxSrc;
```

```
    printRx(src, rxDst, rxDstPan);

    // Hanya PING yang dibalas; PONG milik node lain (tertangkap saat
    // promiscuous) diabaikan agar tidak terjadi balas-membalas tanpa henti
    if (strncmp(rxPayload, "PING ", 5) != 0) return;

    // Balas PONG ke alamat pengirim sebenarnya
    char msg[32];
    snprintf(msg, sizeof(msg), "PONG %s", rxPayload + 5);  // hilangkan "PING "
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, src, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX balasan ke 0x%04X: %s\n", src, msg);
  }
}
```

Kode 9.5: `src/node3/main.cpp` — node ketiga `0x0003` untuk uji broadcast

9.7 Build dan Flash

```
pio device list
pio run -d week07_802154_p2p -e node1 -t upload
pio run -d week07_802154_p2p -e node2 -t upload -t monitor
```

Memantau semua node dari satu komputer. `pio device monitor` hanya membuka satu port. Skrip `monitor_serial.py` (ada di folder `week07_802154_p2p` pada repositori) membuka semua port UART CH343 sekaligus, menampilkan ketiga node dalam satu jendela dengan timestamp bersama, lalu mencetak ringkasan per link (terkirim, diterima, loss, RSSI, LQI) saat berhenti.

```
python week07_802154_p2p/monitor_serial.py                    # deteksi port otomatis
python week07_802154_p2p/monitor_serial.py --duration 120 --log sesi1.txt
python week07_802154_p2p/monitor_serial.py --port COM5 --port COM11 --port COM13
python3 week07_802154_p2p/monitor_serial.py --port /dev/ttyACM0 --port /dev/ttyACM2
```

Nama node dikenali dari banner saat boot, jadi urutan port tidak perlu diingat. Tiap board di-reset sekali saat port dibuka agar hitungan PING dimulai dari 1 (pakai `--no-reset` untuk mengamati tanpa reset). Tutup dulu `pio device monitor` — satu port tidak bisa dibuka dua program sekaligus. Contoh ringkasan pada baseline:

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING       8      8    0.0%    -11.0 (-11..-11)
Node2(0x0002) -> Node1(0x0001)   PONG       8      8    0.0%    -12.0 (-12..-12)
```

PENTING

Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, dua port dicatat dan diisikan di atas.
- ☐ Dua ESP32-H2 terpasang dan terdeteksi.
- ☐ Environment `node1` dan `node2` dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud untuk kedua node.
- ☐ Tidak ada firmware Zigbee atau Thread yang masih berjalan di board yang sama (radio 802.15.4 dipakai langsung).

9.8 Percobaan

9.8.1 EXP-01 — Konfigurasi Radio dan Anatomi Frame

Unggah kedua firmware, verifikasi konfigurasi radio pada baris pertama Serial Monitor, lalu telusuri fungsi `buildFrame()`.

```
frame[0]=Len | [1..2]=FC 0x8801 | [3]=Seq | [4..5]=PAN 0xCAFE
[6..7]=DestAddr | [8..9]=PAN | [10..11]=SrcAddr | [12..]=payload

Len = 11 (MHR) + panjang payload + 2 (FCS)
```

Contoh frame `PING 38` yang benar-benar tertangkap di udara (dump `receive_done` pada ESP32-H2; dua byte terakhir adalah RSSI dan LQI yang menggantikan FCS saat penerimaan).

```
14 01 88 00 FE CA 02 00 FE CA 01 00 50 49 4E 47 20 33 38 E5 0A
|  |     |  |     |     |     |     "P  I  N  G  _  3  8" |  |
|  FC    |  PAN   Dest  PAN   Src                          |  LQI
Len=0x14 Seq      0x0002      0x0001                       RSSI
```

Data capture

Parameter	Hasil
Channel	
PAN ID	
Short address Node1	
Short address Node2	
Ukuran MHR (byte)	
`Len` untuk payload 7 karakter	

Tabel 9.7: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Hitung sendiri nilai `Len` untuk payload `"PING 38"` (7 karakter) dan cocokkan dengan `0x14` pada dump di atas. Lalu jawab: jika FCS dihitung hardware, mengapa panjangnya tetap harus disertakan? Petunjuk: `Len` adalah field PHY, dan PHY menghitung **semua** byte yang mengudara.

CHECKPOINT

Kedua node mencetak baris `Channel 15, PAN 0xCAFE, short addr 0x000X` dengan nilai yang berbeda untuk tiap node. Jika keduanya mencetak alamat yang sama, berarti terjadi kesalahan flash — periksa `upload_port`.

9.8.2 EXP-02 — Pertukaran PING dan PONG

Node1 mengirim `PING n` tiap 2 s; Node2 menerima (callback `esp_ieee802154_receive_done` menyalin payload dan menyetel flag, `loop()` mencetak), lalu membalas `PONG n`; Node1 menerima balasan itu.

```
Node1 loop (tiap 2 s)            Node2 receive_done
TX "PING n" ──────────────────►  RX dari 0x0001: PING n
RX dari 0x0002: PONG n  ◄──────  TX balasan "PONG n"
```

Expected output — Node1

```
Node1 (802.15.4 sender) starting...
Channel 15, PAN 0xCAFE, short addr 0x0001
TX ke 0x0002: PING 1
RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]
TX ke 0x0002: PING 2
RX dari 0x0002: PONG 2  [RSSI -12 dBm, LQI 11]
```

Expected output — Node2

```
Node2 (802.15.4 receiver) starting...
Channel 15, PAN 0xCAFE, short addr 0x0002
RX dari 0x0001: PING 1  [RSSI -11 dBm, LQI 10]
TX balasan ke 0x0001: PONG 1
```

Nilai RSSI dan LQI bergantung pada jarak dan posisi board; angka di atas diambil dengan board berdekatan di meja.

CHECKPOINT

Isi payload harus **terbaca sebagai teks**, bukan karakter acak. Jika muncul sampah tak terbaca, itu gejala buffer TX bukan `static` (lihat bagian Dasar Teori). Jika Node2 hanya mencetak baris konfigurasi dan tidak pernah `RX`, `esp_ieee802154_receive()` tidak dipanggil. Perbaiki dulu, jangan lanjut mengukur.

9.8.3 EXP-03 — Isolasi Channel dan RTT

1. Ubah jarak antar node secara bertahap.
2. Ubah `CHANNEL` pada **salah satu** node (perlu unggah ulang) sehingga berbeda, lalu verifikasi komunikasi berhenti total.
3. Kembalikan ke channel yang sama, lalu ubah `PAN_ID` salah satu node — amati apakah gejalanya sama atau berbeda dengan kasus channel.
4. Ukur round-trip kasar PING ke PONG dari waktu di Serial Monitor.

Data capture

Parameter	Hasil
Perilaku saat channel berbeda	
Perilaku saat PAN ID berbeda	
Round-trip PING ke PONG (kasar)	
Interval kirim PING (s)	
Pesan per menit yang dibalas	

Tabel 9.8: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan **perbedaan** antara gagal karena channel dan gagal karena PAN ID (petunjuk: yang satu radio tidak mendengar sama sekali, yang satu mendengar tetapi menyaring). Ini penting untuk memahami penyaringan Zigbee dan Thread di modul berikutnya.

9.8.4 EXP-04 — Efek Parameter: PAN ID, Channel, Broadcast

Semua efek di bawah cukup dicapai dengan **mengubah `#define` di `src/nodeX/main.cpp`** — tidak ada logika yang diubah. Tiap ubah parameter, unggah ulang node yang bersangkutan.

Uji	`node1`	`node2`	Gejala yang harus terlihat
04-a PAN sama	`PAN_ID 0xCAFE`	`PAN_ID 0xCAFE`	PING–PONG normal (baseline)
04-b PAN beda	`PAN_ID 0xCAFE`	`PAN_ID 0xBEEF`	Node1 tetap cetak `TX`, Node2 **tidak pernah** cetak `RX` — frame *heard but filtered* (disaring hardware MAC)
04-c Channel sama	`CHANNEL 15`	`CHANNEL 15`	PING–PONG normal
04-d Channel beda	`CHANNEL 15`	`CHANNEL 20`	Kedua node sunyi di sisi RX — radio *never heard* (tuli, tidak mendengar sama sekali)
04-e Broadcast	`PEER_ADDR 0xFFFF`	tidak diubah	Node2 **dan** Node3 menerima frame yang sama (`RX dari 0x0001 (ke 0xFFFF, ...)`); keduanya membalas bersamaan sehingga Node1 hanya menerima PONG dari **salah satu** node

Tabel 9.9: Matriks uji efek parameter EXP-04

Perbedaan 04-b dengan 04-d adalah inti yang harus bisa dijelaskan.

PAN ID beda (04-b) radio memang menerima transmisi di channel yang sama, tetapi MAC hardware **menyaring frame** dengan PAN ID asing sebelum sampai ke callback (berlaku karena `setup()` mematikan promiscuous mode; lihat EXP-05). TX di sisi lain tetap berjalan normal — komunikasi terlihat "searah hilang".

Channel beda (04-d) radio tidak berada di frekuensi yang sama, sehingga tidak ada yang diterima secara fisik. Tidak ada filter yang "menolak" karena memang tidak ada yang masuk.

Uji broadcast (04-e) — wajib 3 node. Firmware node ketiga sudah tersedia pada `src/node3` (Kode 9.5) dengan `MY_ADDR 0x0003`; tambahkan `[env:node3]` di `platformio.ini`, lalu set `PEER_ADDR` Node1 menjadi `0xFFFF`. Amati bahwa Node2 dan Node3 menerima frame yang sama, lalu diskusikan apa yang hilang dibanding unicast (tidak ada ACK, tidak ada penyaringan alamat tujuan).

Perhatikan pula **dari alamat mana** PONG di Node1 berasal. Pada uji di perangkat (25 detik, `monitor_serial.py`), Node2 dan Node3 masing-masing menerima 12/12 PING dan membalas semuanya, tetapi seluruh 12 PONG yang dicetak Node1 adalah **`RX dari 0x0003`**. Kedua penerima membalas pada saat yang hampir sama, CCA keduanya melihat channel kosong, dan kedua PONG bertabrakan di udara; Node1 hanya menangkap frame yang lebih kuat (*capture effect*) dan tidak ada ACK yang memberi tahu Node2 bahwa PONG-nya hilang. RSSI menjelaskan siapa yang menang: di Node1, sinyal Node3 sekitar −4 dBm, sedangkan sinyal Node2 sekitar −12 dBm. Tanpa pembacaan `SrcAddr`, kehilangan ini tidak terlihat sama sekali.

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING      12     12    0.0%    -12.0 (-12..-12)
Node1(0x0001) -> Node3(0x0003)   PING      12     12    0.0%     -5.4 (-6..-5)
Node2(0x0002) -> Node1(0x0001)   PONG      12      0  100.0%   -
Node3(0x0003) -> Node1(0x0001)   PONG      12     12    0.0%     -3.8 (-6..-2)
```

Data capture — tabel gejala lintas kelompok

Uji	Konfigurasi	TX Node1?	RX Node2?	Loss	Gejala yang diamati
04-a	PAN sama				
04-b	PAN beda				
04-c	Channel sama				

Uji	Konfigurasi	TX Node1?	RX Node2?	Loss	Gejala yang diamati
04-d	Channel beda				
04-e	Broadcast (3 node)				

Tabel 9.10: Lembar pengamatan EXP-04

CHECKPOINT

Praktikan dapat membedakan 04-b dari 04-d tanpa melihat kode, hanya dari pola log: 04-b ada TX tanpa RX di pasangan; 04-d kedua sisi sunyi. Jika sebuah kelompok di ruangan sama menyalakan radio 802.15.4 di channel yang sama, gejala yang muncul mirip 04-b — itu sebabnya tiap kelompok wajib memakai channel berbeda.

9.8.5 EXP-05 — Promiscuous Mode: Ada dan Tidaknya Filter MAC

EXP-04 menunjukkan bahwa node dengan PAN ID berbeda tidak saling menerima. Penyaringan itu **tidak** terjadi dengan sendirinya: driver ESP-IDF menyalakan promiscuous mode secara default, dan filter PAN ID serta alamat tujuan baru aktif karena `setup()` memanggil satu baris berikut sebelum `esp_ieee802154_set_rx_when_idle(true)`:

```
esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver: true)
```

Percobaan ini membuktikan pengaruh baris tersebut. Seperti EXP-04, yang diubah hanya satu baris di `setup()`; tiap perubahan diikuti unggah ulang node yang bersangkutan. Channel semua node tetap 15.

Uji	PAN (Node1 / Node2)	Baris promiscuous	Gejala yang harus terlihat
05-a	`0xCAFE` / `0xBEEF`	`false` di semua node (kode asli)	Node1 tetap cetak `TX`, Node2 **tidak pernah** cetak `RX` — sama dengan 04-b
05-b	`0xCAFE` / `0xBEEF`	Node2: `true`	Node2 cetak `RX dari 0x0001 (ke 0x0002, PAN 0xCAFE)` untuk setiap PING dan membalas PONG, tetapi Node1 **tidak pernah** cetak `RX`
05-c	`0xCAFE` / `0xBEEF`	Node2: baris **dihapus**	Sama dengan 05-b — tanpa baris ini driver memakai nilai default `true`
05-d	`0xCAFE` / `0xCAFE`	Node3: `true` (Node1, Node2: `false`)	Node3 **ikut** cetak `RX dari 0x0001 (ke 0x0002, ...)` dan membalas; PONG Node2 bertabrakan dengan PONG Node3, sehingga Node1 hanya menerima PONG dari `0x0003` — atau tidak sama sekali

Tabel 9.11: Matriks uji promiscuous mode EXP-05

Ada tiga hal yang harus bisa dijelaskan dari percobaan ini.

Filter bekerja di hardware (05-a vs 05-b) kode callback `esp_ieee802154_receive_done()` memang membaca header, tetapi tidak menyaring berdasarkan PAN ID atau alamat tujuan, sehingga satu-satunya yang membedakan 05-a dan 05-b adalah filter MAC. Begitu promiscuous dinyalakan, PING dari PAN asing langsung sampai ke callback.

Penyaringan terjadi di sisi penerima (05-b) PONG dari Node2 membawa Dest PAN `0xBEEF`, sementara Node1 masih memakai `false` sehingga PONG itu dibuang. Komunikasi hanya "tembus" ke arah node yang promiscuous.

Filter alamat tujuan ikut mati (05-d) promiscuous tidak hanya mengabaikan PAN ID, tetapi juga alamat tujuan. Node3 menerima PING yang ditujukan ke `0x0002` **dan** PONG yang ditujukan ke `0x0001` — keduanya bukan untuknya, padahal Node1 tidak mengirim ke `0xFFFF`. Node3 mencetak `RX dari 0x0001 (ke 0x0002, ...)` lalu ikut membalas PING itu, sehingga PONG-nya bertabrakan dengan PONG Node2. Pada uji 25 detik (`monitor_serial.py`): 12 PING, Node3 menangkap 12/12, dan Node1 hanya menerima 5 PONG — **seluruhnya dari `0x0003`** (RSSI sekitar −3 dBm, jauh di atas sinyal Node2 sekitar −12 dBm); 7 PING lainnya tidak terbalas sama sekali karena kedua PONG rusak. Buffer penerima hanya satu slot, jadi kadang PING di Node3 sudah tertimpa PONG Node2 sebelum sempat dibaca; Node3 lalu mencetak `RX dari 0x0002 (ke 0x0001, ...)`, tidak membalas (bukan PING), dan PONG Node2 lolos ke Node1.

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING      12     12    0.0%    -11.0 (-11..-11)
Node1(0x0001) -> Node3(0x0003)   PING       -     12     n/a     -5.0 (-5..-5)
Node2(0x0002) -> Node1(0x0001)   PONG      12      0  100.0%   -
Node3(0x0003) -> Node1(0x0001)   PONG      12      5   58.3%    -3.2 (-4..-2)
```

Bandingkan dengan XBee: firmware XBee selalu menyaring PAN ID dan alamat tujuan, sehingga perilakunya setara dengan 05-a tanpa perlu diatur. Pada ESP32-H2 yang memakai radio 802.15.4 secara langsung, filter itu harus dinyalakan sendiri.

Catatan. Node2 dan Node3 hanya membalas PING, bukan PONG. Tanpa aturan ini, node yang promiscuous akan membalas PONG milik node lain dan penerimanya membalas balik — terjadi *reply storm* (Serial Monitor dibanjiri puluhan `RX`/`TX` per detik). Dengan aturan ini, menyalakan promiscuous pada Node2 **dan** Node3 bersamaan hanya menghasilkan gejala seperti 04-e: keduanya membalas tiap PING dan Node1 hanya menerima PONG dari salah satunya. Kembalikan semua node ke `false` dan `PAN_ID 0xCAFE` setelah selesai.

Data capture

Uji	Konfigurasi	RX Node1?	RX Node2?	RX Node3?	Gejala yang diamati
05-a	PAN beda, `false`				
05-b	PAN beda, Node2 `true`				
05-c	PAN beda, Node2 tanpa baris				
05-d	PAN sama, Node3 `true`				

Tabel 9.12: Lembar pengamatan EXP-05

CHECKPOINT

Praktikan dapat menjelaskan mengapa 05-b menghasilkan komunikasi **satu arah** (Node2 menerima PING, Node1 tidak menerima PONG), dan mengapa 05-c sama dengan 05-b. Jawaban yang benar menyebut bahwa `esp_ieee802154_set_panid()` hanya mengisi register, sedangkan penyaringan baru aktif bila promiscuous dimatikan. Praktikan juga dapat menjelaskan, dari alamat pengirim dan RSSI di log, mengapa PONG di Node1 pada 05-d berasal dari `0x0003` dan mengapa sebagian PING tidak terbalas sama sekali.

9.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (Node3 menyala tetapi diam), board berdekatan di meja, direkam 17,8 detik dengan `monitor_serial.py`. Baris dengan timestamp sama diurutkan menurut alur protokol.

```
[  0.512] Node1 | Channel 15, PAN 0xCAFE, short addr 0x0001
[  0.512] Node2 | Channel 15, PAN 0xCAFE, short addr 0x0002
[  2.483] Node1 | TX ke 0x0002: PING 1
[  2.483] Node2 | RX dari 0x0001: PING 1  [RSSI -11 dBm, LQI 10]
[  2.483] Node2 | TX balasan ke 0x0001: PONG 1
[  2.483] Node1 | RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]
[  4.498] Node1 | TX ke 0x0002: PING 2
[  4.498] Node2 | RX dari 0x0001: PING 2  [RSSI -11 dBm, LQI 11]
[  4.498] Node2 | TX balasan ke 0x0001: PONG 2
[  4.498] Node1 | RX dari 0x0002: PONG 2  [RSSI -12 dBm, LQI 11]
```

Parameter	Hasil terukur
PING dikirim / diterima Node2	8 / 8
PONG dikirim / diterima Node1	8 / 8
RSSI PING di Node2 / PONG di Node1	−11 dBm / −12 dBm
LQI (mentah hardware)	9–11
Round-trip PING ke PONG	di bawah resolusi timestamp (PING dan PONG tercetak pada milidetik yang sama)

Tabel 9.13: Hasil verifikasi hardware Modul 07

9.10 Pengukuran

Jarak	RSSI (dBm)	Latency RTT (kasar)	Success PONG (%)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 9.14: Lembar pengukuran jarak Modul 07

Pengukuran per-node (pengamatan 2 menit) — pisahkan dua arah.

Node	TX	RX	Loss (%)
Node1 (`0x0001`)			
Node2 (`0x0002`)			

Tabel 9.15: Lembar pengukuran per-node Modul 07

RSSI sudah dicetak di setiap baris RX (disalin dari `frame_info->rssi` di `receive_done`, dicetak di `loop()` — tidak di ISR). Cara termudah mengisi tabel di atas: jalankan `monitor_serial.py --duration 120` pada tiap jarak, lalu ambil RSSI rata-rata (min..max) dan loss per arah dari ringkasannya.

Baseline untuk M16. Catat jarak maksimum yang masih 100 % berhasil pada modul ini. Angka itu adalah jangkauan radio 802.15.4 **tanpa** bantuan mesh — pembanding langsung untuk Zigbee (M10) dan Thread (M12).

9.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana pengaruh jarak terhadap RSSI dan persentase PING yang dibalas PONG?
2. Pada jarak berapa komunikasi mulai gagal, dan apa indikasinya di Serial Monitor?
3. Apakah round-trip latency bertambah signifikan dengan jarak? Mengapa?
4. Berapa packet loss tiap arah (PING hilang dibanding PONG hilang)? Adakah asimetri, dan apa dugaan penyebabnya?
5. Mengapa channel dan PAN ID harus sama, dan apa hubungan 802.15.4 dengan Zigbee pada modul berikutnya?

9.12 Concept Check

1. Apa fungsi field Frame Control dan sequence number pada frame 802.15.4?
2. Apa perbedaan short address dan extended address, dan kapan masing-masing dipakai?
3. Mengapa FCS tidak perlu dihitung di perangkat lunak, tetapi panjangnya tetap harus dihitung?
4. Mengapa callback penerimaan tidak boleh mencetak langsung ke Serial (konteks ISR)?
5. Apa yang terjadi bila dua node memakai PAN ID berbeda — di lapisan mana penyaringan itu terjadi?

9.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Sequence number dan ukuran payload

CH-1 — Sequence number nyata (wajib) Field `frame[3]` saat ini selalu `0`. Isi dengan nomor urut yang naik tiap kirim seperti Kode 9.6, lalu hitung packet loss dari sisi penerima. Contoh: 60 PING dikirim, 57 PONG diterima, sehingga loss = (60 − 57)/60 × 100 % = 5 %.

CH-2 — Pengaruh ukuran payload Perbesar payload menjadi 40 byte teks dan bandingkan success rate terhadap payload pendek pada jarak yang sama. Jelaskan hasilnya dari sisi peluang bit error per frame.

```
// buildFrame(): ganti frame[3] = 0;
static uint8_t seq = 0;
frame[3] = seq++;
```

Kode 9.6: Kerangka CH-1 — sequence number pada MHR

Level 3 — Tantangan · RSSI, jarak, dan broadcast

CH-3 — RSSI terhadap jarak dan *capture effect* (a) Dengan `monitor_serial.py --duration 120`, catat RSSI rata-rata (min..max) kedua arah pada kelima jarak di tabel Pengukuran, lalu buat grafik RSSI terhadap jarak. (b) Ulangi uji broadcast 04-e dua kali: sekali dengan Node3 lebih dekat ke Node1 daripada Node2, sekali sebaliknya. Sebelum menjalankan, **prediksi** PONG siapa yang akan diterima Node1 berdasarkan selisih RSSI, lalu buktikan dengan ringkasan monitor. Berapa selisih RSSI minimum agar PONG yang lebih kuat tetap lolos?

CH-4 — Broadcast Ubah `DestAddr` menjadi `0xFFFF` (broadcast) dan tambahkan node ketiga. Amati apakah kedua penerima menerima frame yang sama, dan diskusikan apa yang hilang (tidak ada ACK, tidak ada penyaringan alamat).

9.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (802.15.4, MHR, channel, PAN ID, short address, tiga aturan API).
3. Konfigurasi — environment `node1` dan `node2`, channel 15, PAN `0xCAFE`, interval 2 s.
4. Hasil eksperimen — log PING–PONG kedua node beserta analisis dump frame heksadesimal.
5. Data pengukuran — tabel jarak, tabel per-node, dan baseline jangkauan untuk M16.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

10. Modul 08 — Zigbee P2P: Join dan Binding

IoT COMMUNICATION LAB · MODUL 08

Join a Zigbee Network — Zigbee P2P: Join dan Binding

ESP32-H2 · Zigbee · ZC ke ZED · level Advanced · 3 × 50 menit · folder kode `week08_zigbee_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 08 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya membentuk jaringan Zigbee, memasukkan satu end device ke dalamnya, dan mengendalikannya lewat binding. Percobaan berjalan sebagai Zigbee P2P antara coordinator yang berperan sebagai switch dan end device yang berperan sebagai lampu, diamati melalui dua terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

10.1 Pendahuluan

Pada M07 frame disusun sendiri: tidak ada identitas jaringan, tidak ada keamanan, tidak ada penemuan perangkat. Zigbee menambahkan ketiganya di atas radio 802.15.4 yang **sama persis**. Yang menarik justru harga yang harus dibayar: firmware jauh lebih besar (butuh tabel partisi khusus), dan perangkat menyimpan keanggotaan jaringan di NVS — sesuatu yang tidak pernah jadi masalah di modul-modul BLE.

Prasyaratnya adalah M07: channel, PAN ID, dan pemahaman bahwa Zigbee berjalan di radio 802.15.4 yang sama. Yang dibangun di sini adalah pembentukan PAN, window join, prosedur find-and-bind, cluster ON/OFF, perbedaan mode build ZCZR dan ED, tabel partisi khusus Zigbee, serta penyimpanan keanggotaan jaringan di NVS. Semuanya dipakai lagi pada M09 ketika satu coordinator melayani banyak end device melalui binding table, M10 ketika router menambah hop, dan M16 ketika Zigbee menjadi salah satu protokol yang dibandingkan.

Peta modul blok Zigbee

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang — frame disusun manual
08 (ini)	**Jaringan Zigbee terbentuk: join, binding, cluster ON/OFF**
09	Satu coordinator melayani banyak end device (binding table)
10	Router menambah hop — routing multi-hop otomatis

Tabel 10.1: Peta modul blok Zigbee

Kontrak data lab ini. Zigbee tidak mengirim "string", melainkan **perintah cluster standar** (`On/Off`). Perbedaan ini penting untuk M16: BLE dan Thread di lab ini mengirim payload bebas, Zigbee mengirim perintah baku. Catat konsekuensinya pada interoperabilitas — perangkat Zigbee merek berbeda bisa saling mengerti, payload BLE buatan sendiri tidak.

10.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membentuk jaringan Zigbee dan mengendalikan lewat binding

1. Membentuk jaringan Zigbee dengan satu coordinator dan memasukkan satu end device melalui window join 180 detik, dibuktikan dari log kedua board.
2. Menjelaskan perbedaan **join** dan **binding** dengan menunjuk baris log yang menandai masing-masing tahap.
3. Mengukur waktu join dan binding serta latency perintah ON/OFF (coordinator sampai LED berubah di end device) pada minimal 4 jarak.
4. Menjelaskan mengapa dua peran membutuhkan `build_flags` dan tabel partisi berbeda, serta akibatnya bila tertukar.

Kriteria keberhasilan

- ☐ Coordinator mencetak `End device ter-binding!`.
- ☐ LED RGB pada end device berubah ON/OFF tiap 5 detik mengikuti perintah.
- ☐ Waktu join dan binding tercatat, minimal 3 percobaan.
- ☐ Tabel jarak–latency–success terisi dari pengukuran sendiri.

10.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (ZDO, APS layer, key management, Zigbee Cluster Library lengkap) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 10.2.

Istilah	Definisi kerja di lab ini
Zigbee	Protokol jaringan di atas IEEE 802.15.4 untuk otomasi; dipakai lewat library `Zigbee` Arduino core 3.x.
Coordinator (ZC)	Membentuk dan mengelola PAN; build flag `-DZIGBEE_MODE_ZCZR`.
End Device (ZED)	Node akhir hemat daya, tidak meneruskan trafik; build flag `-DZIGBEE_MODE_ED`.
Endpoint	"Nomor kamar" fungsi di dalam satu perangkat: switch EP 5, light EP 10.
Join	End device masuk ke jaringan yang dibuka coordinator (`setRebootOpenNetwork(180)`).
Binding	Mengaitkan endpoint switch dengan endpoint light (find-and-bind) agar perintah punya tujuan.
Cluster ON/OFF	Perintah standar Zigbee untuk menyalakan atau mematikan beban (`lightOn()` dan `lightOff()`).
Partisi khusus	`partitions_zczr.csv` dan `partitions_ed.csv` menyediakan `zb_storage` dan `zb_fct`.
NVS jaringan	Keanggotaan Zigbee disimpan di flash — bertahan melewati reset **dan** melewati flash ulang firmware.

Tabel 10.2: Istilah kerja Modul 08

Join berbeda dari binding. Join membuat perangkat menjadi **anggota jaringan** (punya alamat, punya kunci). Binding membuat satu endpoint **tahu harus mengirim ke endpoint mana**. Perangkat bisa sudah join tetapi belum ter-binding — dan perintahnya tidak akan sampai ke mana pun. Dua tahap ini muncul sebagai dua baris log yang berbeda; pastikan keduanya dapat ditunjuk.

Dari XBee ke ESP32-H2: di mana MY, SH, dan SL? Kalau sebelumnya Anda belajar komunikasi dengan modul XBee, semua konsep alamat di sana tetap berlaku — hanya cara mengaksesnya yang berbeda. Di XBee, parameter dibaca dan diatur dengan perintah AT (`ATMY`, `ATSH`, ...) lewat XCTU atau terminal. Di ESP32-H2, nilai yang sama dibaca dari stack ESP-Zigbee oleh kode, lalu dicetak fungsi `printNetworkInfo()`. Padanan lengkapnya ada pada Tabel 10.3.

```
Short address  : 0x7EC7                   ← MY  (alamat logika 16-bit)
IEEE address   : 74:4D:BD:FF:FE:61:E8:C1  ← SH + SL (alamat fisik 64-bit)
                 └── SH ──┘ └── SL ──┘
                 0x744DBDFF  0xFE61E8C1
```

XBee (AT)	Arti	Di ESP32-H2 / log modul ini	Catatan
MY	Alamat **logika** 16-bit (network address)	`Short address` — `esp_zb_get_short_address()`	Di Zigbee **dibagikan coordinator saat join**, hanya bisa dibaca; coordinator selalu `0x0000`. Bisa berubah bila node join ulang.
SH	32 bit **atas** alamat **fisik** 64-bit	4 byte pertama `IEEE address` — `esp_zb_get_long_address()`	Ditanam pabrik, tetap, unik sedunia — sama seperti nomor di label XBee.
SL	32 bit **bawah** alamat fisik 64-bit	4 byte terakhir `IEEE address`	
CH	Channel operasi	`Channel` — `esp_zb_get_current_channel()`	XBee menulisnya heksadesimal: channel 18 = `0x12`.
SC	Daftar channel yang boleh di-scan	`Zigbee.setPrimaryChannelMask()`	Default semua channel 11–26.
ID	Extended PAN ID yang **diminta**	Tidak diatur di kode modul ini	ID = 0 di XBee berarti "coordinator yang memilih"; di sini coordinator memakai IEEE address-nya sendiri.
OP	Extended PAN ID 64-bit yang **dipakai**	`Extended PAN ID` — `esp_zb_get_extended_pan_id()`	
OI	PAN ID 16-bit yang **dipakai**	`PAN ID` — `esp_zb_get_pan_id()`	Dipilih acak oleh coordinator saat membentuk network.
CE	Coordinator Enable (peran node)	Build flag `ZIGBEE_MODE_ZCZR` / `ZIGBEE_MODE_ED` + `Zigbee.begin(ZIGBEE_COORDINATOR)`	Di ESP32 peran dipilih saat **build**, bukan lewat perintah AT.
NJ	Lama network terbuka untuk join	`Zigbee.setRebootOpenNetwork(180)`	Sama-sama dalam detik.
AI	Status asosiasi (0 = sudah join)	`Zigbee.connected()`	
DH / DL	Alamat 64-bit **tujuan** (mode transparan)	**Tidak ada padanan langsung** — tujuan ditentukan **binding table**	Lihat penjelasan di bawah.
Endpoint `0xE8` (DE/SE)	Endpoint data transparan Digi	Endpoint 5 (switch) dan 10 (light)	Zigbee standar memakai endpoint + cluster (On/Off = `0x0006`), bukan satu endpoint data umum.

Tabel 10.3: Padanan parameter XBee dan ESP32-H2 pada Modul 08

Tiga perbedaan yang paling sering membingungkan:

1. **MY tidak Anda isi sendiri.** Pada XBee 802.15.4 (Series 1) dan pada M07, alamat 16-bit ditentukan pengguna (`0x0001`, `0x0002`). Pada Zigbee — baik XBee Zigbee (Series 2/3) maupun ESP32-H2 — alamat

16-bit **diberikan coordinator** saat join. Karena bisa berubah, alamat yang pasti untuk mengenali satu perangkat adalah **SH+SL (IEEE address)**, bukan MY.

2. **Tidak ada DH/DL.** Di XBee mode transparan, Anda menulis alamat tujuan ke DH/DL lalu semua data serial terkirim ke sana. Di Zigbee standar, switch mengirim *command* `On`/`Off` ke **endpoint** yang tercatat di **binding table** — isinya diisi otomatis oleh find-and-bind. Karena itu log coordinator mencetak `End device ter-binding!`, bukan alamat tujuan.
3. **Yang dikirim bukan byte mentah, melainkan command cluster.** XBee transparan meneruskan byte apa pun yang masuk ke UART-nya. `lightOn()` mengirim command ZCL `On` pada cluster On/Off, dan penerimanya harus endpoint yang memang mengerti cluster itu (`ZigbeeLight`).

`ZigbeeSwitch` dan `ZigbeeLight` bukan fitur standar Zigbee. Keduanya class C++ buatan Espressif di library `Zigbee` Arduino core 3.x, yang membungkus *device type* standar (On/Off Light Switch dan On/Off Light) beserta cluster On/Off, endpoint, dan find-and-bind. Yang standar adalah cluster, device type, dan command-nya (`On`, `Off`, `Toggle`); nama method seperti `bound()`, `allowMultipleBinding()`, dan `onLightStateChange()` adalah API Espressif. Pembagian perannya dirangkum pada Tabel 10.4.

Class	Mewakili	Yang disembunyikan class ini
`ZigbeeSwitch`	On/Off Light Switch (cluster On/Off sisi *client*, mengirim command)	Membuat endpoint, mendaftarkan cluster, find-and-bind. `lightOn()` mengirim command ZCL `On` ke tujuan di binding table.
`ZigbeeLight`	On/Off Light (cluster On/Off sisi *server*, menerima command)	Menerima command lalu memanggil callback `onLightChange`.

Tabel 10.4: Class dasar yang dipakai pada Modul 08

Class lain di library `Zigbee`. Semua class endpoint mewarisi `ZigbeeEP` dan dipakai dengan pola yang sama: buat objek dengan nomor endpoint, `setManufacturerAndModel()`, `Zigbee.addEndpoint()`, lalu `Zigbee.begin()`. Objek global `Zigbee` mengelola stack (`begin()`, `connected()`, `setRebootOpenNetwork()`). Daftar class dikelompokkan pada Tabel 10.5.

Kategori	Class
Lampu dan saklar	`ZigbeeLight`, `ZigbeeSwitch`, `ZigbeeColorDimmableLight`, `ZigbeeColorDimmerSwitch`
Sensor	`ZigbeeTempSensor`, `ZigbeeFlowSensor`, `ZigbeePressureSensor`, `ZigbeeOccupancySensor`, `ZigbeeContactSwitch`, `ZigbeeVibrationSensor`, `ZigbeeIlluminanceSensor`, `ZigbeeCarbonDioxideSensor`
Aktuator	`ZigbeePowerOutlet`, `ZigbeeWindowCovering`, `ZigbeeFanControl`, `ZigbeeDoorWindowHandle`
Kontrol dan ukur	`ZigbeeThermostat`, `ZigbeeElectricalMeasurement`
Generik	`ZigbeeAnalog`, `ZigbeeBinary`, `ZigbeeGateway`

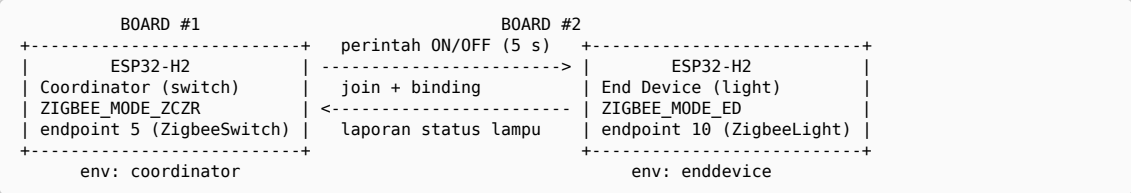
Tabel 10.5: Class endpoint pada library `Zigbee`

> **CATATAN**
> Daftar ini bergantung pada versi Arduino core; sumber pastinya ada di `libraries/Zigbee/src/ep/` pada repositori `espressif/arduino-esp32`. `ZigbeeTempSensor` paling relevan untuk WSN karena mengirim data sensor lewat attribute reporting, bukan hanya perintah On/Off.

Sekuens protokol yang diamati

```
Coordinator boot ⟶ Zigbee.begin(ZIGBEE_COORDINATOR)
   ⟶ network terbuka 180 s ⟶ ED join ⟶ find-and-bind switch↔light
   ⟶ loop 5 s: lightOn() / lightOff() ⟶ ED: setLED ON/OFF
```

10.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch	EP 5
End device	ESP32-H2 DevKitM-1	`enddevice`	ZED dan light	EP 10

Tabel 10.6: Peran tiap node Modul 08

Kedua peran memakai **ESP32-H2 DevKitM-1**. Perbedaannya hanya pada `build_flags` (`-DZIGBEE_MODE_ZCZR` dibanding `-DZIGBEE_MODE_ED`) dan tabel partisi — board-nya identik, jadi board mana pun bisa diberi peran mana pun.

10.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, **dengan LED RGB bawaan** (`RGB_BUILTIN`)	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (library `Zigbee` bawaan core 3.x)	—
5	Tabel partisi	`partitions_zczr.csv` (ZC) dan `partitions_ed.csv` (ED) — sudah ada di folder modul	—

Tabel 10.7: Alat dan bahan Modul 08

10.6 Kode Program

Dua peran, dua tabel partisi. Potongan `platformio.ini` pada Kode 10.1 memperlihatkan perbedaan `build_flags` dan `board_build.partitions` antara kedua environment.

```
[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2
```

Kode 10.1: Potongan `platformio.ini` — dua peran dengan tabel partisi berbeda

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 10.2: `platformio.ini` Modul 08 pada repositori

```
# Name,     Type, SubType, Offset,  Size, Flags
nvs,        data, nvs,     0x9000,  0x5000,
otadata,    data, ota,     0xe000,  0x2000,
app0,       app,  ota_0,   0x10000, 0x140000,
app1,       app,  ota_1,   0x150000,0x140000,
spiffs,     data, spiffs,  0x290000,0x15A000,
zb_storage, data, fat,     0x3EA000,0x4000,
zb_fct,     data, fat,     0x3EE000,0x1000,
rcp_fw,     data, spiffs,  0x3EF000,0x1000,
coredump,   data, coredump,0x3F0000,0x10000,
```

Kode 10.3: `partitions_zczr.csv` — tabel partisi peran coordinator/router

```
# Name,     Type, SubType, Offset,  Size, Flags
nvs,        data, nvs,     0x9000,  0x5000,
otadata,    data, ota,     0xe000,  0x2000,
app0,       app,  ota_0,   0x10000, 0x140000,
app1,       app,  ota_1,   0x150000,0x140000,
spiffs,     data, spiffs,  0x290000,0x15B000,
zb_storage, data, fat,     0x3EB000,0x4000,
zb_fct,     data, fat,     0x3EF000,0x1000,
coredump,   data, coredump,0x3F0000,0x10000,
```

Kode 10.4: `partitions_ed.csv` — tabel partisi peran end device

CATATAN
Kedua berkas partisi di atas identik isinya pada modul M08, M09, dan M10, sehingga listing yang sama tidak diulang pada bab-bab berikutnya.

```ino
// Minggu 8 — Zigbee P2P: Coordinator (switch) mengendalikan End Device (lampu)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);  // endpoint 5

void onLightStateChange(bool state) {
  Serial.printf("Lampu sekarang: %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);

  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(false);   // P2P: hanya satu light
  zbSwitch.onLightStateChange(onLightStateChange);

  Zigbee.addEndpoint(&zbSwitch);

  // Buka network 180 detik agar end device bisa join
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();
```

```
  Serial.println("Menunggu end device ter-binding...");
  while (!zbSwitch.bound()) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nEnd device ter-binding!");
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    if (on) {
      zbSwitch.lightOn();
      Serial.println("Perintah: Lampu ON");
    } else {
      zbSwitch.lightOff();
      Serial.println("Perintah: Lampu OFF");
    }
  }
}
```

Kode 10.5: `src/coordinator/main.cpp` — coordinator sekaligus switch (EP 5)

```
// Minggu 8 — Zigbee P2P: End Device (lampu) dikendalikan Coordinator
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);  // endpoint 10

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Lampu %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);

  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightBulb");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);
```

```
  if (!Zigbee.begin()) {  // default = ZIGBEE_END_DEVICE
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Menunggu bergabung ke network koordinator...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Berhasil bergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 10.6: `src/enddevice/main.cpp` — end device sekaligus light (EP 10)

10.6.1 Channel antar Kelompok (wajib kelas paralel)

Satu ruangan praktikum biasanya menampung beberapa kelompok, sehingga berdiri beberapa jaringan Zigbee sekaligus. Satu node tetap hanya menjadi anggota satu jaringan; yang harus dicegah adalah dua hal: node yang baru di-`erase` dapat **salah join** ke coordinator kelompok lain bila beberapa window join terbuka bersamaan, dan beberapa jaringan pada channel yang sama saling berebut airtime sehingga angka loss serta latency — data pengukuran modul ini — tercemar oleh tetangga.

Channel operasi ditentukan coordinator saat membentuk jaringan (baris **SC** pada Tabel 10.3), jadi nilainya disetel lewat `setPrimaryChannelMask()` sebelum `Zigbee.begin()` pada **kedua** berkas (Kode 10.5 dan Kode 10.6). Jatah untuk enam kelompok ada pada Tabel 10.8 — contoh pada listing memakai jatah Kelompok 1.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 10.8: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

Mask Zigbee adalah satu bit per channel (`1UL << n` berarti **hanya** channel n); mask default `0x07FFF800` mencakup channel 11–26 sehingga coordinator bebas memilih di antaranya.

> **PERINGATAN**
> Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/enddevice/main.cpp` baris yang sama **bukan** memilih channel — end device tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai, supaya hanya jaringan kelompok sendiri yang dapat ditemukan.

> **TIP**
> Channel 15, 20, dan 25 adalah pilihan klasik Zigbee karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah minimal dua kanal. Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di kedua board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada kedua log: `Channel` harus sesuai jatah kelompok dan `Extended PAN ID` harus IEEE address coordinator sendiri. Bila board menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

10.7 Build dan Flash

Coordinator dulu, lalu end device **dalam 180 detik**.

Linux/macOS (bash):

```
pio run -d week08_zigbee_p2p -e enddevice   -t erase     # bersihkan NVS
pio run -d week08_zigbee_p2p -e coordinator -t erase
pio run -d week08_zigbee_p2p -e coordinator -t upload -t monitor
pio run -d week08_zigbee_p2p -e enddevice   -t upload    # jangan lewat 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e enddevice   -t erase  --upload-port COM11   # sesuai pio device list
pio run -e coordinator -t upload --upload-port COM5
pio run -e enddevice   -t upload --upload-port COM11   # dalam 180 s
```

> **CATATAN**
> Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

> **PENTING**
> **Pilih port USB-to-UART, bukan USB native.** Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, dua port dicatat dan diisikan di atas.
- ☐ Channel kelompok disetel pada kedua berkas dan berbeda dari kelompok lain — lihat Tabel 10.8.
- ☐ **Hapus NVS kedua board sebelum flash pertama:** `pio run -d week08_zigbee_p2p -e enddevice -t erase`. Tanpa ini, board yang pernah join jaringan lain akan terus mencari koordinator lama.
- ☐ Environment `coordinator` (ZCZR) dan `enddevice` (ED) dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud untuk kedua board.
- ☐ Endpoint dipahami: switch EP 5, light EP 10.
- ☐ Tidak ada jaringan Zigbee lain aktif di sekitar (hindari interferensi dan salah join).

10.8 Percobaan

10.8.1 EXP-01 — Pembentukan Jaringan (Coordinator)

Unggah environment `coordinator`, buka Serial Monitor, verifikasi: coordinator membentuk network, mencetak parameternya, membuka network 180 detik, lalu menunggu binding.

```
boot ──► begin(ZIGBEE_COORDINATOR) OK
     ──► network terbentuk: cetak channel, PAN ID, alamat
     ──► network terbuka 180 s
     ──► menunggu: . . . . . (titik tiap 500 ms)
```

Channel dan PAN ID **tidak ditulis di kode**: coordinator memilih sendiri saat membentuk network (channel dari hasil scan, PAN ID acak), lalu menyimpannya di NVS. Nilainya dibaca dari stack ESP-Zigbee (`esp_zb_get_current_channel()`, `esp_zb_get_pan_id()`, dan seterusnya) di fungsi `printNetworkInfo()`, karena library Arduino hanya menuliskannya ke log debug.

Data capture

Parameter	Hasil
Pesan awal coordinator	
Channel	
PAN ID / Extended PAN ID	
Short address coordinator	
Lama network terbuka (s)	
Endpoint switch	
Perintah yang dikirim tiap 5 s	

Tabel 10.9: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Bandingkan ukuran firmware modul ini dengan modul BLE (lihat ringkasan `RAM/Flash` di akhir `pio run`). Lalu buka `partitions_zczr.csv` (Kode 10.3) dan temukan partisi `zb_storage` dan `zb_fct`. Jawab: apa yang disimpan di sana, dan mengapa tabel partisi default Arduino tidak cukup?

CHECKPOINT

Coordinator mencetak titik-titik `.` berulang. Jika langsung muncul `Zigbee gagal start!` lalu board restart, tabel partisi atau `build_flags` tidak cocok — perbaiki sebelum lanjut.

10.8.2 EXP-02 — Join dan Binding (End Device)

Unggah environment `enddevice` ke board kedua **sebelum 180 detik habis**. Amati proses join lalu binding, dan verifikasi RGB bawaan berkedip ON/OFF tiap 5 detik.

```
ED boot ──► Zigbee.begin() ──► scan channel
        ──► join network ──► find & bind ke switch
ZC loop 5 s: lightOn() … lightOff() … (bergantian)
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 18
  PAN ID         : 0x4FC6
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu end device ter-binding...
End device ter-binding!
Perintah: Lampu ON
Perintah: Lampu OFF
Perintah: Lampu ON
Perintah: Lampu OFF
```

Bila binding berlangsung lambat, baris titik-titik (`....`) tercetak di bawah `Menunggu end device ter-binding...` selama menunggu.

> **CATATAN**
> Baris `Lampu sekarang: ...` berasal dari callback `onLightStateChange()`, yaitu **laporan balik** dari lampu ke switch. Pada firmware Arduino core 3.3.x baris ini tidak muncul karena `ZigbeeLight` tidak mengonfigurasi attribute reporting ke switch secara otomatis — perintah tetap sampai dan lampu tetap menyala. Mengaktifkan laporan balik justru menjadi CH-2.

Expected output — End device

```
Menunggu bergabung ke network koordinator...
Berhasil bergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 18
  PAN ID         : 0x4FC6
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x7EC7 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
Lampu ON
Lampu OFF
```

Padanan XBee (Tabel 10.3): `Short address` = **MY**, `IEEE address` = **SH + SL**, `Channel` = **CH**, `PAN ID` = **OI**, `Extended PAN ID` = **OP**.

Bandingkan info network kedua board: **Channel, PAN ID, dan Extended PAN ID harus sama** — itulah bukti end device bergabung ke network coordinator ini, bukan network lain. Short address coordinator selalu `0x0000`; short address end device dibagikan coordinator saat join. Bila tidak diatur di kode, Extended PAN ID diambil dari IEEE address coordinator — terlihat pada log di atas.

> **CHECKPOINT**
> Dua hal harus terjadi berurutan: end device mencetak `Berhasil bergabung ke network!` (join), lalu coordinator mencetak `End device ter-binding!` (binding). Jika join berhasil tetapi binding tidak pernah terjadi, perintah tidak akan sampai — jangan lanjut, ulangi dengan menghapus NVS kedua board.

10.8.3 EXP-03 — Keandalan Kontrol

1. Hitung siklus ON/OFF per menit (harapan 12) dan bandingkan status di kedua Serial Monitor.
2. Reset end device — catat waktu join ulang dan verifikasi apakah keanggotaan tersimpan (network terbuka lagi tiap coordinator reboot 180 s).
3. **Cabut daya coordinator selama 30 detik**, lalu nyalakan lagi. Amati apakah end device otomatis kembali atau perlu di-reset.
4. Geser jarak bertahap dan catat kapan perintah mulai tertinggal.

Data capture

Parameter	Hasil
Siklus ON/OFF per menit	
Waktu join ulang setelah reset ED	
Perilaku saat coordinator mati lalu hidup	
Sinkron status ZC dan ED?	
Jarak mulai ada perintah tertinggal (m)	

Tabel 10.10: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa end device yang di-reset bisa kembali **tanpa** window join dibuka lagi (petunjuk: keanggotaan tersimpan di NVS, join hanya diperlukan sekali).

10.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 2 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu agar tidak ada sisa jaringan Zigbee lama di NVS), capture 70 detik.

```
# Coordinator (ESP32-H2, ZCZR)          # End device (ESP32-H2, ED)
[0.401] Menunggu end device ...         [0.401] Menunggu bergabung ...
[3.407] ......                          [3.205] Berhasil bergabung ke network!
[3.407] End device ter-binding!
[5.209] Perintah: Lampu ON              [5.207] Lampu ON
[10.219] Perintah: Lampu OFF            [10.214] Lampu OFF
```

Parameter	Hasil terukur
Waktu join dan binding sejak boot	3,4 s
Perintah dikirim coordinator	13
Aksi terjadi di end device	13 (0 % loss)
Selisih waktu perintah sampai aksi	≈ 2 ms
Siklus ON/OFF per menit	12

Tabel 10.11: Hasil verifikasi hardware Modul 08

10.10 Pengukuran

Jarak	RSSI (dBm)	Latency perintah (kasar)	Success ON/OFF (%)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 10.12: Lembar pengukuran jarak Modul 08

Pengukuran per-node (pengamatan 2 menit, jarak tetap).

Node	Perintah terkirim	Aksi terlaksana	Loss (%)
Coordinator (switch)		—	
End device (light)	—		

Tabel 10.13: Lembar pengukuran per-node Modul 08

Waktu join dan binding (minimal 3 percobaan, hapus NVS tiap kali).

Percobaan	Waktu join (s)	Waktu binding (s)	Total (s)
1			
2			
3			

Tabel 10.14: Lembar pengukuran waktu join dan binding

Bandingkan dengan M07. Pada jarak yang sama, apakah success rate Zigbee lebih baik, sama, atau lebih buruk daripada raw 802.15.4? Radionya identik, jadi selisih apa pun berasal dari lapisan di atasnya.

10.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana pengaruh jarak terhadap keberhasilan perintah ON/OFF diterima lampu?
2. Apakah latency perintah (ZC hingga LED berubah di ED) bertambah pada jarak jauh?
3. Berapa persen perintah gagal dalam 2 menit pengamatan, dan pola gagalnya bagaimana (acak atau berkelompok)?
4. Berapa waktu proses join dan binding dari tiga percobaan, dan apa yang membuatnya bervariasi?
5. Apakah Zigbee (join dan binding otomatis ditambah enkripsi) lebih cocok untuk WSN dibanding raw 802.15.4 M07? Sebutkan apa yang diperoleh dan apa harga yang dibayar.

10.12 Concept Check

1. Apa fungsi coordinator dalam jaringan Zigbee?
2. Apa perbedaan proses join dan binding? Bisakah salah satu terjadi tanpa yang lain?
3. Mengapa end device memakai partisi dan library berbeda (`partitions_ed.csv`, `esp_zb_api.ed`)?
4. Apa yang terjadi bila end device dinyalakan setelah window 180 detik berakhir, dan bagaimana cara memperbaikinya?
5. Mengapa `allowMultipleBinding(false)` dipakai pada skenario P2P ini, dan apa yang berubah di Modul 09?

10.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Mengukur latency dan menutup arah balik

CH-1 — Latency terukur Ukur latency perintah secara objektif: kirim timestamp di payload atau catat `millis()` saat `lightOn()` di coordinator dan saat `setLED()` di end device, lalu bandingkan. Lakukan 20 kali dan hitung rata-rata serta sebarannya.

CH-2 — Laporan balik (menutup celah `onLightStateChange`) Aktifkan attribute reporting pada `ZigbeeLight` sehingga coordinator benar-benar mencetak `Lampu sekarang: ON/OFF`. Bandingkan jumlah perintah terkirim dengan laporan diterima untuk menghitung loss arah balik.

Level 3 — Tantangan · Kontrol dua arah dan ketahanan NVS

CH-3 — Kontrol dari end device Tambahkan tombol pada end device yang mengirim perintah toggle ke coordinator. Diskusikan mengapa ini memerlukan binding arah sebaliknya.

CH-4 — Uji ketahanan NVS Flash ulang end device **tanpa** `-t erase`, lalu amati apakah ia langsung kembali ke jaringan. Jelaskan perbedaannya dengan perangkat BLE di M01–M06 yang tidak menyimpan apa pun.

10.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (Zigbee, ZC dan ZED, endpoint, join dibanding binding, cluster ON/OFF).
3. Konfigurasi — `build_flags`, tabel partisi, endpoint, window join 180 s.
4. Hasil eksperimen — log kedua board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel jarak, tabel per-node, tabel waktu join dan binding, serta perbandingan dengan M07.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

11. Modul 09 — Zigbee Multi-Node dan Binding Table

IoT COMMUNICATION LAB · MODUL 09

Scale the Network — Zigbee Multi-Node dan Binding Table

ESP32-H2 · Zigbee · 1 ZC + 2 ZED · level Intermediate · 3 × 50 menit · folder kode `week09_zigbee_multinode`

PENGANTAR MODUL · Gambaran Umum

Modul 09 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menskalakan satu coordinator ke banyak end device tanpa satu perintah pun hilang. Percobaan berjalan sebagai jaringan Zigbee multi-node dengan satu coordinator dan dua end device, diamati melalui tiga terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

11.1 Pendahuluan

M08 mengikat **satu** lampu ke satu switch. Di sini `allowMultipleBinding(true)` dinyalakan, dan coordinator harus menyimpan **daftar** tujuan — inilah binding table. Masalah yang muncul identik dengan M05 di dunia BLE (satu pusat, banyak sumber), tetapi jawabannya berbeda: BLE memakai objek koneksi per node, Zigbee memakai `endpoint + short address`. Perbandingan dua cara ini adalah bahan analisis modul ini.

Prasyaratnya adalah M08: join, binding, endpoint, dan penghapusan NVS sebelum flash. Yang dibangun di sini adalah pemakaian `allowMultipleBinding(true)`, iterasi binding table melalui `getBoundDevices()`, pengalamatan per endpoint, perhitungan loss per node, serta pengamatan perilaku sistem saat satu node hilang. Semuanya dipakai lagi pada M10 ketika jenis anggota jaringan bertambah dengan hadirnya router, M12 pada komunikasi many-to-many di Thread, dan M16 ketika skalabilitas menjadi kriteria pemilihan protokol.

Peta modul blok Zigbee

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang
08	Jaringan Zigbee terbentuk: join, binding, cluster ON/OFF
09 (ini)	**Satu coordinator melayani banyak end device (binding table)**
10	Router menambah hop — routing multi-hop otomatis

Tabel 11.1: Peta modul blok Zigbee

Kontrak data lab ini. Identitas node di Zigbee adalah pasangan **`endpoint` ditambah `short address`** — bukan prefiks di dalam payload seperti M05 (`A:`, `B:`). Catat perbedaannya: identitas di sini dikelola **jaringan**, bukan aplikasi. Konsekuensinya muncul langsung di modul ini (lihat catatan `0xFFFF` di bagian Percobaan).

11.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menskalakan Zigbee ke banyak end device

1. Membangun jaringan Zigbee satu coordinator dengan dua end device pada satu window join, dan menampilkan isi binding table beserta endpoint tiap node.
2. Mengirim perintah ke node tertentu berdasarkan `endpoint + short address` dan membuktikan node yang dituju bereaksi.
3. Menghitung packet loss **per node** (EP 10 dibanding EP 11) pada jarak yang sama dan pada jarak berbeda.
4. Menjelaskan perilaku coordinator ketika satu end device menghilang, berdasarkan log — termasuk apakah perintah tetap dikirim ke node yang sudah mati.

Kriteria keberhasilan

- ☐ Coordinator mencetak `Total 2 device.` pada daftar binding.
- ☐ Kedua lampu merespons pada siklus yang sama.
- ☐ Loss per node terukur terpisah (EP 10 dibanding EP 11).
- ☐ Skenario satu node dimatikan diuji dan perilakunya tercatat.

11.3 Dasar Teori (Secukupnya)

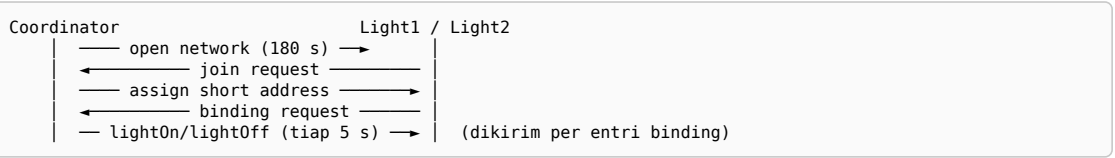
Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (tabel routing Zigbee, addressing mode APS, group addressing) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 11.2.

Istilah	Definisi kerja di lab ini
Coordinator (ZC)	Membentuk jaringan, mengelola PAN, mengizinkan join.
End Device (ZED)	Node akhir (child) rendah daya, tidak meneruskan trafik.
Join	Node bergabung ke jaringan; dibuka 180 detik setelah reboot (`setRebootOpenNetwork(180)`).
Binding table	Daftar tujuan yang dipegang switch; dibaca lewat `getBoundDevices()`.
Endpoint	Titik komunikasi aplikasi; Switch EP 5, Light1 EP 10, Light2 EP 11.
Cluster On/Off	Perintah standar; dipanggil per tujuan lewat `lightOn(ep, addr)`.
Short address	Alamat 16-bit node Zigbee, dicetak coordinator sebagai `0x%04X`.

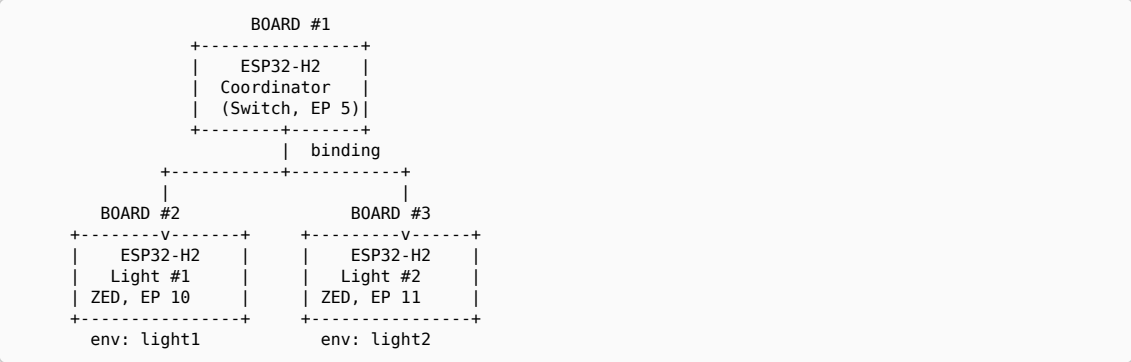
Tabel 11.2: Istilah kerja Modul 09

Mengapa endpoint kedua lampu harus berbeda? Karena binding table mengidentifikasi tujuan dari pasangan `endpoint + address`. Bila dua lampu memakai endpoint yang sama, entri binding menjadi ambigu dan perintah bisa menyasar. Endpoint adalah "nomor kamar" — dua kamar tidak boleh bernomor sama di satu daftar tujuan.

Sekuens protokol yang diamati



11.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch (ZCZR)	EP 5
Light #1	ESP32-H2 DevKitM-1	`light1`	ZED dan light	EP 10
Light #2	ESP32-H2 DevKitM-1	`light2`	ZED dan light	EP 11

Tabel 11.3: Peran tiap node Modul 09

Ketiganya **ESP32-H2 DevKitM-1** dengan radio 802.15.4; peran ditentukan oleh `build_flags` dan tabel partisi, bukan oleh jenis board.

11.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, LED RGB bawaan	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Power bank atau catu daya USB	untuk node yang dipindah saat uji jarak	2
5	Tabel partisi	`partitions_zczr.csv` (ZC), `partitions_ed.csv` (ED) — sudah ada di folder modul	—

Tabel 11.4: Alat dan bahan Modul 09

11.6 Kode Program

Tiga environment (Kode 11.1).

```
[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv
upload_port = /dev/ttyACM0

[env:light1]
build_src_filter = +<light1/*.cpp>
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port = /dev/ttyACM2

[env:light2]
build_src_filter = +<light2/*.cpp>
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port = /dev/ttyACM4
```

Kode 11.1: Potongan `platformio.ini` — tiga environment Modul 09

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:light1]
build_src_filter = +<light1/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv

[env:light2]
build_src_filter = +<light2/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 11.2: `platformio.ini` Modul 09 pada repositori

Tabel partisi `partitions_zczr.csv` dan `partitions_ed.csv` pada modul ini identik dengan milik Modul 08 (Kode 10.3 dan Kode 10.4), sehingga tidak diulang di sini.

```
// Minggu 9 — Zigbee Multi-Node: 1 Coordinator mengendalikan beberapa End Device
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
```

```
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);

  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(true);   // izinkan banyak light ter-bind

  Zigbee.addEndpoint(&zbSwitch);
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();

  Serial.println("Menunggu light ter-binding (join dalam 180 detik)...");
  // Tunggu minimal 1 light ter-bind
  while (!zbSwitch.bound()) {
    delay(500);
  }
  delay(5000);  // beri waktu light lain join & bind

  Serial.println("Daftar device ter-bind:");
  std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
  for (auto d : bound) {
    Serial.printf(" - endpoint %d, short addr 0x%04X\n", d->endpoint, d->short_addr);
  }
  Serial.printf("Total %d device.\n", (int)bound.size());
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
    for (auto d : bound) {
      if (on) {
        zbSwitch.lightOn(d->endpoint, d->short_addr);
        Serial.printf("-> Light 0x%04X ON\n", d->short_addr);
      } else {
        zbSwitch.lightOff(d->endpoint, d->short_addr);
        Serial.printf("-> Light 0x%04X OFF\n", d->short_addr);
      }
    }
  }
}
```

Kode 11.3: `src/coordinator/main.cpp` — coordinator dengan binding table

```ino
// Minggu 9 — Zigbee Multi-Node: Light #1 (end device)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Light1 %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLight1");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Light1 menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Light1 tergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 11.4: `src/light1/main.cpp` — Light #1 pada endpoint 10

```ino
// Minggu 9 — Zigbee Multi-Node: Light #2 (end device)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;
```

```
ZigbeeLight zbLight = ZigbeeLight(11);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Light2 %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 11 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLight2");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Light2 menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Light2 tergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 11.5: `src/light2/main.cpp` — Light #2 pada endpoint 11

11.6.1 Channel antar Kelompok (wajib kelas paralel)

Seperti diperkenalkan pada Modul 08, setiap kelompok memakai **channel operasi yang berbeda** supaya node yang baru di-`erase` tidak salah join ke coordinator kelompok lain saat window join terbuka bersamaan, dan supaya angka loss serta latency tidak tercemar trafik kelompok tetangga pada channel yang sama.

Channel ditentukan coordinator saat membentuk jaringan; ketiga berkas pada listing di atas (Kode 11.3, Kode 11.4, Kode 11.5) memakai mask yang sama supaya kedua light hanya dapat menemukan jaringan kelompoknya sendiri. Jatah untuk enam kelompok ada pada Tabel 11.5.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 11.5: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

PERINGATAN
Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/light1/main.cpp` dan `src/light2/main.cpp` baris yang sama **bukan** memilih channel — kedua light tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai.

TIP
Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di ketiga board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada ketiga log: nilainya harus sama dengan coordinator kelompok sendiri. Bila sebuah light menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

11.7 Build dan Flash

Linux/macOS (bash):

```
for e in coordinator light1 light2; do pio run -d week09_zigbee_multinode -e $e -t erase; done
pio run -d week09_zigbee_multinode -e coordinator -t upload -t monitor
pio run -d week09_zigbee_multinode -e light1 -t upload      # dalam 180 s
pio run -d week09_zigbee_multinode -e light2 -t upload      # dalam 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e light1      -t erase  --upload-port COM11   # sesuai pio device list
pio run -e light2      -t erase  --upload-port COM12
pio run -e coordinator -t upload --upload-port COM5
pio run -e light1      -t upload --upload-port COM11   # dalam 180 s
pio run -e light2      -t upload --upload-port COM12   # dalam 180 s
```

CATATAN
Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- [ ] `pio device list` dijalankan, tiga port dicatat dan diisikan di atas.
- [ ] Channel kelompok disetel pada ketiga berkas dan berbeda dari kelompok lain — lihat Tabel 11.5.
- [ ] **Hapus NVS ketiga board** (`-t erase`) sebelum flash pertama.
- [ ] Flash `coordinator` (ZCZR, `partitions_zczr.csv`).
- [ ] Flash `light1` dan `light2` (ED, `partitions_ed.csv`) **dalam window 180 detik**.
- [ ] Serial Monitor 115200 baud pada tiap board.
- [ ] Tabel pencatat jarak, RSSI, dan latency siap.

11.8 Percobaan

11.8.1 EXP-01 — Pembentukan Jaringan dan Join Ganda

Nyalakan coordinator lebih dulu. Jaringan terbuka 180 detik setelah reboot; dalam window ini nyalakan `light1` dan `light2` agar join dan binding.

```
[ZC reboot] ──► open network 180 s ──► [ZED1 join] [ZED2 join] ──► bound
```

Data capture

Parameter	Hasil
Waktu join Light1 (detik)	
Waktu join Light2 (detik)	
Status LED saat join	
Apakah keduanya masuk dalam satu window?	

Tabel 11.6: Lembar pengamatan EXP-01

CHECKPOINT

Kedua light mencetak `tergabung ke network!`. Jika hanya satu, window join sudah habis untuk yang kedua — reboot coordinator (window terbuka lagi) lalu ulangi. Jangan lanjut dengan satu node saja.

11.8.2 EXP-02 — Binding Table dan Kontrol Otomatis

Setelah minimal satu light ter-bind, coordinator menunggu 5 detik tambahan (agar light kedua sempat join dan bind), mencetak daftar device ter-bind, lalu men-toggle semua light setiap 5 detik.

```
[ZC] getBoundDevices() ──► endpoint + short addr
[ZC] lightOn(ep, addr) / lightOff(ep, addr)  tiap 5 s
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu light ter-binding (join dalam 180 detik)...
Daftar device ter-bind:
 - endpoint 10, short addr 0xFFFF
 - endpoint 11, short addr 0xFFFF
Total 2 device.
-> Light 0xFFFF ON
-> Light 0xFFFF ON
-> Light 0xFFFF OFF
-> Light 0xFFFF OFF
```

CATATAN

Satu atau kedua `short addr` di daftar bound sering tercetak `0xFFFF` (pada uji ini keduanya), padahal short address asli kedua light terlihat di blok info network masing-masing. Itu **bukan** kegagalan binding — lihat catatan pada bagian Verifikasi Hardware di bawah.

Expected output — Light1

```
Light1 menunggu join ke network...
Light1 tergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x82C3 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
Light1 ON
Light1 OFF
```

Expected output — Light2

```
Light2 menunggu join ke network...
Light2 tergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x127F (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:F3:97
  Endpoint       : 11 (light)
Light2 ON
Light2 OFF
```

Info network ketiga board harus sama: **Channel, PAN ID, dan Extended PAN ID** (di sini 26, `0x1890`, dan IEEE address coordinator). Short address tiap light berbeda dan **dibagikan coordinator** saat join. Padanan XBee: `Short address` = MY, `IEEE address` = SH + SL, `Channel` = CH, `PAN ID` = OI, `Extended PAN ID` = OP — lihat tabel padanan XBee di Modul 08.

BUKA ABSTRAKSINYA

`getBoundDevices()` mengembalikan `std::list` berisi `zb_device_params_t`. Cetak **seluruh** field struct itu (bukan hanya endpoint dan short address) dan cocokkan dengan `IEEE address` (MAC 64-bit) yang dicetak tiap light pada blok `Info network`. Jawab: entri mana yang benar-benar unik dan stabil — short address atau alamat IEEE?

CHECKPOINT

Baris `Total 2 device.` muncul, dan setelah itu ada **dua** baris `-> Light 0x....` untuk tiap siklus ON dan tiap siklus OFF. Jika hanya satu baris per siklus, binding kedua gagal.

11.8.3 EXP-03 — Jarak dan Kehilangan Node

1. Letakkan Light2 pada jarak bertambah (1, 3, 5, lalu 10 m) dari coordinator; amati apakah perintah tetap diterima sementara Light1 tetap dekat.
2. Matikan (cabut daya) Light1 saat sistem berjalan; amati apakah coordinator **masih mencetak** `-> Light 0xXXXX ON` untuk node yang sudah mati.
3. Nyalakan kembali Light1 setelah window 180 detik berlalu; catat apakah dapat kembali sendiri dan apa yang perlu dilakukan.

Data capture

Parameter	Hasil
Success Light2 pada 10 m	
Apakah loss Light1 terpengaruh oleh Light2 yang jauh?	
Perilaku coordinator saat Light1 mati	
Light1 kembali otomatis? (ya/tidak) beserta alasan	

Tabel 11.7: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa coordinator tetap mengirim perintah ke node yang sudah mati (petunjuk: binding table adalah daftar statis, bukan daftar node yang sedang hidup). Ini temuan penting untuk desain sistem nyata.

11.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu), capture 70 detik.

```
# Coordinator (ESP32-H2, ZCZR)
[0.401] Menunggu light ter-binding (join dalam 180 detik)...
[5.409] Daftar device ter-bind:
[5.409]  - endpoint 10, short addr 0xFFFF
[5.409]  - endpoint 11, short addr 0x1DA3
[5.409] Total 2 device.
[5.409] -> Light 0xFFFF ON
[5.409] -> Light 0x1DA3 ON

# Light1 (ESP32-H2, ED)            # Light2 (ESP32-H2, ED)
[0.602] Light1 tergabung ...       [3.208] Light2 tergabung ...
[5.411] Light1 ON                  [5.411] Light2 ON
[10.421] Light1 OFF                [10.419] Light2 OFF
```

Parameter	Hasil terukur
Device ter-bind terdeteksi	2 (EP 10 dan EP 11)
Perintah dikirim per light	14
Aksi terjadi di Light1 dan Light2	14 / 14 (0 % loss)

Parameter	Hasil terukur
Waktu join Light1 dan Light2	0,6 s dan 3,2 s

Tabel 11.8: Hasil verifikasi hardware Modul 09

> **CATATAN**
> **Short addr `0xFFFF` itu normal.** Entri binding yang dibuat lewat alamat IEEE (bukan alamat pendek) disimpan library dengan `short_addr = 0xFFFF`. `lightOn(ep, 0xFFFF)` tetap sampai ke node yang benar — buktinya Light1 tetap menyala. Yang perlu dicatat pada laporan adalah jumlah device terbind dan keberhasilan aksinya, bukan nilai alamatnya.

11.10 Pengukuran

Tabel jarak (geser Light2, Light1 tetap di 1 m; amati LED dan Serial Monitor).

Jarak Light2	RSSI	Latency	Success Light2	Success Light1
1 m			… / 10	… / 10
3 m			… / 10	… / 10
5 m			… / 10	… / 10
10 m			… / 10	… / 10
15 m			… / 10	… / 10

Tabel 11.9: Lembar pengukuran jarak Modul 09

Tabel per-node (jarak tetap 3 m, amati 20 perintah).

Node	RSSI	Paket diterima	Loss (%)
Light1 (EP 10)		… / 20	
Light2 (EP 11)		… / 20	

Tabel 11.10: Lembar pengukuran per-node Modul 09

Bandingkan dengan M05. Isi Tabel 11.11 memakai data BLE multi-node.

Aspek	BLE bintang (M05)	Zigbee multi-node (M09)
Cara pusat membedakan node		
Loss node dekat saat node lain jauh		
Perilaku saat satu node mati		
Batas jumlah node (perkiraan beserta alasan)		

Tabel 11.11: Tabel pembanding BLE bintang dan Zigbee multi-node

11.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa waktu rata-rata proses join dari menyala hingga pesan `tergabung ke network`?
2. Apakah kedua light menerima perintah ON/OFF pada siklus yang sama? Buktikan dari urutan baris log coordinator.
3. Bagaimana pengaruh jarak terhadap success rate perintah ON/OFF, dan apakah node yang dekat ikut terdampak?
4. Apa yang terjadi di coordinator ketika salah satu light dimatikan? Masihkah perintah dikirim ke node itu, dan apa implikasinya untuk sistem nyata?
5. Mengapa endpoint light1 dan light2 harus berbeda (10 dibanding 11)? Apa yang akan terjadi jika sama?

11.12 Concept Check

1. Apa fungsi `allowMultipleBinding(true)` pada switch coordinator?
2. Apa yang dimaksud window join 180 detik pada `setRebootOpenNetwork(180)`, dan mengapa tidak dibuka selamanya?
3. Jelaskan perbedaan peran ZC dan ZED dalam topologi multi-node ini.
4. Bagaimana coordinator mengidentifikasi setiap light secara unik (endpoint ditambah short address)?
5. Apa keuntungan multi-binding dibanding broadcast ke semua node?

11.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah node dan menghitung loss per endpoint

CH-1 — Node ketiga Tambahkan **light3** (endpoint 12): salin `src/light2`, ubah `ZigbeeLight(11)` menjadi `ZigbeeLight(12)` dan teks `Light2` menjadi `Light3`, lalu tambahkan env `light3` di `platformio.ini`. Amati `Total 3 device.` pada coordinator dan periksa apakah interval siklus bergeser.

CH-2 — Loss per node otomatis Catat jumlah perintah TX (`-> Light 0xXXXX ON/OFF`) dibanding jumlah `LightX ON/OFF` yang diterima tiap node selama 2 menit. Contoh: TX = 24, RX = 22, sehingga loss = (24 − 22)/24 × 100 % = 8,33 %. Sajikan terpisah per endpoint.

Level 3 — Tantangan · Kontrol selektif dan deteksi node hilang

CH-3 — Kontrol selektif Ubah coordinator agar menyalakan Light1 dan mematikan Light2 pada saat yang sama (bukan toggle serentak). Ini membuktikan perintah benar-benar dialamatkan per node, bukan disiarkan.

CH-4 — Deteksi node hilang Tambahkan mekanisme di coordinator untuk menandai node yang tidak merespons, misalnya menghitung berapa siklus berturut-turut tanpa laporan balik dari CH-2 Modul 08. Cetak `Node 0xXXXX tidak merespons` setelah 3 siklus.

11.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (Zigbee multi-node, ZC dan ZED, binding table, endpoint, cluster).
3. Konfigurasi — env `coordinator`, `light1`, `light2`, endpoint 5, 10, 11, serta interval 5 detik.

4. Hasil eksperimen — log join, daftar binding, log ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel jarak, tabel per-node, dan tabel perbandingan dengan M05.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

12. Modul 10 — Zigbee Mesh: Routing Multi-Hop

IoT COMMUNICATION LAB · MODUL 10

Route Through the Mesh — Zigbee Mesh: Routing Multi-Hop

ESP32-H2 · Zigbee · ZC ke ZR ke ZED · level Advanced · 3 × 50 menit · folder kode `week10_zigbee_mesh`

PENGANTAR MODUL · Gambaran Umum

Modul 10 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya membuktikan router benar-benar mengangkut trafik milik node lain, sekaligus mengukur harga hop keduanya. Percobaan berjalan sebagai mesh multi-hop dari coordinator ke router lalu ke end device, diamati melalui tiga terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

12.1 Pendahuluan

M06 sudah memperkenalkan hop — tetapi jalurnya ditulis sendiri di dalam kode. Di sini jalur ditentukan **stack**: end device memilih parent terbaik sendiri, dan router meneruskan trafik tanpa satu baris kode penerusan pun di sketch. Membandingkan dua pendekatan ini (relay manual M06 dibanding routing otomatis M10) adalah inti analisis modul ini, dan bekalnya dipakai lagi saat Thread melakukan hal serupa di atas IPv6 (M12).

Prasyaratnya ada dua: M09 untuk binding table, multi-node, dan penghapusan NVS; serta M06 untuk konsep hop dan loss per hop. Yang dibangun di sini adalah peran Router (ZR), pemilihan parent secara otomatis, routing multi-hop, perbandingan latency satu hop dengan dua hop, dan pengenalan status orphan. Semuanya dipakai lagi pada M12 ketika mesh Thread menambahkan role election, M13 ketika gateway berperan sebagai tepi jaringan, dan M16 ketika jangkauan mesh menjadi kriteria pemilihan protokol.

Peta modul blok Zigbee (penutup blok)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang
08	Jaringan Zigbee terbentuk: join, binding
09	Satu coordinator, banyak end device (binding table)
10 (ini)	**Router meneruskan trafik — jangkauan melampaui satu hop**
11	Pindah ke Thread: IPv6 di atas radio 802.15.4 yang sama

Tabel 12.1: Peta modul blok Zigbee

Kontrak data lab ini. Ukur **latency 1 hop dan 2 hop** pada modul ini. Angka itu adalah pembanding langsung untuk latency relay manual M06 dan untuk mesh Thread M12 — tiga cara berbeda menyelesaikan masalah yang sama, di radio yang sama.

12.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membuktikan dan mengukur routing multi-hop Zigbee

1. Membangun jaringan Zigbee tiga peran (ZC, ZR, ZED) dan menunjukkan dari log bahwa ketiganya berada dalam satu jaringan.
2. Membuktikan end device mencapai coordinator **melalui** router pada formasi garis, dengan cara memutus jalur langsung dan mengamati akibatnya.
3. Mengukur selisih latency 1 hop (formasi dekat) dan 2 hop (formasi garis) lalu menyajikannya sebagai angka.
4. Menjelaskan status *orphan* pada end device saat router dimatikan, serta apakah dan bagaimana jaringan memulihkan diri.

Kriteria keberhasilan

- ☐ `Total device ter-bind: 2` tercetak di coordinator.
- ☐ EndLight tetap merespons pada formasi garis (ZC jauh dari ZED).
- ☐ Selisih latency 1 hop dan 2 hop terukur dan tercatat.
- ☐ Skenario router dimatikan diuji, perilaku ZED tercatat.

12.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (algoritma routing AODV Zigbee, tabel routing, many-to-one routing) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 12.2.

Istilah	Definisi kerja di lab ini
Coordinator (ZC)	Membentuk jaringan, membuka window join 180 detik, sumber perintah ON/OFF.
Router (ZR)	Node penuh yang **meneruskan paket node lain** (`Zigbee.begin(ZIGBEE_ROUTER)`); juga punya aplikasi lampu (EP 10).
End Device (ZED)	Child yang bergabung lewat parent (bisa router), tidak meneruskan trafik.
Parent selection	ZED memilih parent dengan kualitas tautan terbaik — bisa ZC, bisa ZR.
Mesh	Jalur alternatif otomatis; EndLight bisa mencapai ZC lewat ZR (2 hop) atau langsung (1 hop).
Orphan	Status ZED yang kehilangan parent dan belum menemukan pengganti.
Endpoint/Cluster	EP 5 (switch), EP 10 (router light), EP 11 (ED light), semuanya pada cluster On/Off.

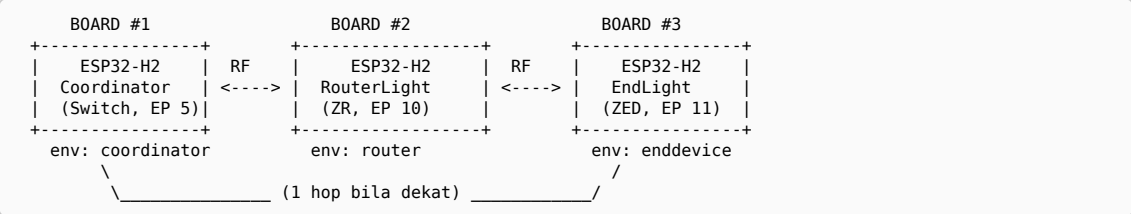
Tabel 12.2: Istilah kerja Modul 10

Router bukan sekadar "node yang juga menyala". Perbedaan sesungguhnya: router **selalu mendengarkan** (tidak tidur) dan menyimpan tabel routing. Itulah mengapa ZR memakai firmware ZCZR (lebih besar, partisi berbeda) dan mengapa ZR tidak cocok untuk node baterai. Catat konsekuensi daya ini — ia menjadi salah satu argumen di M16.

Sekuens protokol yang diamati

```
ZC ──(lightOn/lightOff)──► ZR (EP10: nyala)
                            │
                            └── relay ──► ZED (EP11: nyala)
```

12.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch (ZCZR)	EP 5
RouterLight	ESP32-H2 DevKitM-1	`router`	ZR dan light (ZCZR)	EP 10
EndLight	ESP32-H2 DevKitM-1	`enddevice`	ZED dan light (ED)	EP 11

Tabel 12.3: Peran tiap node Modul 10

Ketiganya **ESP32-H2 DevKitM-1**. Router memakai firmware ZCZR (`partitions_zczr.csv`) sedangkan end device memakai ED (`partitions_ed.csv`) — satu-satunya perbedaan perangkat keras adalah peran yang diberikan, bukan chip.

Rencana penempatan: ZC, jarak, ZR, jarak, ZED, dalam satu garis. Formasi meja (semua kurang dari 1 m) hanya untuk pemanasan; ia **tidak** membuktikan multi-hop.

12.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, LED RGB bawaan	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE	1
4	Power bank atau catu daya USB	wajib — ZC dan ZED harus bisa dijauhkan	3
5	Ruang uji	lorong atau ruang panjang untuk formasi garis	—

Tabel 12.4: Alat dan bahan Modul 10

12.6 Kode Program

Tiga peran, dua tabel partisi (Kode 12.1).

```
[env:coordinator]
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:router]                ; ZR juga memakai firmware ZCZR
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 12.1: Potongan `platformio.ini` — tiga peran Modul 10

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:router]
build_src_filter = +<router/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 12.2: `platformio.ini` Modul 10 pada repositori

Tabel partisi `partitions_zczr.csv` dan `partitions_ed.csv` pada modul ini identik dengan milik Modul 08 (Kode 10.3 dan Kode 10.4).

```
// Minggu 10 — Zigbee Mesh: Coordinator mengendalikan lampu lewat Router & ED
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);
```

```
  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(true);

  Zigbee.addEndpoint(&zbSwitch);
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();

  Serial.println("Menunggu router & end device ter-binding...");
  while (!zbSwitch.bound()) {
    delay(500);
  }
  delay(8000);  // beri waktu router dan ED join & bind

  std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
  Serial.printf("Total device ter-bind: %d\n", (int)bound.size());
  for (auto d : bound) {
    Serial.printf(" - endpoint %d, short addr 0x%04X\n", d->endpoint, d->short_addr);
  }
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
    for (auto d : bound) {
      if (on) {
        zbSwitch.lightOn(d->endpoint, d->short_addr);
        Serial.printf("-> 0x%04X ON\n", d->short_addr);
      } else {
        zbSwitch.lightOff(d->endpoint, d->short_addr);
        Serial.printf("-> 0x%04X OFF\n", d->short_addr);
      }
    }
  }
}
```

Kode 12.3: `src/coordinator/main.cpp` — coordinator sekaligus switch (EP 5)

```
// Minggu 10 — Zigbee Mesh: Router (lampu yang juga meneruskan trafik)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("RouterLight %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
```

```
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : Router (ZR)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightRouter");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  // Peran ROUTER: bisa meneruskan trafik ke node lain (mesh)
  if (!Zigbee.begin(ZIGBEE_ROUTER)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Router menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Router tergabung (role=ROUTER).");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 12.4: `src/router/main.cpp` — RouterLight (ZR, EP 10)

```
// Minggu 10 — Zigbee Mesh: End Device (child, dapat join lewat router)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(11);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("EndLight %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
```

```
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 11 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightED");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {  // default = ZIGBEE_END_DEVICE
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("End device menunggu join (bisa lewat router)...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("End device tergabung (role=END_DEVICE).");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 12.5: `src/enddevice/main.cpp` — EndLight (ZED, EP 11)

12.6.1 Channel antar Kelompok (wajib kelas paralel)

Satu ruangan praktikum biasanya menampung beberapa kelompok, sehingga berdiri beberapa jaringan Zigbee sekaligus. Satu node tetap hanya menjadi anggota satu jaringan; yang harus dicegah adalah dua hal: node yang baru di-`erase` dapat **salah join** ke coordinator kelompok lain bila beberapa window join terbuka bersamaan, dan beberapa jaringan pada channel yang sama saling berebut airtime sehingga angka loss serta latency — data utama modul ini — tercemar oleh tetangga.

Channel operasi ditentukan coordinator saat membentuk jaringan (parameter **SC**, lihat Modul 08), jadi nilainya disetel lewat `setPrimaryChannelMask()` sebelum `Zigbee.begin()` — pada **ketiga** berkas. Tiga listing di atas sudah memuat baris itu (Kode 12.3, Kode 12.4, Kode 12.5) dengan jatah Kelompok 1; setiap kelompok mengganti angka `15` sesuai Tabel 12.5.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 12.5: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

Mask Zigbee adalah satu bit per channel (`1UL << n` berarti **hanya** channel n). Mask default `0x07FFF800` mencakup channel 11–26 sehingga coordinator bebas memilih di antaranya; dengan jatah di Tabel 12.5, tiap kelompok mengunci satu channel saja.

PERINGATAN
Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/router/main.cpp` dan `src/enddevice/main.cpp` baris yang sama **bukan** memilih channel — ZR dan ZED tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai, supaya keduanya hanya dapat menemukan jaringan kelompoknya sendiri.

TIP
Channel 15, 20, dan 25 adalah pilihan klasik Zigbee karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah minimal dua kanal. Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di ketiga board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada ketiga log: `Channel` harus sesuai jatah kelompok dan `Extended PAN ID` harus IEEE address coordinator sendiri. Bila sebuah board menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

12.7 Build dan Flash

Urutan penting: ZC, lalu ZR, lalu ZED.

Linux/macOS (bash):

```
for e in coordinator router enddevice; do pio run -d week10_zigbee_mesh -e $e -t erase; done
pio run -d week10_zigbee_mesh -e coordinator -t upload -t monitor
pio run -d week10_zigbee_mesh -e router      -t upload    # dalam 180 s
pio run -d week10_zigbee_mesh -e enddevice   -t upload    # dalam 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e router      -t erase  --upload-port COM11   # sesuai pio device list
pio run -e enddevice   -t erase  --upload-port COM12
pio run -e coordinator -t upload --upload-port COM5
pio run -e router      -t upload --upload-port COM11   # dalam 180 s
pio run -e enddevice   -t upload --upload-port COM12   # dalam 180 s
```

CATATAN
Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat dan diisikan pada tiap env.
- ☐ Channel kelompok disetel pada ketiga berkas dan berbeda dari kelompok lain — lihat Tabel 12.5.
- ☐ **Hapus NVS ketiga board** (`-t erase`) sebelum flash pertama.
- ☐ Flash `coordinator` (ZCZR), `router` (ZCZR, mulai sebagai `ZIGBEE_ROUTER`), dan `enddevice` (ED) — semuanya dalam window 180 detik.
- ☐ Serial Monitor 115200 baud pada ketiga board.
- ☐ Formasi penempatan (meja dulu, lalu garis) sudah direncanakan.

12.8 Percobaan

12.8.1 EXP-01 — Join Bertingkat

Nyalakan coordinator, lalu router, lalu end device (semua dalam window 180 detik). Router join langsung ke ZC; end device memilih parent terbaik.

```
[ZC on] ──► open 180 s ──► [ZR join ke ZC] ──► [ZED join (parent = ZR/ZC)]
```

Data capture

Parameter	Hasil
Waktu join Router (detik)	
Waktu join End Device (detik)	
Total device ter-bind di ZC	
Role yang dicetak Router dan ZED	

Tabel 12.6: Lembar pengamatan EXP-01

CHECKPOINT

Router mencetak `role=ROUTER` dan end device mencetak `role=END_DEVICE`. Jika router mencetak `END_DEVICE`, `build_flags`-nya salah — perbaiki sebelum lanjut, karena tanpa ZR tidak ada hop kedua.

12.8.2 EXP-02 — Kontrol Multi-Hop

Coordinator menunggu binding (ditambah 8 detik), mencetak daftar device, lalu men-toggle semua device tiap 5 detik. Perintah ke EndLight diteruskan router bila ZED memilih ZR sebagai parent.

```
[ZC] ─── ON/OFF (5 s) ──► [ZR: RouterLight ON/OFF]
                │
                └─ relay ──► [ZED: EndLight ON/OFF]
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu router & end device ter-binding...
Total device ter-bind: 2
 - endpoint 10, short addr 0xFFFF
 - endpoint 11, short addr 0xBB28
-> 0xFFFF ON
-> 0xBB28 ON
-> 0xFFFF OFF
-> 0xBB28 OFF
```

Expected output — Router

```
Router menunggu join ke network...
Router tergabung (role=ROUTER).
Info network:
  Peran          : Router (ZR)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x33F4 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
RouterLight ON
RouterLight OFF
```

Expected output — End device

```
End device menunggu join (bisa lewat router)...
End device tergabung (role=END_DEVICE).
Info network:
  Peran          : End Device (ED)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0xBB28 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:F3:97
  Endpoint       : 11 (light)
EndLight ON
EndLight OFF
```

Info network ketiga board harus sama: **Channel, PAN ID, dan Extended PAN ID** (di sini 21, `0x4D92`, dan IEEE address coordinator). Router juga mendapat short address dari coordinator, sama seperti end device. Short address end device (`0xBB28`) sama dengan yang tercetak di daftar bound coordinator; router tercetak `0xFFFF` di sana. Padanan XBee: `Short address` = MY, `IEEE address` = SH + SL, `Channel` = CH, `PAN ID` = OI, `Extended PAN ID` = OP — lihat tabel padanan XBee di Modul 08.

BUKA ABSTRAKSINYA

Cari di `src/router/main.cpp` baris kode yang **meneruskan** perintah ke end device. Baris itu tidak akan ditemukan: penerusan dikerjakan stack Zigbee, bukan aplikasi. Bandingkan dengan `src/nodeb/main.cpp` di Modul 06, tempat penerusan ditulis eksplisit. Tuliskan perbandingan itu — ia adalah jawaban pertanyaan analisis nomor 5.

CHECKPOINT

Dua baris `-> 0x....` muncul tiap siklus dan kedua LED berubah. Jika EndLight tidak ikut berubah padahal ter-bind, dekatkan dulu semuanya (formasi meja) sebelum mencoba formasi garis.

12.8.3 EXP-03 — Varian Topologi (inti modul)

1. **Formasi garis** (ZC, 5 m, ZR, 5 m, ZED): EndLight menerima perintah lewat router (2 hop). Amati apakah tetap ON/OFF sinkron.
2. **Formasi dekat**: letakkan ZED bersebelahan ZC; ZED mungkin memilih ZC sebagai parent (1 hop). Bandingkan latency nyala LED.
3. **Hilangkan router**: matikan ZR pada formasi garis; amati apakah ZED menjadi *orphan*, lalu apakah dapat re-join ke ZC bila jarak masih terjangkau.
4. Nyalakan ZR kembali dan catat waktu pemulihan.

Data capture

Parameter	Hasil
Latency EndLight, formasi dekat (1 hop)	
Latency EndLight, formasi garis (2 hop)	
Selisih latency 1 hop dan 2 hop	
Perilaku ZED saat ZR dimatikan	
Waktu pemulihan setelah ZR hidup lagi	

Tabel 12.7: Lembar pengamatan EXP-03

CHECKPOINT

Tersedia **dua angka latency** dari dua formasi berbeda, bukan satu. Tanpa keduanya, klaim "multi-hop berhasil" tidak bisa dibuktikan.

12.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu; ketiga board berdekatan, jadi ZED kemungkinan besar memilih ZC sebagai parent, yaitu 1 hop), capture 80 detik.

```
# Coordinator (ESP32-H2, ZCZR)
[0.401] Menunggu router & end device ter-binding...
[8.415] Total device ter-bind: 2
[8.415]  - endpoint 10, short addr 0xFFFF
[8.415]  - endpoint 11, short addr 0x727D
[8.415] -> 0xFFFF ON
[8.415] -> 0x727D ON

# Router (ESP32-H2, ZR)           # End device (ESP32-H2, ZED)
[0.401] Router tergabung          [3.206] End device tergabung
        (role=ROUTER).                    (role=END_DEVICE).
[8.416] RouterLight ON            [8.415] EndLight ON
[13.425] RouterLight OFF          [13.423] EndLight OFF
```

Parameter	Hasil terukur
Waktu join router dan end device	0,4 s dan 3,2 s
Semua node ter-bind	2 light (EP 10 dan EP 11) pada detik 8,4
Perintah dikirim per light	15
Aksi terjadi di RouterLight dan EndLight	15 / 15 (0 % loss)

Tabel 12.8: Hasil verifikasi hardware Modul 10

Formasi meja (semua node kurang dari 1 m) **belum membuktikan** routing multi-hop. Untuk itu jalankan EXP-03 formasi garis dan matikan jalur langsung ZC ke ZED.

12.10 Pengukuran

Tabel jarak ZC ke ZED (dengan ZR di tengah).

Jarak total	RSSI	Latency	Success
1 m			… / 10
3 m			… / 10
5 m			… / 10
10 m			… / 10
15 m			… / 10

Tabel 12.9: Lembar pengukuran jarak Modul 10

Tabel per-hop atau per-node (20 perintah).

Node atau Hop	RSSI	Paket diterima	Loss (%)
Router (EP 10, hop 1)		… / 20	
EndLight (EP 11, hop 2)		… / 20	

Tabel 12.10: Lembar pengukuran per-hop Modul 10

Tabel pembanding hop — wajib. Isi dari data sendiri.

Cara mencapai node jauh	Modul	Latency terukur	Siapa yang menentukan jalur
Relay manual (BLE)	M06		kode aplikasi
Routing otomatis (Zigbee)	M10		stack Zigbee

Tabel 12.11: Tabel pembanding relay manual dan routing otomatis

12.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa lama waktu join router dan end device? Apakah urutan penyalaan memengaruhi keberhasilan?
2. Berdasarkan formasi dekat dibanding garis, kapan end device memilih router sebagai parent? Bagaimana hal itu dibuktikan dari log?
3. Bandingkan latency nyala EndLight pada 1 hop dan 2 hop; berapa selisihnya dan apakah sebanding dengan tambahan jangkauan?
4. Apa yang terjadi pada EndLight ketika router dimatikan? Sebutkan istilah status node tersebut dan berapa lama pemulihannya.
5. Bandingkan dengan relay manual M06: apa yang **diperoleh** dari routing otomatis, dan apa yang **hilang** (kendali, keterlihatan, ukuran firmware)?

12.12 Concept Check

1. Apa perbedaan utama mode build ZCZR dan ED (lihat `build_flags` di `platformio.ini`)?
2. Mengapa router memakai `partitions_zczr.csv` sedangkan end device `partitions_ed.csv`?
3. Bagaimana mesh Zigbee menemukan jalur baru jika salah satu router mati?
4. Jelaskan hubungan endpoint 5, 10, dan 11 dengan binding pada percobaan ini.
5. Apa konsekuensi daya dari node yang berperan sebagai router dibanding end device, dan apa artinya untuk node bertenaga baterai?

12.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memperpanjang rantai dan mengukur loss per hop

CH-1 — Rantai dua router Tambahkan **router2** (salin `src/router`, tetap `ZigbeeLight(10)` tetapi ubah model string) dan susun rantai ZC, ZR1, ZR2, ZED. Amati berapa hop maksimum yang masih berfungsi dan berapa tambahan latency per hop.

CH-2 — Packet loss per hop Catat TX di ZC dan RX di tiap lampu selama 3 menit. Contoh: TX = 36, RX EndLight = 33, sehingga loss = (36 − 33)/36 × 100 % = 8,33 %. Sajikan terpisah untuk hop 1 dan hop 2.

Level 3 — Tantangan · Melacak parent dan menguji self-healing

CH-3 — Lacak parent Amati alamat parent end device dengan memindahkan ZED bertahap, lalu simpulkan pada jarak berapa jalur mulai di-router. Sajikan sebagai tabel jarak terhadap parent.

CH-4 — Uji self-healing Pada formasi garis, matikan ZR lalu ukur berapa lama ZED butuh untuk kembali (baik lewat ZC langsung maupun setelah ZR hidup lagi). Bandingkan dengan perilaku relay manual M06 yang tidak pulih sendiri.

12.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (mesh Zigbee, ZR sebagai relayer, parent selection, orphan).
3. Konfigurasi — env `coordinator`, `router`, `enddevice`, endpoint 5, 10, 11, interval 5 detik, dan window 180 s.
4. Hasil eksperimen — log ketiga node, skema formasi (meja dan garis), serta checkpoint.
5. Data pengukuran — tabel jarak, tabel per-hop, dan tabel pembanding hop M06 dengan M10.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

13. Modul 11 — Thread: Datagram UDP di atas IPv6

IoT COMMUNICATION LAB · MODUL 11

Speak IPv6 over Thread — Datagram UDP di atas IPv6

ESP32-H2 · Thread · UDP/IPv6 · level Intermediate · 3 × 50 menit · folder kode `week11_thread_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 11 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya memberi tiap node alamat IPv6 sungguhan dan mengirim datagram UDP di antara keduanya. Percobaan berjalan sebagai pertukaran P2P berbasis IPv6, dengan PING dikirim secara multicast dan PONG dibalas secara unicast, diamati melalui dua terminal Serial Monitor pada 115200 baud.

13.1 Pendahuluan

Zigbee memakai alamat 16-bit milik jaringannya sendiri; untuk keluar ke Internet ia butuh penerjemah. Thread memakai **IPv6** — alamat yang bentuknya sama dengan alamat Internet — di atas radio 802.15.4 yang sama persis dengan M07–M10. Inilah alasan Thread bisa disambungkan ke Wi-Fi di M13 hampir tanpa penerjemahan: paketnya sudah IP sejak dari node sensor.

Prasyaratnya ada dua: M07 untuk channel, PAN ID, dan radio 802.15.4; serta M08–M10 untuk pengalaman menghadapi jaringan yang mengelola dirinya sendiri. Yang dibangun di sini adalah Active Dataset Thread, mesh-local prefix beserta EID, proses attach role secara otomatis, socket UDP IPv6, dan perbedaan multicast realm-local dengan unicast. Semuanya dipakai lagi pada M12 untuk mesh many-to-many, M13 ketika dataset yang sama dipakai gateway C6, M15 ketika payload UDP ini berakhir di MQTT, dan M16 ketika Thread dibandingkan dengan BLE dan Zigbee.

Peta modul blok Thread

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang
08–10	Zigbee: alamat 16-bit, join dan binding, mesh
11 (ini)	**Thread: tiap node punya alamat IPv6, datagram UDP**
12	Mesh Thread many-to-many, role election
13	Thread disambungkan ke Wi-Fi lewat gateway C6

Tabel 13.1: Peta modul blok Thread

Kontrak data lab ini. Grup multicast `ff03::abcd` dan port **5050** dipakai sama persis di M11, M12, M13, dan M15. Karena itu firmware penerima modul mana pun bisa dijadikan alat bantu diagnosis untuk modul lainnya — teknik yang benar-benar dipakai saat menguji M13 tanpa board C6.

13.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membentuk jaringan Thread dan mengirim datagram UDP IPv6

1. Menyusun Active Dataset Thread lengkap (nama, channel, PAN ID, ext PAN ID, network key, mesh-local prefix) dan menjelaskan mengapa keenam parameter harus identik di semua node.
2. Membentuk jaringan Thread dua node dan menunjukkan mesh-local EID masing-masing beserta **awalan yang sama**.
3. Mengirim datagram UDP IPv6 multicast dan membalasnya unicast, dibuktikan dari alamat sumber yang tercetak di log.
4. Mengukur latency PING ke PONG dan packet loss pada minimal 4 jarak, serta menjelaskan gejala khas bila prefix mesh-local berbeda.

Kriteria keberhasilan

- ☐ Kedua node attach (satu Leader, satu Child) dan mencetak mesh-local EID dengan **awalan yang sama**.
- ☐ PING multicast dibalas PONG unicast secara periodik.
- ☐ Latency PING ke PONG rata-rata dihitung dari sekurangnya 10 sampel.
- ☐ Tabel jarak–RSSI–latency–loss terisi dari pengukuran sendiri.

13.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (MLE, Router ID assignment, MPL forwarding, keamanan DTLS atau commissioning) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 13.2.

Istilah	Definisi kerja di lab ini
Thread	Protokol jaringan mesh berbasis IPv6 di atas IEEE 802.15.4.
Leader	Node yang mengelola router ID dan dataset jaringan; terpilih otomatis.
Child	Node yang attach ke parent (Leader atau Router) untuk berkomunikasi.
Active Dataset	Parameter jaringan — nama `ESP_OT_P2P`, channel 15, PAN ID `0xABCD`, ext PAN ID `DE:AD:00:BE:EF:00:CA:FE`, network key 16 byte, **mesh-local prefix `fdde:ad00:beef::/64`**.
Mesh-Local prefix	Prefix `/64` milik seluruh jaringan; semua alamat mesh-local diturunkan darinya. Harus **sama persis** di setiap node.
Mesh-Local EID	Alamat IPv6 mesh-local tiap node (`fdde:ad00:beef:0:...`), dicetak via `getMeshLocalEid()`.
Link-local (`fe80::`)	Alamat per-interface untuk komunikasi 1 hop antar tetangga.
Multicast `ff03::abcd`	Realm-local: diteruskan ke seluruh mesh, bukan hanya tetangga langsung.
“ping”	Di lab ini diwakili pola PING/PONG aplikasi di atas UDP, bukan ICMPv6 echo.

Tabel 13.2: Istilah kerja Modul 11

Mengapa dataset di-commit ulang tiap boot. `DataSet::initNew()` memanggil `otDatasetCreateNewNetwork()`, yang **mengacak** network key, ext PAN ID, dan mesh-local prefix. Kode modul ini menimpa empat field pertama dengan konstanta, lalu memaksa prefix mesh-local lewat `otThreadSetMeshLocalPrefix()` sebelum `OThread.start()` seperti Kode 13.1.

```
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] =
    {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};   // fdde:ad00:beef::/64
```

Kode 13.1: Konstanta mesh-local prefix yang dipaksakan di setiap node

Tanpa langkah itu tiap board memakai prefix acak sendiri. Gejalanya menyesatkan: kedua node **tetap attach** (MLE hanya mencocokkan channel, PAN ID, ext PAN ID, dan network key) dan Serial Monitor tampak normal, tetapi tidak satu pun paket multicast `ff03::` sampai — karena penerusan multicast realm-local terikat pada prefix mesh-local jaringan. Cirinya: dua node dengan awalan Mesh-Local EID berbeda, misalnya `fdcd:8b6:7a73:...` di satu sisi dan `fd99:6dd0:2d68:...` di sisi lain.

Alamat di Thread: di mana MY, SH, dan SL? Setelah attach, tiap node mencetak blok `Info network Thread:` (fungsi `printNetworkInfo()`). Isinya bisa dipetakan ke parameter XBee dan Zigbee (Modul 08) seperti pada Tabel 13.3.

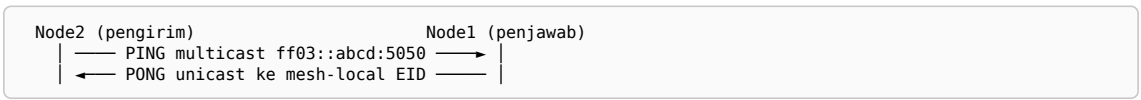
XBee	Zigbee (M08)	Thread	Catatan
MY	`Short address`	`RLOC16`	Alamat 16-bit yang dibagikan jaringan. Di Thread nilainya menyandi **Router ID + Child ID**, jadi berubah bila peran atau parent berubah.
SH + SL	`IEEE address`	`EUI-64`	Alamat pabrik 64-bit, tetap, unik — padanan nomor di label XBee.
—	(sama dengan IEEE)	`Extended addr`	Alamat MAC 64-bit yang **diacak** Thread demi privasi, jadi **tidak sama** dengan EUI-64.
CH	`Channel`	`Channel`	Di Thread diambil dari dataset yang ditulis kode (15), bukan dipilih jaringan.
OI	`PAN ID`	`PAN ID`	Dari dataset (`0xABCD`).
OP	`Extended PAN ID`	`Extended PAN ID`	Dari dataset (`DE:AD:00:BE:EF:00:CA:FE`).
—	—	`Network name`	Nama jaringan dari dataset (`ESP_OT_P2P`). XBee tidak punya padanannya (NI adalah nama *node*, bukan nama jaringan).
—	—	`Mesh-Local EID`	Alamat **IPv6** node di dalam mesh — alamat yang muncul di baris `RX [...]` dan dipakai untuk unicast balik.

Tabel 13.3: Padanan alamat XBee, Zigbee, dan Thread

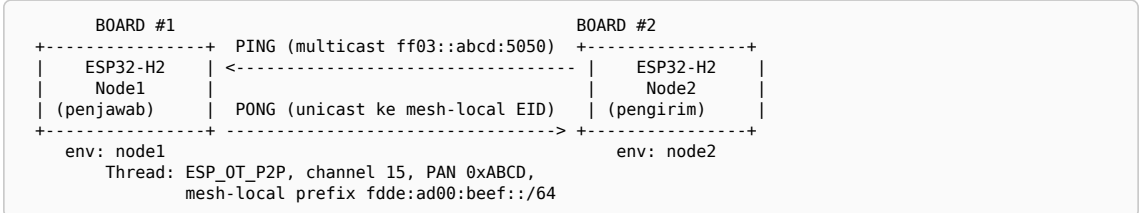
Dua perbedaan yang paling penting: **(1)** parameter jaringan (channel, PAN ID, Extended PAN ID, network name) di Thread **ditentukan dataset di kode**, sedangkan di Zigbee dipilih coordinator; **(2)** aplikasi Thread berbicara dengan **alamat IPv6** (Mesh-Local EID), bukan alamat 16-bit seperti DH/DL di XBee atau binding di Zigbee.

Membaca RLOC16. 6 bit atas adalah Router ID, 9 bit bawah adalah Child ID. Router/Leader punya Child ID 0 (mis. `0xE800` = Router ID 58); child memakai Router ID parent-nya (mis. `0xE801` = child nomor 1 dari router `0xE800`). Jadi dari RLOC16 saja terlihat siapa parent sebuah child.

Sekuens protokol yang diamati



13.4 Topologi



Node	Board	Environment	Role Thread	Aksi
Node1	ESP32-H2 DevKitM-1	`node1`	Leader **atau** Child (dinamis)	bind multicast dan unicast, balas `PONG`
Node2	ESP32-H2 DevKitM-1	`node2`	Leader **atau** Child (dinamis)	kirim `PING` multicast tiap 3 s

Tabel 13.4: Peran tiap node Modul 11

Keduanya **ESP32-H2 DevKitM-1** (radio 802.15.4). ESP32-C6 baru masuk di Modul 13 saat Thread perlu disambungkan ke Wi-Fi.

> **CATATAN**
> **Role Thread tidak ditentukan oleh nama environment.** Leader dipilih otomatis oleh stack — biasanya board yang selesai boot lebih dulu. Pada uji referensi, `node2` justru menjadi Leader dan `node1` menjadi Child, dan PING/PONG tetap berjalan normal. Yang harus sama di kedua board adalah **dataset**, bukan role.

13.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan OpenThread (`OThread` dan `OThreadUDP`).

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1 (env `node1`, penjawab PONG)	1
2	Board ESP32-H2	DevKitM-1 (env `node2`, pengirim PING)	1
3	Kabel USB data	kabel data, bukan *charge-only*	2
4	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
5	Power bank atau catu daya USB	untuk uji jarak	2

Tabel 13.5: Alat dan bahan Modul 11

Tidak ada library eksternal — `OThread` dan `OThreadUDP` bawaan Arduino core 3.x.

Parameter jaringan yang wajib identik

Parameter	Nilai
Network name	`ESP_OT_P2P`
Channel dan PAN ID	15 dan `0xABCD`
Ext PAN ID	`DE:AD:00:BE:EF:00:CA:FE`
Network key	`00 11 22 … ee ff` (16 byte)
Mesh-local prefix	`fdde:ad00:beef::/64`
Grup multicast dan port	`ff03::abcd` : 5050

Tabel 13.6: Parameter jaringan Thread Modul 11

13.6 Kode Program

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>
```

Kode 13.2: `platformio.ini` Modul 11 pada repositori

```
// Minggu 11 — Thread P2P/IP: Node1 (Leader) menjawab PING dengan PONG
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

const char OT_NETWORK_NAME[] = "ESP_OT_P2P";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama untuk semua node. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printNetworkInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
```

```
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node1 (Thread Leader) starting...");

  OThread.begin(false);  // init tanpa autostart agar dataset bisa di-set dulu

  // Dataset selalu ditulis ulang agar seluruh node memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu attach...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
  printNetworkInfo();

  OtUdp.beginMulticast(GROUP, PORT);
  Serial.printf("Mendengarkan [%s]:%d (dan unicast)\n", GROUP.toString().c_str(), PORT);
}

void loop() {
  while (int n = OtUdp.parsePacket()) {
    char buf[32];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    IPAddress src = OtUdp.remoteIP();
    uint16_t sp = OtUdp.remotePort();
    Serial.printf("RX [%s]:%u -> '%s'\n", src.toString().c_str(), sp, buf);

    const char *resp = "PONG";
    OtUdp.beginPacket(src, sp);
    OtUdp.write((const uint8_t *)resp, strlen(resp));
    OtUdp.endPacket();
    Serial.println("TX PONG (unicast ke pengirim)");
  }
  delay(10);
}
```

Kode 13.3: `src/node1/main.cpp` — penjawab PONG

```
// Minggu 11 — Thread P2P/IP: Node2 (Child) mengirim PING dan menunggu PONG
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

const char OT_NETWORK_NAME[] = "ESP_OT_P2P";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
```

```
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama untuk semua node. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printNetworkInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node2 (Thread Child) starting...");

  OThread.begin(false);

  // Dataset selalu ditulis ulang agar seluruh node memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu join ke network Leader...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
  printNetworkInfo();

  OtUdp.begin(PORT);  // bind untuk menerima balasan unicast
}

void loop() {
```

```
  static unsigned long last = 0;
  if (millis() - last > 3000) {
    last = millis();
    const char *msg = "PING";
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.println("TX PING (multicast)");
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[32];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX [%s]:%u -> '%s'\n", OtUdp.remoteIP().toString().c_str(), OtUdp.remotePort(), buf);
  }
  delay(10);
}
```

Kode 13.4: `src/node2/main.cpp` — pengirim PING multicast

13.6.1 Channel antar Kelompok (wajib kelas paralel)

Seperti pada blok Zigbee (Modul 08–10), satu ruangan praktikum berisi beberapa kelompok sehingga berdiri beberapa jaringan Thread sekaligus. Pada Thread, channel adalah bagian dari **Active Dataset** yang ditulis firmware ke tiap node sebelum `OThread.start()`; jaringan terbentuk pada channel dataset itu, dan node lain attach ke sana. Karena network name, PAN ID, dan network key dibuat sama di semua kelompok, pemisah antar kelompok yang andal adalah **channel yang berbeda** — beda channel berarti radio tuli total.

Jatah untuk enam kelompok sama dengan blok Zigbee. Ubah satu baris `OT_CHANNEL` pada **kedua** berkas modul ini (Kode 13.3 dan Kode 13.4):

Kelompok	Channel	Baris yang diubah
1	15	`const uint8_t OT_CHANNEL = 15;`
2	20	`const uint8_t OT_CHANNEL = 20;`
3	25	`const uint8_t OT_CHANNEL = 25;`
4	11	`const uint8_t OT_CHANNEL = 11;`
5	17	`const uint8_t OT_CHANNEL = 17;`
6	22	`const uint8_t OT_CHANNEL = 22;`

Tabel 13.7: Jatah channel Thread untuk enam kelompok — satu nilai untuk semua node kelompok

PERINGATAN
Semua node dalam satu kelompok harus memakai nilai `OT_CHANNEL` yang sama. Bila Leader dan Child berbeda channel, keduanya tidak akan pernah saling dengar dan Child berhenti di `Menunggu join ke network Leader...`. Setelah upload, cocokkan baris `Channel` pada `Info network Thread` semua node.

TIP
Mengganti `OT_CHANNEL` tidak memerlukan erase NVS: dataset ditulis ulang setiap boot (`commitDataSet`), jadi cukup build + upload ulang seluruh board kelompok. Channel 15, 20, dan 25 adalah pilihan klasik karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah.

13.7 Build dan Flash

```
pio run -d week11_thread_p2p -e node1 -t erase
pio run -d week11_thread_p2p -e node2 -t erase
pio run -d week11_thread_p2p -e node1 -t upload
pio run -d week11_thread_p2p -e node2 -t upload -t monitor
```

Pre-flight checklist

- ☐ `pio device list` dijalankan, dua port dicatat dan diisikan pada tiap env.
- ☐ Channel kelompok disetel (`OT_CHANNEL`) sama di kedua berkas dan berbeda dari kelompok lain — lihat Tabel 13.7.
- ☐ Flash environment `node1` dan `node2` (role Leader atau Child dipilih otomatis oleh stack, bukan oleh nama environment).
- ☐ Mesh-local prefix di kedua firmware sama: `fdde:ad00:beef::/64`.
- ☐ **Hapus dataset lama** bila board pernah dipakai modul Thread lain: `pio run -e node1 -t erase`.
- ☐ Serial Monitor 115200 baud pada kedua board.
- ☐ Tabel pencatat jarak, RSSI, latency, dan loss siap.

13.8 Percobaan

13.8.1 EXP-01 — Pembentukan Jaringan dan Attach

Nyalakan kedua board. Keduanya men-commit dataset identik (termasuk prefix mesh-local), start Thread, dan menunggu role minimal CHILD. Board yang selesai boot lebih dulu umumnya menjadi Leader — urutannya boleh berbeda tiap percobaan.

```
[board A] dataset (+ ML prefix) ──► start ──► Leader
[board B] dataset (+ ML prefix) ──► start ──► scan ──► attach ──► Child
```

Data capture

Parameter	Hasil
Role Node1	… (Leader atau Child)
Role Node2	… (Leader atau Child)
Awalan Mesh-Local EID sama di kedua node?	… (harus `fdde:ad00:beef:0:`)
Mesh-Local EID Node1	
Mesh-Local EID Node2	
Waktu attach tiap node (detik)	

Tabel 13.8: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Komentari baris `applyMeshLocalPrefix();` pada **salah satu** node, flash ulang, dan amati: kedua node tetap mencetak `Attached as:`, tetapi tidak ada satu pun baris `RX`. Bandingkan awalan EID keduanya — di situlah bukti kegagalannya. Kembalikan kodenya. Percobaan ini mengajarkan sesuatu yang jarang terlihat: **jaringan bisa "terbentuk" tetapi tidak berfungsi**, dan log status saja tidak cukup untuk menyimpulkan keberhasilan.

CHECKPOINT

Kedua node mencetak `Attached as: ...` **dan** awalan EID keduanya sama (`fdde:ad00:beef:0:`). Jika awalannya berbeda, jangan lanjut — tidak akan ada paket yang sampai.

13.8.2 EXP-02 — PING Multicast dan PONG Unicast

Node1 bind grup multicast port 5050 dan juga unicast; Node2 mengirim `PING` multicast tiap 3 detik lalu Node1 membalas `PONG` unicast.

```
[N2] TX PING (multicast ff03::abcd:5050) ──3 s──►
     [N1] RX PING ──► TX PONG (unicast ke EID N2)
[N2] RX PONG dari EID Node1
```

Expected output — Node1 (dari uji di board; alamat dan peran di board Anda akan berbeda)

```
Node1 (Thread Leader) starting...
Menunggu attach...
Attached as: Child         <- bisa juga Leader, tergantung urutan boot
Info network Thread:
  Peran          : Child
  Network name   : ESP_OT_P2P
  Channel        : 15
  PAN ID         : 0xABCD
  Extended PAN ID: DE:AD:00:BE:EF:00:CA:FE
  RLOC16         : 0xE801
  Extended addr  : FA:FE:07:23:A6:B1:A9:FB
  EUI-64         : 74:4D:BD:FF:FE:61:E6:2C
  Mesh-Local EID : fdde:ad00:beef:0:e54c:17e1:80ad:62a4
Mendengarkan [ff03::abcd]:5050 (dan unicast)
RX [fdde:ad00:beef:0:cbc4:ae06:ccea:2768]:5050 -> 'PING'
TX PONG (unicast ke pengirim)
```

Expected output — Node2

```
Node2 (Thread Child) starting...
Menunggu join ke network Leader...
Attached as: Leader        <- bisa juga Child, tergantung urutan boot
Info network Thread:
  Peran          : Leader
  Network name   : ESP_OT_P2P
  Channel        : 15
  PAN ID         : 0xABCD
  Extended PAN ID: DE:AD:00:BE:EF:00:CA:FE
  RLOC16         : 0xE800
  Extended addr  : 3E:CC:8F:12:00:85:DF:E5
  EUI-64         : 74:4D:BD:FF:FE:61:E8:C1
  Mesh-Local EID : fdde:ad00:beef:0:cbc4:ae06:ccea:2768
TX PING (multicast)
TX PING (multicast)
TX PING (multicast)
RX [fdde:ad00:beef:0:e54c:17e1:80ad:62a4]:5050 -> 'PONG'
```

Blok info network kedua node harus sama pada **Network name, Channel, PAN ID, dan Extended PAN ID**. Pada uji ini Node2 yang menjadi Leader (`RLOC16 0xE800`) dan Node1 menjadi child-nya (`0xE801`) — peran tidak ditentukan kode. Perhatikan juga `Extended addr` ≠ `EUI-64` (Tabel 13.3).

CHECKPOINT

Alamat yang tercetak pada baris `RX` di Node2 harus **sama persis** dengan Mesh-Local EID Node1. Jika tidak cocok, ada node lain di ruangan yang ikut membalas — catat, itu temuan menarik untuk analisis.

13.8.3 EXP-03 — Jarak, Kehilangan Peer, dan Latency

1. **Jarak** — geser Node2 dari 1 sampai 15 m; hitung jumlah PONG diterima per 10 PING pada tiap jarak.
2. **Kehilangan peer** — matikan Node1 selama 30 detik; amati Node2 tetap mencetak `TX PING (multicast)` tanpa `RX PONG`, lalu nyalakan lagi dan amati pemulihan **tanpa** intervensi manual.

3. **Latency** — ukur selisih waktu antara `TX PING` di Node2 dan `RX ... 'PONG'` pada 10 sampel, lalu hitung rata-ratanya.

Data capture

Parameter	Hasil
PONG diterima per 10 PING (1 m)	
Perilaku Node2 saat Node1 mati	
Waktu pemulihan setelah Node1 hidup lagi	
Latency rata-rata (10 sampel)	

Tabel 13.9: Lembar pengamatan EXP-03

CHECKPOINT

Setelah Node1 dinyalakan lagi, `RX PONG` kembali muncul di Node2 **tanpa** Node2 direset sama sekali. Bandingkan dengan M06 (relay BLE) yang tidak pulih sendiri — perbedaan ini adalah nilai jual Thread.

13.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 2 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu), capture 45 detik.

```
# Node1 (ESP32-H2, env node1)              # Node2 (ESP32-H2, env node2)
[9.416] Attached as: Child                 [7.414] Attached as: Leader
[9.416] Mesh-Local EID:                    [7.414] Mesh-Local EID:
        fdde:ad00:beef:0:9fd0:c175:...             fdde:ad00:beef:0:407e:e905:...
[9.416] Mendengarkan [ff03::abcd]:5050     [7.414] TX PING (multicast)
[10.418] RX [fdde:ad00:beef:0:407e:...]    [10.419] TX PING (multicast)
         :5050 -> 'PING'                   [10.419] RX [fdde:ad00:beef:0:9fd0:...]
[10.418] TX PONG (unicast ke pengirim)             :5050 -> 'PONG'
```

Parameter	Hasil terukur
Waktu attach Node2 dan Node1	7,4 s dan 9,4 s sejak boot
Awalan Mesh-Local EID kedua node	`fdde:ad00:beef:0:` (identik — syarat wajib)
PING dikirim / PONG diterima	12 / 12 (0 % loss)
Latency PING ke PONG	< 1 ms terukur di Serial (satu hop)

Tabel 13.10: Hasil verifikasi hardware Modul 11

CATATAN

Baris `E OT_STATE: handle_ot_role_change(105): Failed to get the active dataset` muncul sekali saat boot dan tidak berbahaya: role sempat berubah sebelum dataset selesai dibaca netif. Jaringan tetap terbentuk normal sesudahnya.

13.10 Pengukuran

Jarak	RSSI	Latency (ms)	Success
1 m			… / 10

Jarak	RSSI	Latency (ms)	Success
3 m			… / 10
5 m			… / 10
10 m			… / 10
15 m			… / 10

Tabel 13.11: Lembar pengukuran jarak Modul 11

Success adalah jumlah PONG diterima dari 10 PING; latency adalah waktu PING ke PONG dalam milidetik.

Bandingkan dengan M07 dan M08. Radionya sama (802.15.4 channel 15), jadi selisih jangkauan berasal dari lapisan di atasnya (Tabel 13.12).

Protokol	Modul	Jarak 100 % berhasil	Overhead per pesan
802.15.4 raw	M07		11 byte MHR
Zigbee	M08		
Thread (IPv6/UDP)	M11		header IPv6 dan UDP

Tabel 13.12: Tabel pembanding jangkauan dan overhead M07, M08, dan M11

13.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa latency rata-rata round-trip PING ke PONG pada jarak 1 m dan 10 m?
2. Bagaimana tren RSSI dan success rate terhadap peningkatan jarak?
3. Mengapa PING dikirim multicast sedangkan PONG unicast? Apa keuntungannya untuk jaringan berisi banyak node?
4. Apa perbedaan alamat mesh-local EID (`fd...`) dan link-local (`fe80::`) pada jaringan Thread, dan yang mana yang muncul pada log hasil percobaan?
5. Apa yang terjadi di Node2 ketika Node1 dimatikan? Mengapa PING tetap terkirim, dan apa artinya bagi desain aplikasi?

13.12 Concept Check

1. Apa itu Active Dataset dan parameter apa saja yang di-set sebelum `OThread.start()`?
2. Bagaimana sebuah node menjadi Leader pada jaringan Thread?
3. Jelaskan perbedaan alamat realm-local multicast `ff03::abcd` dengan unicast mesh-local EID.
4. Mengapa komunikasi Thread memakai IPv6 dibanding alamat 16-bit seperti Zigbee? Apa untungnya untuk Modul 13?
5. Apa fungsi `OtUdp.begin(PORT)` pada Node2 dibanding `beginMulticast()` pada Node1?

13.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah node dan menghitung loss

CH-1 — Node ketiga Tambah `node3` (salin `src/node2`, tambahkan env di `platformio.ini`). Amati bahwa PONG dari Node1 tetap hanya dikirim ke pengirim aslinya, dan bandingkan EID ketiga node — awalannya harus sama, sisanya berbeda.

CH-2 — Packet loss (wajib) Beri nomor urut pada PING (`PING:1`, `PING:2`, dan seterusnya) lalu hitung loss di sisi penerima. Contoh: 60 PING dikirim, 55 PONG diterima, sehingga loss = (60 − 55)/60 × 100 % = 8,33 %. Pisahkan: PING yang hilang dibanding PONG yang hilang.

Level 3 — Tantangan · Statistik latency dan kegagalan senyap

CH-3 — Statistik latency Ukur latency 30 sampel, hitung minimum, maksimum, rata-rata, dan simpangannya. Bandingkan dengan latency Zigbee M08 pada jarak yang sama.

CH-4 — Prefix salah, sengaja Ubah `OT_ML_PREFIX` di **satu** node saja (misalnya byte kedua menjadi `0xdd`), flash, lalu dokumentasikan gejalanya secara lengkap: apa yang tetap normal, apa yang gagal, dan bagaimana diagnosisnya ditegakkan dari log. Ini melatih membaca *kegagalan senyap*.

13.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (Thread, Leader dan Child, Active Dataset, mesh-local prefix dan EID, multicast realm-local).
3. Konfigurasi — env `node1` dan `node2`, dataset lengkap, port 5050, grup `ff03::abcd`, serta interval 3 s.
4. Hasil eksperimen — log attach, EID kedua node, sesi PING–PONG, dan hasil percobaan “buka abstraksinya”.
5. Data pengukuran — tabel bagian Pengukuran beserta tabel pembanding M07, M08, dan M11.
6. Analisis dan concept check.
7. Challenge — minimal CH-2 dan CH-4.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

14. Modul 12 — Mesh Thread IPv6 (Many-to-Many)

IoT COMMUNICATION LAB · MODUL 12

Mesh the Internet — Mesh Thread IPv6 (Many-to-Many)

ESP32-H2 · Thread · mesh IPv6 · level Advanced · 3 × 50 menit · folder kode `week12_thread_mesh`

PENGANTAR MODUL · Gambaran Umum

Modul 12 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya membangun mesh IPv6 yang mengatur dirinya sendiri, lalu merusaknya dengan sengaja untuk melihat apakah ia pulih. Percobaan berjalan sebagai mesh IPv6 tiga node dengan komunikasi many-to-many melalui multicast, diamati melalui tiga terminal Serial Monitor pada 115200 baud.

14.1 Pendahuluan

M11 membuktikan dua node Thread bisa saling kirim datagram. Di sini jumlahnya tiga, dan **tidak ada peran yang ditentukan manusia** — firmware ketiga node identik kecuali `NODE_ID`. Bandingkan dengan Zigbee M10, tempat peran ZC, ZR, dan ZED dipilih saat kompilasi: perbedaan ini adalah salah satu argumen terkuat Thread, dan angka pemulihannya menjadi bahan M16.

Prasyaratnya ada dua: M11 untuk Active Dataset, mesh-local prefix, dan socket UDP IPv6; serta M10 untuk konsep routing multi-hop. Yang dibangun di sini adalah role election otomatis, komunikasi many-to-many melalui multicast, deteksi loss berbasis counter, serta uji kegagalan node perantara beserta pemulihannya. Semuanya dipakai lagi pada M13 ketika mesh ini disambungkan ke Wi-Fi lewat gateway, M15 pada pipeline end-to-end, dan M16 ketika keandalan mesh menjadi kriteria pemilihan protokol.

Peta modul blok Thread

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
11	Thread P2P: alamat IPv6, datagram UDP
12 (ini)	**Mesh many-to-many, role dipilih sendiri, self-healing**
13	Mesh Thread bertemu Wi-Fi lewat gateway ESP32-C6

Tabel 14.1: Peta modul blok Thread

Kontrak data lab ini. Payload `NODE<n>:<counter>` membawa **identitas sumber dan nomor urut** sekaligus. Format ini yang membuat loss bisa dihitung tanpa alat bantu apa pun — cukup mencari lompatan angka di log. Pola yang sama dipakai di M13 dan M15 sebagai `suhu:XX.X,#<seq>`.

14.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membangun mesh IPv6 yang mengatur dan memulihkan dirinya

1. Membangun mesh Thread tiga node dengan dataset identik dan mencatat role yang dipilih stack untuk masing-masing (Leader, Router, atau Child).
2. Membuktikan komunikasi many-to-many: setiap node menerima pesan dari dua node lainnya, dicocokkan lewat alamat sumber dan counter.
3. Menghitung packet loss per node dari lompatan counter, dan mengukur latency 1 hop dibanding 2 hop pada formasi garis.
4. Mengevaluasi self-healing: mengukur berapa lama mesh pulih setelah node relayer dimatikan, dan membandingkannya dengan relay manual M06 serta Zigbee M10.

Kriteria keberhasilan

- ☐ Ketiga node saling menerima pesan (many-to-many).
- ☐ Role tiap node teridentifikasi dan tercatat, termasuk perubahannya setelah node dimatikan.
- ☐ Perilaku mesh saat relayer mati terdokumentasi dengan counter, bukan dengan kesan.
- ☐ Awalan Mesh-Local EID ketiga node sama.

14.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (MLE, Router ID assignment, partition merge, MPL) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 14.2.

Istilah	Definisi kerja di lab ini
Thread mesh	Setiap node Router dapat meneruskan paket node lain; jalur dibentuk otomatis.
Leader	Satu node terpilih mengelola jaringan; lainnya Router atau Child (`otGetDeviceRole()`).
Mesh-Local EID (`fd..`)	Alamat IPv6 unik per node dalam domain mesh (`fdde:ad00:beef:0:...`).
Mesh-Local prefix	Prefix `/64` milik jaringan; wajib identik di ketiga node, jika tidak multicast `ff03::` tidak diteruskan.
Link-local (`fe80::`)	Alamat per-interface antar tetangga 1 hop.
Multicast `ff03::abcd`	Realm-local; paket diteruskan ke seluruh mesh, semua anggota grup menerimanya.
UDP port 5050	Port aplikasi kirim dan terima (`OtUdp.beginMulticast(GROUP, PORT)`).
Sequence/counter	Pesan `NODE<n>:<count>` sehingga paket hilang terdeteksi dari lompatan counter.

Tabel 14.2: Istilah kerja Modul 12

Dataset harus identik, termasuk prefix mesh-local. `DataSet::initNew()` mengacak prefix mesh-local tiap board. Ketiga firmware karena itu memaksa prefix yang sama sebelum `OThread.start()` seperti Kode 14.1.

```
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] =
    {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};   // fdde:ad00:beef::/64
otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
```

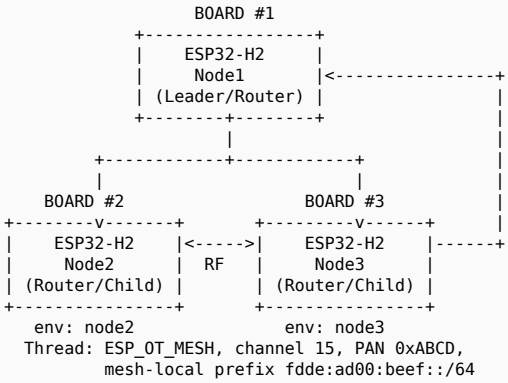
Kode 14.1: Prefix mesh-local yang dipaksakan di ketiga node

Bila dilewatkan, node tetap attach dan `Attached as: ...` tetap tercetak, tetapi tidak ada satu pun baris `RX` — gejala yang mudah disalahartikan sebagai masalah jarak atau interferensi. Periksa awalan Mesh-Local EID: harus sama di semua node.

Sekuens protokol yang diamati

```
Node1 ──TX "NODE1:1" (multicast)──► Node2, Node3
Node2 ──TX "NODE2:1" (multicast)──► Node1, Node3   (tiap 5 detik, semua node)
Node3 ──TX "NODE3:1" (multicast)──► Node1, Node2
```

14.4 Topologi



Node	Board	Env	`NODE_ID`	Role Thread	Aksi
Node1	ESP32-H2	`node1`	1	Leader/Router/Child (dinamis)	TX `NODE1:<n>` multicast tiap 5 s
Node2	ESP32-H2	`node2`	2	Leader/Router/Child (dinamis)	TX `NODE2:<n>` multicast tiap 5 s
Node3	ESP32-H2	`node3`	3	Leader/Router/Child (dinamis)	TX `NODE3:<n>` multicast tiap 5 s

Tabel 14.3: Peran tiap node Modul 12

Ketiganya **ESP32-H2 DevKitM-1** dengan firmware yang sama kecuali `NODE_ID`. Role Thread dipilih otomatis oleh stack dan bisa berbeda tiap kali dinyalakan — bukan ditentukan oleh nama environment.

Formasi awal: segitiga dengan jarak antar node ± 3 m. Formasi garis (untuk membuktikan multi-hop) dipakai di EXP-03.

14.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan OpenThread (`OThread` dan `OThreadUDP`).

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1 (env `node1`, `node2`, `node3`)	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Power bank atau catu daya USB	wajib untuk formasi garis	3
5	Ruang uji	area untuk segitiga ±3 m dan garis panjang	—

Tabel 14.4: Alat dan bahan Modul 12

Parameter jaringan yang wajib identik di ketiga node

Parameter	Nilai
Network name	`ESP_OT_MESH`
Channel dan PAN ID	15 dan `0xABCD`
Ext PAN ID	`DE:AD:00:BE:EF:00:CA:FE`
Mesh-local prefix	`fdde:ad00:beef::/64`
Grup multicast dan port	`ff03::abcd` : 5050
Interval TX	5000 ms

Tabel 14.5: Parameter jaringan Thread Modul 12

14.6 Kode Program

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>

[env:node3]
build_src_filter = +<node3/*.cpp>
```

Kode 14.2: `platformio.ini` Modul 12 pada repositori

```
// Minggu 12 — Thread Mesh: Node 1
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

#define NODE_ID 1

const char OT_NETWORK_NAME[] = "ESP_OT_MESH";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama untuk semua node. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
```

```
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printNetworkInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

void setup() {
  Serial.begin(115200);
  Serial.printf("Node%d (Thread) starting...\n", NODE_ID);

  OThread.begin(false);

  // Dataset selalu ditulis ulang agar seluruh node memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
  printNetworkInfo();

  OtUdp.beginMulticast(GROUP, PORT);
  Serial.println("Bergabung ke mesh, siap kirim/terima.");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (millis() - last > 5000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "NODE%d:%lu", NODE_ID, (unsigned long)++count);
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.printf("TX multicast: %s\n", msg);
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[32];
```

```
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX [%s]: %s\n", OtUdp.remoteIP().toString().c_str(), buf);
  }
  delay(10);
}
```

Kode 14.3: `src/node1/main.cpp` — anggota mesh dengan `NODE_ID` 1

```
// Minggu 12 — Thread Mesh: Node 2
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

#define NODE_ID 2

const char OT_NETWORK_NAME[] = "ESP_OT_MESH";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama untuk semua node. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printNetworkInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

void setup() {
```

```
  Serial.begin(115200);
  Serial.printf("Node%d (Thread) starting...\n", NODE_ID);

  OThread.begin(false);

  // Dataset selalu ditulis ulang agar seluruh node memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
  printNetworkInfo();

  OtUdp.beginMulticast(GROUP, PORT);
  Serial.println("Bergabung ke mesh, siap kirim/terima.");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (millis() - last > 5000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "NODE%d:%lu", NODE_ID, (unsigned long)++count);
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.printf("TX multicast: %s\n", msg);
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[32];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX [%s]: %s\n", OtUdp.remoteIP().toString().c_str(), buf);
  }
  delay(10);
}
```

Kode 14.4: `src/node2/main.cpp` — anggota mesh dengan `NODE_ID` 2

```
// Minggu 12 — Thread Mesh: Node 3
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

#define NODE_ID 3

const char OT_NETWORK_NAME[] = "ESP_OT_MESH";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama untuk semua node. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai.
```

```
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printNetworkInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

void setup() {
  Serial.begin(115200);
  Serial.printf("Node%d (Thread) starting...\n", NODE_ID);

  OThread.begin(false);

  // Dataset selalu ditulis ulang agar seluruh node memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
  printNetworkInfo();

  OtUdp.beginMulticast(GROUP, PORT);
  Serial.println("Bergabung ke mesh, siap kirim/terima.");
}

void loop() {
  static unsigned long last = 0;
  static uint32_t count = 0;
  if (millis() - last > 5000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "NODE%d:%lu", NODE_ID, (unsigned long)++count);
```

```
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.printf("TX multicast: %s\n", msg);
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[32];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX [%s]: %s\n", OtUdp.remoteIP().toString().c_str(), buf);
  }
  delay(10);
}
```

Kode 14.5: `src/node3/main.cpp` — anggota mesh dengan `NODE_ID` 3

14.6.1 Channel antar Kelompok (wajib kelas paralel)

Seperti pada blok Zigbee (Modul 08–10), satu ruangan praktikum berisi beberapa kelompok sehingga berdiri beberapa jaringan Thread sekaligus. Pada Thread, channel adalah bagian dari **Active Dataset** yang ditulis firmware ke tiap node sebelum `OThread.start()`; jaringan terbentuk pada channel dataset itu, dan node lain attach ke sana. Karena network name, PAN ID, dan network key dibuat sama di semua kelompok, pemisah antar kelompok yang andal adalah **channel yang berbeda** — beda channel berarti radio tuli total.

Jatah untuk enam kelompok sama dengan blok Zigbee. Ubah satu baris `OT_CHANNEL` pada **ketiga** berkas modul ini (Kode 14.3, Kode 14.4, dan Kode 14.5):

Kelompok	Channel	Baris yang diubah
1	15	`const uint8_t OT_CHANNEL = 15;`
2	20	`const uint8_t OT_CHANNEL = 20;`
3	25	`const uint8_t OT_CHANNEL = 25;`
4	11	`const uint8_t OT_CHANNEL = 11;`
5	17	`const uint8_t OT_CHANNEL = 17;`
6	22	`const uint8_t OT_CHANNEL = 22;`

Tabel 14.6: Jatah channel Thread untuk enam kelompok — satu nilai untuk semua node kelompok

PERINGATAN
Semua node dalam satu kelompok harus memakai nilai `OT_CHANNEL` yang sama. Bila satu node berbeda channel, ia tidak akan pernah attach dan berhenti menunggu selamanya. Setelah upload, cocokkan baris `Channel` pada `Info`
`network Thread` semua node.

TIP
Mengganti `OT_CHANNEL` tidak memerlukan erase NVS: dataset ditulis ulang setiap boot (`commitDataSet`), jadi cukup build + upload ulang seluruh board kelompok. Channel 15, 20, dan 25 adalah pilihan klasik karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah.

14.7 Build dan Flash

```
for e in node1 node2 node3; do pio run -d week12_thread_mesh -e $e -t erase; done
pio run -d week12_thread_mesh -e node1 -t upload
pio run -d week12_thread_mesh -e node2 -t upload
pio run -d week12_thread_mesh -e node3 -t upload -t monitor
```

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat dan diisikan pada tiap env.
- ☐ Channel kelompok disetel (`OT_CHANNEL`) sama di ketiga berkas dan berbeda dari kelompok lain — lihat Tabel 14.6.
- ☐ **Hapus dataset lama** ketiga board (`-t erase`) bila pernah dipakai modul Thread lain.
- ☐ Flash environment `node1`, `node2`, dan `node3` (firmware sama, `NODE_ID` 1, 2, 3).
- ☐ Serial Monitor 115200 baud pada ketiga board (3 terminal).
- ☐ Dataset identik: `ESP_OT_MESH`, channel 15, PAN `0xABCD`, port 5050, prefix `fdde:ad00:beef::/64`.
- ☐ Formasi segitiga ± 3 m sudah disiapkan.

14.8 Percobaan

14.8.1 EXP-01 — Pembentukan Mesh dan Pemilihan Role

Nyalakan ketiga board. Tiap node men-commit dataset (termasuk prefix mesh-local), start, menunggu attach (role sekurangnya Child), lalu join grup multicast. Board yang selesai boot lebih dulu umumnya terpilih menjadi Leader.

```
[board pertama] dataset (+ ML prefix) ──► start ──► Leader
[board kedua]   dataset (+ ML prefix) ──► start ──► attach ──► Router/Child
[board ketiga]  dataset (+ ML prefix) ──► start ──► attach ──► Router/Child
```

Data capture

Parameter	Hasil
Role Node1	… (Leader/Router/Child)
Role Node2	… (Leader/Router/Child)
Role Node3	… (Leader/Router/Child)
Awalan Mesh-Local EID sama di ketiga node?	… (harus `fdde:ad00:beef:0:`)
Waktu attach tiap node (detik)	

Tabel 14.7: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Matikan board yang menjadi Leader, tunggu, lalu amati role kedua node sisanya di Serial Monitor. Salah satunya akan **naik menjadi Leader** tanpa satu baris kode tambahan. Catat berapa lama peralihan itu. Bandingkan dengan M10: di Zigbee, coordinator tidak bisa digantikan — matinya ZC berarti matinya jaringan.

CHECKPOINT

Ketiga node mencetak `Attached as: ...` dan awalan EID ketiganya sama. Jika ada yang berbeda, node itu berada di jaringan lain — hapus NVS-nya dan flash ulang.

14.8.2 EXP-02 — Multicast Many-to-Many

Setiap 5 detik tiap node mengirim `NODEn:<counter>` ke `ff03::abcd:5050` dan menerima pesan node lain; alamat sumber mesh-local tercetak pada baris RX.

```
[tiap node] TX multicast ──► [semua node lain] RX [fdde:ad00:beef:0:...]: NODEx:c
```

Expected output — contoh Node1 (dari uji di board; alamat dan peran di board Anda akan berbeda)

```
Node1 (Thread) starting...
Attached as: Router        <- bisa Leader atau Child, tergantung urutan boot
Info network Thread:
  Peran          : Router
  Network name   : ESP_OT_MESH
  Channel        : 15
  PAN ID         : 0xABCD
  Extended PAN ID: DE:AD:00:BE:EF:00:CA:FE
  RLOC16         : 0x1C00
  Extended addr  : 56:22:E4:9A:19:AC:56:CA
  EUI-64         : 74:4D:BD:FF:FE:61:E6:2C
  Mesh-Local EID : fdde:ad00:beef:0:d44d:fcf1:5002:f8c7
Bergabung ke mesh, siap kirim/terima.
TX multicast: NODE1:1
RX [fdde:ad00:beef:0:6c6f:63ae:ba2c:a98d]: NODE2:1
RX [fdde:ad00:beef:0:f915:1df2:1a01:26c9]: NODE3:2
TX multicast: NODE1:2
RX [fdde:ad00:beef:0:6c6f:63ae:ba2c:a98d]: NODE2:2
```

Blok info network ketiga node harus sama pada **Network name, Channel, PAN ID, dan Extended PAN ID**. Dari `RLOC16` terbaca topologinya: pada uji ini Node3 Leader `0x6C00`, Node1 Router `0x1C00`, dan Node2 Child `0x1C01` — Router ID-nya sama dengan Node1, jadi **parent Node2 adalah Node1**. Padanan alamat XBee/Zigbee/Thread ada di Modul 11.

CHECKPOINT

Di **setiap** Serial Monitor muncul baris RX dari **dua** sumber berbeda (bukan satu). Jika hanya satu, node ketiga belum masuk mesh atau prefix-nya berbeda. Cocokkan juga counter: harus berurutan tanpa lompat.

14.8.3 EXP-03 — Perubahan Topologi dan Kegagalan Node

1. **Formasi garis** N1, N2, N3 (N2 di tengah, N1 dan N3 dijauhkan sampai tidak saling terjangkau): amati apakah pesan NODE3 tetap diterima N1 (multi-hop) dengan counter berlanjut.
2. **Relayer mati** — pada formasi garis, matikan N2; amati apakah N1 dan N3 saling kehilangan paket. Catat counter terakhir yang diterima.
3. Nyalakan N2 lagi dan catat **waktu pemulihan** sampai counter kembali mengalir.
4. **Jarak** — regangkan jarak antar node dan amati kenaikan loss dari lompatan counter.

Data capture

Parameter	Hasil
Counter NODE3 terakhir diterima N1 sebelum N2 mati	
Apakah N1 dan N3 benar-benar putus saat N2 mati?	
Waktu pemulihan setelah N2 hidup lagi (s)	
Latency 1 hop dibanding 2 hop	
Role setelah N2 kembali — berubah?	

Tabel 14.8: Lembar pengamatan EXP-03

CHECKPOINT

Tersedia bukti kuantitatif untuk dua klaim terpisah: (a) pesan N3 mencapai N1 lewat N2, dan (b) mesh pulih sendiri. Klaim (a) hanya sah bila N1 dan N3 memang **tidak** saling terjangkau langsung — buktikan dulu dengan mematikan N2 dan melihat aliran berhenti.

14.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu, formasi meja kurang dari 1 m), capture 50 detik.

```
# Node1 (ESP32-H2)              # Node3 (ESP32-H2)
[9.617] Attached as: Router     [7.410] Attached as: Leader
[9.817] TX multicast: NODE1:1   [7.410] TX multicast: NODE3:1
                                [9.814] RX [...:385a:...]: NODE1:1
                                [11.016] RX [...:390e:...]: NODE2:1
```

Parameter	Hasil terukur
Role terpilih (Node1, Node2, Node3)	Router, Child, Leader
Awalan Mesh-Local EID ketiga node	`fdde:ad00:beef:0:` (identik)
Pesan diterima tiap node dari 2 node lain	9 + 9, counter berurutan tanpa lompatan
Loss	0 % pada jarak meja

Tabel 14.9: Hasil verifikasi hardware Modul 12

Karena semua node saling terjangkau langsung, hasil ini **belum** membuktikan routing multi-hop — itu tugas EXP-03 formasi garis.

14.10 Pengukuran

Tabel jarak (amati RX pada node terjauh, 10 pesan sumber).

Jarak	RSSI	Latency	Success
1 m			… / 10
3 m			… / 10
5 m			… / 10
10 m			… / 10
15 m			… / 10

Tabel 14.10: Lembar pengukuran jarak Modul 12

Tabel per-node (amati 1 menit, tiap node ± 12 TX).

Node	RSSI	Paket diterima	Loss (%)
Node1 (dari N2 dan N3)		… / 24	
Node2 (dari N1 dan N3)		… / 24	
Node3 (dari N1 dan N2)		… / 24	

Tabel 14.11: Lembar pengukuran per-node Modul 12

Tabel pembanding self-healing — wajib. Isi dari data tiga modul.

Pendekatan	Modul	Penentu jalur	Pulih sendiri?	Waktu pemulihan
Relay manual (BLE)	M06	kode aplikasi		
Routing Zigbee	M10	stack Zigbee		
Mesh Thread	M12	stack Thread		

Tabel 14.12: Tabel pembanding self-healing M06, M10, dan M12

14.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Role apa yang dipegang tiap node setelah attach? Apakah berubah setelah node dimatikan lalu dinyalakan?
2. Pada formasi garis, berapa hop yang ditempuh pesan NODE3 ke NODE1, dan apa buktinya dari latency atau loss?
3. Bagaimana pola loss tiap node ketika jarak diperbesar? Apakah merata atau ada node yang lebih rentan?
4. Apa yang terjadi pada komunikasi N1 dan N3 ketika N2 (relayer) dimatikan? Jelaskan dari counter yang berhenti atau berlanjut.
5. Bandingkan keandalan mesh ini dengan multi-node Zigbee M09 dan M10 serta relay manual M06, memakai tabel pembanding self-healing.

14.12 Concept Check

1. Bagaimana Thread memilih Leader dan apa tugas Leader dalam mesh?
2. Mengapa pesan multicast `ff03::abcd` dapat diterima semua node tanpa alamat tujuan per node?
3. Apa perbedaan alamat mesh-local EID (`fd...`) dan link-local (`fe80::`) dalam forwarding mesh?
4. Apa yang membuat node Router dapat meneruskan paket sedangkan Child tidak?
5. Bagaimana format pesan `NODEn:counter` membantu deteksi paket hilang, dan apa yang tidak bisa ia deteksi?

14.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah node dan menghitung loss dari counter

CH-1 — Node keempat Tambah `node4`: salin `src/node3`, ubah `NODE_ID` menjadi 4, lalu tambahkan env `node4` di `platformio.ini`. Amati total pesan RX per menit di tiap node dan periksa apakah loss naik.

CH-2 — Packet loss dari counter (wajib) Hitung loss otomatis dari lompatan counter per sumber. Contoh: Node2 mengirim `NODE2:1..60`, Node1 menerima 54, sehingga loss = (60 − 54)/60 × 100 % = 10 %.

Level 3 — Tantangan · Memetakan hop dan mengukur self-healing

CH-3 — Pemetaan hop Pindahkan Node3 sehingga harus 2 hop ke Node1; bandingkan latency 1 hop dan 2 hop dari stempel waktu TX dan RX. Sajikan sebagai tabel jarak, hop, dan latency.

CH-4 — Ukur self-healing secara kuantitatif Matikan Leader (bukan relayer), catat berapa detik sampai ada node lain mengambil alih dan aliran pesan kembali normal. Ulangi 5 kali, lalu laporkan rata-rata dan sebarannya.

14.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (mesh Thread, role election, multicast realm-local, forwarding, mesh-local prefix).
3. Konfigurasi — env `node1` sampai `node3`, dataset `ESP_OT_MESH`, port 5050, grup `ff03::abcd`, serta interval 5 s.
4. Hasil eksperimen — log ketiga node, skema formasi (segitiga dan garis), serta hasil percobaan "buka abstraksinya".
5. Data pengukuran — tabel jarak, tabel per-node, dan tabel pembanding self-healing M06, M10, dan M12.
6. Analisis dan concept check.
7. Challenge — minimal CH-2 dan CH-4.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

15. Modul 13 — Gateway Thread ke Wi-Fi (H2 + C6)

IoT COMMUNICATION LAB · MODUL 13

Bridge Thread to Wi-Fi — Gateway Thread ke Wi-Fi (H2 + C6)

ESP32-H2 + ESP32-C6 · Thread ke HTTP · level Advanced · 3 × 50 menit · folder kode `week13_thread_wifi_gateway`

PENGANTAR MODUL · Gambaran Umum

Modul 13 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya menjembatani dua radio dalam satu perangkat dan membawa data sensor keluar ke jaringan IP. Percobaan berjalan dalam mode gateway dwi-radio, diamati melalui dua terminal Serial Monitor pada 115200 baud beserta kode respons HTTP sebagai bukti data benar-benar sampai.

15.1 Pendahuluan

Sampai M12 data tidak pernah keluar dari jaringan 802.15.4. Modul ini adalah **titik keluar**: ESP32-C6 menjalankan dua stack sekaligus dan meneruskan payload apa adanya ke server HTTP. Ini juga modul pertama yang benar-benar membutuhkan **dua jenis board** — sebuah batasan perangkat keras, bukan firmware.

Prasyaratnya ada dua: M11–M12 untuk Active Dataset, mesh-local prefix, dan UDP multicast; serta M04 dan M06 untuk konsep transparent forwarding. Yang dibangun di sini adalah gateway dwi-radio yang menjalankan Thread dan Wi-Fi bersamaan, penerusan UDP menjadi HTTP POST, pemakaian tabel partisi `huge_app.csv`, dan penanganan saat Wi-Fi terputus. Semuanya dipakai lagi pada M14 ketika sisi IP diganti MQTT tanpa Thread, M15 ketika kedua sisi digabung menjadi pipeline penuh, dan M16 ketika arsitektur gateway dipakai untuk membandingkan protokol.

Peta modul blok integrasi

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
11–12	Thread: IPv6 dan mesh — data masih di dalam 802.15.4
13 (ini)	**Gateway: Thread bertemu Wi-Fi, data keluar ke IP (HTTP)**
14	Sisi IP diperdalam: MQTT publish/subscribe di C6
15	M12, M13, dan M14 digabung: sensor ke Thread ke C6 ke MQTT ke dashboard

Tabel 15.1: Peta modul blok integrasi

Kontrak data lab ini. Payload `suhu:XX.X` diteruskan **tanpa diubah** dari Thread ke HTTP; gateway hanya membungkusnya dalam JSON. Prinsip ini (*transparent forwarding*, sama seperti relay M06) membuat M15 bisa mengganti HTTP dengan MQTT tanpa menyentuh firmware node sensor sama sekali.

15.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menjembatani Thread dan Wi-Fi dalam satu perangkat

1. Mengonfigurasi satu ESP32-C6 sebagai Thread Leader **dan** Wi-Fi STA secara bersamaan, serta menunjukkan urutan inisialisasi keduanya di kode.
2. Menerima datagram UDP multicast Thread dari node ESP32-H2 di sisi gateway dan menampilkan alamat sumbernya.
3. Meneruskan payload ke server HTTP dan memverifikasi keberhasilannya dari kode respons (HTTP 200).
4. Menghitung packet loss pada dua hop terpisah — Thread (H2 ke C6) dan Wi-Fi/HTTP (C6 ke server) — dan menentukan hop mana yang menjadi penyebab bila ada data hilang.

Kriteria keberhasilan

- ☐ Pesan `suhu:XX.X` dari H2 tiba di server HTTP dengan status **HTTP 200**.
- ☐ Latency end-to-end terukur pada tiga skenario jarak.
- ☐ Packet loss Thread ke gateway terhitung terpisah dari loss Wi-Fi ke server.
- ☐ Perilaku saat Wi-Fi diputus diuji dan tercatat.

15.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (Thread Border Router penuh, SRP/DNS-SD, NAT64, koeksistensi radio) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 15.2.

Istilah	Definisi kerja di lab ini
Gateway	Perangkat yang menghubungkan dua jaringan berbeda protokol (Thread dan Wi-Fi) dan meneruskan data antar keduanya.
Border Router	Router tepi jaringan Thread yang menyediakan konektivitas ke jaringan IP eksternal — C6 di sini berfungsi sebagai versi sederhananya.
Thread Leader	Perangkat Thread yang mengelola dataset aktif jaringan.
UDP Multicast	Pesan dikirim ke grup IPv6 `ff03::abcd` port 5050 sehingga semua anggota grup menerimanya.
Wi-Fi STA	Mode station: C6 bergabung ke access point dan memperoleh alamat IP.
HTTP POST	Pengiriman data ke server; payload dibungkus JSON `{"sensor":"h2","data":"..."}`.
`huge_app.csv`	Tabel partisi besar; firmware Thread, Wi-Fi, dan HTTP melebihi partisi app default 1,25 MB.

Tabel 15.2: Istilah kerja Modul 13

Mengapa H2 dan C6 wajib memakai prefix mesh-local yang sama. `DataSet::initNew()` mengacak network key, ext PAN ID, dan **prefix mesh-local** di tiap board. Kedua firmware karena itu menimpa field-field tersebut dengan konstanta dan memaksa prefix lewat `otThreadSetMeshLocalPrefix()` sebelum `OThread.start()` (Kode 15.1).

```
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] =
    {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};   // fdde:ad00:beef::/64
```

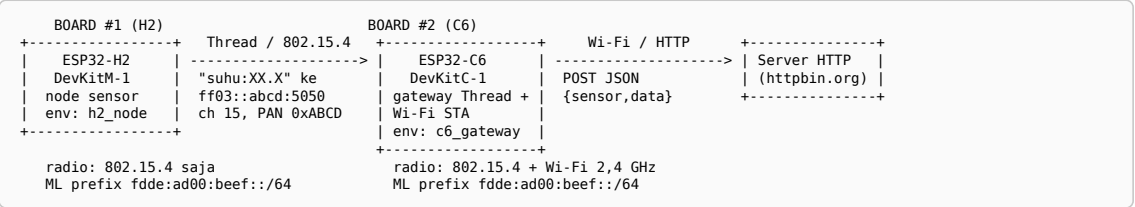
Kode 15.1: Prefix mesh-local yang wajib identik di H2 dan C6

Tanpa itu H2 dan C6 tetap attach dan Serial Monitor keduanya tampak sehat, tetapi gateway tidak pernah mencetak satu pun baris `RX via Thread` — paket multicast `ff03::abcd` tidak diteruskan antar prefix mesh-local yang berbeda. Pemeriksaan cepat: awalan Mesh-Local EID kedua board harus sama persis.

Sekuens protokol yang diamati

```
[ H2 sensor ] ──(Thread UDP)──► [ C6 gateway ] ──(HTTP POST JSON)──► [ Server ]
   readSensor      OtUdp            parsePacket       HTTPClient
   "suhu:25.3"     multicast        forwardToWifi()   http.POST()
```

15.4 Topologi



Node	Board	Environment	Peran	Aksi
Node sensor	**ESP32-H2** DevKitM-1	`h2_node`	Thread node (Child)	TX `suhu:XX.X` multicast tiap 3 s
Gateway	**ESP32-C6** DevKitC-1	`c6_gateway`	Thread Leader dan Wi-Fi STA	RX Thread lalu HTTP POST

Tabel 15.3: Peran tiap node Modul 13

Inilah modul pertama yang benar-benar butuh dua jenis board. ESP32-H2 punya radio 802.15.4 tetapi **tidak punya Wi-Fi**; ESP32-C6 punya keduanya, sehingga hanya C6 yang bisa memegang kaki Thread dan kaki Wi-Fi sekaligus. Menukar peran (H2 sebagai gateway) tidak mungkin — bukan soal firmware, tetapi soal radio yang tersedia di chip. Karena itu `platformio.ini` modul ini memakai `board` berbeda per environment (`esp32-h2-devkitm-1` dibanding `esp32-c6-devkitc-1`), tidak seperti Modul 01–12 yang seragam ESP32-H2.

15.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (node Thread) dan ESP32-C6 (gateway Thread dan Wi-Fi).

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1 — node sensor Thread	1
2	Board ESP32-C6	DevKitC-1 — gateway Thread ke Wi-Fi	1
3	Kabel USB data	kabel data, bukan *charge-only*	2
4	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
5	Wi-Fi atau hotspot	2,4 GHz, ada akses internet (C6 tidak mendukung 5 GHz)	1
6	Server HTTP tujuan	default `http://httpbin.org/post`, atau server lokal (`http_sink.py`)	1

Tabel 15.4: Alat dan bahan Modul 13

Konfigurasi jaringan

Parameter	Nilai
Thread network	`ESP_OT_GW`, channel 15, PAN `0xABCD`
Mesh-local prefix	`fdde:ad00:beef::/64` (identik di H2 dan C6)
Grup multicast dan port	`ff03::abcd` : 5050
Interval telemetri	3000 ms
Topic atau URL tujuan	`SERVER_URL` di `src/c6_gateway/main.cpp`

Tabel 15.5: Konfigurasi jaringan Modul 13

15.6 Kode Program

Dua board berbeda dalam satu proyek (Kode 15.2).

```
[env:h2_node]
board = esp32-h2-devkitm-1
build_src_filter = +<h2_node/*.cpp>
upload_port  = /dev/ttyACM0

[env:c6_gateway]
board = esp32-c6-devkitc-1
board_build.partitions = huge_app.csv    ; firmware Thread+Wi-Fi+HTTP > 1,25 MB
build_src_filter = +<c6_gateway/*.cpp>
upload_port  = /dev/ttyACM2
```

Kode 15.2: Potongan `platformio.ini` — dua jenis board dalam satu proyek

```
[platformio]
default_envs = c6_gateway

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
monitor_speed = 115200

[env:h2_node]
board = esp32-h2-devkitm-1
build_src_filter = +<h2_node/*.cpp>

[env:c6_gateway]
board = esp32-c6-devkitc-1
build_src_filter = +<c6_gateway/*.cpp>
; Thread/BLE + Wi-Fi + MQTT/HTTP melebihi partisi app default (1,25 MB)
board_build.partitions = huge_app.csv
```

Kode 15.3: `platformio.ini` Modul 13 pada repositori

```
// Minggu 13 — Thread -> Wi-Fi Gateway: Node sensor (H2) kirim telemetry via Thread
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

const char OT_NETWORK_NAME[] = "ESP_OT_GW";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
```

```
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama di H2 dan C6. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai ke gateway.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printThreadInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Sensor H2 (Thread node) starting...");

  OThread.begin(false);
  // Dataset selalu ditulis ulang agar H2 dan C6 memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu join ke gateway (C6)...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());
```

```
  printThreadInfo();

  OtUdp.begin(PORT);
}

void loop() {
  static unsigned long last = 0;
  if (millis() - last > 3000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "suhu:%.1f", readSensor());
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.printf("TX via Thread: %s\n", msg);
  }
  delay(10);
}
```

Kode 15.4: `src/h2_node/main.cpp` — node sensor Thread di ESP32-H2

```
// Minggu 13 — Thread -> Wi-Fi Gateway: C6 terima telemetry Thread, teruskan via Wi-Fi
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>
#include "esp_netif.h"
#include "esp_coexist.h"

const char *WIFI_SSID = "SprH-3";
const char *WIFI_PASS = "yourpassword!";

// Batas waktu menunggu Wi-Fi/MQTT saat boot; melewati batas ini setup() tetap
// dilanjutkan agar kegagalan sisi IP tidak menyembunyikan sisi radio lain.
const unsigned long WIFI_TIMEOUT_MS = 30000;
// Jeda antar percobaan sambung ulang. Harus lebih panjang dari durasi
// asosiasi pada sinyal lemah, kalau tidak percobaan berikutnya justru
// membatalkan yang sedang berjalan.
const unsigned long WIFI_RETRY_MS = 20000;
const unsigned long MQTT_TIMEOUT_MS = 15000;

// Endpoint tujuan di sisi Wi-Fi (ganti sesuai server Anda)
const char *SERVER_URL = "http://192.168.1.5:8080/post";

const char OT_NETWORK_NAME[] = "ESP_OT_GW";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama di H2 dan C6. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai ke gateway.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

// Board ini punya dua netif sekaligus (Wi-Fi STA dan OpenThread). Pastikan
// default netif lwIP tetap Wi-Fi STA supaya trafik IPv4 keluar (HTTP/MQTT)
// punya rute yang benar. Catatan hasil uji: ini praktik yang benar, tetapi
// BUKAN obat untuk kegagalan TCP karena starvation koeksistensi — lihat
// Bagian 7 README.
```

```
static void restoreWifiAsDefaultNetif() {
  esp_netif_t *sta = esp_netif_get_handle_from_ifkey("WIFI_STA_DEF");
  if (sta == nullptr) {
    Serial.println("Netif Wi-Fi STA tidak ditemukan; rute IPv4 mungkin salah.");
    return;
  }
  esp_err_t err = esp_netif_set_default_netif(sta);
  Serial.printf("Default netif dikembalikan ke Wi-Fi STA (err=%d)\n", (int)err);
}

OThreadUDP OtUdp;

// Cetak 8 byte sebagai XX:XX:...:XX (Extended PAN ID, alamat 64-bit).
static void printHex8(const char *label, const uint8_t *b) {
  Serial.print(label);
  if (b == nullptr) {
    Serial.println("?");
    return;
  }
  for (int i = 0; i < 8; i++) {
    Serial.printf(i ? ":%02X" : "%02X", b[i]);
  }
  Serial.println();
}

// Cetak parameter jaringan Thread yang sedang dipakai node ini. Getter
// OThread sudah mengunci stack OpenThread sendiri.
static void printThreadInfo() {
  uint8_t eui64[8];
  bool hasEui = OThread.getEui64(eui64);

  Serial.println("Info network Thread:");
  Serial.printf("  Peran          : %s\n", OThread.otGetStringDeviceRole());
  Serial.printf("  Network name   : %s\n", OThread.getNetworkName().c_str());
  Serial.printf("  Channel        : %u\n", OThread.getChannel());
  Serial.printf("  PAN ID         : 0x%04X\n", OThread.getPanId());
  printHex8("  Extended PAN ID: ", OThread.getExtendedPanId());
  Serial.printf("  RLOC16         : 0x%04X\n", OThread.getRloc16());
  printHex8("  Extended addr  : ", OThread.getExtendedAddress());
  printHex8("  EUI-64         : ", hasEui ? eui64 : nullptr);
  Serial.printf("  Mesh-Local EID : %s\n", OThread.getMeshLocalEid().toString().c_str());
}

// Cetak parameter sisi Wi-Fi gateway. Channel Wi-Fi dicetak karena berbagi
// pita 2,4 GHz dengan channel 802.15.4 Thread (koeksistensi).
static void printWifiInfo() {
  Serial.println("Info network Wi-Fi:");
  if (WiFi.status() != WL_CONNECTED) {
    Serial.printf("  Status         : belum tersambung (status=%d)\n", (int)WiFi.status());
    Serial.printf("  MAC            : %s\n", WiFi.macAddress().c_str());
    return;
  }
  Serial.printf("  SSID           : %s\n", WiFi.SSID().c_str());
  Serial.printf("  IP             : %s\n", WiFi.localIP().toString().c_str());
  Serial.printf("  RSSI           : %d dBm\n", (int)WiFi.RSSI());
  Serial.printf("  Channel Wi-Fi  : %d\n", (int)WiFi.channel());
  Serial.printf("  BSSID (AP)     : %s\n", WiFi.BSSIDstr().c_str());
  Serial.printf("  MAC            : %s\n", WiFi.macAddress().c_str());
}

// SIMULASI sensor H2 (tidak ada board fisik): bangkitkan nilai suhu sintetis
// agar alur gateway (forward -> Wi-Fi -> HTTP) tetap bisa diuji end-to-end.
static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);

  // URUTAN PENTING: Thread dinyalakan LEBIH DULU, baru Wi-Fi.
  // Wrapper OpenThread Arduino memanggil esp_event_loop_create_default() dan
  // menganggap ESP_ERR_INVALID_STATE sebagai kegagalan fatal. Kalau Wi-Fi
  // diinisialisasi duluan, event loop default sudah ada, sehingga
  // OThread.begin() gagal dan board panic
  // ("assert failed: otTaskletsSignalPending"). Sebaliknya, stack Wi-Fi
  // Arduino memang mentoleransi event loop yang sudah dibuat.

  // 1) Thread sebagai leader
```

```
  OThread.begin(false);
  // Dataset selalu ditulis ulang agar H2 dan C6 memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  // 2) Wi-Fi disambungkan SEBELUM OThread.start()
  WiFi.mode(WIFI_STA);
  // JANGAN setSleep(false) di board yang juga menjalankan 802.15.4:
  // koeksistensi Wi-Fi + 802.15.4 pada ESP32-C6 mengandalkan modem sleep
  // untuk membagi airtime. Mematikannya membuat Wi-Fi meminta radio 100 %
  // sementara Thread juga selalu RX, sehingga TCP keluar hampir selalu gagal
  // (gejala: HTTP -1 / MQTT rc=-2 padahal IP sudah didapat).
  WiFi.setSleep(true);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.printf("Konek Wi-Fi %s", WIFI_SSID);
  // Batasi penantian: kalau SSID/password salah atau AP tidak ada, setup()
  // tetap lanjut supaya sisi radio lain (Thread/BLE) tetap bisa diamati.
  // Wi-Fi dicoba ulang di loop().
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < WIFI_TIMEOUT_MS) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nWi-Fi OK, IP: %s | RSSI: %d dBm\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.printf("\nWi-Fi GAGAL setelah %lu s (status=%d). Lanjut tanpa Wi-Fi;\n"
                  "periksa WIFI_SSID/WIFI_PASS dan pastikan AP memancar di 2,4 GHz.\n",
                  WIFI_TIMEOUT_MS / 1000UL, (int)WiFi.status());
  }

  // Beri prioritas radio ke Wi-Fi sebelum 802.15.4 dinyalakan.
  esp_err_t coexErr = esp_coex_preference_set(ESP_COEX_PREFER_WIFI);
  Serial.printf("coex preference = WIFI (err=%d)\n", (int)coexErr);

  // 3) Baru radio Thread dinyalakan
  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu attach Thread...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Thread attached as: %s\n", OThread.otGetStringDeviceRole());

  restoreWifiAsDefaultNetif();
  printThreadInfo();
  printWifiInfo();

  OtUdp.beginMulticast(GROUP, PORT);


  Serial.println("Gateway siap (Thread -> Wi-Fi).");
}

void forwardToWifi(const char *payload) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Wi-Fi terputus, skip forward");
    return;
  }
  HTTPClient http;
  // Timeout dilonggarkan: saat berbagi radio dengan 802.15.4, balasan server
  // bisa datang jauh lebih lambat dari default 5 detik.
  http.setConnectTimeout(8000);
  http.setTimeout(8000);
  http.begin(SERVER_URL);
  http.addHeader("Content-Type", "application/json");

  String body = String("{\"sensor\":\"h2\",\"data\":\"") + payload + "\"}";
  int code = http.POST(body);
  Serial.printf("Forward via Wi-Fi -> %s | HTTP %d\n", SERVER_URL, code);
  http.end();
}
```

```
// Sambung ulang Wi-Fi berkala; tanpa ini gateway tidak pernah pulih setelah AP
// sempat mati, padahal sisi Thread-nya tetap jalan.
static void maintainWifi() {
  static unsigned long lastTry = 0;
  if (WiFi.status() == WL_CONNECTED) return;
  if (millis() - lastTry < WIFI_RETRY_MS) return;
  lastTry = millis();
  // JANGAN WiFi.disconnect() di sini: pada sinyal lemah, asosiasi bisa perlu
  // > 10 detik, dan memutusnya tiap percobaan membuat Wi-Fi tidak pernah
  // selesai menyambung.
  Serial.printf("Wi-Fi belum tersambung (status=%d), mencoba lagi...\n",
                (int)WiFi.status());
  WiFi.begin(WIFI_SSID, WIFI_PASS);
}

void loop() {
  maintainWifi();

  // SIMULASI sensor: bangkitkan telemetri tiap 3 detik lalu teruskan via HTTP.
  static unsigned long lastSim = 0;
  if (millis() - lastSim > 3000) {
    lastSim = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "suhu:%.1f", readSensor());
    Serial.printf("SIM sensor (Thread): %s\n", msg);
    forwardToWifi(msg);
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[64];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX via Thread [%s]: %s\n", OtUdp.remoteIP().toString().c_str(), buf);
    forwardToWifi(buf);
  }
  delay(10);
}
```

Kode 15.5: `src/c6_gateway/main.cpp` — gateway dwi-radio di ESP32-C6

15.6.1 Channel antar Kelompok (wajib kelas paralel)

Seperti pada blok Zigbee (Modul 08–10), satu ruangan praktikum berisi beberapa kelompok sehingga berdiri beberapa jaringan Thread sekaligus. Pada Thread, channel adalah bagian dari **Active Dataset** yang ditulis firmware ke tiap node sebelum `OThread.start()`; jaringan terbentuk pada channel dataset itu, dan node lain attach ke sana. Karena network name, PAN ID, dan network key dibuat sama di semua kelompok, pemisah antar kelompok yang andal adalah **channel yang berbeda** — beda channel berarti radio tuli total.

Jatah untuk enam kelompok sama dengan blok Zigbee. Ubah satu baris `OT_CHANNEL` pada **kedua** firmware modul ini (Kode 15.4 dan Kode 15.5):

Kelompok	Channel	Baris yang diubah
1	15	`const uint8_t OT_CHANNEL = 15;`
2	20	`const uint8_t OT_CHANNEL = 20;`
3	25	`const uint8_t OT_CHANNEL = 25;`
4	11	`const uint8_t OT_CHANNEL = 11;`
5	17	`const uint8_t OT_CHANNEL = 17;`
6	22	`const uint8_t OT_CHANNEL = 22;`

Tabel 15.6: Jatah channel Thread untuk enam kelompok — satu nilai untuk kedua firmware

PERINGATAN
Node H2 dan gateway C6 harus memakai nilai `OT_CHANNEL` yang sama. Bila berbeda, node sensor tidak akan pernah attach dan berhenti di `Menunggu join ke gateway (C6)...`. Setelah upload, cocokkan baris `Channel` pada `Info network Thread` kedua board.

TIP
Mengganti `OT_CHANNEL` tidak memerlukan erase NVS: dataset ditulis ulang setiap boot (`commitDataSet`), jadi cukup build + upload ulang kedua board. Pada gateway C6, channel Thread juga berpengaruh pada koeksistensi dengan Wi-Fi 2,4 GHz — 15, 20, dan 25 adalah pilihan klasik; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah.

15.6.2 Server HTTP Lokal — `http_sink.py`

Bila `httpbin.org` terblokir (gejala di Serial Monitor: `HTTP -1`), gunakan server HTTP lokal yang sudah disertakan di folder modul ini (Kode 15.6). Server ini mendengarkan POST di `0.0.0.0:8080`, mencetak tiap POST yang masuk dengan timestamp, lalu membalas HTTP 200 beserta JSON — berfungsi sebagai pengganti `httpbin.org` sekaligus bukti bahwa hop Wi-Fi/HTTP benar-benar sampai.

```
#!/usr/bin/env python3
"""Server HTTP penerima POST untuk Modul 13, pengganti httpbin.org.

Dipakai bila jaringan memblokir akses keluar (gejala: `HTTP -1` pada Serial
Monitor gateway). Jalankan di laptop, lalu arahkan SERVER_URL firmware ke
alamat IP laptop:

    python3 tools/http_sink.py                # dengar di 0.0.0.0:8080
    // src/c6_gateway/main.cpp
    const char *SERVER_URL = "http://192.168.110.74:8080/post";

Setiap POST dicetak dengan timestamp dan isinya, lalu dibalas HTTP 200 + JSON,
sehingga `http.POST()` di firmware mengembalikan 200 seperti httpbin.
"""
import json, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

HOST, PORT = "0.0.0.0", 8080
t0 = time.time()
count = 0


class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        global count
        n = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(n).decode("utf-8", "replace")
        count += 1
        print("[%8.3f] #%-4d POST %s from %s  ->  %s"
              % (time.time() - t0, count, self.path, self.client_address[0], body),
              flush=True)
        resp = json.dumps({"ok": True, "seq": count, "echo": body}).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(resp)))
        self.end_headers()
        self.wfile.write(resp)

    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(b"http_sink aktif; kirim POST ke /post\n")

    def log_message(self, *args):
        pass          # senyapkan log bawaan, kita cetak sendiri


print("[   0.000] http_sink siap di %s:%d (POST -> HTTP 200)" % (HOST, PORT), flush=True)
ThreadingHTTPServer((HOST, PORT), Handler).serve_forever()
```

Kode 15.6: `http_sink.py` — server HTTP lokal penerima POST

```
# 1. cari IP laptop di Wi-Fi yang sama dengan board
ip -4 addr show | grep inet        # mis. 192.168.1.5

# 2. jalankan server (dengar di 0.0.0.0:8080)
python3 http_sink.py
```

Arahkan `SERVER_URL` di `src/c6_gateway/main.cpp` ke IP laptop.

```
const char *SERVER_URL = "http://192.168.1.5:8080/post";
```

Kode 15.7: Mengarahkan gateway ke server HTTP lokal

Contoh keluaran setiap POST yang sampai:

```
[   0.000] http_sink siap di 0.0.0.0:8080 (POST -> HTTP 200)
[ 480.308] #1    POST /post from 192.168.1.39  ->  {"sensor":"h2","data":"suhu:23.1"}
```

CATATAN
Karena hop Wi-Fi/HTTP di gateway dwi-radio ini paling rapuh (koeksistensi Thread dan Wi-Fi), sebagian besar POST bisa tercetak `HTTP -1` di Serial Monitor sedangkan server tidak menerima apa pun. Tugas pada modul ini menghitung berapa `RX via Thread` yang berhasil sampai ke server (lihat bagian Pengukuran).

CATATAN
Windows Firewall. Bila sink sudah berjalan tetapi board selalu mencetak `HTTP -1` dan sink tidak menerima satu POST pun, periksa firewall Windows — koneksi masuk ke python bisa diblokir. Di mesin uji lab ini ditemukan rule **Block** inbound untuk python PlatformIO (`~\.platformio\python3\python.exe`). Dua solusi: jalankan sink dengan python lain yang sudah diizinkan (misalnya python miniconda), atau tambahkan rule izin dari PowerShell **Administrator**:

```
New-NetFirewallRule -DisplayName WSN-http_sink-8080 -Direction Inbound -Protocol TCP -LocalPort 8080 -
Action Allow -Profile Private
```

Uji berhasil bila gateway mencetak `HTTP 200` dan sink mencetak baris `POST /post from …` beserta isi suhunya.

15.7 Build dan Flash

```
pio run -d week13_thread_wifi_gateway -e c6_gateway -t upload -t monitor
pio run -d week13_thread_wifi_gateway -e h2_node    -t upload
```

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

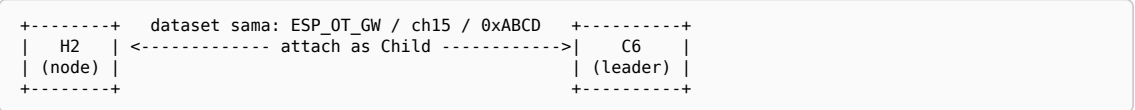
- ☐ ESP32-H2 dan ESP32-C6 terhubung ke PC via kabel USB, port dicatat lewat `pio device list`.
- ☐ Channel kelompok disetel (`OT_CHANNEL`) sama di kedua berkas dan berbeda dari kelompok lain — lihat Tabel 15.6.
- ☐ `WIFI_SSID`, `WIFI_PASS`, dan `SERVER_URL` pada `src/c6_gateway/main.cpp` sudah disesuaikan.
- ☐ Hotspot atau Wi-Fi **2,4 GHz** aktif dan kedua board berada dalam jangkauan sinyal.

- ☐ Server HTTP tujuan dapat diakses (cek `httpbin.org` dari browser, atau jalankan `python3 http_sink.py` untuk server lokal).
- ☐ Firmware `h2_node` dan `c6_gateway` berhasil di-build.
- ☐ Env gateway memakai `board_build.partitions = huge_app.csv`.
- ☐ Serial Monitor 115200 dibuka untuk masing-masing board.

15.8 Percobaan

15.8.1 EXP-01 — Menyalakan Jaringan Thread

Deploy firmware `h2_node` ke ESP32-H2 dan `c6_gateway` ke ESP32-C6. Kedua board memakai dataset Thread identik. Gateway menjadi Leader, node menunggu hingga role mencapai Child.



Data capture

Parameter	Hasil
Nama jaringan Thread	
Channel dan PAN ID	
Role H2 setelah attach	
Role C6 setelah attach	
Awalan Mesh-Local EID H2 dan C6 sama?	… (harus `fdde:ad00:beef:0:`)
Waktu hingga attach (± detik)	

Tabel 15.7: Lembar pengamatan EXP-01

CHECKPOINT

Kedua board mencetak `Attached as: ...` **dan** awalan EID keduanya sama. Jika berbeda, gateway tidak akan pernah menerima apa pun meski keduanya tampak "terhubung" — perbaiki dulu.

15.8.2 EXP-02 — Telemetri lewat Thread

Setelah attach, H2 mengirim `suhu:XX.X` ke grup multicast tiap 3 detik. Amati pasangan TX dan RX pada kedua Serial Monitor.

```
[H2 loop] ──tiap 3 s──► OtUdp.beginPacket(GROUP,5050) ──► "TX via Thread: suhu:25.3"
[C6 loop] ──────────────► OtUdp.parsePacket()          ──► "RX via Thread [<EID sumber>]: suhu:25.3"
```

Alamat yang tercetak gateway adalah `OtUdp.remoteIP()` — mesh-local EID **node pengirim** (`fdde:ad00:...`), bukan alamat grup `ff03::abcd` yang dituju.

Expected output — H2 (dari uji di board; alamat, peran, dan IP di board Anda akan berbeda)

```
Sensor H2 (Thread node) starting...
Menunggu join ke gateway (C6)...
Attached as: Leader        <- bisa juga Child, tergantung urutan boot
Info network Thread:
  Peran          : Leader
  Network name   : ESP_OT_GW
  Channel        : 15
  PAN ID         : 0xABCD
  Extended PAN ID: DE:AD:00:BE:EF:00:CA:FE
  RLOC16         : 0x6C00
  Extended addr  : D6:48:6C:1E:E0:BC:16:6B
  EUI-64         : 74:4D:BD:FF:FE:61:E6:2C
  Mesh-Local EID : fdde:ad00:beef:0:7c1b:2b3:3487:1195
TX via Thread: suhu:24.4
TX via Thread: suhu:25.0
```

Expected output — C6

```
Konek Wi-Fi SprH-3......
Wi-Fi OK, IP: 192.168.1.109 | RSSI: -63 dBm
coex preference = WIFI (err=0)
Menunggu attach Thread...
Thread attached as: Child
Default netif dikembalikan ke Wi-Fi STA (err=0)
Info network Thread:
  Peran          : Child
  Network name   : ESP_OT_GW
  Channel        : 15
  PAN ID         : 0xABCD
  Extended PAN ID: DE:AD:00:BE:EF:00:CA:FE
  RLOC16         : 0x6C01
  Extended addr  : BE:21:63:FE:AC:75:5F:60
  EUI-64         : 40:4C:CA:FF:FE:5E:C5:40
  Mesh-Local EID : fdde:ad00:beef:0:15c:9037:1e71:3135
Info network Wi-Fi:
  SSID           : SprH-3
  IP             : 192.168.1.109
  RSSI           : -64 dBm
  Channel Wi-Fi  : 8
  BSSID (AP)     : 24:C0:13:CB:4B:27
  MAC            : 40:4C:CA:5E:C5:40
Gateway siap (Thread -> Wi-Fi).
SIM sensor (Thread): suhu:25.8
Forward via Wi-Fi -> http://192.168.1.5:8080/post | HTTP -1
RX via Thread [fdde:ad00:beef:0:7c1b:2b3:3487:1195]: suhu:24.8
Forward via Wi-Fi -> http://192.168.1.5:8080/post | HTTP -1
```

Baris `SIM sensor (Thread): ...` berasal dari simulasi sensor di firmware gateway (`readSensor()`), yang memanggil jalur `forwardToWifi()` yang sama dengan data Thread asli; baris `RX via Thread [...]` adalah telemetri H2 yang sebenarnya.

CATATAN

Membaca info network. Kedua board harus sama pada **Network name, Channel, PAN ID, dan Extended PAN ID** Thread. Peran dipilih jaringan: pada uji ini H2 attach lebih dulu dan menjadi **Leader** (`RLOC16 0x6C00`), gateway C6 menjadi **Child**-nya (`0x6C01`) — gateway tidak harus Leader. Dua hal khusus modul ini:

- **EUI-64 C6 diturunkan dari MAC Wi-Fi-nya**: MAC `40:4C:CA:5E:C5:40` → EUI-64 `40:4C:CA:FF:FE:5E:C5:40` (disisipi `FF:FE` di tengah). Satu chip, satu identitas pabrik untuk kedua radio.
- **Channel Wi-Fi vs channel Thread.** Wi-Fi memakai channel AP (di sini 8, ±2447 MHz), Thread channel 15 (2425 MHz). Walau frekuensinya tidak bertumpuk, keduanya tetap **berbagi satu radio dan satu antena** di C6 — itu sumber masalah koeksistensi, bukan tumpang tindih frekuensi.

CATATAN

Mengapa banyak telemetri H2 hilang. Pada rekaman ini `SERVER_URL` tidak menunjuk ke server yang hidup, sehingga setiap `http.POST()` gagal (`HTTP -1`) setelah timeout ±8 s. Selama itu `loop()` gateway tertahan dan tidak membaca socket UDP: hanya satu paket yang tertampung, sisanya dibuang. Hasilnya hanya 8 dari 22 telemetri H2 yang tercetak `RX via Thread`, masing-masing tertunda beberapa detik. Kehilangan ini terjadi di **aplikasi gateway**, bukan di radio Thread — hop Thread-nya sendiri sehat. Jalankan `http_sink.py` dan arahkan `SERVER_URL` ke IP laptop untuk melihat perbedaannya.

BUKA ABSTRAKSINYA

Perhatikan urutan di `setup()` gateway: `OThread.begin()` dulu, lalu Wi-Fi, baru `OThread.start()`. Coba dua variasi, flash, dan catat gejalanya masing-masing.

1. Pindahkan blok Wi-Fi ke paling atas, maka board **panic** (`Failed to create OpentThread event loop`, lalu `assert failed: otTaskletsSignalPending`).
2. Pindahkan blok Wi-Fi ke bawah `OThread.start()`, maka board hidup tetapi Wi-Fi **tidak pernah** asosiasi (`status=6` terus).

Telusuri sebab pertama hingga menemukan `esp_event_loop_create_default()`: siapa yang membuatnya lebih dulu, dan mengapa satu stack menerima kondisi "sudah ada" sedangkan yang lain menganggapnya fatal? Untuk yang kedua, kaitkan dengan *radio coexistence*: kedua radio berbagi satu antena 2,4 GHz.

CHECKPOINT

Jumlah baris `TX via Thread` di H2 dan `RX via Thread` di C6 harus sama dalam periode pengamatan yang sama. Selisihnya adalah loss hop Thread — catat, jangan diabaikan.

15.8.3 EXP-03 — Penerusan ke Wi-Fi (HTTP POST)

Setiap pesan yang diterima diteruskan gateway ke `SERVER_URL` sebagai JSON `{"sensor":"h2","data":"suhu:XX.X"}` dengan header `Content-Type: application/json`. Verifikasi kode respons HTTP (200 berarti sukses). Biarkan sistem berjalan 2–3 menit.

```
"RX via Thread" ──► forwardToWifi(buf) ──► HTTPClient ──► http.POST(JSON)
                ──► "Forward via Wi-Fi ... | HTTP 200"
```

```
Forward via Wi-Fi -> http://httpbin.org/post | HTTP 200
```

Variasi wajib: **matikan hotspot ± 15 detik** saat sistem berjalan. Amati baris `Wi-Fi terputus, skip forward` dan catat apakah data Thread yang datang selama itu hilang atau tertahan.

Data capture

Parameter	Hasil
IP Wi-Fi gateway	
URL server tujuan	
HTTP status code yang diterima	
RSSI Wi-Fi gateway (`WiFi.RSSI()`)	
Paket Thread diterima dibanding di-POST (2 menit)	
Nasib data saat Wi-Fi putus	

Tabel 15.8: Lembar pengamatan EXP-03

CHECKPOINT

Tersedia **tiga** angka untuk periode yang sama: paket dikirim H2, paket diterima C6, dan POST berhasil (HTTP 200). Selisih antar ketiganya menunjukkan hop mana yang bermasalah — inilah yang membedakan laporan yang bisa dipertanggungjawabkan dari sekadar "sistem berjalan".

15.9 Verifikasi Hardware — Log Referensi

Dijalankan pada **ESP32-H2 DevKitM-1** dan **ESP32-C6 DevKitC-1** asli.

```
# H2 (env h2_node)                # C6 (env c6_gateway)
[0.401] Sensor H2 starting...     [2.004] Konek Wi-Fi ...
[1.203] Attached as: Router       [2.004] Wi-Fi OK, IP: 192.168.110.197 | RSSI: -83 dBm
[3.407] TX via Thread: suhu:25.6  [2.805] Thread attached as: Router
                                  [2.805] Default netif dikembalikan ke Wi-Fi STA (err=0)
                                  [3.406] RX via Thread [fdde:ad00:beef:0:385a:...]: suhu:25.6
                                  [8.413] Forward via Wi-Fi -> ... | HTTP -1
```

Bagian rantai	Status
Build `h2_node` dan `c6_gateway` (`huge_app.csv`)	berhasil
C6 attach ke Thread `ESP_OT_GW`	berhasil, 2,8 s sejak boot
Telemetri `suhu:XX.X` H2 ke C6 lewat Thread	berhasil, 13/13 paket, 0 % loss
C6 asosiasi Wi-Fi sambil Thread jalan	berhasil, 2,0 s (setelah perbaikan urutan)
HTTP POST ke server	sampai di server, tetapi balasan sering timeout (`HTTP -11`)
Penanganan Wi-Fi gagal	berhasil, `setup()` lanjut dan dicoba ulang berkala di `loop()`

Tabel 15.9: Hasil verifikasi hardware Modul 13

15.9.1 Koeksistensi Wi-Fi dan 802.15.4

Gateway ini menjalankan dua radio pada satu antena 2,4 GHz. Tanpa penanganan khusus, C6 tetap asosiasi Wi-Fi dan dapat IP yang benar, tetapi **seluruh TCP keluar gagal** — bahkan ke router sendiri.

```
diag: status=3 rssi=-82 ip=... gw=...
diag: tcp ke router = 0 (4002 ms)     <- paket tidak lewat sama sekali
```

Dua hal yang menyelesaikannya, keduanya sudah ada di kode: **urutan inisialisasi** — Wi-Fi disambungkan di antara `OThread.begin()` dan `OThread.start()`; serta **prioritas radio ke Wi-Fi** sebelum 802.15.4 aktif seperti Kode 15.8.

```
#include "esp_coexist.h"
esp_coex_preference_set(ESP_COEX_PREFER_WIFI);   // sebelum OThread.start()
```

Kode 15.8: Memberi prioritas radio kepada Wi-Fi sebelum stack Thread aktif

Setelah keduanya diterapkan, POST benar-benar sampai di server (`{"sensor":"h2","data":"suhu:22.9"}` tercatat di server penerima). Namun hop Wi-Fi tetap yang paling rapuh: **balasan** HTTP sering tidak kembali tepat waktu, tercetak sebagai `HTTP -11` (read timeout) walaupun datanya sudah diterima server. Karena itu `http.setConnectTimeout()` dan `setTimeout()` dinaikkan ke 8 detik.

Yang sudah dicoba dan tidak menyelesaikan: mengganti channel 802.15.4 (15 menjadi 25), memakai AP di kanal Wi-Fi yang tidak bertetangga, `WiFi.setSleep()` kedua nilainya, dan memaksa default netif ke Wi-Fi STA.

Cara membaca kegagalan HTTP

Kode	Arti	Tindakan
`200`	POST diterima server	—
`-1`	koneksi TCP gagal terbentuk	periksa server terjangkau; bila server lokal, pastikan satu subnet
`-11`	request terkirim, **balasan** timeout	perbesar `http.setTimeout()`; cek apakah data sudah masuk di sisi server

Tabel 15.10: Arti kode kegagalan HTTP pada gateway

Yang harus dilakukan praktikan: catat RSSI Wi-Fi gateway, dan isi tabel loss per hop di bagian Pengukuran dengan membandingkan `RX via Thread`, kode HTTP, dan log di sisi server. Bila `RX via Thread` normal tetapi POST bocor, itu bukan kegagalan praktikum — itu justru data yang diminta bagian Analisis nomor 1.

15.9.2 Delapan Perbaikan Kode yang Lahir dari Uji Ini

Masalah di perangkat nyata	Perbaikan
C6 **panic** saat boot: `Failed to create OpentThread event loop` lalu `assert failed: otTaskletsSignalPending`	`OThread.begin()` dipanggil **sebelum** Wi-Fi
Wi-Fi tidak pernah asosiasi bila `OThread.start()` sudah jalan	Wi-Fi disambungkan **di antara** `OThread.begin()` dan `OThread.start()`
`while (WiFi.status() != WL_CONNECTED)` menggantung selamanya bila SSID atau password salah	dibatasi `WIFI_TIMEOUT_MS`, `setup()` lanjut, dicoba ulang di `loop()`
Gateway tidak pernah pulih setelah AP sempat mati	`maintainWifi()` menyambung ulang berkala
`WiFi.disconnect()` tiap percobaan ulang membatalkan asosiasi yang sedang berjalan	hanya `WiFi.begin()` ulang, jeda `WIFI_RETRY_MS` lebih panjang dari durasi asosiasi
Dua netif aktif (Wi-Fi dan OpenThread) tanpa default yang pasti	`restoreWifiAsDefaultNetif()` memastikan rute IPv4 lewat Wi-Fi STA
Seluruh TCP keluar gagal saat stack Thread aktif	`esp_coex_preference_set(ESP_COEX_PREFER_WIFI)` sebelum `OThread.start()`
Balasan HTTP timeout (`HTTP -11`) padahal data sudah sampai server	`http.setConnectTimeout(8000)` dan `http.setTimeout(8000)`

Tabel 15.11: Delapan perbaikan kode hasil pengujian perangkat nyata

Urutan inisialisasi yang benar — inilah hasil terpenting modul ini.

```
1. OThread.begin(false)      // OT membuat event loop default
2. commit dataset + ML prefix
3. WiFi.mode()/begin()       // asosiasi selesai selagi radio 802.15.4 belum aktif
4. OThread.networkInterfaceUp() + OThread.start()
```

Membalik langkah 1 dan 3 membuat board **panic**; menaruh langkah 3 setelah langkah 4 membuat Wi-Fi **tidak pernah** asosiasi.

15.10 Pengukuran

Skenario / Jarak H2–C6	RSSI Wi-Fi (C6)	Latency end-to-end (TX ke HTTP, ms)	Success (paket / 40)
1 m, garis pandang			
5 m, garis pandang			
5 m + penghalang dinding			

Tabel 15.12: Lembar pengukuran end-to-end Modul 13

Latency diukur manual: cap waktu baris `TX via Thread` pada H2 dibandingkan `Forward via Wi-Fi ... HTTP 200` pada C6.

Tabel loss per hop — wajib. Inilah yang membedakan modul ini dari sekadar demo.

Hop	Dikirim	Diterima	Loss (%)
H2 ke C6 (Thread)			
C6 ke server (Wi-Fi/HTTP)			
H2 ke server (ujung-ke-ujung)			

Tabel 15.13: Lembar pengukuran loss per hop Modul 13

15.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Mengapa ESP32-C6 dapat menjadi Thread Leader dan Wi-Fi STA secara bersamaan, dan apa konsekuensinya (ukuran firmware, pembagian waktu radio)?
2. Bagaimana pengaruh jarak atau penghalang terhadap jumlah paket Thread yang diterima gateway?
3. Berapa latency tambahan yang diperkenalkan hop Wi-Fi/HTTP dibanding bila data berhenti di gateway? Bandingkan dengan latency Thread murni M11.
4. Apa yang terjadi pada baris `Wi-Fi terputus, skip forward` — dan apakah data Thread tersebut hilang? Bagaimana cara memperbaikinya?
5. Kapan arsitektur gateway Thread ke Wi-Fi lebih tepat dipakai dibanding node sensor Wi-Fi langsung? Jawab dari sisi daya, jangkauan, dan jumlah node.

15.12 Concept Check

1. Apa perbedaan gateway dan border router dalam konteks jaringan Thread?
2. Mengapa komunikasi Thread pada praktikum ini memakai UDP multicast, bukan unicast?
3. Apa fungsi dataset (channel, PAN ID, network key, mesh-local prefix) dalam pembentukan jaringan Thread?
4. Bagaimana gateway mengetahui alamat sumber pesan Thread? Perhatikan keluaran `remoteIP`.
5. Apa kelemahan forwarding via HTTP POST dibanding MQTT untuk telemetri periodik? Jawaban ini adalah jembatan ke Modul 14.

15.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memperkaya payload dan menambah sumber

CH-1 — Dua sensor Tambahkan node Thread H2 kedua dengan payload berbeda (misalnya `hum:XX`) dan pastikan gateway mem-forward keduanya. Bagaimana server membedakan sumbernya?

CH-2 — Payload lebih kaya Sertakan RSSI Thread dan nomor urut di dalam payload (`suhu:25.3,rssi:-62,#42`). Diskusikan: apakah ini masih *transparent forwarding*?

Level 3 — Tantangan · Menghitung loss per hop dan menyelamatkan data

CH-3 — Loss per hop (wajib) Hitung packet loss dengan sequence number pada payload. Contoh: 100 paket dikirim H2, 97 diterima gateway, sehingga loss Thread = 3 %. Lalu bandingkan dengan jumlah HTTP 200 untuk mendapatkan loss hop Wi-Fi.

CH-4 — Buffer saat Wi-Fi putus Ubah gateway agar menyimpan pesan Thread yang datang saat Wi-Fi mati (misalnya antrean 20 pesan) dan mengirimkannya setelah koneksi pulih. Ukur berapa pesan yang berhasil diselamatkan.

15.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (gateway, border router, Thread, Wi-Fi STA, UDP multicast, mesh-local prefix).
3. Konfigurasi — dataset Thread, SSID, URL server, port, dan `huge_app.csv`.
4. Hasil eksperimen — log kedua board (EXP-01 sampai EXP-03 beserta checkpoint), termasuk uji Wi-Fi diputus.
5. Data pengukuran — tabel bagian Pengukuran **dan** tabel loss per hop.
6. Analisis dan concept check.
7. Challenge — minimal CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

16. Modul 14 — MQTT: Publish dan Subscribe

IoT COMMUNICATION LAB · MODUL 14

Publish to the Cloud — MQTT: Publish dan Subscribe

ESP32-C6 · Wi-Fi / MQTT · pub-sub · level Intermediate · 3 × 50 menit · folder kode `week14_mqtt`

PENGANTAR MODUL · Gambaran Umum

Modul 14 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menempatkan perangkat pada sebuah broker dan membuatnya bicara dua arah tanpa perlu tahu alamat lawan bicaranya. Percobaan berjalan dalam model client–broker dengan pola publish/subscribe, diamati melalui Serial Monitor 115200 baud serta perintah `mosquitto_sub` dan `mosquitto_pub` di sisi PC.

16.1 Pendahuluan

M13 mengeluarkan data ke jaringan IP lewat HTTP POST — satu arah, satu tujuan tetap, satu koneksi per pesan. MQTT membalik modelnya: perangkat menempel pada **broker**, telemetri naik dan perintah turun lewat koneksi yang sama, dan tidak ada pihak yang perlu tahu alamat pihak lain. Modul ini sengaja **tanpa Thread** supaya sisi IP bisa dipelajari terpisah; keduanya baru disatukan di M15.

Prasyaratnya adalah M13: Wi-Fi STA pada ESP32-C6, konsep gateway, dan transparent forwarding. Yang dibangun di sini adalah koneksi ke broker, penyusunan topic hierarkis, publish berkala, subscribe beserta callback-nya, tingkat QoS, pesan retained, dan perilaku saat koneksi tersambung ulang. Semuanya dipakai lagi pada M15 ketika HTTP POST milik M13 diganti publish MQTT ini, serta M16 ketika seluruh protokol bermuara ke topic yang sama agar perbandingannya adil.

Peta modul blok integrasi

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
13	Gateway: Thread bertemu Wi-Fi, data keluar lewat HTTP
14 (ini)	**Sisi IP diperdalam: publish/subscribe lewat broker MQTT**
15	M12, M13, dan M14 digabung: sensor ke Thread ke C6 ke MQTT ke dashboard
16	Protokol sensor diganti-ganti, muara MQTT-nya tetap sama

Tabel 16.1: Peta modul blok integrasi

Kontrak data lab ini. Topic dibagi dua: **`praktikum/h2/telemetri`** untuk data naik dan **`praktikum/h2/perintah`** untuk perintah turun. Pemisahan ini adalah kelanjutan langsung dari pola M03 (characteristic data dibanding characteristic perintah) dan M08 (cluster On/Off dibanding laporan status). Topic telemetri yang sama dipakai lagi di M15 dan M16 — itulah yang membuat data ketiga modul bisa dibandingkan.

16.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Bicara dua arah lewat broker MQTT

1. Menghubungkan ESP32-C6 ke Wi-Fi dan broker MQTT dengan client ID tertentu, lalu membuktikan koneksinya dari sisi PC memakai `mosquitto_sub`.
2. Mem-publish telemetri berkala ke topic tertentu dan memverifikasi kedatangannya di subscriber eksternal, minimal 20 pesan berturut-turut.
3. Menerima perintah dari topic lain melalui callback dan menunjukkan aksi nyata yang dipicunya di perangkat.
4. Mengukur latency publish sampai terima dan menjelaskan perilaku sistem saat Wi-Fi atau broker terputus, berdasarkan log — termasuk nasib subscription setelah reconnect.

Kriteria keberhasilan

- ☐ 20 publish berturut-turut tiba di subscriber eksternal.
- ☐ Perintah dari `mosquitto_pub` memicu callback `onMessage()` dan aksi nyata (CH-2).
- ☐ Perilaku reconnect Wi-Fi dan MQTT terverifikasi dan tercatat, termasuk return code kegagalan.
- ☐ Tabel latency dan success rate terisi dari pengukuran sendiri.

16.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (paket kontrol MQTT, session state, last will and testament, MQTT 5) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 16.2.

Istilah	Definisi kerja di lab ini
Broker	Server pusat yang menerima pesan dari publisher dan meneruskannya ke subscriber topic terkait.
Publish/Subscribe	Model komunikasi tak langsung — pengirim dan penerima tidak saling tahu alamat, hanya terikat topic.
Topic	Label hierarkis pemisah aliran data: `praktikum/h2/telemetri` (data), `praktikum/h2/perintah` (kontrol).
Wildcard	`+` cocok satu level, `#` cocok sisa level — dipakai subscriber, bukan publisher.
QoS	Tingkat jaminan pengiriman — 0 (at most once), 1 (at least once), 2 (exactly once).
Retained	Broker menyimpan pesan terakhir tiap topic dan mengirimkannya ke subscriber baru.
Client ID	Identitas unik klien pada broker; dua klien dengan ID sama akan saling menendang.
Keep alive dan reconnect	Klien menjaga koneksi; kode ini otomatis reconnect Wi-Fi dan MQTT bila terputus.

Tabel 16.2: Istilah kerja Modul 14

PENTING

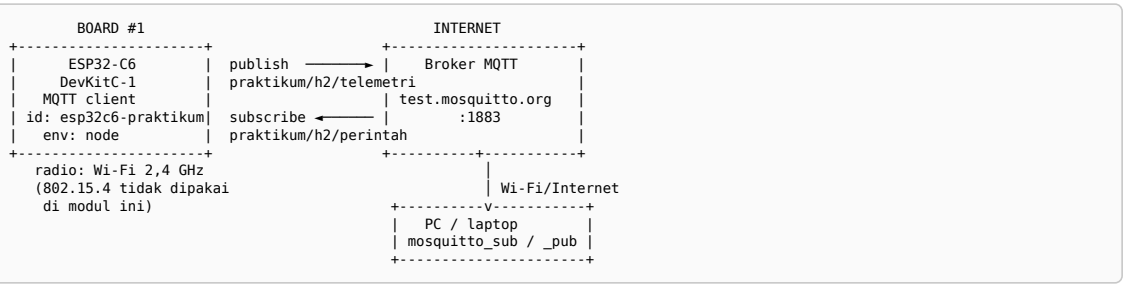
Batasan PubSubClient yang wajib diketahui. Library ini hanya melakukan **publish QoS 0**. Argumen ketiga `mqtt.publish(topic, payload, true)` adalah flag *retained*, **bukan** QoS. Artinya: setiap klaim tentang QoS 1 atau 2 pada laporan harus berasal dari library lain (misalnya `arduino-mqtt` atau `AsyncMqttClient`), bukan dari percobaan ini. Menyebut "QoS 1" tanpa mengganti library adalah kesalahan yang sering muncul di laporan.

Mengapa modul ini tanpa Thread? Supaya kegagalan bisa dilokalisasi. Jika MQTT dan Thread dinyalakan bersamaan sejak awal (seperti M15), pesan yang tidak sampai bisa berasal dari mana saja. Di sini hanya ada satu hop — sehingga angka latency dan loss yang diperoleh adalah **milik MQTT saja**, dan dapat dikurangkan dari angka M15 nanti.

Sekuens protokol yang diamati

```
[C6] ──publish──► [Broker] ──push──► [Subscriber / dashboard]
[C6] ◄──push─────  [Broker] ◄──publish── [mosquitto_pub di PC]
        callback onMessage()          route by topic
```

16.4 Topologi



Elemen	Board atau alat	Identitas	Peran
Klien MQTT	**ESP32-C6** DevKitC-1 (env `node`)	client ID `esp32c6-praktikum`	publish telemetri dan subscribe perintah
Broker	layanan publik	`test.mosquitto.org:1883`	routing berbasis topic
Verifikator	PC/laptop	`mosquitto_sub` dan `mosquitto_pub`	pembuktian independen dari sisi luar

Tabel 16.3: Elemen sistem Modul 14

Mengapa ESP32-C6, bukan H2? ESP32-H2 tidak punya radio Wi-Fi sama sekali, jadi ia tidak bisa menjadi klien MQTT. Modul ini hanya memakai kaki Wi-Fi C6 — radio 802.15.4-nya menganggur, dan baru dipakai bersamaan di M15.

16.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-C6 (Arduino core 3.x) dengan Wi-Fi dan PubSubClient.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-C6	DevKitC-1	1
2	Kabel USB data	kabel data, bukan *charge-only*	1
3	PC/Laptop	PlatformIO Core/IDE dan paket `mosquitto-clients`	1
4	Wi-Fi atau hotspot	**2,4 GHz**, ada akses internet	1
5	Broker MQTT	`test.mosquitto.org:1883` (publik) atau Mosquitto lokal	1
6	Library PubSubClient	`knolleary/PubSubClient@^2.8` — otomatis via `lib_deps`	—

Tabel 16.4: Alat dan bahan Modul 14

Konfigurasi

Parameter	Nilai
Client ID	`esp32c6-praktikum`
Topic telemetri (publish)	`praktikum/h2/telemetri`
Topic perintah (subscribe)	`praktikum/h2/perintah`
Interval publish	5000 ms
QoS efektif	0 (batasan PubSubClient)

Tabel 16.5: Konfigurasi MQTT Modul 14

16.6 Kode Program

```
[platformio]
default_envs = node

[env:node]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-c6-devkitc-1
monitor_speed = 115200
lib_deps =
    knolleary/PubSubClient@^2.8
```

Kode 16.1: `platformio.ini` Modul 14 pada repositori

```
// Minggu 14 — MQTT: C6 publish & subscribe ke broker (Wi-Fi)
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>

const char *WIFI_SSID = "SprH-3";
const char *WIFI_PASS = "yourpassword!";

// Batas waktu menunggu Wi-Fi/MQTT saat boot; melewati batas ini setup() tetap
// dilanjutkan agar kegagalan sisi IP tidak menyembunyikan sisi radio lain.
const unsigned long WIFI_TIMEOUT_MS = 30000;
// Jeda antar percobaan sambung ulang. Harus lebih panjang dari durasi
// asosiasi pada sinyal lemah, kalau tidak percobaan berikutnya justru
// membatalkan yang sedang berjalan.
const unsigned long WIFI_RETRY_MS = 20000;
const unsigned long MQTT_TIMEOUT_MS = 15000;

// Broker publik (bisa diganti broker lokal, mis. Mosquitto)
// Broker MQTT. Banyak jaringan kampus memblokir port 1883 keluar sehingga
// test.mosquitto.org tidak terjangkau (gejala: rc=-2 dan "Host is unreachable").
// Bila itu terjadi, jalankan `python3 tools/mqtt_broker.py` di laptop lalu ganti
// baris di bawah dengan alamat IP laptop tersebut, mis. "192.168.110.74".
const char *MQTT_BROKER = "192.168.1.5";
const uint16_t MQTT_PORT = 1884;
const char *TOPIC_TELEM = "praktikum/h2/telemetri";
const char *TOPIC_CMD   = "praktikum/h2/perintah";

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

void reconnectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);   // modem sleep memperburuk asosiasi saat sinyal lemah
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.printf("Konek Wi-Fi %s", WIFI_SSID);
  // Batasi penantian: kalau SSID/password salah atau AP tidak ada, setup()
  // tetap lanjut supaya sisi radio lain (Thread/BLE) tetap bisa diamati.
  // Wi-Fi dicoba ulang di loop().
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < WIFI_TIMEOUT_MS) {
    delay(500);
    Serial.print(".");
```

```
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nWi-Fi OK, IP: %s | RSSI: %d dBm\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.printf("\nWi-Fi GAGAL setelah %lu s (status=%d). Lanjut tanpa Wi-Fi;\n"
                  "periksa WIFI_SSID/WIFI_PASS dan pastikan AP memancar di 2,4 GHz.\n",
                  WIFI_TIMEOUT_MS / 1000UL, (int)WiFi.status());
  }
}

void reconnectMQTT() {
  // Berbatas waktu: kalau broker tidak terjangkau, loop() tetap jalan sehingga
  // status Wi-Fi/MQTT tetap terlihat dan tidak tampak seperti board hang.
  unsigned long start = millis();
  while (!mqtt.connected() && millis() - start < MQTT_TIMEOUT_MS) {
    if (WiFi.status() != WL_CONNECTED) return;   // percuma tanpa Wi-Fi
    Serial.printf("Konek MQTT %s:%d ...\n", MQTT_BROKER, MQTT_PORT);
    if (mqtt.connect("esp32c6-praktikum")) {
      Serial.println("MQTT terhubung");
      mqtt.subscribe(TOPIC_CMD);
      Serial.printf("Subscribe: %s\n", TOPIC_CMD);
      return;
    }
    Serial.printf("Gagal (rc=%d), coba lagi 2 detik\n", mqtt.state());
    delay(2000);
  }
}

// Callback saat ada pesan masuk di topic yang di-subscribe
void onMessage(char *topic, byte *payload, unsigned int length) {
  char buf[64];
  unsigned int n = length < sizeof(buf) - 1 ? length : sizeof(buf) - 1;
  memcpy(buf, payload, n);
  buf[n] = '\0';
  Serial.printf("RX MQTT [%s]: %s\n", topic, buf);
}

static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);
  Serial.println("MQTT Node (C6) starting...");

  reconnectWiFi();
  mqtt.setServer(MQTT_BROKER, MQTT_PORT);
  mqtt.setCallback(onMessage);
  reconnectMQTT();
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) reconnectWiFi();
  if (!mqtt.connected()) reconnectMQTT();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 5000) {
    last = millis();
    char payload[32];
    snprintf(payload, sizeof(payload), "%.1f", readSensor());
    mqtt.publish(TOPIC_TELEM, payload);
    Serial.printf("TX MQTT [%s]: %s\n", TOPIC_TELEM, payload);
  }
}
```

Kode 16.2: `src/main.cpp` — klien MQTT di ESP32-C6

16.7 Build dan Flash

```
# terminal 1 - subscriber, jalankan lebih dulu
mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v

# terminal 2 - flash & monitor
pio run -d week14_mqtt -e node -t upload -t monitor
```

Pre-flight checklist

- ☐ ESP32-C6 terhubung ke PC via kabel USB, port dicatat lewat `pio device list`.
- ☐ `WIFI_SSID` dan `WIFI_PASS` pada `src/main.cpp` sudah disesuaikan (hotspot **2,4 GHz**).
- ☐ Hotspot atau Wi-Fi aktif dan board memperoleh akses internet.
- ☐ Broker dapat dijangkau dari PC: `mosquitto_sub -h test.mosquitto.org -t "praktikum/#" -v`.
- ☐ Firmware environment `node` berhasil di-build (`pio run -e node`).
- ☐ Serial Monitor 115200 baud dibuka.
- ☐ **Ganti prefix topic** menjadi unik per kelompok bila memakai broker publik (lihat catatan keamanan di bagian Concept Check).

16.8 Percobaan

16.8.1 EXP-01 — Koneksi Wi-Fi dan Broker

Deploy firmware `node` ke ESP32-C6. Node terhubung ke Wi-Fi (`reconnectWiFi`), lalu melakukan koneksi MQTT dengan client ID `esp32c6-praktikum` dan subscribe topic perintah.

```
[C6] ──► WiFi.begin ──► IP didapat ──► mqtt.connect("esp32c6-praktikum")
     ──► subscribe TOPIC_CMD
```

Data capture

Parameter	Hasil
SSID yang digunakan	
IP Wi-Fi C6	
Alamat dan port broker	
RSSI Wi-Fi (`WiFi.RSSI()`)	
Topic yang di-subscribe	
Waktu boot sampai `MQTT terhubung` (s)	

Tabel 16.6: Lembar pengamatan EXP-01

CHECKPOINT

Serial Monitor mencetak `Wi-Fi OK, IP: ...` **lalu** `MQTT terhubung`. Jika berhenti di titik-titik Wi-Fi, periksa apakah hotspot 2,4 GHz (C6 tidak mendukung 5 GHz). Jika Wi-Fi OK tetapi MQTT gagal, catat `rc=` yang tercetak — angka itu jawaban soal analisis.

16.8.2 EXP-02 — Publish Berkala

Pada `loop()`, setiap 5 detik node membaca sensor (simulasi suhu 20–40 °C) dan melakukan `mqtt.publish(TOPIC_TELEM, "XX.X")`. Ukur interval publish aktual pada log.

```
loop() ──► millis()-last > 5000 ──► readSensor() ──► mqtt.publish
       ──► "TX MQTT [praktikum/h2/telemetri]: 26.4"
```

Expected output

```
MQTT Node (C6) starting...
Konek Wi-Fi SprH-3...
Wi-Fi OK, IP: 192.168.1.39 | RSSI: -70 dBm
Konek MQTT 192.168.1.5:1884 ...
MQTT terhubung
Subscribe: praktikum/h2/perintah
TX MQTT [praktikum/h2/telemetri]: 25.3
TX MQTT [praktikum/h2/telemetri]: 25.5
```

Verifikasi di PC:

```
mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v
```

BUKA ABSTRAKSINYA

Jalankan subscriber dengan wildcard `mosquitto_sub -h test.mosquitto.org -t "praktikum/#" -v`. Data kelompok lain — bahkan data orang asing — mungkin ikut terlihat pada broker publik yang sama. Jelaskan dari sini: apa yang **tidak** dikerjakan broker, dan mengapa produksi nyata tidak pernah memakai broker publik tanpa autentikasi.

CHECKPOINT

Baris yang muncul di `mosquitto_sub` **sama persis** dengan baris `TX MQTT` di Serial Monitor, termasuk nilainya. Jika Serial mencetak tetapi subscriber diam, publish gagal di sisi jaringan — bukan di sisi kode.

16.8.3 EXP-03 — Perintah Turun dan Pemulihan

Injeksi perintah dari PC:

```
mosquitto_pub -h test.mosquitto.org -t "praktikum/h2/perintah" -m "LED_ON"
```

Verifikasi callback `onMessage()`:

```
RX MQTT [praktikum/h2/perintah]: LED_ON
```

Variasi wajib:

1. Kirim beberapa perintah berbeda (`LED_ON`, `LED_OFF`, `RESET`) dan amati callback.
2. **Matikan hotspot ± 10 detik** lalu nyalakan lagi; amati `reconnectWiFi` dan `reconnectMQTT` beserta keluaran `Gagal (rc=...)`.
3. Setelah reconnect, kirim perintah lagi — apakah masih diterima? Ini menguji apakah subscription bertahan.

Data capture

Parameter	Hasil
Perintah terkirim dibanding diterima	
Delay kirim sampai `RX MQTT` (ms)	
Perilaku saat Wi-Fi terputus	
Return code saat gagal connect	

Parameter	Hasil
Apakah subscription bertahan setelah reconnect?	

Tabel 16.7: Lembar pengamatan EXP-03

CHECKPOINT

Setelah hotspot dinyalakan lagi, perintah baru **tetap** sampai ke perangkat. Jika tidak, subscription hilang saat reconnect — temuan penting, dan jawabannya ada di apakah `subscribe()` dipanggil ulang di dalam fungsi reconnect.

16.9 Verifikasi Hardware — Log Referensi

Dijalankan pada **ESP32-C6 DevKitC-1** asli dengan broker lokal (`tools/mqtt_broker.py`) karena `test.mosquitto.org:1883` diblokir jaringan uji.

```
# ESP32-C6 (env node)                    # Broker (tools/mqtt_broker.py)
[0.200] MQTT Node (C6) starting...       CONNECT   id=esp32c6-praktikum
[2.405] Konek Wi-Fi myrouter....                   from 192.168.110.197
[2.405] Wi-Fi OK, IP: 192.168.110.197    SUBSCRIBE id=esp32c6-praktikum
        | RSSI: -83 dBm                             -> praktikum/h2/perintah
[2.606] MQTT terhubung                   PUBLISH   praktikum/h2/telemetri 25.4
[2.606] Subscribe: praktikum/h2/perintah PUBLISH   praktikum/h2/telemetri 24.5
[5.210] TX MQTT [praktikum/h2/telemetri]: 25.4
[6.009] RX MQTT [praktikum/h2/perintah]: LED_ON     <- dari mosquitto_pub/paho di PC
[10.215] RX MQTT [praktikum/h2/perintah]: LED_OFF
[13.219] RX MQTT [praktikum/h2/perintah]: RESET
```

Parameter	Hasil terukur
Waktu boot sampai `Wi-Fi OK`	2,4 s (RSSI −83 dBm)
Waktu boot sampai `MQTT terhubung`	2,6 s
Publish dikirim / tiba di broker	8 / 8 (0 % loss)
Interval publish terukur	5,00 s ± 0,01
Perintah dikirim / memicu `onMessage()`	3 / 3 (`LED_ON`, `LED_OFF`, `RESET`)
Subscription bertahan setelah reboot board	ya, re-subscribe 0,2 s setelah connect

Tabel 16.8: Hasil verifikasi hardware Modul 14

CATATAN

`test.mosquitto.org` sering tidak terjangkau dari jaringan kampus (port 1883 keluar diblokir). Gejalanya: `Gagal (rc=-2)` disertai `hostByName(): DNS Failed` dan `Host is unreachable`. Solusinya ada di `tools/README.md` — jalankan broker lokal dan arahkan `MQTT_BROKER` ke IP laptop.

Perbaikan kode yang lahir dari uji ini

Masalah di perangkat nyata	Perbaikan
`while (WiFi.status() != WL_CONNECTED)` menggantung selamanya bila SSID atau password salah	penantian dibatasi `WIFI_TIMEOUT_MS`, lalu lanjut dan dicoba ulang di `loop()`
`reconnectMQTT()` memblokir `loop()` tanpa batas saat broker tak terjangkau, board tampak hang	dibatasi `MQTT_TIMEOUT_MS` dan langsung keluar bila Wi-Fi belum siap

Tabel 16.9: Perbaikan kode hasil pengujian perangkat nyata

16.10 Pengukuran

Skenario	Interval (s)	QoS	Latency publish ke terima (ms)	Success / 20 pesan
Dekat AP (1 m)	5	0 (default)		
Jauh AP (5 m)	5	0		
Jauh AP + dinding	5	0		
Interval 1 s (modifikasi)	1	0		

Tabel 16.10: Lembar pengukuran latency dan success rate Modul 14

Latency diukur dengan cap waktu log publish C6 dibandingkan kemunculan pesan pada `mosquitto_sub` (gunakan `mosquitto_sub -F '%t %p'` atau timestamp terminal).

Tabel pembanding transport — wajib. Isi kolom M13 dari data modul sebelumnya.

Aspek	HTTP POST (M13)	MQTT publish (M14)
Koneksi per pesan		
Arah komunikasi		
Perlu tahu alamat penerima?		
Latency rata-rata		
Perilaku saat jaringan putus		

Tabel 16.11: Tabel pembanding HTTP POST dan MQTT publish

16.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Mengapa model publish/subscribe lebih cocok untuk telemetri IoT dibanding HTTP request/response? Dukung dengan tabel pembanding transport.
2. Apa fungsi broker dalam sistem ini dan apa yang terjadi jika broker tidak dapat dijangkau?
3. Bagaimana pengaruh RSSI Wi-Fi terhadap latency dan keberhasilan publish?
4. Apa perbedaan QoS 0, 1, dan 2 — dan QoS berapa yang **sebenarnya** dipakai kode ini? Jelaskan batasan PubSubClient.
5. Mengapa topic telemetri dan perintah dipisah, bukan digabung satu topic? Kaitkan dengan pola yang sama di M03 dan M08.

16.12 Concept Check

1. Jelaskan perbedaan pesan MQTT dan pesan HTTP dari sisi overhead dan arah komunikasi.
2. Apa yang dimaksud topic hierarkis, dan bagaimana wildcard # dan + bekerja?
3. Apa fungsi client ID `esp32c6-praktikum` pada `mqtt.connect()`, dan apa yang terjadi bila dua board memakai ID sama?
4. Apa yang terjadi pada subscription bila koneksi MQTT terputus lalu reconnect?

5. Mengapa praktikum ini menggunakan broker publik, dan apa risikonya? Sebutkan minimal dua, beserta cara menguranginya.

16.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Retained message dan aksi nyata

CH-1 — Retained message Aktifkan flag *retained* (`mqtt.publish(topic, payload, true)`), lalu jalankan subscriber **setelah** publish berjalan dan amati apakah pesan terakhir langsung diterima. Catatan: argumen ketiga adalah `retained`, **bukan** QoS. Untuk menguji QoS 1 diperlukan library lain (misalnya `arduino-mqtt` atau `AsyncMqttClient`); jelaskan konsekuensinya di laporan.

CH-2 — Aksi nyata Tambahkan aksi pada perintah: toggle LED bawaan berdasarkan `LED_ON` dan `LED_OFF`, lalu publish balik status LED ke `praktikum/h2/status`. Ini melengkapi lingkaran perintah, aksi, dan laporan.

Level 3 — Tantangan · Menghitung loss dan menskalakan topic

CH-3 — Packet loss (wajib) Hitung packet loss dengan sequence number pada payload (`26.4,#15`). Contoh: 50 publish, 48 diterima subscriber, sehingga loss = (50 − 48)/50 × 100 % = 4 %.

CH-4 — Topic per perangkat Ubah topic menjadi `praktikum/<client_id>/telemetri` dan subscribe dengan wildcard `praktikum/+/telemetri` di PC. Diskusikan bagaimana skema ini menskalakan ke 50 perangkat.

16.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (MQTT, broker, publish/subscribe, topic, QoS, retained, batasan PubSubClient).
3. Konfigurasi — SSID, broker, client ID, topic, dan library PubSubClient.
4. Hasil eksperimen — log Serial Monitor **dan** log `mosquitto_sub` serta `mosquitto_pub` (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel bagian Pengukuran **dan** tabel pembanding HTTP dengan MQTT.
6. Analisis dan concept check.
7. Challenge — minimal CH-2 dan CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

17. Modul 15 — Pipeline IoT End-to-End

IoT COMMUNICATION LAB · MODUL 15

Build End-to-End IoT System — Pipeline IoT End-to-End

ESP32-H2 + ESP32-C6 · Thread ke MQTT · level Advanced · 3 × 50 menit · folder kode `week15_e2e_iot`

PENGANTAR MODUL · Gambaran Umum

Modul 15 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya merangkai seluruh vertikal — dari sensor sampai dashboard — dan menunjukkan di hop mana pesan hilang apabila memang hilang. Percobaan berjalan sebagai rantai end-to-end empat hop, diamati melalui dua Serial Monitor 115200 baud dan `mosquitto_sub` di sisi PC.

17.1 Pendahuluan

Modul ini **tidak memperkenalkan protokol baru**. Seluruh komponennya sudah dibangun: sensor Thread (M11), mesh (M12), gateway dwi-radio (M13), dan klien MQTT (M14). Yang baru adalah menyatukannya — dan menghadapi masalah yang hanya muncul saat sistem punya banyak hop: **ketika data tidak sampai, hop mana yang salah?** Karena tiap hop sudah diukur sendiri pada modul sebelumnya, angka pembanding untuk menjawabnya sudah tersedia.

Prasyaratnya adalah M11–M12 untuk Thread beserta dataset-nya, M13 untuk gateway dwi-radio, dan M14 untuk MQTT beserta `mosquitto_sub`. Yang dibangun di sini adalah integrasi tiga stack pada satu gateway, pengukuran latency dan loss per hop pada rantai empat hop, isolasi kesalahan, serta verifikasi dari sisi luar sistem. Semuanya dipakai lagi pada M16 ketika pipeline ini menjadi kerangka baku dan hanya protokol hop pertama yang diganti-ganti agar perbandingannya adil.

```
H2 → Thread → C6 → Wi-Fi → MQTT → Dashboard
```

Peta modul blok integrasi (penutup blok)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
13	Gateway: Thread bertemu Wi-Fi, keluar lewat HTTP
14	Sisi IP diperdalam: publish/subscribe MQTT
15 (ini)	**Semuanya disatukan: rantai lengkap sensor ke dashboard**
16	Hop pertama diganti BLE, Zigbee, atau Thread untuk dibandingkan

Tabel 17.1: Peta modul blok integrasi

Kontrak data lab ini. Payload `suhu:XX.X` berjalan **tanpa diubah** dari node H2 sampai subscriber di PC — gateway tidak mem-parsing ulang, hanya memindahkan dari satu transport ke transport lain. Karena itu topic `praktikum/h2/telemetri` di sini identik dengan M14, dan datanya bisa langsung dibandingkan dengan M16.

17.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Merangkai dan mendiagnosis pipeline IoT empat hop

1. Merangkai pipeline lengkap sensor Thread, gateway dwi-radio, broker MQTT, dan subscriber, lalu menunjukkan jejak satu pesan yang sama di keempat titik.
2. Mengonfigurasi ESP32-C6 menjalankan Thread Leader, Wi-Fi STA, dan klien MQTT bersamaan, serta menjelaskan urutan inisialisasinya.
3. Mengukur latency end-to-end pada tiga skenario jarak dan memecahnya menjadi kontribusi tiap hop memakai data M11–M14.
4. Mengidentifikasi hop penyebab packet loss dengan membandingkan counter di tiap tahap, bukan dengan menebak.

Kriteria keberhasilan

- ☐ Pesan `suhu:XX.X` dari H2 muncul di subscriber MQTT tiap ± 3 detik.
- ☐ Latency end-to-end terukur pada tiga skenario jarak.
- ☐ Hop penyebab loss teridentifikasi dengan membandingkan counter tiap tahap.
- ☐ Satu pesan yang sama dapat ditunjukkan jejaknya di H2, C6, dan `mosquitto_sub`.

17.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (arsitektur referensi IoT, edge dibanding cloud processing, skema QoS berlapis) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 17.2.

Istilah	Definisi kerja di lab ini
Pipeline end-to-end	Rantai lengkap sensor, WSN, gateway, backbone IP, broker, sampai konsumen data.
Gateway dwi-radio	C6 menjalankan stack Thread dan Wi-Fi bersamaan, menerjemahkan UDP multicast Thread menjadi publish MQTT.
Hop	Satu perpindahan antar tahap; rantai ini punya 4 hop yang bisa gagal secara terpisah.
Transparent forwarding	String `suhu:XX.X` diteruskan tanpa diubah sepanjang rantai.
Topic	Data masuk ke `praktikum/h2/telemetri`; konsumen cukup berlangganan tanpa tahu sensornya.
QoS	Jaminan pengiriman MQTT (0, 1, atau 2); kode ini memakai QoS 0 (batasan PubSubClient).

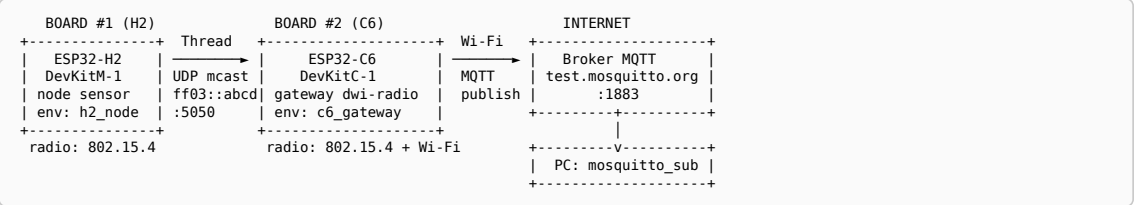
Tabel 17.2: Istilah kerja Modul 15

Isolasi kesalahan adalah inti modul ini. Empat hop berarti empat tempat pesan bisa hilang. Cara membedakannya: hitung **counter di tiap tahap** — berapa yang dikirim H2, berapa yang tercetak `RX via Thread` di C6, berapa yang tercetak `Publish MQTT`, dan berapa yang muncul di `mosquitto_sub`. Selisih antar tahap menunjuk hop yang bermasalah. Tanpa counter, semua kegagalan terlihat sama: "datanya tidak muncul".

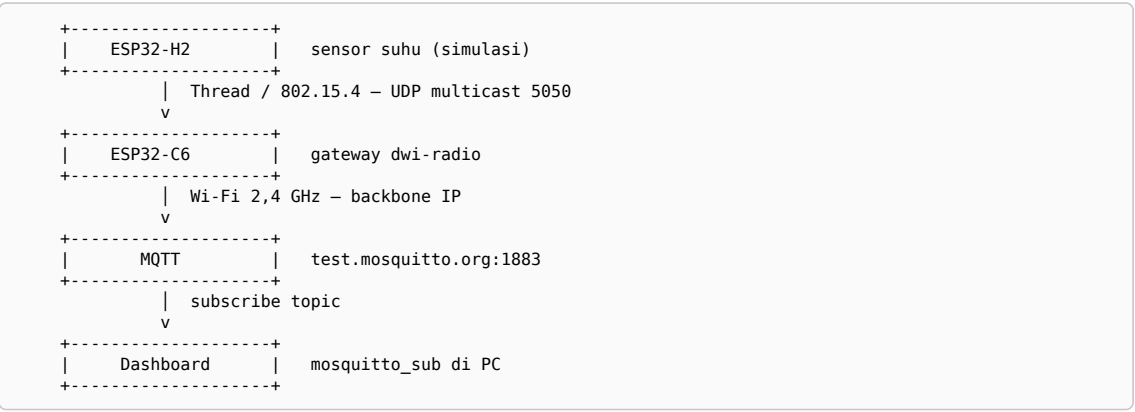
Sekuens protokol yang diamati

```
[H2] readSensor ⟶ "suhu:25.3" ⟶ OtUdp multicast
[C6] OtUdp.parsePacket ⟶ "RX via Thread" ⟶ mqtt.publish(TOPIC_TELEM)
                                          ⟶ "Publish MQTT [...]"
[PC] mosquitto_sub -t praktikum/h2/telemetri ⟶ praktikum/h2/telemetri suhu:25.3
```

17.4 Topologi



Rantai vertikal per lapis:



Node	Board	Environment	Peran
Sensor	**ESP32-H2** DevKitM-1	`h2_node`	Sensor Thread, TX `suhu:XX.X` tiap 3 s
Gateway	**ESP32-C6** DevKitC-1	`c6_gateway`	Thread Leader, Wi-Fi STA, dan klien MQTT (`esp32c6-gateway`)
Konsumen	PC/laptop	—	`mosquitto_sub`, verifikasi independen

Tabel 17.3: Peran tiap elemen Modul 15

Sama seperti M13, **dua jenis board wajib**: ESP32-H2 tidak punya Wi-Fi, jadi hanya ESP32-C6 yang bisa memegang Thread dan Wi-Fi sekaligus.

17.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (sensor Thread) dan ESP32-C6 (gateway Thread dan Wi-Fi/MQTT).

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1 — node sensor Thread	1
2	Board ESP32-C6	DevKitC-1 — gateway Thread ke MQTT	1
3	Kabel USB data	kabel data, bukan *charge-only*	2
4	PC/Laptop	PlatformIO Core/IDE dan `mosquitto-clients`	1
5	Wi-Fi atau hotspot	**2,4 GHz**, ada akses internet	1
6	Broker MQTT	`test.mosquitto.org:1883` atau Mosquitto lokal	1
7	Library PubSubClient	`knolleary/PubSubClient@^2.8` via `lib_deps`	—

Tabel 17.4: Alat dan bahan Modul 15

Konfigurasi jaringan

Parameter	Nilai
Thread network	`ESP_OT_E2E`, channel 15, PAN `0xABCD`
Mesh-local prefix	`fdde:ad00:beef::/64` (identik di H2 dan C6)
Grup multicast dan port	`ff03::abcd` : 5050
Topic MQTT	`praktikum/h2/telemetri`
Client ID gateway	`esp32c6-gateway`
Interval telemetri	3000 ms

Tabel 17.5: Konfigurasi jaringan Modul 15

17.6 Kode Program

```
[platformio]
default_envs = c6_gateway

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
monitor_speed = 115200

[env:h2_node]
board = esp32-h2-devkitm-1
build_src_filter = +<h2_node/*.cpp>

[env:c6_gateway]
board = esp32-c6-devkitc-1
build_src_filter = +<c6_gateway/*.cpp>
; Thread/BLE + Wi-Fi + MQTT/HTTP melebihi partisi app default (1,25 MB)
board_build.partitions = huge_app.csv
lib_deps =
    knolleary/PubSubClient@^2.8
```

Kode 17.1: `platformio.ini` Modul 15 pada repositori

```
// Minggu 15 — End-to-End IoT: sensor H2 kirim telemetri via Thread
#include <Arduino.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>

const char OT_NETWORK_NAME[] = "ESP_OT_E2E";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama di H2 dan C6. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai ke gateway.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}
```

```
OThreadUDP OtUdp;

static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Sensor H2 (Thread node) starting...");

  OThread.begin(false);
  // Dataset selalu ditulis ulang agar H2 dan C6 memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu join ke gateway (C6)...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Attached as: %s\n", OThread.otGetStringDeviceRole());

  OtUdp.begin(PORT);
}

void loop() {
  static unsigned long last = 0;
  if (millis() - last > 3000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "suhu:%.1f", readSensor());
    OtUdp.beginPacket(GROUP, PORT);
    OtUdp.write((const uint8_t *)msg, strlen(msg));
    OtUdp.endPacket();
    Serial.printf("TX via Thread: %s\n", msg);
  }
  delay(10);
}
```

Kode 17.2: `src/h2_node/main.cpp` — node sensor Thread di ESP32-H2

```
// Minggu 15 — End-to-End IoT: gateway C6 terima Thread lalu publish ke MQTT
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include "OThread.h"
#include "OThreadUDP.h"
#include "esp_openthread.h"
#include "esp_openthread_lock.h"
#include <openthread/thread.h>
#include "esp_netif.h"
#include "esp_coexist.h"

const char *WIFI_SSID = "SprH-3";
const char *WIFI_PASS = "yourpassword!";

// Batas waktu menunggu Wi-Fi/MQTT saat boot; melewati batas ini setup() tetap
// dilanjutkan agar kegagalan sisi IP tidak menyembunyikan sisi radio lain.
const unsigned long WIFI_TIMEOUT_MS = 30000;
// Jeda antar percobaan sambung ulang. Harus lebih panjang dari durasi
// asosiasi pada sinyal lemah, kalau tidak percobaan berikutnya justru
// membatalkan yang sedang berjalan.
const unsigned long WIFI_RETRY_MS = 20000;
```

```
// Jeda percobaan MQTT dipisah dari jeda Wi-Fi: saat Wi-Fi sudah sehat,
// tidak ada alasan menunggu selama asosiasi ulang.
const unsigned long MQTT_RETRY_MS = 4000;
const unsigned long MQTT_TIMEOUT_MS = 15000;

// Broker MQTT. Banyak jaringan kampus memblokir port 1883 keluar sehingga
// test.mosquitto.org tidak terjangkau (gejala: rc=-2 dan "Host is unreachable").
// Bila itu terjadi, jalankan `python3 tools/mqtt_broker.py` di laptop lalu ganti
// baris di bawah dengan alamat IP laptop tersebut, mis. "192.168.110.74".
const char *MQTT_BROKER = "192.168.1.5";
const uint16_t MQTT_PORT = 1884;
const char *TOPIC_TELEM = "praktikum/h2/telemetri";

const char OT_NETWORK_NAME[] = "ESP_OT_E2E";
const uint8_t  OT_CHANNEL = 15;
const uint16_t OT_PAN_ID  = 0xABCD;
const uint8_t  OT_EXTPANID[OT_EXT_PAN_ID_SIZE] = {0xDE, 0xAD, 0x00, 0xBE, 0xEF, 0x00, 0xCA, 0xFE};
const uint8_t  OT_NETKEY[OT_NETWORK_KEY_SIZE] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99,
0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

const uint16_t PORT = 5050;
const uint8_t  GROUP_BYTES[16] = {0xff, 0x03, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xab, 0xcd};
const IPAddress GROUP(IPv6, GROUP_BYTES);

// Prefix mesh-local dipaksa sama di H2 dan C6. DataSet::initNew() mengacak
// prefix ini per board; node dengan prefix berbeda tetap bisa attach, tetapi
// trafik multicast mesh (ff03::/16) tidak akan sampai ke gateway.
const uint8_t OT_ML_PREFIX[OT_MESH_LOCAL_PREFIX_SIZE] = {0xfd, 0xde, 0xad, 0x00, 0xbe, 0xef, 0x00, 0x00};

// Harus dipanggil sebelum OThread.start() (stack masih berhenti).
static void applyMeshLocalPrefix() {
  otMeshLocalPrefix prefix;
  memcpy(prefix.m8, OT_ML_PREFIX, OT_MESH_LOCAL_PREFIX_SIZE);
  esp_openthread_lock_acquire(portMAX_DELAY);
  otThreadSetMeshLocalPrefix(esp_openthread_get_instance(), &prefix);
  esp_openthread_lock_release();
}

// Board ini punya dua netif sekaligus (Wi-Fi STA dan OpenThread). Pastikan
// default netif lwIP tetap Wi-Fi STA supaya trafik IPv4 keluar (HTTP/MQTT)
// punya rute yang benar. Catatan hasil uji: ini praktik yang benar, tetapi
// BUKAN obat untuk kegagalan TCP karena starvation koeksistensi — lihat
// Bagian 7 README.
static void restoreWifiAsDefaultNetif() {
  esp_netif_t *sta = esp_netif_get_handle_from_ifkey("WIFI_STA_DEF");
  if (sta == nullptr) {
    Serial.println("Netif Wi-Fi STA tidak ditemukan; rute IPv4 mungkin salah.");
    return;
  }
  esp_err_t err = esp_netif_set_default_netif(sta);
  Serial.printf("Default netif dikembalikan ke Wi-Fi STA (err=%d)\n", (int)err);
}

OThreadUDP OtUdp;
WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

// SIMULASI sensor H2 (tidak ada board fisik): bangkitkan nilai suhu sintetis
// agar rantai gateway -> MQTT tetap bisa diuji end-to-end.
static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);

  // URUTAN PENTING: Thread dinyalakan LEBIH DULU, baru Wi-Fi.
  // Wrapper OpenThread Arduino memanggil esp_event_loop_create_default() dan
  // menganggap ESP_ERR_INVALID_STATE sebagai kegagalan fatal. Kalau Wi-Fi
  // diinisialisasi duluan, event loop default sudah ada, sehingga
  // OThread.begin() gagal dan board panic
  // ("assert failed: otTaskletsSignalPending"). Sebaliknya, stack Wi-Fi
  // Arduino memang mentoleransi event loop yang sudah dibuat.

  // 1) Thread leader
  OThread.begin(false);
  // Dataset selalu ditulis ulang agar H2 dan C6 memakai parameter identik,
  // termasuk saat board masih menyimpan dataset dari modul sebelumnya.
```

```
  DataSet ds;
  ds.initNew();
  ds.setNetworkName(OT_NETWORK_NAME);
  ds.setChannel(OT_CHANNEL);
  ds.setPanId(OT_PAN_ID);
  ds.setExtendedPanId(OT_EXTPANID);
  ds.setNetworkKey(OT_NETKEY);
  OThread.commitDataSet(ds);
  applyMeshLocalPrefix();

  // 2) Wi-Fi disambungkan SEBELUM OThread.start()
  WiFi.mode(WIFI_STA);
  // JANGAN setSleep(false) di board yang juga menjalankan 802.15.4:
  // koeksistensi Wi-Fi + 802.15.4 pada ESP32-C6 mengandalkan modem sleep
  // untuk membagi airtime. Mematikannya membuat Wi-Fi meminta radio 100 %
  // sementara Thread juga selalu RX, sehingga TCP keluar hampir selalu gagal
  // (gejala: HTTP -1 / MQTT rc=-2 padahal IP sudah didapat).
  WiFi.setSleep(true);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.printf("Konek Wi-Fi %s", WIFI_SSID);
  // Batasi penantian: kalau SSID/password salah atau AP tidak ada, setup()
  // tetap lanjut supaya sisi radio lain (Thread/BLE) tetap bisa diamati.
  // Wi-Fi dicoba ulang di loop().
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < WIFI_TIMEOUT_MS) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nWi-Fi OK, IP: %s | RSSI: %d dBm\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.printf("\nWi-Fi GAGAL setelah %lu s (status=%d). Lanjut tanpa Wi-Fi;\n"
                  "periksa WIFI_SSID/WIFI_PASS dan pastikan AP memancar di 2,4 GHz.\n",
                  WIFI_TIMEOUT_MS / 1000UL, (int)WiFi.status());
  }


  // Beri prioritas radio ke Wi-Fi sebelum 802.15.4 dinyalakan. Tanpa ini,
  // trafik TCP keluar hampir selalu gagal saat stack Thread aktif.
  esp_err_t coexErr = esp_coex_preference_set(ESP_COEX_PREFER_WIFI);
  Serial.printf("coex preference = WIFI (err=%d)\n", (int)coexErr);

  // 3) Baru radio Thread dinyalakan
  OThread.networkInterfaceUp();
  OThread.start();

  Serial.println("Menunggu attach Thread...");
  while (OThread.otGetDeviceRole() < OT_ROLE_CHILD) {
    delay(250);
  }
  Serial.printf("Thread attached as: %s\n", OThread.otGetStringDeviceRole());

  restoreWifiAsDefaultNetif();

  OtUdp.beginMulticast(GROUP, PORT);

  // 4) Terakhir MQTT (butuh Wi-Fi sudah siap)
  mqtt.setServer(MQTT_BROKER, MQTT_PORT);
  unsigned long mqttStart = millis();
  while (!mqtt.connected() && millis() - mqttStart < MQTT_TIMEOUT_MS) {
    Serial.printf("Konek MQTT %s:%d ...\n", MQTT_BROKER, MQTT_PORT);
    if (!mqtt.connect("esp32c6-gateway")) {
      Serial.printf("Gagal (rc=%d)\n", mqtt.state());
      delay(2000);
    }
  }
  if (mqtt.connected()) {
    Serial.println("MQTT terhubung");
  } else {
    Serial.println("MQTT belum terhubung. Lanjut; dicoba ulang di loop().");
  }


  Serial.println("Gateway siap (H2 -> Thread -> C6 -> MQTT).");
}


// Pemeliharaan koneksi IP, dijalankan berkala dari loop().
// Rate-limited: tanpa ini, mqtt.connect() dipanggil tiap iterasi loop dan
// membanjiri Serial dengan kegagalan DNS saat Wi-Fi/broker tidak tersedia.
static void maintainNetwork(const char *clientId) {
```

```
  static unsigned long lastWifiTry = 0;
  static unsigned long lastMqttTry = 0;

  if (WiFi.status() == WL_CONNECTED) {
    // Wi-Fi sehat: percobaan MQTT tidak boleh ikut terkunci jeda panjang milik
    // Wi-Fi, kalau tidak kesempatan menyambung jadi jauh lebih sedikit.
    if (mqtt.connected() || millis() - lastMqttTry < MQTT_RETRY_MS) return;
    lastMqttTry = millis();
    if (mqtt.connect(clientId)) {
      Serial.println("MQTT terhubung");
    } else {
      Serial.printf("MQTT gagal (rc=%d), coba lagi %lu detik\n",
                    mqtt.state(), MQTT_RETRY_MS / 1000UL);
    }
    return;
  }

  if (millis() - lastWifiTry < WIFI_RETRY_MS) return;
  lastWifiTry = millis();
  {
    // JANGAN WiFi.disconnect() di sini: pada sinyal lemah, asosiasi bisa perlu
    // > 10 detik, dan memutusnya tiap percobaan membuat Wi-Fi tidak pernah
    // selesai menyambung. Cukup picu ulang WiFi.begin() dengan jeda panjang.
    Serial.printf("Wi-Fi belum tersambung (status=%d), mencoba lagi...\n",
                  (int)WiFi.status());
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    return;                       // MQTT percuma dicoba tanpa Wi-Fi
  }
}

void loop() {
  maintainNetwork("esp32c6-gateway");
  mqtt.loop();

  // SIMULASI sensor: bangkitkan telemetri tiap 3 detik lalu publish ke MQTT.
  static unsigned long lastSim = 0;
  if (millis() - lastSim > 3000) {
    lastSim = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "suhu:%.1f", readSensor());
    Serial.printf("SIM sensor (Thread): %s\n", msg);
    if (mqtt.connected() && mqtt.publish(TOPIC_TELEM, msg)) {
      Serial.printf("Publish MQTT [%s]: %s\n", TOPIC_TELEM, msg);
    } else {
      Serial.printf("Publish MQTT GAGAL (mqtt=%d): %s\n", mqtt.state(), msg);
    }
  }

  while (int n = OtUdp.parsePacket()) {
    char buf[64];
    int got = OtUdp.read((uint8_t *)buf, (n < (int)sizeof(buf) - 1) ? n : (int)sizeof(buf) - 1);
    buf[got] = '\0';
    Serial.printf("RX via Thread: %s\n", buf);

    // Laporkan apa adanya: hop Thread bisa sukses walau hop MQTT gagal.
    if (mqtt.connected() && mqtt.publish(TOPIC_TELEM, buf)) {
      Serial.printf("Publish MQTT [%s]: %s\n", TOPIC_TELEM, buf);
    } else {
      Serial.printf("Publish MQTT GAGAL (mqtt=%d), pesan Thread tidak diteruskan: %s\n",
                    mqtt.state(), buf);
    }
  }
  delay(10);
}
```

Kode 17.3: `src/c6_gateway/main.cpp` — gateway Thread, Wi-Fi, dan MQTT

17.7 Build dan Flash

```
# terminal 1 - subscriber, jalankan lebih dulu
mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v

# terminal 2 & 3 - gateway dulu, baru node sensor
pio run -d week15_e2e_iot -e c6_gateway -t upload -t monitor
pio run -d week15_e2e_iot -e h2_node    -t upload -t monitor
```

Pre-flight checklist

- ☐ ESP32-H2 dan ESP32-C6 terhubung ke PC via kabel USB, port dicatat.
- ☐ `WIFI_SSID` dan `WIFI_PASS` pada `src/c6_gateway/main.cpp` sudah disesuaikan (hotspot **2,4 GHz**).
- ☐ Broker MQTT dapat dijangkau dari PC (tes dengan `mosquitto_sub`).
- ☐ Firmware `h2_node` dan `c6_gateway` berhasil di-build.
- ☐ Env gateway memakai `board_build.partitions = huge_app.csv` (firmware lebih dari 1,25 MB).
- ☐ Dua Serial Monitor (115200) dibuka, satu untuk tiap board.
- ☐ Prefix mesh-local kedua firmware sama: `fdde:ad00:beef::/64`.
- ☐ Data latency dan loss dari M11 (Thread) dan M14 (MQTT) sudah di tangan sebagai pembanding per hop.

17.8 Percobaan

17.8.1 EXP-01 — Inisialisasi Gateway

Deploy firmware kedua board. Gateway melakukan tiga tahap setup: konek Wi-Fi, konek MQTT (client ID `esp32c6-gateway`), lalu membentuk jaringan Thread `ESP_OT_E2E` sebagai Leader dan join grup multicast. Ukur waktu total setup.

```
[C6 setup] WiFi.begin ──► mqtt.connect ──► OThread dataset ──► leader
           ──► beginMulticast(ff03::abcd, 5050)
```

Data capture

Parameter	Hasil
SSID dan IP Wi-Fi gateway	
Broker dan port	
Nama jaringan Thread	
Role C6 setelah attach	
Awalan Mesh-Local EID H2 dan C6 sama?	
Waktu total setup hingga `Gateway siap`	

Tabel 17.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Gateway ini menjalankan **tiga** stack di satu chip: Thread, Wi-Fi, dan MQTT. Bandingkan ukuran firmware `c6_gateway` modul ini dengan `c6_gateway` M13 (Thread, Wi-Fi, dan HTTP) dari ringkasan `pio run`, lalu periksa `huge_app.csv` untuk melihat berapa besar partisi app yang tersedia. Jawab: berapa persen partisi sudah terpakai, dan apa yang akan terjadi bila ditambahkan satu stack lagi (misalnya BLE seperti di M16)? Ini metrik yang jarang diukur orang tetapi menentukan apakah sebuah gateway bisa dikembangkan lagi.

CHECKPOINT

Gateway mencetak **ketiga** tanda kesiapan berurutan: `Wi-Fi OK, IP: ...`, `MQTT terhubung`, dan `Thread attached as: leader`. Jika salah satu tidak muncul, hentikan di sini — mendiagnosis satu stack jauh lebih mudah daripada mendiagnosis tiga sekaligus.

17.8.2 EXP-02 — Thread TX lalu Forward MQTT

H2 mengirim `suhu:XX.X` tiap 3 detik; C6 menerima paket dan langsung mem-publish payload yang sama ke broker.

```
[H2] ──3 s──► "TX via Thread: suhu:25.3"
[C6] ──► "RX via Thread: suhu:25.3" ──► "Publish MQTT [praktikum/h2/telemetri]: suhu:25.3"
```

Expected output — H2

```
Sensor H2 (Thread node) starting...
Menunggu join ke gateway (C6)...
Attached as: Child
TX via Thread: suhu:25.2
```

Expected output — C6

```
Konek Wi-Fi SprH-3......
Wi-Fi OK, IP: 192.168.1.39 | RSSI: -67 dBm
coex preference = WIFI (err=0)
Menunggu attach Thread...
Thread attached as: Leader
Default netif dikembalikan ke Wi-Fi STA (err=0)
Konek MQTT 192.168.1.5:1884 ...
Gagal (rc=-2)
Konek MQTT 192.168.1.5:1884 ...
Gagal (rc=-2)
Konek MQTT 192.168.1.5:1884 ...
Gagal (rc=-2)
MQTT belum terhubung. Lanjut; dicoba ulang di loop().
Gateway siap (H2 -> Thread -> C6 -> MQTT).
MQTT gagal (rc=-2), coba lagi 4 detik
SIM sensor (Thread): suhu:25.8
Publish MQTT GAGAL (mqtt=-2): suhu:25.8
```

CATATAN

Log di atas direkam **tanpa board H2**: baris `SIM sensor (Thread): ...` berasal dari simulasi sensor di firmware gateway. Bila board H2 terpasang, baris `RX via Thread: suhu:...` ikut muncul. Pada rekaman ini hop MQTT gagal total (`rc=-2`, koneksi TCP keluar tidak terbentuk), sehingga setiap nilai dicetak `Publish MQTT GAGAL`. Saat MQTT tersambung, baris yang sama menjadi `Publish MQTT [praktikum/h2/telemetri]: suhu:....`

CHECKPOINT

Untuk **satu** nilai suhu yang sama (misalnya `25.2`), tiga baris log berurutan harus dapat ditunjuk: `TX via Thread` di H2, `RX via Thread` di C6, dan `Publish MQTT` di C6. Jika baris kedua ada tetapi ketiga tidak, masalahnya di MQTT — bukan di Thread.

17.8.3 EXP-03 — Verifikasi End-to-End

Jalankan subscriber di PC:

```
mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v
```

Verifikasi bahwa tiap ± 3 detik muncul:

```
praktikum/h2/telemetri suhu:25.2
```

Variasi wajib:

1. Jauhkan H2 dari C6 (1 m, 5 m, lalu di balik dinding) dan hitung pesan yang sampai ke subscriber.
2. Matikan hotspot sesaat untuk melihat reconnect MQTT (kode melakukan `mqtt.connect` ulang di `loop`). Catat berapa pesan Thread yang datang selama itu dan apa nasibnya.
3. Reset H2 saat sistem berjalan; ukur berapa lama sampai data muncul lagi di subscriber.

Data capture

Parameter	Hasil
Interval pesan di subscriber (s)	
Jumlah pesan per 2 menit	
RSSI Wi-Fi gateway	
Perilaku saat MQTT terputus	
Waktu pulih setelah H2 di-reset	

Tabel 17.7: Lembar pengamatan EXP-03

CHECKPOINT

Nilai suhu yang muncul di `mosquitto_sub` sama persis dengan yang dicetak H2. Jika berbeda atau tertukar urutannya, ada pihak lain yang mem-publish ke topic yang sama — ganti prefix topic menjadi unik per kelompok.

17.9 Verifikasi Hardware — Log Referensi

Dijalankan pada **ESP32-H2 DevKitM-1** dan **ESP32-C6 DevKitC-1** asli.

```
# H2 (env h2_node)                # C6 (env c6_gateway)
[1.002] Attached as: Router       [2.003] Wi-Fi OK, IP: 192.168.110.197 | RSSI: -84 dBm
[3.406] TX via Thread: suhu:24.7  [3.004] Thread attached as: Router
[6.412] TX via Thread: suhu:24.5  [3.004] Default netif dikembalikan ke Wi-Fi STA (err=0)
                                  [18.025] RX via Thread: suhu:23.4
                                  [18.025] Publish MQTT GAGAL (mqtt=-2) ...
```

Bagian rantai	Status
Build `h2_node` dan `c6_gateway` (`huge_app.csv`)	berhasil
C6 attach ke Thread `ESP_OT_E2E`	berhasil, 3,0 s sejak boot
Hop Thread H2 ke C6 (`RX via Thread`)	berhasil, seluruh paket, 0 % loss
C6 asosiasi Wi-Fi sambil Thread jalan	berhasil, 2,0 s (setelah perbaikan urutan)
Publish MQTT ke broker	berjalan tetapi tidak andal (0–36 % end-to-end, tergantung AP)

Tabel 17.8: Hasil verifikasi hardware Modul 15

17.9.1 Rantai Penuh Terbukti — dengan Satu Syarat Wajib

Pipeline H2, Thread, C6, Wi-Fi, MQTT, hingga subscriber berhasil dijalankan utuh, **tetapi hanya setelah prioritas radio diberikan ke Wi-Fi** sebelum stack 802.15.4 dinyalakan (Kode 17.4).

```
#include "esp_coexist.h"
...
esp_coex_preference_set(ESP_COEX_PREFER_WIFI);   // sebelum OThread.start()
```

Kode 17.4: Prioritas radio ke Wi-Fi sebelum stack Thread dinyalakan

Tanpa baris itu, C6 tetap asosiasi Wi-Fi dan dapat IP, tetapi **seluruh TCP keluar gagal** — bahkan ke router sendiri (probe timeout 4 detik berulang).

Diuji pada tiga jaringan Wi-Fi berbeda. Hop Thread selalu sehat; hop Wi-Fi/MQTT yang bervariasi dan selalu menjadi penyumbang kerugian (Tabel 17.9).

AP uji	Kanal	RSSI	H2 TX ke C6 RX	Publish ke broker	End-to-end
AP-1	1 (2412 MHz)	−82 dBm	berhasil	0	0 %
AP-2	12 (2467 MHz)	−81 dBm	32/33 (97 %)	12/12	**36 %**
AP-3	9 (2452 MHz)	−71 dBm	30/39 (77 %)	2/7	**5 %**

Tabel 17.9: Hasil uji pada tiga access point berbeda

Sinyal yang lebih kuat (AP-3) **tidak** menghasilkan hasil terbaik — jadi penyebabnya bukan link budget, melainkan pembagian airtime antara Wi-Fi dan 802.15.4 pada satu antena. Mengganti channel 802.15.4 (15 menjadi 25, dan 15 menjadi 11 untuk menjauh maksimum dari kanal Wi-Fi AP-3) juga tidak menolong.

> **PENTING**
> **Pelajaran penting dari AP-3: `publish()` bernilai `true` bukan bukti sampai.** Pada run itu gateway mencetak 7 baris `Publish MQTT [...]` sementara broker hanya menerima **2**. Pada QoS 0, `PubSubClient::publish()` hanya menulis ke buffer socket; bila koneksi TCP sudah setengah mati, tidak ada yang memberi tahu. Karena itu bagian Pengukuran mewajibkan kolom "Broker menerima" diisi dari **log broker**, bukan dari log gateway. Ini juga jawaban konkret untuk Concept Check nomor 3.

Pembanding yang paling menentukan. Modul 16 (BLE dan Wi-Fi, bukan Thread dan Wi-Fi) diukur pada AP-3, jarak, dan broker yang sama persis (Tabel 17.10).

Pipeline	Modul	Hop sensor	Hop Wi-Fi/MQTT	End-to-end
BLE ke C6 ke MQTT	M16	47/48 (98 %)	47/47 (100 %)	**98 %**
Thread ke C6 ke MQTT	M15	30/39 (77 %)	2/7 (29 %)	**5 %**

Tabel 17.10: Pembanding pipeline BLE dan Thread pada kondisi jaringan identik

Kondisi jaringan identik; yang berbeda hanya radio hop pertama. Ini bahan utama untuk analisis Modul 16: pada gateway satu-chip satu-antena, **BLE dengan Wi-Fi nyaris tanpa ongkos koeksistensi, sedangkan Thread dengan Wi-Fi sangat mahal**. Kesimpulan yang tepat bukan "Thread lebih buruk dari BLE", melainkan bahwa arsitektur gateway satu-chip tidak cocok untuk Thread dan Wi-Fi — border router dua-chip akan mengubah angka ini sepenuhnya.

Yang sudah dicoba dan tidak menyelesaikan: mengganti channel 802.15.4 (15 menjadi 25), memakai AP di kanal Wi-Fi yang tidak bertetangga, `WiFi.setSleep()` kedua nilainya, dan memaksa default netif ke Wi-Fi STA. Yang **berhasil** hanya kombinasi urutan inisialisasi, `esp_coex_preference_set(ESP_COEX_PREFER_WIFI)`, dan memisahkan jeda retry MQTT dari jeda retry Wi-Fi. Bahkan setelah itu, hop Wi-Fi tetap tidak andal — laporkan angkanya apa adanya.

Yang harus dilakukan praktikan: catat RSSI Wi-Fi gateway di laporan dan isi tabel counter per tahap di bagian Pengukuran. Rantai yang bocor di satu hop dengan sebab yang dapat ditunjuk bernilai lebih tinggi daripada demo mulus tanpa data.

17.9.2 Perbaikan Kode yang Lahir dari Uji Ini

Masalah di perangkat nyata	Perbaikan
C6 **panic** saat boot (`Failed to create OpentThread event loop`, lalu `assert failed: otTaskletsSignalPending`)	`OThread.begin()` dipanggil **sebelum** Wi-Fi; wrapper OpenThread tidak mentoleransi event loop default yang sudah dibuat Wi-Fi
Wi-Fi tidak pernah asosiasi bila `OThread.start()` sudah jalan	Wi-Fi disambungkan **di antara** `OThread.begin()` dan `OThread.start()`, sehingga asosiasi selesai sebelum radio 802.15.4 aktif
`while (WiFi.status() != WL_CONNECTED)` dan loop MQTT menggantung selamanya	dibatasi `WIFI_TIMEOUT_MS` dan `MQTT_TIMEOUT_MS`, lalu lanjut dan dicoba ulang berkala
`mqtt.connect()` dipanggil tiap iterasi `loop()`, sehingga banjir `DNS Failed` menenggelamkan log	`maintainNetwork()` dengan jeda `WIFI_RETRY_MS` dan cek Wi-Fi lebih dulu
`WiFi.disconnect()` di tiap percobaan ulang membatalkan asosiasi yang sedang berjalan	hanya `WiFi.begin()` ulang, dengan jeda lebih panjang dari durasi asosiasi
`Publish MQTT [...]` dicetak walau broker tidak terhubung — **laporan palsu**	publish hanya bila `mqtt.connected()`, kegagalan dicetak apa adanya beserta `mqtt.state()`
Seluruh TCP keluar gagal saat stack Thread aktif, padahal Wi-Fi sudah dapat IP	`esp_coex_preference_set(ESP_COEX_PREFER_WIFI)` dipanggil sebelum `OThread.start()`
Percobaan MQTT ikut terkunci jeda retry Wi-Fi (20 s) padahal Wi-Fi sudah sehat	timer retry Wi-Fi dan MQTT dipisah (`WIFI_RETRY_MS` dibanding `MQTT_RETRY_MS`)

Tabel 17.11: Perbaikan kode hasil pengujian perangkat nyata Modul 15

17.10 Pengukuran

Skenario / Jarak H2–C6	RSSI Wi-Fi (C6)	Latency end-to-end (TX H2 ke subscriber, ms)	Success / 40 pesan
1 m, garis pandang			
5 m, garis pandang			
5 m + penghalang dinding			

Tabel 17.12: Lembar pengukuran end-to-end Modul 15

Latency end-to-end diukur dari cap waktu `TX via Thread` pada H2 dibandingkan kemunculan pesan di `mosquitto_sub` (gunakan `mosquitto_sub -F '%t %p'` atau timestamp terminal).

Tabel counter per tahap — inti modul ini. Amati 2 menit (kira-kira 40 pesan).

Tahap	Baris log yang dihitung	Jumlah	Loss terhadap tahap sebelumnya
1. H2 mengirim	`TX via Thread`		—
2. C6 menerima	`RX via Thread`		
3. C6 mem-publish	`Publish MQTT`		
4. Subscriber menerima	baris di `mosquitto_sub`		

Tabel 17.13: Lembar counter per tahap Modul 15

Dekomposisi latency — wajib. Isi dari data modul sebelumnya.

Hop	Modul sumber angka	Latency
H2 ke C6 (Thread)	M11	
C6 ke broker (MQTT)	M14	
Jumlah keduanya	—	
Latency end-to-end terukur (M15)	modul ini	
Selisih (overhead integrasi)	—	

Tabel 17.14: Lembar dekomposisi latency per hop

17.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Apa peran gateway C6 dalam pipeline ini, dan protokol apa saja yang ia jalankan bersamaan?
2. Berapa latency tambahan karena hop Thread ke MQTT dibanding pengiriman Thread satu hop (M11 dan M13)? Gunakan tabel dekomposisi latency.
3. Apakah payload berubah di sepanjang rantai? Jelaskan desain transparent forwarding ini dan apa untungnya untuk M16.
4. Bagian mana dari rantai yang paling banyak kehilangan pesan? Tunjukkan dari tabel counter per tahap — bukan dari dugaan.
5. Bandingkan arsitektur end-to-end ini dengan node sensor Wi-Fi dan MQTT langsung (M14) dari sisi konsumsi daya, jangkauan, dan jumlah node yang bisa dilayani.

17.12 Concept Check

1. Gambarkan kembali arsitektur H2, Thread, C6, dan MQTT lalu jelaskan fungsi tiap elemen.
2. Mengapa gateway melakukan publish dengan payload asli, bukan mem-parsing ulang datanya?
3. Apa yang terjadi pada data Thread yang sedang dikirim saat koneksi MQTT gateway terputus?
4. Bagaimana cara memonitor topic ini dari dashboard (misalnya MQTT Explorer) dan apa yang perlu disiapkan?
5. Keuntungan apa yang diperoleh dengan memisahkan jaringan sensor (Thread) dan backbone (Wi-Fi/MQTT), dibanding satu jaringan Wi-Fi untuk semuanya?

17.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah sumber dan menaikkan jaminan pengiriman

CH-1 — Dua sensor Tambahkan node Thread H2 kedua dengan payload ber-ID (`suhu:25.3,node2`) sehingga sumber data dapat dibedakan di subscriber. Diskusikan alternatifnya: memberi ID di payload dibanding memberi topic sendiri per node.

CH-2 — QoS sungguhan PubSubClient hanya mendukung publish QoS 0. Ganti library gateway ke yang mendukung QoS 1 (misalnya `arduino-mqtt`), ukur apakah success rate meningkat, dan bandingkan overhead-nya (ukuran firmware, latency).

Level 3 — Tantangan · Isolasi loss per hop dan dashboard sungguhan

CH-3 — Loss per hop (wajib) Hitung packet loss end-to-end dengan sequence number (`suhu:25.3,#42`). Contoh: H2 mengirim 100, subscriber menerima 94, sehingga loss = 6 %. Identifikasi hop penyebab dengan membandingkan counter `RX via Thread` dan `Publish MQTT` di C6.

CH-4 — Dashboard sungguhan Sambungkan topic ini ke MQTT Explorer, Node-RED, atau Grafana, lalu tampilkan grafik suhu terhadap waktu. Lampirkan tangkapan layarnya di laporan.

17.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (pipeline IoT, gateway dwi-radio, transparent forwarding, isolasi kesalahan per hop).
3. Konfigurasi — dataset Thread `ESP_OT_E2E`, SSID, broker, topic, client ID, dan PubSubClient.
4. Hasil eksperimen — log H2, C6, dan `mosquitto_sub`, termasuk jejak satu pesan yang sama di tiga titik.
5. Data pengukuran — tabel jarak, **tabel counter per tahap**, dan tabel dekomposisi latency.
6. Analisis dan concept check.
7. Challenge — minimal CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

18. Modul 16 — Proyek: Benchmark Protokol IoT

IoT COMMUNICATION LAB · MODUL 16

Prove Your Protocol — Proyek Benchmark Protokol IoT

ESP32-H2 + ESP32-C6 · BLE / Zigbee / Thread ke MQTT · level Project · 3 × 50 menit · folder kode `week16_comparative`

PENGANTAR MODUL · Gambaran Umum

Modul 16 dirancang untuk tiga pertemuan (3 × 50 menit) dan berstatus proyek. Misinya membuktikan dengan data — bukan dengan demo — protokol mana yang paling sesuai untuk sebuah skenario IoT. Percobaan berjalan sebagai proyek komparatif, diamati melalui dua Serial Monitor 115200 baud, `mosquitto_sub`, serta seluruh data pengukuran yang dikumpulkan sejak Modul 05.

18.1 Pendahuluan

Ini adalah modul **panen**. Lima belas modul sebelumnya menghasilkan angka; di sini angka-angka itu dikumpulkan menjadi satu tabel pembanding. Modul ini membangun pipeline **BLE ke gateway C6 ke MQTT** sebagai pasangan yang setara dengan pipeline Thread di M15 — dengan payload, interval, gateway, dan broker yang **sama persis**, sehingga hanya protokol hop pertama yang menjadi variabel.

Prasyaratnya adalah M04–M06 untuk BLE telemetry dan mesh, M08–M10 untuk Zigbee, M11–M13 untuk Thread, serta M14–M15 untuk MQTT dan pipeline — **beserta seluruh data pengukurannya**. Yang dibangun di sini adalah pipeline BLE ke MQTT sebagai pembanding pipeline Thread M15, metodologi perbandingan yang adil melalui variabel kontrol, dan rekomendasi yang berpijak pada data. Modul ini menutup seluruh seri: dari satu tautan radio pada M01 sampai keputusan arsitektur yang berbasis bukti.

PENTING

Yang dinilai bukan "sistem menyala". Yang dinilai adalah kemampuan membuktikan, dengan angka hasil pengukuran sendiri, protokol mana yang paling sesuai untuk sebuah kasus — dan mempertahankan rekomendasi itu saat ditanya balik.

Peta seluruh seri — dari mana angka hasil pengukuran berasal

Blok	Modul	Angka yang dipanen di sini
BLE	01–06	RSSI terhadap jarak, loss, latency notify, batas multi-node, latency relay
802.15.4	07	jangkauan radio telanjang (baseline)
Zigbee	08–10	waktu join dan binding, latency 1 hop dibanding 2 hop, loss per node
Thread	11–12	latency PING/PONG, loss mesh, waktu self-healing
Integrasi	13–15	latency dan loss per hop, latency end-to-end pipeline Thread
Proyek	**16 (ini)**	**latency dan loss end-to-end pipeline BLE — pasangan pembanding M15**

Tabel 18.1: Peta seluruh seri dan asal angka pengukuran

Kontrak data lab ini — inilah yang membuat perbandingan sah.

Variabel	Status	Nilai
Payload	**dikontrol**	`suhu:XX.X`
Interval kirim	**dikontrol**	2000 ms
Gateway	**dikontrol**	ESP32-C6
Broker dan topic	**dikontrol**	`test.mosquitto.org:1883`, `praktikum/h2/telemetri`
Jarak dan penghalang	**dikontrol**	1 m, 5 m, 5 m + dinding
Protokol hop pertama	**variabel bebas**	BLE (modul ini) dibanding Thread (M15) dibanding Zigbee (M08–M10)

Tabel 18.2: Variabel kontrol dan variabel bebas benchmark

Angka yang diukur pada kondisi berbeda **tidak boleh** dimasukkan ke tabel pembanding. Jika data M15 diambil pada interval 3 detik sementara modul ini 2 detik, ulangi salah satunya — atau nyatakan perbedaan itu secara eksplisit sebagai keterbatasan.

18.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membandingkan protokol dengan data yang dapat diaudit

1. Membangun pipeline BLE ke gateway C6 ke MQTT dengan variabel kontrol yang identik dengan pipeline Thread M15.
2. Mengukur RSSI, latency end-to-end, packet loss, dan throughput untuk pipeline BLE, lalu menempatkannya dalam satu tabel bersama data Thread dan Zigbee dari modul sebelumnya.
3. Menunjukkan asal setiap angka dalam tabel perbandingan (modul, tanggal, kondisi pengukuran) sehingga hasilnya dapat diaudit.
4. Merumuskan rekomendasi protokol untuk minimal tiga kasus nyata, dengan justifikasi yang menunjuk baris tertentu pada tabel — dan menyebutkan keterbatasan datanya.

Kriteria keberhasilan

- ☐ Tabel perbandingan protokol terisi metrik terukur (RSSI, latency, packet loss, throughput).
- ☐ Setiap angka dapat ditunjukkan asal pengukurannya (modul beserta kondisi).
- ☐ Rekomendasi protokol dapat dipertanggungjawabkan dengan data, bukan opini.
- ☐ Keterbatasan metodologi dinyatakan secara eksplisit.

18.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (model konsumsi daya, analisis kapasitas kanal, metodologi benchmark jaringan) berada di buku teori terpisah. Karakteristik kerja tiap protokol dirangkum pada Tabel 18.3.

Protokol atau istilah	Karakteristik kerja
BLE	Jaringan bintang, koneksi GATT, notify untuk push — daya sangat rendah, jangkauan pendek, tanpa mesh pada mode ini.
Zigbee	Mesh 802.15.4 dengan ZC, ZR, dan ZED — jangkauan luas via multi-hop, perintah baku antar-vendor.
Thread	Mesh 802.15.4 berbasis IPv6 — self-healing, role dipilih otomatis, langsung nyambung ke dunia IP.

Protokol atau istilah	Karakteristik kerja
MQTT pub/sub	Muara bersama semua protokol; QoS 0 pada lab ini.
Variabel kontrol	Faktor yang sengaja disamakan agar perbandingan adil.
Variabel bebas	Faktor yang sengaja diubah — di sini protokol hop pertama.
Metrik	RSSI, latency end-to-end, packet loss, throughput (pesan per menit).

Tabel 18.3: Istilah kerja Modul 16

Empat metrik, empat pertanyaan berbeda. Jangan dicampur.

Metrik	Menjawab pertanyaan
RSSI	Seberapa kuat sinyalnya di titik ini?
Latency	Berapa lama data sampai?
Packet loss	Berapa banyak yang tidak sampai sama sekali?
Throughput	Berapa banyak yang bisa lewat per satuan waktu?

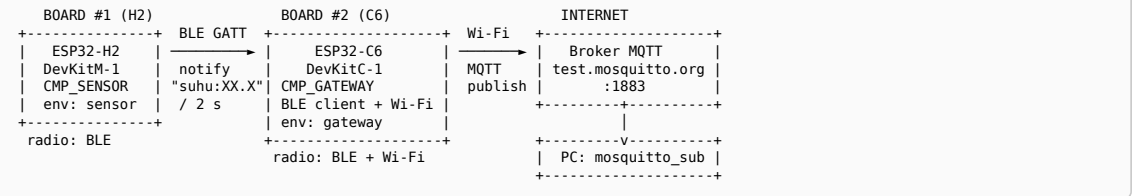
Tabel 18.4: Empat metrik dan pertanyaan yang dijawabnya

Protokol bisa unggul di satu metrik dan kalah di lainnya — itu justru inti pekerjaan modul ini. Rekomendasi yang baik menyebut **metrik mana** yang menentukan untuk kasus yang dibahas.

Sekuens protokol yang diamati

```
[H2 sensor] NimBLE server ⟶ characteristic NOTIFY
[C6] scan "CMP_SENSOR" ⟶ connect ⟶ subscribe
[C6] onTelemetry() ⟶ mqtt.publish("praktikum/h2/telemetri", "suhu:25.3")
[Broker] ⟶ [Subscriber di PC]
```

18.4 Topologi



Node	Board	Environment	Peran
Sensor	**ESP32-H2** DevKitM-1	`sensor`	BLE server `CMP_SENSOR`, notify tiap 2 s
Gateway	**ESP32-C6** DevKitC-1	`gateway`	BLE client `CMP_GATEWAY` dan Wi-Fi/MQTT publisher
Konsumen	PC/laptop	—	`mosquitto_sub`, verifikasi independen

Tabel 18.5: Peran tiap elemen Modul 16

Perhatikan: **gateway-nya sama** dengan M15 (ESP32-C6), **broker dan topic-nya sama**, yang berbeda hanya hop pertama — BLE di sini, Thread di M15. Itulah sebabnya kedua hasil bisa diletakkan berdampingan.

UUID penting: service `4fafc201-1fb5-459e-8fcc-c5c9c331914b`, characteristic telemetri `beb5483e-36e1-4688-b7f5-ea07361b26b2` — sama dengan characteristic telemetri Modul 04, jadi sensor M04 bisa dipakai ulang di sini bila perlu.

18.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (sensor) dan ESP32-C6 (gateway BLE, Wi-Fi, dan MQTT).

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1 — sensor BLE `CMP_SENSOR`	1
2	Board ESP32-C6	DevKitC-1 — gateway `CMP_GATEWAY`	1
3	Kabel USB data	kabel data, bukan *charge-only*	2
4	PC/Laptop	PlatformIO Core/IDE dan `mosquitto-clients`	1
5	Wi-Fi atau hotspot	**2,4 GHz**, ada akses internet	1
6	Broker MQTT	`test.mosquitto.org:1883` atau lokal	1
7	**Data Modul 05–15**	tabel pengukuran BLE, Zigbee, Thread, dan pipeline M15	wajib
8	Meteran atau penanda jarak	agar posisi 1 m dan 5 m benar-benar sama dengan pengukuran modul lain	1

Tabel 18.6: Alat dan bahan Modul 16

18.6 Kode Program

```
[platformio]
default_envs = gateway

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
monitor_speed = 115200

[env:sensor]
board = esp32-h2-devkitm-1
build_src_filter = +<sensor/*.cpp>
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3

[env:gateway]
board = esp32-c6-devkitc-1
build_src_filter = +<gateway/*.cpp>
; Thread/BLE + Wi-Fi + MQTT/HTTP melebihi partisi app default (1,25 MB)
board_build.partitions = huge_app.csv
lib_deps =
    h2zero/NimBLE-Arduino@^2.2.3
    knolleary/PubSubClient@^2.8
```

Kode 18.1: `platformio.ini` Modul 16 pada repositori

```
// Minggu 16 — Proyek Komparatif: sensor H2 kirim telemetri via BLE (Notify)
#include <Arduino.h>
#include <NimBLEDevice.h>

#define SERVICE_UUID    "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TELEM_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b2"

static bool deviceConnected = false;
static NimBLECharacteristic *pTelemetry = nullptr;

class ServerCallbacks : public NimBLEServerCallbacks {
public:
  void onConnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo) override {
    deviceConnected = true;
    Serial.println("Gateway terhubung");
  }
  void onDisconnect(NimBLEServer *pServer, NimBLEConnInfo &connInfo, int reason) override {
```

```
    deviceConnected = false;
    Serial.println("Gateway terputus, advertise ulang");
    NimBLEDevice::startAdvertising();
  }
};

static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Sensor (BLE telemetry) starting...");

  NimBLEDevice::init("CMP_SENSOR");

  NimBLEServer *pServer = NimBLEDevice::createServer();
  pServer->setCallbacks(new ServerCallbacks());

  NimBLEService *pService = pServer->createService(SERVICE_UUID);
  pTelemetry = pService->createCharacteristic(CHAR_TELEM_UUID, NIMBLE_PROPERTY::NOTIFY);
  pService->start();

  NimBLEAdvertising *pAdvertising = NimBLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  // NimBLE 2.x tidak lagi menyisipkan nama device secara otomatis, dan scan
  // response default-nya mati. Nama ditaruh di scan response agar muat
  // berdampingan dengan Service UUID 128-bit dan terbaca oleh active scan.
  pAdvertising->enableScanResponse(true);
  pAdvertising->setName("CMP_SENSOR");
  pAdvertising->start();

  Serial.println("Menunggu gateway...");
}

void loop() {
  static unsigned long last = 0;
  if (deviceConnected && millis() - last > 2000) {
    last = millis();
    char payload[32];
    snprintf(payload, sizeof(payload), "suhu:%.1f", readSensor());
    pTelemetry->setValue((const uint8_t *)payload, strlen(payload));
    pTelemetry->notify();
    Serial.printf("Notify: %s\n", payload);
  }
}
```

Kode 18.2: `src/sensor/main.cpp` — sensor BLE `CMP_SENSOR` di ESP32-H2

```
// Minggu 16 — Proyek Komparatif: gateway C6 terima BLE lalu publish ke MQTT
#include <Arduino.h>
#include <NimBLEDevice.h>
#include <WiFi.h>
#include <PubSubClient.h>

#define SERVICE_UUID    "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_TELEM_UUID "beb5483e-36e1-4688-b7f5-ea07361b26b2"

const char *WIFI_SSID = "SprH-3";
const char *WIFI_PASS = "yourpassword!";

// Batas waktu menunggu Wi-Fi/MQTT saat boot; melewati batas ini setup() tetap
// dilanjutkan agar kegagalan sisi IP tidak menyembunyikan sisi radio lain.
const unsigned long WIFI_TIMEOUT_MS = 30000;
// Jeda antar percobaan sambung ulang. Harus lebih panjang dari durasi
// asosiasi pada sinyal lemah, kalau tidak percobaan berikutnya justru
// membatalkan yang sedang berjalan.
const unsigned long WIFI_RETRY_MS = 20000;
// Jeda percobaan MQTT dipisah dari jeda Wi-Fi: saat Wi-Fi sudah sehat,
// tidak ada alasan menunggu selama asosiasi ulang.
const unsigned long MQTT_RETRY_MS = 4000;
const unsigned long MQTT_TIMEOUT_MS = 15000;

// Broker MQTT. Banyak jaringan kampus memblokir port 1883 keluar sehingga
```

```
// test.mosquitto.org tidak terjangkau (gejala: rc=-2 dan "Host is unreachable").
// Bila itu terjadi, jalankan `python3 tools/mqtt_broker.py` di laptop lalu ganti
// baris di bawah dengan alamat IP laptop tersebut, mis. "192.168.110.74".
const char *MQTT_BROKER = "192.168.1.5";
const uint16_t MQTT_PORT = 1884;
const char *TOPIC_TELEM = "praktikum/h2/telemetri";

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

// SIMULASI sensor BLE (tidak ada board H2 fisik): bangkitkan nilai suhu
// sintetis agar rantai gateway -> MQTT tetap bisa diuji end-to-end.
static float readSensor() {
  static float suhu = 25.0;
  suhu += (random(0, 20) - 10) / 10.0;
  if (suhu > 40.0) suhu = 25.0;
  if (suhu < 20.0) suhu = 25.0;
  return suhu;
}

// Alamat disalin (bukan pointer hasil scan) agar tetap valid setelah scan berhenti
static NimBLEAddress sensorAddr;
static bool doConnect = false;
static bool connected = false;

class ClientCallbacks : public NimBLEClientCallbacks {
public:
  void onConnect(NimBLEClient *pClient) override { connected = true; Serial.println("BLE: terhubung ke
sensor"); }
  void onDisconnect(NimBLEClient *pClient, int reason) override { connected = false; Serial.println("BLE:
terputus"); }
};

// Callback notify NimBLE 2.x: sebuah fungsi, bukan objek callback
static void onTelemetry(NimBLERemoteCharacteristic *pChar, uint8_t *pData,
                        size_t length, bool isNotify) {
  char payload[64];
  size_t n = length < sizeof(payload) - 1 ? length : sizeof(payload) - 1;
  memcpy(payload, pData, n);
  payload[n] = '\0';

  Serial.printf("RX BLE: %s\n", payload);
  if (mqtt.connected()) {
    mqtt.publish(TOPIC_TELEM, payload);
    Serial.printf("Publish MQTT [%s]: %s\n", TOPIC_TELEM, payload);
  }
}

class ScanCallbacks : public NimBLEScanCallbacks {
public:
  void onResult(const NimBLEAdvertisedDevice *d) override {
    if (d->getName() == "CMP_SENSOR") {
      Serial.println("Sensor BLE ditemukan");
      sensorAddr = d->getAddress();
      doConnect = true;
      NimBLEDevice::getScan()->stop();
    }
  }
};

static bool connectToSensor() {
  NimBLEClient *pClient = NimBLEDevice::createClient();
  pClient->setClientCallbacks(new ClientCallbacks());

  if (!pClient->connect(sensorAddr)) {
    Serial.println("BLE: gagal terhubung");
    NimBLEDevice::deleteClient(pClient);
    return false;
  }

  NimBLERemoteService *pService = pClient->getService(SERVICE_UUID);
  if (pService == nullptr) {
    Serial.println("BLE: service tidak ditemukan");
    pClient->disconnect();
    return false;
  }

  NimBLERemoteCharacteristic *pChar = pService->getCharacteristic(CHAR_TELEM_UUID);
  if (pChar == nullptr) {
    Serial.println("BLE: characteristic tidak ditemukan");
    pClient->disconnect();
    return false;
```

```
  }

  if (pChar->canNotify()) {
    pChar->subscribe(true, onTelemetry);
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Gateway (BLE -> MQTT) starting...");

  // Wi-Fi + MQTT
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);   // modem sleep memperburuk asosiasi saat sinyal lemah
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.printf("Konek Wi-Fi %s", WIFI_SSID);
  // Batasi penantian: kalau SSID/password salah atau AP tidak ada, setup()
  // tetap lanjut supaya sisi radio lain (Thread/BLE) tetap bisa diamati.
  // Wi-Fi dicoba ulang di loop().
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < WIFI_TIMEOUT_MS) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nWi-Fi OK, IP: %s | RSSI: %d dBm\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.printf("\nWi-Fi GAGAL setelah %lu s (status=%d). Lanjut tanpa Wi-Fi;\n"
                  "periksa WIFI_SSID/WIFI_PASS dan pastikan AP memancar di 2,4 GHz.\n",
                  WIFI_TIMEOUT_MS / 1000UL, (int)WiFi.status());
  }

  mqtt.setServer(MQTT_BROKER, MQTT_PORT);

  // BLE
  NimBLEDevice::init("CMP_GATEWAY");
  NimBLEScan *pScan = NimBLEDevice::getScan();
  pScan->setScanCallbacks(new ScanCallbacks());
  pScan->setActiveScan(true);
  pScan->start(5000, false);  // NimBLE 2.x: durasi dalam milidetik -> 5 detik
  Serial.println("Scanning sensor BLE...");
}


// Pemeliharaan koneksi IP, dijalankan berkala dari loop().
// Rate-limited: tanpa ini, mqtt.connect() dipanggil tiap iterasi loop dan
// membanjiri Serial dengan kegagalan DNS saat Wi-Fi/broker tidak tersedia.
static void maintainNetwork(const char *clientId) {
  static unsigned long lastWifiTry = 0;
  static unsigned long lastMqttTry = 0;

  if (WiFi.status() == WL_CONNECTED) {
    // Wi-Fi sehat: percobaan MQTT tidak boleh ikut terkunci jeda panjang milik
    // Wi-Fi, kalau tidak kesempatan menyambung jadi jauh lebih sedikit.
    if (mqtt.connected() || millis() - lastMqttTry < MQTT_RETRY_MS) return;
    lastMqttTry = millis();
    if (mqtt.connect(clientId)) {
      Serial.println("MQTT terhubung");
    } else {
      Serial.printf("MQTT gagal (rc=%d), coba lagi %lu detik\n",
                    mqtt.state(), MQTT_RETRY_MS / 1000UL);
    }
    return;
  }

  if (millis() - lastWifiTry < WIFI_RETRY_MS) return;
  lastWifiTry = millis();
  {
    // JANGAN WiFi.disconnect() di sini: pada sinyal lemah, asosiasi bisa perlu
    // > 10 detik, dan memutusnya tiap percobaan membuat Wi-Fi tidak pernah
    // selesai menyambung. Cukup picu ulang WiFi.begin() dengan jeda panjang.
    Serial.printf("Wi-Fi belum tersambung (status=%d), mencoba lagi...\n",
                  (int)WiFi.status());
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    return;                       // MQTT percuma dicoba tanpa Wi-Fi
  }
}

void loop() {
  maintainNetwork("esp32c6-gateway");
```

```
  mqtt.loop();

  // SIMULASI sensor: bangkitkan telemetri tiap 2 detik lalu publish ke MQTT.
  static unsigned long lastSim = 0;
  if (millis() - lastSim > 2000) {
    lastSim = millis();
    char payload[32];
    snprintf(payload, sizeof(payload), "suhu:%.1f", readSensor());
    Serial.printf("SIM sensor BLE: %s\n", payload);
    if (mqtt.connected()) {
      mqtt.publish(TOPIC_TELEM, payload);
      Serial.printf("Publish MQTT [%s]: %s\n", TOPIC_TELEM, payload);
    } else {
      Serial.printf("Publish MQTT GAGAL (mqtt=%d): %s\n", mqtt.state(), payload);
    }
  }

  if (doConnect) {
    if (connectToSensor()) Serial.println("BLE: koneksi berhasil");
    doConnect = false;
  }
  delay(10);
}
```

Kode 18.3: `src/gateway/main.cpp` — gateway BLE ke MQTT di ESP32-C6

18.7 Build dan Flash

```
mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v   # terminal 1
pio run -d week16_comparative -e sensor  -t upload -t monitor        # terminal 2
pio run -d week16_comparative -e gateway -t upload -t monitor        # terminal 3
```

Pre-flight checklist

- ☐ ESP32-H2 dan ESP32-C6 terhubung ke PC via kabel USB, port dicatat.
- ☐ `WIFI_SSID` dan `WIFI_PASS` pada `src/gateway/main.cpp` sudah disesuaikan (hotspot **2,4 GHz**).
- ☐ Env `gateway` memakai `board_build.partitions = huge_app.csv` (firmware BLE, Wi-Fi, dan MQTT lebih dari 1,25 MB).
- ☐ Broker dapat dijangkau (`mosquitto_sub -h test.mosquitto.org -t "praktikum/#" -v`).
- ☐ Firmware `sensor` dan `gateway` berhasil di-build.
- ☐ Dua Serial Monitor (115200) dibuka, satu untuk tiap board.
- ☐ **Data Thread M15 tersedia** sebagai pembanding, beserta catatan kondisi pengukurannya.
- ☐ Posisi uji (1 m, 5 m, 5 m + dinding) ditandai fisik agar bisa diulang identik.

18.8 Percobaan

18.8.1 EXP-01 — Sensor BLE dan Advertising

Deploy firmware `sensor` ke H2. NimBLE membuat server GATT dengan characteristic NOTIFY lalu mengadvertise nama `CMP_SENSOR`. Gateway belum dinyalakan.

```
[H2] NimBLE init "CMP_SENSOR" ──► createService ──► characteristic NOTIFY
     ──► startAdvertising
```

Data capture

Parameter	Hasil
Nama perangkat BLE	
Service dan Characteristic UUID	
Interval kirim data (s)	

Parameter	Hasil
Properti characteristic	
Status serial saat advertising	

Tabel 18.7: Lembar pengamatan EXP-01

CHECKPOINT

H2 mencetak `Menunggu gateway...` dan belum mengirim notify apa pun. Ini kondisi awal yang benar sebelum gateway dinyalakan.

18.8.2 EXP-02 — Gateway: Scan, Connect, Subscribe

Deploy firmware `gateway` ke C6. Gateway konek Wi-Fi, inisialisasi BLE sebagai `CMP_GATEWAY`, active scan 5 detik, berhenti saat menemukan `CMP_SENSOR`, lalu connect dan subscribe notify.

```
[C6] WiFi.begin ⟶ NimBLE init ⟶ scan(5s) ⟶ "Sensor BLE ditemukan"
    ⟶ connect ⟶ subscribe(notify)
[H2] onConnect ⟶ "Gateway terhubung" ⟶ notify "suhu:XX.X" tiap 2 s
```

Expected output — H2

```
Sensor (BLE telemetry) starting...
Menunggu gateway...
Gateway terhubung
Notify: suhu:25.1
Notify: suhu:25.9
```

Expected output — C6

```
Gateway (BLE -> MQTT) starting...
Konek Wi-Fi SprH-3......
Wi-Fi OK, IP: 192.168.1.39 | RSSI: -67 dBm
Scanning sensor BLE...
SIM sensor BLE: suhu:24.2
Publish MQTT GAGAL (mqtt=-1): suhu:24.2
MQTT terhubung
SIM sensor BLE: suhu:24.0
Publish MQTT [praktikum/h2/telemetri]: suhu:24.0
SIM sensor BLE: suhu:24.8
Publish MQTT [praktikum/h2/telemetri]: suhu:24.8
```

CATATAN

Log di atas direkam **tanpa board H2**: baris `SIM sensor BLE: ...` berasal dari simulasi sensor di firmware gateway. Bila board H2 terpasang, baris `Sensor BLE ditemukan`, `BLE: terhubung ke sensor`, `BLE: koneksi berhasil`, dan `RX BLE: suhu:...` ikut muncul. Publish pertama gagal (`mqtt=-1`) karena MQTT belum tersambung; setelah `MQTT terhubung`, seluruh publish berhasil.

BUKA ABSTRAKSINYA

Perhatikan bahwa gateway C6 di modul ini menjalankan **BLE dan Wi-Fi**, sedangkan gateway C6 di M15 menjalankan **Thread dan Wi-Fi**. Bandingkan ukuran firmware keduanya dari ringkasan `pio run`. Lalu jawab: mana yang lebih besar, dan apa artinya untuk perangkat gateway dengan flash terbatas? Ini adalah metrik kelima yang jarang diukur orang, tetapi nyata biayanya.

CHECKPOINT

Untuk **satu** nilai suhu yang sama, tiga baris berikut harus dapat ditunjuk: `Notify:` di H2, `RX BLE:` di C6, dan `Publish MQTT` di C6. Persis seperti M15 — struktur pengamatannya sengaja dibuat identik.

18.8.3 EXP-03 — Verifikasi End-to-End dan Variasi

Jalankan `mosquitto_sub -h test.mosquitto.org -t "praktikum/h2/telemetri" -v` dan pastikan tiap ± 2 detik muncul `praktikum/h2/telemetri suhu:25.1`.

Variasi wajib:

1. **Jarak H2 ke C6**: 1 m, 5 m, lalu di balik dinding — posisi harus sama persis dengan pengukuran M15. Catat apakah koneksi BLE bertahan dan pesan tetap sampai.
2. **Putus koneksi**: reset H2, amati `onDisconnect` sehingga H2 advertise ulang; catat apakah gateway menyambung kembali sendiri atau perlu di-reset.
3. **Ulangi pengukuran pipeline Thread M15** pada kondisi jarak yang sama bila sempat — ini menghilangkan keraguan terbesar pada tabel pembanding.

Data capture

Parameter	Hasil
Interval pesan di subscriber (s)	
Pesan per 2 menit	
RSSI Wi-Fi gateway	
RSSI BLE (dari `getRssi()` di C6)	
Perilaku saat sensor reset	

Tabel 18.8: Lembar pengamatan EXP-03

CHECKPOINT

Tersedia **tiga baris data BLE** (1 m, 5 m, 5 m + dinding) dengan kondisi yang dapat dibuktikan sama dengan data M15. Tanpa itu, tabel perbandingan di bagian Pengukuran tidak sah — dan itulah yang paling sering membuat laporan modul ini gugur.

18.9 Verifikasi Hardware — Log Referensi

Dijalankan pada **ESP32-H2 DevKitM-1** dan **ESP32-C6 DevKitC-1** asli, broker lokal (`tools/mqtt_broker.py`), jarak ±20 cm, capture 60 detik.

```
# H2 (env sensor)             # C6 (env gateway)              # Broker
[..] Gateway terhubung        [2.004] Wi-Fi OK, IP: ...197    CONNECT id=esp32c6-gateway
[..] Notify: suhu:24.7                | RSSI: -84 dBm         PUBLISH praktikum/h2/telemetri
[..] Notify: suhu:23.9        [2.204] Sensor BLE ditemukan             suhu:23.7
                              [3.606] BLE: koneksi berhasil   PUBLISH praktikum/h2/telemetri
                              [4.608] RX BLE: suhu:24.7               suhu:22.7
                              [20.434] MQTT terhubung
                              [20.434] Publish MQTT [...]: suhu:23.7
```

Parameter	Hasil terukur
Waktu boot sampai Wi-Fi OK (C6)	2,0 s (RSSI −84 dBm)

Parameter	Hasil terukur
Waktu boot sampai BLE tersambung ke sensor	3,6 s
Notify BLE dikirim H2 / diterima C6	28 / 28 (0 % loss)
Publish MQTT tiba di broker	47/47 — diverifikasi dari log broker, bukan dari log gateway
BLE, Wi-Fi, dan MQTT jalan bersamaan di satu C6	stabil

Tabel 18.9: Hasil verifikasi hardware Modul 16

Pengukuran pembanding langsung BLE dan Thread — pipeline M16 dan M15 diukur pada AP, jarak, dan broker yang sama persis, masing-masing 100 detik (Tabel 18.10).

Pipeline	Modul	Hop sensor	Hop Wi-Fi/MQTT	End-to-end
BLE ke C6 ke MQTT	M16 (ini)	47/48 (98 %)	47/47 (100 %)	**98 %**
Thread ke C6 ke MQTT	M15	30/39 (77 %)	2/7 (29 %)	**5 %**

Tabel 18.10: Pembanding langsung pipeline BLE dan Thread

Pada jaringan lain (AP kanal 12) M15 mencapai 36 % dan M16 mencapai 82 % — urutannya tetap sama, selisihnya tetap besar.

Yang membedakan bukan protokol di atas kertas, melainkan **biaya koeksistensi radio pada satu chip satu antena**: BLE duty-cycled sehingga berbagi antena dengan Wi-Fi nyaris gratis; 802.15.4 pada peran Router selalu RX sehingga menekan airtime Wi-Fi sampai TCP tidak lewat.

> **CATATAN**
> Masukkan temuan ini ke analisis nomor 4 dan ke kolom “kondisi ukur” tabel perbandingan. Kesimpulan yang tepat bukan “Thread lebih buruk dari BLE”, melainkan “pada gateway satu-chip, Thread dengan Wi-Fi berbagi radio jauh lebih mahal daripada BLE dengan Wi-Fi” — gateway dua-chip atau border router khusus akan mengubah angka ini sepenuhnya.

> **PENTING**
> **Jangan percaya `publish()` yang mengembalikan `true`.** Pada run M15 di atas, gateway mencetak 7 baris `Publish MQTT [...]` sementara broker hanya menerima
>
> 2. Pada QoS 0 itu wajar: `publish()` hanya menulis ke buffer socket. Kolom
>
> “Broker menerima” pada tabel hasil pengukuran **wajib** diisi dari log broker atau subscriber.

Perbaikan kode yang lahir dari uji ini

Masalah di perangkat nyata	Perbaikan
`mqtt.connect()` dipanggil tiap iterasi `loop()` tanpa jeda, sehingga ribuan baris `DNS Failed` membanjiri Serial dan menenggelamkan log BLE	`maintainNetwork()` dengan jeda dan pengecekan status Wi-Fi lebih dulu
`while (WiFi.status() != WL_CONNECTED)` menggantung selamanya	dibatasi `WIFI_TIMEOUT_MS`, lalu lanjut; kaki BLE tetap bisa diamati

Tabel 18.11: Perbaikan kode hasil pengujian perangkat nyata Modul 16

18.10 Pengukuran

Pengukuran BLE (proyek ini)

Skenario / Jarak	RSSI BLE	Latency end-to-end (Notify ke subscriber, ms)	Success / 40
1 m			
5 m			
5 m + dinding			

Tabel 18.12: Lembar pengukuran pipeline BLE Modul 16

Tabel perbandingan protokol — deliverable utama modul ini. Setiap sel wajib disertai asal datanya.

Protokol	Sumber	Kondisi ukur	RSSI (dBm)	Latency (ms)	Loss (%)	Throughput (pesan/mnt)
BLE ke MQTT	M16 (ini)					
Thread ke MQTT	M15					
Zigbee (hop sensor)	M08–M10					
802.15.4 raw (baseline)	M07					

Tabel 18.13: Tabel perbandingan protokol — deliverable utama Modul 16

PENTING
Kolom **"Kondisi ukur"** wajib diisi (jarak, interval, ada atau tidaknya penghalang, serta tanggal). Sel yang kondisinya berbeda dari baris lain harus ditandai dan disebut sebagai keterbatasan di bagian Analisis — bukan disembunyikan.

Zigbee tidak punya pipeline MQTT di lab ini, jadi angkanya adalah **latency hop sensor saja**, bukan end-to-end. Menuliskannya sebagai end-to-end adalah kesalahan yang akan langsung terlihat saat sidang.

Metrik pendukung (opsional tetapi bernilai)

Aspek	BLE (M16)	Thread (M15)
Ukuran firmware gateway		
Waktu dari boot sampai data pertama sampai		
Pemulihan setelah sensor di-reset		

Tabel 18.14: Metrik pendukung pembanding BLE dan Thread

18.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran, dan **tunjuk barisnya**.

1. Berdasarkan data, protokol mana yang paling andal (packet loss terkecil) pada jarak terjauh, dan mengapa?

2. Bagaimana latency end-to-end BLE ke MQTT dibandingkan Thread ke MQTT untuk payload yang sama? Berapa selisihnya, dan dari hop mana selisih itu berasal?
3. Mengapa BLE connection-based lebih sensitif terhadap jarak atau penghalang dibanding mesh Thread? Kaitkan dengan data M06 (relay BLE) dan M12 (mesh Thread).
4. Protokol mana yang paling efisien untuk node baterai dengan interval kirim jarang? Jelaskan dengan data, termasuk konsekuensi peran router (M10).
5. Untuk kasus (a) rumah pintar 15 node, (b) sensor tunggal dekat gateway, dan (c) gedung bertingkat — protokol apa yang direkomendasikan? Dasarkan tiap jawaban pada baris tabel tertentu.
6. **Apa keterbatasan terbesar dari perbandingan ini?** Sebutkan minimal dua, dan jelaskan apa yang perlu diukur untuk mengatasinya.

18.12 Concept Check

1. Jelaskan perbedaan topologi BLE (star atau connection), Zigbee (mesh), dan Thread (mesh IPv6).
2. Apa itu GATT notify dan mengapa dipakai, bukan indication atau polling read?
3. Bagaimana gateway menemukan dan memilih sensor yang tepat saat scanning? Perhatikan `onResult`.
4. Apa peran MQTT dalam proyek ini, dan mengapa perbandingan protokol dilakukan di hop sensor ke gateway, bukan di hop MQTT?
5. Jika throughput naik 10 kali (interval 200 ms), protokol mana yang paling mungkin bermasalah lebih dulu? Mengapa, dan data modul mana yang mendukung dugaan awal?

18.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Rekomendasi berbasis data dan loss per hop

CH-1 — Rekomendasi berbasis data (wajib) Tulis mini-laporan satu halaman yang memilih protokol untuk **satu** kasus nyata, dengan tabel bagian Pengukuran sebagai justifikasi. Sertakan satu paragraf "kapan rekomendasi ini salah" — kondisi yang akan membalikkan kesimpulan yang diambil.

CH-3 — Packet loss (wajib) Hitung packet loss dengan sequence number pada payload (`suhu:25.3,#21`). Contoh: 60 notify terkirim, 57 sampai di subscriber, sehingga loss = (60 − 57)/60 × 100 % = 5 %. Pisahkan loss hop BLE dan hop MQTT.

Level 3 — Tantangan · Menambah sumber dan melengkapi baris Zigbee

CH-2 — Dua sensor BLE Tambahkan sensor BLE kedua (`CMP_SENSOR2`, UUID berbeda) dan buat gateway mem-forward keduanya ke topic berbeda. Ukur apakah latency berubah saat gateway melayani dua koneksi — bandingkan dengan temuan M05.

CH-4 — Pipeline Zigbee Lengkapi baris Zigbee pada tabel perbandingan dengan pipeline yang setara (Zigbee ke gateway ke MQTT), sehingga ketiga protokol diukur end-to-end pada kondisi yang sama. Ini pekerjaan terbesar — dan yang membuat tabel hasil pengukuran benar-benar utuh.

18.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (BLE/GATT, Zigbee, Thread, MQTT, serta metodologi perbandingan: variabel kontrol dibanding variabel bebas).
3. Konfigurasi — UUID, nama perangkat, SSID, broker, topic, NimBLE-Arduino, dan `huge_app.csv`.

4. Hasil eksperimen — log H2, C6, dan `mosquitto_sub` (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel BLE **dan tabel perbandingan protokol lengkap dengan kolom "kondisi ukur"**.
6. Analisis dan concept check, termasuk pernyataan keterbatasan.
7. Challenge — CH-1 dan CH-3 wajib.
8. **Kesimpulan dan rekomendasi protokol** — ditulis sendiri, dengan setiap klaim menunjuk baris data.

A. Skematik Board ESP32-H2

Lampiran ini memuat skematik board Waveshare ESP32-H2-DEV-KIT-N4 yang dipakai pada seluruh modul berbasis ESP32-H2. Skematik diperlukan setiap kali sebuah modul menyebut nomor GPIO: rujukan pin pada tiap modul berasal dari lembar ini, bukan dari datasheet chip. Gambar selengkapnya diberikan pada Gambar A.1, yang ditempatkan pada halaman tersendiri dengan orientasi mendatar agar seluruh blok rangkaian tetap terbaca.

A.1 Pin yang Dipakai di Buku Ini

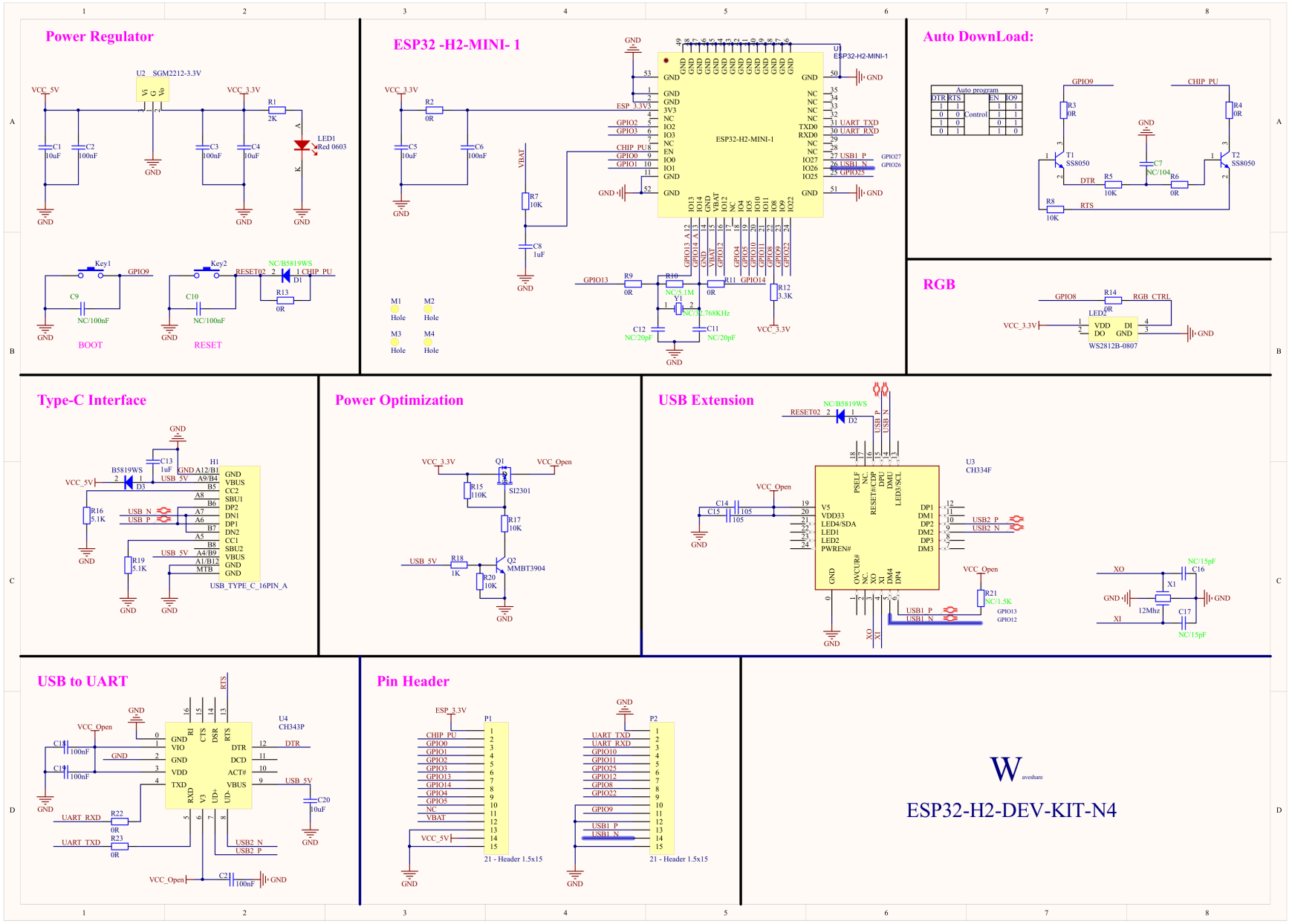
Dua pin berikut dipakai langsung pada modul warm-up dan dirujuk kembali pada modul-modul selanjutnya.

Fungsi	GPIO	Nama di skematik	Keterangan
LED RGB WS2812	GPIO8	`RGB_CTRL`	satu pixel, urutan byte RGB
Tombol BOOT	GPIO9	`Key2`	terhubung ke GND saat ditekan, pull-up 10K di board, sehingga *active low*; sekaligus *strapping pin* mode download

Tabel A.1: Pin board yang dipakai pada Modul 00A dan 00B

CATATAN
Board Waveshare ESP32-H2-DEV-KIT-N4 memakai modul ESP32-H2-MINI-1 dan pinout yang setara dengan ESP32-H2-DevKitM-1, sehingga definisi board `esp32-h2-devkitm-1` pada `platformio.ini` dapat dipakai apa adanya.

CATATAN
Gambar pada lampiran ini dihasilkan dari berkas `skematik/ESP32-H2-DEV-KIT-N4.pdf` pada repositori praktikum, dirender pada resolusi 300 dpi. Berkas PDF aslinya tetap menjadi acuan bila diperlukan pembesaran lebih jauh.



Gambar A.1: Skematik Waveshare ESP32-H2-DEV-KIT-N4

B. Modul 00A — Blinky ESP32-H2

IoT COMMUNICATION LAB · MODUL 00A

Verify the Toolchain — Blinky ESP32-H2 (modul pemanasan)

Waveshare ESP32-H2-DEV-KIT-N4 · satu node · level Basic · 1 × 50 menit · folder kode `week00_blinky`

PENGANTAR MODUL · Gambaran Umum

Modul 00A adalah modul pemanasan yang dikerjakan sebelum M01, dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat dasar. Misinya tunggal dan sempit: memastikan toolchain PlatformIO, board, dan jalur flash benar-benar bekerja sebelum modul komunikasi dimulai. Percobaan berjalan pada satu node dengan LED RGB onboard yang berkedip sebagai penanda keberhasilan, dan Serial Monitor pada 115200 baud sebagai satu-satunya instrumen pengamatan.

B.1 Pendahuluan

Modul ini tidak mengajarkan protokol apa pun. Fungsinya menutup satu sumber kebingungan yang berulang di laboratorium: ketika sebuah modul komunikasi gagal, penyebabnya bisa berada di protokol, di firmware, atau di rantai kerja paling dasar — toolchain, board, kabel, dan port. Dengan menuntaskan modul ini lebih dahulu, kemungkinan terakhir dapat dicoret sejak awal.

Bekal yang diperlukan hanya dasar bahasa C dan PlatformIO Core/IDE yang sudah terpasang; tidak ada modul yang mendahuluinya. Yang dibangun di sini ada tiga, dan ketiganya dipakai terus-menerus sesudahnya: struktur proyek PlatformIO, alur kerja build–flash–monitor, serta kendali LED RGB WS2812. Ketiganya langsung dipakai kembali pada M00B ketika jalur masukan ditambahkan, sedangkan rantai build–flash–monitor yang sama menjadi dasar kerja seluruh modul M01 hingga M16.

B.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membuktikan rantai kerja PlatformIO ke ESP32-H2

1. Menjelaskan posisi ESP32-H2 dalam ekosistem ESP32 (inti RISC-V, radio BLE dan IEEE 802.15.4) dan kesetaraan pinout Waveshare ESP32-H2-DEV-KIT-N4 dengan ESP32-H2-DevKitM-1.
2. Membuat proyek PlatformIO menggunakan fork pioarduino, serta menjelaskan alasan platform resmi belum dapat dipakai untuk board ESP32-H2.
3. Melakukan build, flash, dan monitor firmware, serta memverifikasi LED RGB WS2812 pada GPIO8 bekerja sesuai program.
4. Membaca keluaran Serial Monitor sebagai instrumen verifikasi, bukan sekadar catatan tambahan.

Kriteria keberhasilan

- ☐ Proses `pio run -t upload` selesai tanpa galat.
- ☐ LED RGB onboard berkedip dengan periode 1 detik.
- ☐ Serial Monitor menampilkan pesan startup tepat satu kali setelah reset.
- ☐ Warna LED berhasil diubah melalui `strip.Color(r, g, b)`.

B.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Istilah kerja yang diperlukan dirangkum pada Tabel B.1.

Istilah	Definisi kerja di lab ini
ESP32-H2	SoC berinti RISC-V dengan radio BLE 5 dan IEEE 802.15.4, tanpa Wi-Fi.
pioarduino	Fork platform `espressif32` yang menyediakan definisi board ESP32-H2, yang belum tersedia pada platform resmi PlatformIO.
WS2812	LED RGB beralamat: warna dikirim sebagai deretan bit pada satu jalur data, bukan diatur oleh tegangan pin seperti LED biasa.
Urutan byte warna	Urutan pengiriman komponen warna. WS2812 umumnya GRB; pada board ini pengamatan empiris menunjukkan urutan **RGB**, sehingga dipakai `NEO_RGB`.
Environment PlatformIO	Konfigurasi build bernama pada `platformio.ini`; dipilih dengan opsi `-e`.

Tabel B.1: Istilah kerja Modul 00A

B.4 Alat yang Digunakan

Seluruh percobaan dijalankan pada Waveshare ESP32-H2-DEV-KIT-N4 (modul ESP32-H2-MINI-1) dengan Arduino core 3.x di atas PlatformIO.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	Waveshare ESP32-H2-DEV-KIT-N4	1
2	Kabel USB-C	kabel data, bukan *charge-only*	1
3	PC/Laptop	PlatformIO Core/IDE terpasang	1

Tabel B.2: Alat dan bahan Modul 00A

Fungsi	GPIO	Keterangan (dari skematik board)
LED RGB WS2812	GPIO8	`RGB_CTRL`, satu pixel, urutan byte RGB

Tabel B.3: Pemetaan pin Modul 00A

Skematik lengkap board tersedia pada Bab A.

Struktur proyek

```
week00_blinky/
├── platformio.ini
└── src/
    └── main.cpp
```

B.5 Kode Program

```
[platformio]
default_envs = node

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    adafruit/Adafruit NeoPixel@^1.15.5

[env:node]
```

Kode B.1: `platformio.ini` Modul 00A

Berkas `src/main.cpp` berisi keseluruhan program modul ini (Kode B.2). Program menyalakan satu pixel WS2812 pada GPIO8 dengan warna merah selama 500 ms, lalu memadamkannya selama 500 ms, sehingga periode kedipnya 1 detik.

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h>

// LED RGB onboard Waveshare ESP32-H2-DEV-KIT-N4: WS2812 pada GPIO8,
// order byte RGB (bukan GRB standar WS2812) sesuai pengamatan empiris
#define LED_PIN 8
#define NUM_LEDS 1

Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_RGB + NEO_KHZ800);

void setup() {
  strip.begin();
  strip.setBrightness(50);
  strip.show();
  Serial.begin(115200);
  Serial.println("Blinky RGB WS2812 ESP32-H2-DEV-KIT-N4 dimulai");
}

void loop() {
  strip.setPixelColor(0, strip.Color(255, 0, 0));
  strip.show();
  delay(500);
  strip.setPixelColor(0, strip.Color(0, 0, 0));
  strip.show();
  delay(500);
}
```

Kode B.2: `src/main.cpp` — blinky LED RGB WS2812

B.6 Build dan Flash

```
pio run -d week00_blinky -e node -t upload --upload-port /dev/ttyACM0
pio device monitor -p /dev/ttyACM0 -b 115200
```

PENTING

Pilih port USB-to-UART, bukan USB native. Board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Flash dilakukan lewat **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- [ ] `pio device list` dijalankan dan port jembatan UART (`1A86:55D3`) tercatat.
- [ ] Kabel yang dipakai dipastikan kabel data.
- [ ] Environment `node` dikenali PlatformIO.

B.7 Percobaan

B.7.1 EXP-01 — Build dan Flash Pertama

Bangun dan unggah firmware, lalu amati LED serta Serial Monitor.

Expected output

```
Blinky RGB WS2812 ESP32-H2-DEV-KIT-N4 dimulai
```

Data capture

Parameter	Hasil
Port serial yang terdeteksi	
Waktu build pertama (s)	
Pemakaian Flash / RAM dari ringkasan build	
Periode kedip terukur (s)	

Tabel B.4: Lembar pengamatan EXP-01

CHECKPOINT

LED berkedip **dan** pesan startup muncul tepat satu kali. Pesan yang muncul berulang menandakan board melakukan reset berkala; hentikan dan periksa catu daya serta kabel sebelum melanjutkan.

B.7.2 EXP-02 — Warna sebagai Data

Ubah argumen `strip.Color(r, g, b)`, unggah ulang, dan amati perubahannya. Percobaan ini menegaskan perbedaan mendasar LED beralamat dari LED biasa: yang dikirim adalah **nilai**, bukan sekadar keadaan nyala atau padam.

Nilai `Color(r, g, b)`	Warna yang teramati
`(255, 0, 0)`	
`(0, 255, 0)`	
`(0, 0, 255)`	
`(255, 255, 255)`	

Tabel B.5: Lembar pengamatan EXP-02 — pemetaan nilai warna

CHECKPOINT

Warna yang teramati sesuai dengan urutan RGB. Apabila `(255, 0, 0)` justru menghasilkan hijau, urutan byte board berbeda dan konstanta `NEO_RGB` perlu diganti `NEO_GRB`.

B.8 Analisis

1. Mengapa nilai `strip.setBrightness(50)` memengaruhi seluruh warna secara proporsional, sedangkan `strip.Color()` menentukan komposisinya?
2. Apa yang terjadi apabila `strip.show()` tidak dipanggil setelah `setPixelColor()`? Jelaskan berdasarkan cara kerja WS2812.
3. Berdasarkan ringkasan build, berapa persen Flash yang sudah terpakai oleh program sesederhana ini? Apa penyebabnya?

B.9 Concept Check

1. Apa perbedaan LED digital biasa dan LED beralamat seperti WS2812?
2. Mengapa proyek ini memerlukan fork pioarduino, bukan platform `espressif32` resmi?
3. Apa fungsi `build_src_filter` dan `default_envs` pada `platformio.ini`?
4. Serial Monitor menampilkan pesan startup hanya sekali. Apa arti pengamatan itu terhadap alur `setup()` dan `loop()`?

B.10 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memodifikasi perilaku LED

CH-1 — Pola napas Ubah kedipan menjadi transisi terang–redup bertahap (*breathing*) menggunakan `setBrightness()` di dalam `loop()`.

CH-2 — Non-blocking Ganti `delay()` dengan penjadwalan berbasis `millis()`, lalu jelaskan mengapa pola ini wajib pada modul komunikasi berikutnya.

CH-3 — Indikator status Rancang tiga warna sebagai kode status (misalnya hijau = siap, kuning = menunggu, merah = galat) dan terapkan pada urutan `setup()`.

B.11 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Konfigurasi — isi `platformio.ini` beserta alasan pemilihan platform.
3. Hasil eksperimen — tangkapan Serial Monitor dan foto atau video LED (EXP-01 dan EXP-02).
4. Tabel pengamatan warna.
5. Analisis dan concept check.
6. Challenge — minimal CH-1.
7. Kesimpulan yang disusun sendiri berdasarkan hasil pengujian.

C. Modul 00B — Tombol BOOT ke LED

IoT COMMUNICATION LAB · MODUL 00B

Read the First Input — Tombol BOOT ke LED (modul pemanasan)

Waveshare ESP32-H2-DEV-KIT-N4 · satu node · level Basic · 1 × 50 menit · folder kode `week00_btn`

PENGANTAR MODUL · Gambaran Umum

Modul 00B adalah modul pemanasan lanjutan dari `week00_blinky`, dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat dasar. Misinya menambahkan **input** pada board: keadaan LED tidak lagi ditentukan timer, melainkan oleh penekanan tombol. Percobaan berjalan pada satu node dengan tombol BOOT mengendalikan LED RGB onboard, dan Serial Monitor 115200 baud sebagai instrumen pengamatan.

C.1 Pendahuluan

Modul 00A membuktikan jalur **keluaran** bekerja. Modul ini melengkapinya dengan jalur **masukan**, sehingga board memiliki dua unsur minimal sebuah simpul sensor: sesuatu yang dibaca dari lingkungan, dan sesuatu yang ditampilkan sebagai tanggapan. Keduanya dipakai kembali secara langsung pada M05B, ketika tombol yang sama berperan sebagai simulasi *proximity switch*.

Prasyaratnya adalah M00A: rantai build–flash–monitor dan kendali LED WS2812. Yang dibangun di sini adalah pembacaan digital input, logika *active low*, pengenalan *bounce* kontak mekanis, dan pemetaan masukan ke keluaran di dalam satu `loop()`. Keempatnya dipakai lagi pada M05B ketika tombol BOOT berperan sebagai proximity switch simulasi, dan pada setiap modul yang memerlukan pemicu manual saat pengujian.

C.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membaca masukan digital dan memetakannya ke keluaran

1. Membaca digital input pada ESP32-H2 menggunakan `pinMode(..., INPUT_PULLUP)` dan `digitalRead()`.
2. Menjelaskan logika **active low**: tombol tidak ditekan bernilai `HIGH`, tombol ditekan bernilai `LOW`, beserta peran resistor pull-up.
3. Menghubungkan masukan (tombol) dengan keluaran (LED RGB WS2812) di dalam satu `loop()`.
4. Menjelaskan gejala *bounce* kontak mekanis dan cara paling sederhana meredamnya.
5. Menjelaskan risiko GPIO9 sebagai *strapping pin* dan implikasinya saat pengujian.

Kriteria keberhasilan

- ☐ LED menyala selama tombol BOOT ditahan dan padam saat dilepas.
- ☐ Serial Monitor mencetak tepat satu baris untuk setiap perubahan keadaan.
- ☐ Perilaku *strapping pin* GPIO9 dapat dijelaskan berdasarkan pengamatan sendiri.

C.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Digital input	Pembacaan tegangan pin sebagai dua keadaan logika, `HIGH` atau `LOW`.
Pull-up	Resistor yang menahan pin pada tegangan tinggi ketika tidak ada yang menariknya turun, sehingga pin tidak mengambang. Pada board ini bernilai 10K.
Active low	Konvensi ketika keadaan **aktif** diwakili tegangan rendah. Tombol BOOT terhubung ke GND saat ditekan, sehingga ditekan berarti `LOW`.
Bounce	Pantulan kontak mekanis selama beberapa milidetik saat tombol ditekan; tanpa penyaringan, satu penekanan terbaca sebagai banyak kejadian.
Strapping pin	Pin yang keadaannya dibaca chip **pada saat reset** untuk menentukan mode boot. GPIO9 termasuk di dalamnya.

Tabel C.1: Istilah kerja Modul 00B

> **CATATAN**
> **Mengapa pencetakan dibatasi pada perubahan keadaan?** `loop()` berjalan ribuan kali per detik. Mencetak pada setiap iterasi akan membanjiri Serial Monitor dan menyembunyikan informasi yang justru dicari, yaitu **kapan** keadaan berubah. Pola "cetak hanya saat berubah" ini dipakai kembali pada seluruh modul komunikasi.

C.4 Alat yang Digunakan

Modul ini dijalankan di atas Waveshare ESP32-H2-DEV-KIT-N4 (ESP32-H2-MINI-1, Arduino core 3.x) dengan PlatformIO.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	Waveshare ESP32-H2-DEV-KIT-N4	1
2	Kabel USB-C	kabel data, bukan *charge-only*	1
3	PC/Laptop	PlatformIO Core/IDE terpasang	1

Tabel C.2: Alat dan bahan Modul 00B

Fungsi	GPIO	Keterangan (dari skematik board)
LED RGB WS2812	GPIO8	`RGB_CTRL`, satu pixel, urutan byte RGB
Tombol BOOT	GPIO9	`Key2`, terhubung ke GND saat ditekan, pull-up 10K di board, sehingga **active low**

Tabel C.3: Pemetaan pin Modul 00B

Tidak diperlukan pengawatan tambahan: tombol dan LED sudah terpasang di board. Skematik lengkap board tersedia pada Bab A.

Struktur proyek

```
week00_btn/
├── platformio.ini
└── src/
    └── main.cpp
```

C.5 Kode Program

```
[platformio]
default_envs = node

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200
lib_deps =
    adafruit/Adafruit NeoPixel@^1.15.5

[env:node]
```

Kode C.1: `platformio.ini` Modul 00B

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h>

// LED RGB onboard Waveshare ESP32-H2-DEV-KIT-N4: WS2812 pada GPIO8,
// order byte RGB (bukan GRB standar WS2812) sesuai pengamatan empiris
#define LED_PIN 8
#define NUM_LEDS 1

// Tombol BOOT (Key2 pada skematik) terhubung ke GPIO9 dengan pull-up 10K di
// board: tidak ditekan = HIGH, ditekan = LOW (active low)
#define BTN_PIN 9

Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_RGB + NEO_KHZ800);

void setup() {
  strip.begin();
  strip.setBrightness(50);
  strip.show();

  // INPUT_PULLUP: pull-up internal menyusul pull-up eksternal agar pin tidak
  // mengambang bila resistor board tidak terpasang
  pinMode(BTN_PIN, INPUT_PULLUP);

  Serial.begin(115200);
  Serial.println("Tombol BOOT (GPIO9) -> LED RGB WS2812 (GPIO8) dimulai");
}

void loop() {
  bool pressed = (digitalRead(BTN_PIN) == LOW);

  strip.setPixelColor(0, pressed ? strip.Color(0, 255, 0) : strip.Color(0, 0, 0));
  strip.show();

  // Cetak hanya saat status berubah, supaya Serial Monitor tidak banjir
  static bool last = false;
  if (pressed != last) {
    last = pressed;
    Serial.println(pressed ? "Tombol DITEKAN  -> LED nyala" : "Tombol DILEPAS  -> LED mati");
  }

  delay(20);  // sekaligus meredam pantulan kontak (bounce) tombol
}
```

Kode C.2: `src/main.cpp` — tombol BOOT mengendalikan LED RGB

C.6 Build dan Flash

```
pio run -d week00_btn -e node -t upload --upload-port /dev/ttyACM0
pio device monitor -p /dev/ttyACM0 -b 115200
```

PENTING

Pilih port USB-to-UART, bukan USB native. Board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Flash dilakukan lewat **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

PERINGATAN

Peringatan operasional. GPIO9 juga merupakan *strapping pin* mode download. Menahan tombol **pada saat board direset** akan membuat board masuk mode flash dan program tidak berjalan. Tombol hanya ditekan setelah firmware berjalan.

C.7 Percobaan

C.7.1 EXP-01 — Input Mengendalikan Output

Unggah firmware, buka Serial Monitor, lalu tekan dan lepas tombol BOOT beberapa kali.

Expected output

```
Tombol BOOT (GPIO9) -> LED RGB WS2812 (GPIO8) dimulai
Tombol DITEKAN  -> LED nyala
Tombol DILEPAS  -> LED mati
```

Data capture

Parameter	Hasil
Nilai `digitalRead()` saat tombol dilepas	
Nilai `digitalRead()` saat tombol ditekan	
Jumlah baris Serial untuk satu kali tekan–lepas	
Apakah LED padam tepat saat tombol dilepas?	

Tabel C.4: Lembar pengamatan EXP-01

CHECKPOINT

Satu kali tekan–lepas menghasilkan **tepat dua** baris. Munculnya baris berlipat menandakan *bounce* belum teredam; catat gejalanya, karena hal itu menjadi bahan CH-3.

C.7.2 EXP-02 — GPIO9 sebagai Strapping Pin

Tahan tombol BOOT, tekan tombol RESET, lalu lepaskan keduanya. Amati Serial Monitor.

Data capture

Parameter	Hasil
Keluaran Serial setelah reset dengan tombol ditahan	
Apakah program aplikasi berjalan?	

Parameter	Hasil
Cara mengembalikan board ke mode normal	

Tabel C.5: Lembar pengamatan EXP-02

CHECKPOINT

Board masuk mode download dan pesan startup **tidak** muncul. Pengamatan ini menjelaskan mengapa satu pin dapat memiliki dua peran berbeda, bergantung pada waktu pembacaannya.

C.8 Analisis

1. Mengapa `INPUT_PULLUP` tetap digunakan meskipun board sudah menyediakan pull-up 10K secara perangkat keras?
2. Apa yang terjadi pada pembacaan pin apabila pull-up dihilangkan seluruhnya? Gunakan istilah *floating* dalam penjelasan.
3. `delay(20)` pada akhir `loop()` memiliki dua fungsi sekaligus. Sebutkan keduanya dan jelaskan konsekuensinya bila nilai tersebut diperbesar menjadi 500 ms.
4. Berdasarkan EXP-02, mengapa perancang board menempatkan tombol BOOT pada pin yang juga dipakai program aplikasi? Apa keuntungan dan risikonya?

C.9 Concept Check

1. Apa arti *active low*, dan bagaimana hal itu tampak pada kode?
2. Apa perbedaan `INPUT` dan `INPUT_PULLUP`?
3. Mengapa keluaran Serial dibatasi hanya pada perubahan keadaan?
4. Apa yang dimaksud *bounce*, dan mengapa gejalanya lebih menonjol pada sakelar mekanis dibanding sensor elektronik?
5. Sebutkan satu contoh sensor nyata yang secara listrik berperilaku sama dengan tombol ini.

C.10 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memodifikasi logika masukan dan keluaran

CH-1 — Toggle Ubah program sehingga LED berganti nyala atau padam pada setiap **tepi turun** (saat tombol mulai ditekan), bukan mengikuti selama tombol ditahan.

CH-2 — Siklus warna Setiap penekanan menggeser warna: merah, hijau, biru, kembali ke merah.

CH-3 — Debounce terukur Hapus `delay(20)`, catat berapa baris ganda yang muncul pada 10 kali penekanan, lalu terapkan debounce berbasis `millis()` dan bandingkan hasilnya dalam satu tabel.

CH-4 — Pengukuran durasi Cetak lama tombol ditahan dalam milidetik pada saat dilepas, dan bandingkan hasilnya dengan hitungan manual menggunakan stopwatch.

C.11 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (active low, pull-up, bounce, strapping pin).

3. Konfigurasi — pin GPIO8 dan GPIO9 beserta rujukan skematik.
4. Hasil eksperimen — tangkapan Serial Monitor EXP-01 dan EXP-02 beserta checkpoint.
5. Analisis dan concept check.
6. Challenge — minimal CH-1 dan CH-3, disertai tabel pembanding debounce.
7. Kesimpulan yang disusun sendiri berdasarkan hasil pengujian.